

Fondamentaux de la vision par ordinateur

VNStudio

CHAPITRE

0

Table des matières

1. Décrire une forme avec des nombres	2
1.1. Circularité : à quel point est-ce rond ?	5
1.2. Élongation : trapu ou étiré ?	7
1.3. Excentricité : l'allongement vu par la masse	8
1.4. Solidité : la forme est-elle pleine ?	9
1.5. Convexité : la même enveloppe, vue par le périmètre	11
1.6. Étendue : remplir une boîte qui ne tourne pas	12
1.7. Diamètre équivalent : convertir une aire en taille	13
1.8. Rectangularité : remplir une boîte qui, elle, tourne	14
1.9. Rondeur : la forme d'ensemble, sans le bruit du bord	15
1.10. Tableau récapitulatif — ce que chaque descripteur voit et ignore	16
1.11. Une invariance est une information qu'on accepte de perdre	16
2. Moments d'image	17
2.1. Moments bruts	19
2.2. Définition	19
2.3. L'idée	20
2.4. Exemple numérique — le masque jouet du chapitre	20
2.5. Piège : en image, y descend	21
2.6. Piège : un masque, pas une image	21
2.7. Code	21
2.8. Centroïde	22
2.9. Définition	22
2.10. L'idée	22

2.11. Exemple numérique	22
2.12. Applications	22
2.13. Subtilité importante : le centroïde peut tomber hors de l'objet	23
2.14. Code	23
2.15. Piège : (x, y) ou (row, col) ?	23
2.16. Moments centraux	24
2.17. Définition	24
2.18. L'idée	24
2.19. Exemple numérique (suite du masque jouet)	24
2.20. Les moments d'ordre 3 mesurent l'asymétrie	25
2.21. Code	25
2.22. Moments centraux normalisés	25
2.23. Définition	25
2.24. L'idée	25
2.25. Exemple numérique	26
2.26. Piège : l'invariance d'échelle est exacte en théorie, approchée en pratique	26
2.27. Code	26
2.28. Orientation principale	26
2.29. Définition	26
2.30. L'idée	27
2.31. Piège : arctan2, pas arctan	27
2.32. Exemple numérique	27
2.33. Applications	27
2.34. Piège : l'orientation d'un objet rond n'existe pas	28
2.35. Code	28
2.36. Ellipse équivalente	29
2.37. Définition	29
2.38. L'idée	29
2.39. Exemple numérique (suite et fin du masque jouet)	29
2.40. Le piège central : l'ellipse ne mesure pas la taille de l'objet	30
2.41. Code	30
2.42. Piège de bibliothèque : des noms et une convention qui ont changé	30
2.43. Les sept invariants de Hu	31
2.44. L'idée	31
2.45. Pourquoi ces combinaisons résistent-elles à la rotation ?	31
2.46. Deux propriétés à connaître	31
2.47. Exemple : reconnaissance de caractères	32
2.48. Hu aujourd'hui : encore utile ?	32

2.49. Code	32
2.50. Moments pondérés par l'intensité	33
2.51. Code	33
2.52. Tableau récapitulatif — quel moment pour quelle question ?	34
3. Distances et similarités	36
3.1. Distances de Minkowski (L_p)	38
3.2. Définition	38
3.3. L'idée	39
3.4. Géométrie : la forme de la « boule unité »	39
3.5. Exemple numérique	39
3.6. Piège : la malédiction de la dimension	39
3.7. Code	40
3.8. Distance de Mahalanobis	40
3.9. Définition	40
3.10. L'idée	40
3.11. Exemple numérique	41
3.12. Applications	41
3.13. Piège : Σ singulière ou mal estimée	41
3.14. Code	42
3.15. Similarité cosinus	42
3.16. Définition	42
3.17. L'idée	42
3.18. Quand la préférer à l'eulidienne	42
3.19. Lien avec l'eulidienne sur vecteurs normalisés	43
3.20. Exemple numérique	43
3.21. Code	43
3.22. Distances entre histogrammes	43
3.23. Distance du χ^2	44
3.24. Distance de Bhattacharyya	44
3.25. Exemple numérique	44
3.26. Piège : sensibilité au découpage en cases	44
3.27. Piège de bibliothèque : <code>cv2.compareHist</code> ne calcule pas ces formules	45
3.28. Code	45
3.29. Distance de transport (Wasserstein / EMD)	45
3.30. Définition	45
3.31. L'idée fondatrice	46
3.32. Cas 1-D : une formule simple	46
3.33. Exemple numérique (cas 1-D)	46

3.34. Applications	46
3.35. Piège : coût de calcul	47
3.36. Code	47
3.37. Distance de Hausdorff	47
3.38. Définition	47
3.39. L'idée	47
3.40. Exemple numérique	48
3.41. Piège : sensibilité aux valeurs aberrantes	48
3.42. Applications	48
3.43. Code	48
3.44. Tableau récapitulatif — choisir sa mesure	49
4. Métriques de segmentation et détection	51
4.1. IoU — Intersection over Union (indice de Jaccard)	52
4.2. Coefficient de Dice (F1 sur pixels)	54
4.3. Précision, rappel et F1	56
4.4. Average Precision (AP) et mAP	58
4.5. Panoptic Quality (PQ)	61
4.6. Boundary F1 (BF score)	62
4.7. Tableau récapitulatif — quelle métrique pour quoi ?	66
4.8. aucune métrique unique ne capture tout, et c'est inévitable	66
5. Filtrage et convolution	68
5.1. La convolution 2D	69
5.2. Le noyau gaussien	72
5.3. Différence de gaussiennes (DoG) et Laplacien de gaussienne (LoG)	74
5.4. Le filtre bilatéral	77
5.5. Le filtre de Gabor	79
5.6. Tableau récapitulatif — le filtre comme a priori sur le signal	82
5.7. un filtre est un a priori, et choisir c'est supposer	83
6. Gradients et contours	85
6.1. Le gradient d'image	86
6.2. Les opérateurs de Sobel et Scharr	88
6.3. Le détecteur de Canny	90
6.4. Le tenseur de structure	93
6.5. Détecteurs de coins : Harris et Shi-Tomasi	94
6.6. Tableaux récapitulatif — du gradient à la structure	97
6.7. choisir une échelle, c'est choisir ce qu'on regarde	98
7. Couleur et photométrie	100
7.1. Luminance : du RGB au niveau de gris	101

7.2. RGB $\rightarrow$ HSV / HSL	103
7.3. CIELAB et la mesure perceptuelle	106
7.4. Gamut et conversion RGB $\rightarrow$ CMYK	108
7.5. Égalisation d'histogramme et CLAHE	111
7.6. Correction gamma et balance des blancs	113
7.7. Tableau récapitulatif — quel espace pour quelle tâche ?	116
7.8. l'espace encode une hypothèse	116
8. Géométrie projective et caméra	118
8.1. Coordonnées homogènes : l'astuce fondatrice	119
8.2. Le modèle sténopé (pinhole)	121
8.3. Calibration de caméra	124
8.4. L'homographie	126
8.5. La géométrie épipolaire	129
8.6. Stéréovision et triangulation	132
8.7. Distorsion de l'objectif et erreur de reprojection	135
8.8. Tableau récapitulatif — quelle relation pour quelle situation ?	138
8.9. l'astuce homogène et la perte de profondeur : le même geste	139
9. Flot optique et mouvement	141
9.1. La contrainte du flot optique	142
9.2. Le problème d'ouverture	145
9.3. Lucas-Kanade : la solution locale	146
9.4. Horn-Schunck : la solution globale	150
9.5. Flot épars vs flot dense : choisir	154
9.6. Tableau récapitulatif — combler l'équation manquante	157
9.7. le problème mal posé, l'a priori salvateur, et le méta-fil du livre	157
10. Transformées	159
10.1. La transformée de Fourier discrète (DFT)	160
10.2. Le théorème de convolution	163
10.3. La transformée en cosinus discrète (DCT)	165
10.4. La transformée de Hough	168
10.5. La transformée de distance	172
10.6. Tableau récapitulatif — quelle transformée pour quel but ?	174
10.7. changer de base, c'est choisir où le problème est simple	175
11. Morphologie mathématique	177
11.1. Érosion et dilatation	179
11.2. Ouverture et fermeture	181
11.3. Gradient morphologique	184
11.4. Top-hat (chapeau haut-de-forme) et black-hat	186

11.5.	Transformée tout-ou-rien et squelette	188
11.6.	Tableau récapitulatif — chaque opérateur est un choix de sonde	190
11.7.	tester une forme, c’est déclarer ce qui compte	191
12.	Seuillage et segmentation classique	193
12.1.	Seuillage d’Otsu — la décision sur l’histogramme	194
12.2.	Seuillage adaptatif — la décision locale	197
12.3.	K-means — la décision dans l’espace de caractéristiques	199
12.4.	Mean-shift — la décision par les modes de densité	201
12.5.	Contours actifs (snakes) — la décision sur une frontière	202
12.6.	Coupe de graphe (graph cut) — la décision globale et cohérente	205
12.7.	Tableau récapitulatif — méthode, a priori, forces, limites, usage	207
12.8.	données contre régularité, le curseur de toute segmentation	208
13.	Texture	210
13.1.	Statistiques du premier ordre — la texture que l’histogramme ne capte pas ...	212
13.2.	Matrice de cooccurrence (GLCM) — la statistique du second ordre	214
13.3.	Descripteurs d’Haralick — compresser la GLCM en quelques nombres	217
13.4.	Local Binary Pattern (LBP) — le micro-motif local	219
13.5.	Bancs de filtres et énergie de Gabor — la texture comme spectre orienté	222
13.6.	Tableau récapitulatif — quelle relation, quelle échelle, quel résumé	224
13.7.	la texture est une relation, pas une valeur	225
14.	Qualité d’image	227
14.1.	MSE et PSNR — la fidélité, ou l’observateur aveugle à la structure	229
14.2.	SSIM — modéliser l’observateur : luminance, contraste, structure	231
14.3.	Entropie d’image — la qualité sans référence, ou le contenu informationnel ...	234
14.4.	Mesures de netteté sans référence — variance du Laplacien, énergie de gradient	237
14.5.	Métriques perceptuelles apprises (LPIPS) — l’observateur ajusté sur l’humain	240
14.6.	Tableau récapitulatif — quel observateur, quelle référence, quel angle mort ...	243
14.7.	une métrique de qualité est un observateur déguisé	244
15.	Apprentissage profond (fonctions de coût)	245
15.1.	Entropie croisée et softmax — quand le gradient est l’erreur elle-même	247
15.2.	Dice loss — transformer une métrique de segmentation en objectif	249
15.3.	Focal loss — déplacer le gradient vers les exemples difficiles	252
15.4.	Smooth L1 (Huber) — le compromis entre stabilité et robustesse	254
15.5.	IoU loss et GIoU — optimiser la métrique, et combler ses zones plates	257
15.6.	Loss contrastive (InfoNCE) — structurer un espace de représentation	260
15.7.	Tableau récapitulatif — quel coût pour quel objectif	263

15.8. le coût n'est pas la cible, c'est la pente vers elle	263
16. Statistiques robustes	265
16.1. Médiane et point de rupture — la robustesse comme influence bornée	267
16.2. MAD — l'échelle robuste qui définit l'aberration	269
16.3. M-estimateurs — de Huber à Tukey, doser l'influence	271
16.4. IRLS — comment on résout un M-estimateur en pratique	274
16.5. RANSAC — la robustesse par consensus (et son compte d'itérations)	277
16.6. Au-delà du RANSAC « vanilla » — situer chaque variante	280
16.7. Tableau récapitulatif — qui a le droit de déplacer l'estimation ?	283
16.8. être robuste, c'est décider à l'avance ce qu'on refuse de croire	283

Décrire une forme avec des nombres

1



Table des matières

1.1.	Circularité : à quel point est-ce rond ?	5
1.1.1.	L'intention	6
1.1.2.	La forme recherchée	6
1.1.3.	Ce qu'elle mesure, et son angle mort	7
1.2.	Élongation : trapu ou étiré ?	7
1.2.1.	L'intention	7
1.2.2.	La forme recherchée	7
1.2.3.	Ce qu'elle mesure, et son angle mort	8
1.3.	Excentricité : l'allongement vu par la masse	8
1.3.1.	L'intention	8
1.3.2.	La forme recherchée	8
1.3.3.	Différence avec l'élongation	9
1.4.	Solidité : la forme est-elle pleine ?	9
1.4.1.	L'intention	10
1.4.2.	La forme recherchée	10
1.4.3.	Ce qu'elle détecte le mieux	10
1.5.	Convexité : la même enveloppe, vue par le périmètre	11
1.5.1.	L'intention	11
1.5.2.	La forme recherchée	11
1.5.3.	Le binôme avec la solidité	11
1.6.	Étendue : remplir une boîte qui ne tourne pas	12
1.6.1.	L'intention	12
1.6.2.	La forme recherchée	13
1.6.3.	Sa force et sa faiblesse assumées	13
1.7.	Diamètre équivalent : convertir une aire en taille	13
1.7.1.	L'intention	13
1.7.2.	La forme recherchée	13
1.8.	Rectangularité : remplir une boîte qui, elle, tourne	14
1.8.1.	L'intention	14
1.8.2.	La forme recherchée	14
1.8.3.	Son angle mort surprenant	14
1.9.	Rondeur : la forme d'ensemble, sans le bruit du bord	15
1.9.1.	L'intention	15
1.9.2.	La forme recherchée	15
1.9.3.	Ce que l'écart avec la circularité révèle	15
1.10.	Tableau récapitulatif — ce que chaque descripteur voit et ignore	16

1.11. Une invariance est une information qu'on accepte de perdre	16
--	----

Trier des milliers de formes sans jamais les regarder une à une : il faut d'abord les réduire à quelques nombres bien choisis. Chaque nombre voit une chose et en ignore une autre.

Un pipeline de vision par ordinateur commence presque toujours par découper les objets du fond — la silhouette d'une cellule, d'une pièce mécanique, d'un caractère imprimé, isolée comme on découperait une forme aux ciseaux dans une feuille. Vient ensuite la vraie question : comment trier, comparer et classer ces objets par milliers, sans qu'un humain les regarde un par un ? Une machine ne sait pas dire « cet objet est rond » ou « celui-là est allongé ». Il faut lui donner des nombres.

C'est le rôle d'un **descripteur de forme** : un nombre, ou une petite poignée de nombres, qui résume une silhouette selon un aspect précis. Rond, étiré, troué, dentelé — chaque trait géométrique a son descripteur.

Le fil de ce chapitre tient en une phrase : **chaque descripteur choisit ce qu'il voit et ce qu'il oublie**. La circularité repère un bord rugueux mais ignore l'orientation. La rectangULARITÉ voit comment les coins sont remplis mais reste aveugle à l'allongement. Comprendre un descripteur, c'est d'abord connaître son angle mort — ce qu'il refuse de voir.

Deux mots de vocabulaire avant de commencer. Une **région** désigne l'ensemble des pixels d'un objet, livrés sous forme de **masque binaire** : une image où chaque pixel vaut 1 s'il appartient à l'objet, 0 sinon. On note **A** l'aire de la région (son nombre de pixels) et **P** son périmètre (la longueur de son contour).



La plupart des descripteurs de ce chapitre sont des **rapports** de deux grandeurs. C'est un choix : un rapport efface la taille absolue et ne garde qu'une proportion, ce qui rend la mesure insensible à l'échelle. Voir la forme **a/b**, annexe C.

1.1. Circularité : à quel point est-ce rond ?

Le ballon de baudruche qui cherche la forme la plus économe

Observation 1.1 — circularité ≠ rondeur (le bord vs la forme d'ensemble)

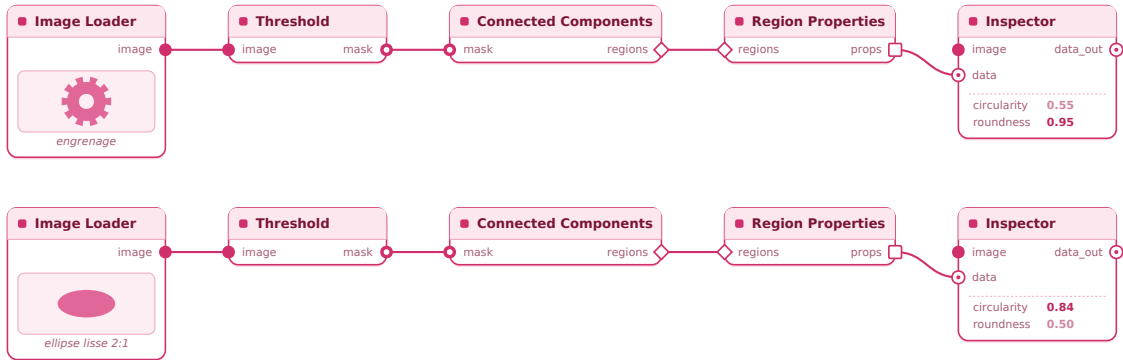


Figure 1.1 – Un disque lisse et un disque dentelé : la circularité chute pour le second, la rondeur (§1.9) non.

1.1.1. L'intention

On veut séparer automatiquement les objets ronds des objets allongés ou déchiquetés. En reconnaissance de caractères, distinguer un « O » d'un « I ». En biologie, repérer les cellules saines (rondes) parmi des débris. Il nous faut un nombre qui vaut son maximum pour un cercle parfait, et qui chute dès que la forme s'éloigne du disque.

1.1.2. La forme recherchée

Le cercle a une propriété remarquable : pour une longueur de contour donnée, c'est lui qui enferme le plus de surface. Un ballon de baudruche qu'on gonfle prend spontanément une forme ronde, parce que c'est la façon la plus économe de contenir l'air avec la peau de plastique disponible. Cette propriété s'appelle l'**inégalité isopérimétrique**.

On cherche donc un nombre qui compare l'aire à la longueur du contour, et qu'on cale pour qu'il vaille exactement **1 au cercle** et moins partout ailleurs.



$$C = 4 \cdot \pi \cdot A / P^2$$

Le facteur 4π n'a rien d'arbitraire : il est choisi précisément pour que le disque tombe sur 1. Pour un cercle de rayon r , on a $A = \pi r^2$ et $P = 2\pi r$, et la formule donne $4\pi \cdot \pi r^2 / (2\pi r)^2 = 1$. Toute autre forme, plus étirée ou plus froissée, a relativement trop de contour pour son aire, et C descend vers 0.

1.1.3. Ce qu'elle mesure, et son angle mort

La circularité baisse pour **deux raisons** que rien ne permet de distinguer : la forme s'allonge, ou son bord se dentelle. Une forme lisse mais en cigare, et un disque parfait au bord en lame de scie, peuvent décrocher la même valeur. La circularité mêle ces deux causes en un seul nombre — c'est son angle mort. (La rondeur, §1.9, séparera les deux.)

Elle reste en revanche insensible à la translation, à la rotation et à l'échelle : un cercle est un cercle, où qu'il soit, quelle que soit sa taille.



Un « O » bien formé donne environ $C \approx 0,9$. Un « I » modélisé par une barre de 40 px de haut sur 4 de large : $C = 4\pi \cdot 160 / 88^2 \approx 0,26$. Un simple seuil à 0,5 sépare d'emblée les caractères ronds des caractères linéaires.



Sur une grille de pixels, le contour d'un cercle est fait de petites marches d'escalier. Ce chemin en escalier est toujours plus long que la courbe lisse qu'il imite. Mesuré naïvement, le périmètre d'un cercle discret est gonflé d'environ **27 %**, ce qui écrase la circularité à $C \approx 0,79$ au lieu de 1 — un résultat faux, et silencieux. La méthode de calcul du périmètre doit donc être fixée et connue pour que les valeurs soient comparables d'une mesure à l'autre.



Canvas : Image Source → Threshold → Find Contours → Shape Descriptors. Le nœud de descripteurs affiche aire, périmètre et circularité dans l'inspecteur, et peut router les objets selon un seuil.

1.2. Élongation : trapu ou étiré ?

Le plus petit carton dans lequel l'objet rentre, quitte à pencher la tête

1.2.1. L'intention

En microscopie, on veut séparer d'un coup d'œil les bactéries rondes des bâtonnets et des longues fibres. Une mesure grossière mais immédiate de l'allongement suffit.

1.2.2. La forme recherchée

On enferme l'objet dans le plus petit rectangle possible, puis on regarde le rapport de ses deux côtés. Détail décisif : ce rectangle doit pouvoir **pivoter** librement pour épouser la forme au plus près. Un crayon posé en diagonale, enfermé dans une boîte strictement

horizontale, donnerait une boîte presque carrée et une mesure trompeuse. En autorisant la boîte à pencher la tête, on rend la mesure indépendante de l'orientation.



$$E = L_{\max} / L_{\min}$$

où $L_{\max}$ et $L_{\min}$ sont la longueur et la largeur de la **boîte englobante orientée**.

1.2.3. Ce qu'elle mesure, et son angle mort

L'élongation dit si la forme générale est trapue ou étirée, rien de plus. Son angle mort est béant : elle ignore tout ce qui se passe à *l'intérieur* de la boîte. Une équerre en « L » et une barre en diagonale peuvent partager la même boîte orientée, donc la même élongation, alors que leur matière est répartie de façons radicalement différentes.



Un globule rouge : $E \approx 1,0$. Une bactérie en bâtonnet : $E \approx 5$ à 8 . Une fibre : souvent $E > 20$.



`cv2.minAreaRect` renvoie les deux dimensions de la boîte dans un ordre quelconque. Un tri explicite du max et du min est nécessaire avant de calculer le rapport, sans quoi l'élongation peut sortir inversée.



Canvas : Find Contours $\rightarrow$ Min Area Rect $\rightarrow$ Shape Descriptors. La boîte orientée est dessinée en surimpression sur l'objet, ce qui rend l'inclinaison visible directement.

1.3. Excentricité : l'allongement vu par la masse

Un nuage de points, et la direction où il s'étire le plus

1.3.1. L'intention

L'élongation se laisse berner par un seul pixel de bruit, qui suffit à élargir la boîte englobante. On veut la même information — l'allongement — mais mesurée sur la répartition de *tous* les pixels, pour qu'un point aberrant ne pèse presque rien.

1.3.2. La forme recherchée

L'image mentale utile est celle d'un **nuage de points** : une silhouette numérique n'est qu'un amas de pixels. On cherche la direction dans laquelle ce nuage est le plus étalé, et la

direction perpendiculaire où il l'est le moins. Un nuage parfaitement circulaire s'étale pareil dans toutes les directions ; un nuage en fin pinceau s'étire dans une seule.

L'outil mathématique qui analyse ce nuage en sort deux nombres, λ_1 et λ_2 . Inutile de maîtriser l'algèbre matricielle qui les calcule : visuellement, **λ_1 mesure l'envergure du nuage dans sa direction la plus longue, et λ_2 dans sa direction la plus courte**. Ce sont les deux « rayons » d'une ellipse qui résumerait le nuage.



$$e = \sqrt{1 - \lambda_2 / \lambda_1}$$

Le rapport λ_2/λ_1 compare les deux envergures. S'il vaut 1 (nuage rond), e tombe à 0. S'il tend vers 0 (nuage en aiguille), e tend vers 1. L'excentricité varie donc entre 0 (cercle) et 1 (segment).

1.3.3. Différence avec l'élongation

Pourquoi deux outils pour mesurer l'allongement ? Parce qu'ils ne regardent pas la même chose. Une forme en croix a une boîte englobante carrée — élongation 1, l'air trapu — mais l'excentricité, en pesant la répartition réelle des pixels des deux barres, en révèle la vraie géométrie. Et surtout, l'excentricité est **stable** : un pixel de bruit isolé change la boîte de l'élongation, mais ne déplace presque pas la masse globale.



Un nuage deux fois et demie plus étalé en longueur ($\lambda_1 = 2500$) qu'en largeur ($\lambda_2 = 400$) : $e = \sqrt{1 - 400/2500} = \sqrt{0,84} \approx 0,916$.



Canvas : Find Contours → Region Properties. Le nœud calcule l'ellipse d'inertie et l'affiche par-dessus l'objet ; l'inspecteur donne l'excentricité et l'orientation.

1.4. Solidité : la forme est-elle pleine ?

La pellicule plastique tendue qui ignore les creux

Observation 1.2 — solidité $\neq$ convexité (aires vs périmètres de l'enveloppe)

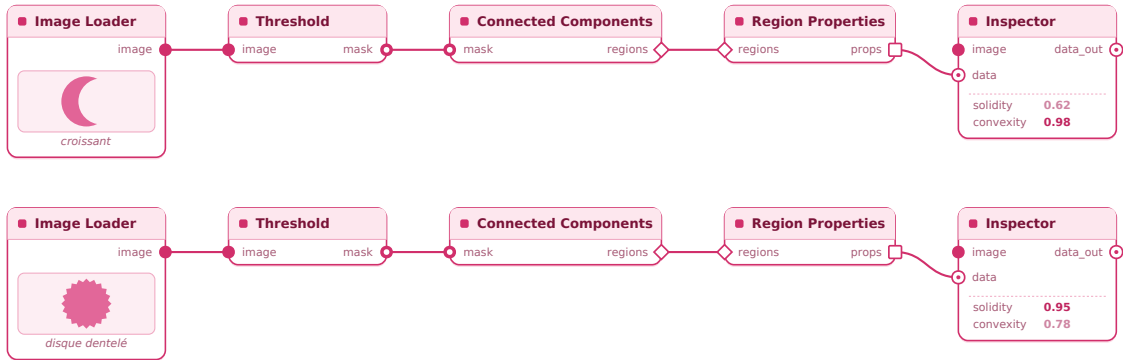


Figure 1.2 – Une forme en « 8 » et son enveloppe convexe : le vide central fait chuter la solidité.

1.4.1. L'intention

Quand deux cellules se touchent, le traitement d'image les fond parfois en une seule forme en « 8 ». On veut détecter ces fusions accidentelles — repérer qu'une forme a un creux profond, là où on attendait un objet plein.

1.4.2. La forme recherchée

Le concept clé est l'**enveloppe convexe** : la pellicule qu'on tendrait autour de l'objet. Elle s'appuie sur toutes les parties saillantes et passe au-dessus des creux sans jamais y entrer, comme un film plastique tiré sur un objet bosselé. Là où l'objet a un renforcement, le film reste tendu droit par-dessus, emprisonnant un vide.

On compare alors la surface réelle de l'objet à la surface enfermée sous cette enveloppe. Sans creux, les deux coïncident. Avec un grand renforcement, l'enveloppe couvre beaucoup de vide en plus.



$$S = A / A_{\text{convexe}}$$

Sans aucune cavité, l'objet touche son enveloppe convexe partout et $S = 1$. Plus les creux sont profonds, plus S s'effondre.

1.4.3. Ce qu'elle détecte le mieux

C'est le descripteur des fusions d'objets. Deux cellules collées en « 8 » : l'enveloppe convexe recouvre le creux central, ce vide fait chuter la solidité autour de 0,85, bien en dessous de

ce qu'on attend d'une cellule isolée. Son angle mort : la rugosité fine du bord, qui retire très peu de surface et la laisse donc presque insensible.



Une forme à deux lobes : aire $A = 3100 \text{ px}^2$, enveloppe convexe $A_{\text{convexe}} = 3720 \text{ px}^2$. Solidité $S = 3100 / 3720 \approx 0,83$.



Les trous entièrement internes — l'intérieur d'un anneau, par exemple — sont comptés dans la surface ou non selon la bibliothèque. Le résultat de la solidité en dépend, et la convention employée doit être vérifiée avant d'interpréter les valeurs.



Canvas : Find Contours → Convex Hull → Shape Descriptors. L'enveloppe convexe se superpose en pointillés sur l'objet, et le creux comblé saute aux yeux.

1.5. Convexité : la même enveloppe, vue par le périmètre

Le chemin direct par-dessus les creux, contre le contour qui épouse tout

1.5.1. L'intention

On veut maintenant distinguer un objet simplement abîmé en surface (un galet ébréché, au bord rugueux mais à la forme intacte) d'un objet vraiment déformé. La solidité, qui regarde les surfaces, n'y est pas sensible. Il faut regarder le contour.

1.5.2. La forme recherchée

On reprend l'**enveloppe convexe** du §1.4, mais on compare cette fois les **périmètres** plutôt que les aires. L'enveloppe, qui va au plus direct par-dessus les creux, a toujours un contour plus court que la silhouette réelle, laquelle doit épouser tous les détours. Une multitude de minuscules dentelures rallonge énormément le contour réel sans presque rien retirer à la surface sous l'enveloppe — c'est exactement ce que la convexité capte et que la solidité manque.



$$C_v = P_{\text{convexe}} / P$$

1.5.3. Le binôme avec la solidité

Les deux descripteurs travaillent en équipe, et c'est leur combinaison qui informe :

- Un contour très granuleux mais sans gros renforcement : **convexité basse, solidité haute** → un bord abîmé, une structure saine.
- Un croissant de lune parfaitement lisse : **convexité haute, solidité basse** → un bord net, mais une grande concavité de masse.



Un galet ébréché : contour réel tortueux $P = 380$ px, enveloppe directe $P_{\text{convexe}} = 322$ px. Convexité $C_v = 322 / 380 \approx 0,84$. Couplée à une solidité élevée, cette valeur dit à la machine : surface détériorée, structure intacte.



Canvas : Find Contours → Convex Hull → Shape Descriptors. Mêmes nœuds qu'en 1.4 ; il suffit de lire la sortie convexité au lieu de la solidité, les deux venant de la même enveloppe.

1.6. Étendue : remplir une boîte qui ne tourne pas

Le rectangle rigide, aligné sur l'image, qui assume sa naïveté

Observation 1.3 — l'étendue dépend de l'orientation, pas la rectangularté

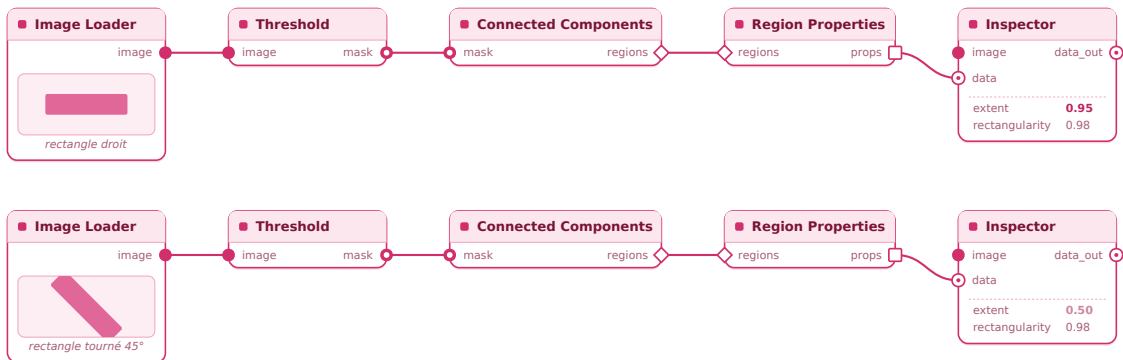


Figure 1.3 – Boîte droite contre boîte orientée : le taux de remplissage s'effondre dès que l'objet penche.

1.6.1. L'intention

Sur une chaîne de tri où les composants arrivent toujours dans la même orientation, on veut une mesure de remplissage instantanée, sans le coût d'extraire et d'analyser des contours complexes.

1.6.2. La forme recherchée

On enferme l'objet dans sa boîte englobante **strictement alignée** sur l'image — aucune rotation permise — et on regarde quelle fraction de cette boîte il occupe. C'est rapide, parce qu'il suffit des coordonnées extrêmes de l'objet.



$$\text{Ext} = A / (W_{\text{boîte}} \cdot H_{\text{boîte}})$$

1.6.3. Sa force et sa faiblesse assumées

Un rectangle posé à plat remplit sa boîte à presque 100 % ($\text{Ext} \approx 0,95$). Inclinez-le à 45° : la boîte horizontale-verticale qui le contient devient bien plus grande, à moitié vide ($\text{Ext} \approx 0,5$). L'étendue change donc radicalement si l'objet tourne — elle n'est pas invariante à la rotation. Elle ne vaut que là où l'orientation est fixe et connue : tri de composants alignés, lecture de caractères sur une même ligne.



Canvas : Find Contours $\rightarrow$ Bounding Rect $\rightarrow$ Shape Descriptors. La boîte droite s'affiche sur l'objet ; on voit immédiatement l'espace vide se creuser dès qu'une pièce arrive de travers.

1.7. Diamètre équivalent : convertir une aire en taille

Refondre l'objet en un disque, et mesurer ce disque

1.7.1. L'intention

En granulométrie — classer des grains de métal, des cellules, du sable par taille — on veut transformer une aire en pixels en une longueur physique unique, comparable et convertible en micromètres.

1.7.2. La forme recherchée

L'idée : si on refondait la matière de l'objet en un disque parfait de même surface, quel serait son diamètre ? On part de la formule de l'aire du disque et on isole le diamètre. Une aire (en pixels carrés) devient ainsi une longueur (en pixels), qu'on convertit ensuite en taille réelle dès qu'on connaît le calibrage de la caméra.



$$D_{\text{eq}} = \sqrt{(4 \cdot A / \pi)}$$



Un grain de sable à $A = 3100 \text{ px}^2$ sous un microscope réglé à $0,5 \text{ }\mu\text{m/pixel}$: $D_{eq} = \sqrt{(4 \cdot 3100 / \pi)} \approx 62,8 \text{ px}$, soit environ **31 μm** de diamètre réel. C'est ce descripteur qui dresse les courbes granulométriques.

Contrairement aux précédents, le diamètre équivalent **dépend de l'échelle** — c'est tout son intérêt. Il mesure une taille absolue, là où les rapports l'effaçaient.



Canvas : Find Contours → Shape Descriptors (sortie diamètre équivalent) → Scale Calibration. Le nœud de calibrage convertit les pixels en unité physique si la résolution est renseignée.

1.8. Rectangularité : remplir une boîte qui, elle, tourne

Le carton ajusté au plus près, et la part de vide qui reste

1.8.1. L'intention

On veut reconnaître les formes manufacturées — cartes, composants, bâtiments vus du ciel — qui remplissent bien un rectangle, quelle que soit leur inclinaison.

1.8.2. La forme recherchée

Comme l'étendue (§1.6), on mesure un taux de remplissage de boîte. Mais cette fois la boîte est **orientée** : elle pivote pour s'ajuster au plus près de l'objet, comme en 1.2. On regarde quelle fraction de cette boîte ajustée l'objet occupe vraiment.



$$R = A / A_{\text{minRect}}$$

1.8.3. Son angle mort surprenant

La rectangularité est totalement aveugle à l'allongement. Prenez n'importe quelle ellipse — un ovale presque rond ou un ovale étiré comme un lacet : elle remplit toujours sa boîte ajustée à la même proportion, $\pi/4 \approx 0,785$. La rectangularité ne lit que l'occupation des **coins**, pas les proportions d'ensemble. Un descripteur qui vaut 0,785 signale donc « bords arrondis, coins vides », sans rien dire de la forme générale.



Un composant rectangulaire $A = 4800 \text{ px}^2$ dans une boîte ajustée de $122 \times 42 \text{ px}$ (5124 px^2) : $R = 4800 / 5124 \approx 0,93$ — une forme franchement rectangulaire.



Canvas : Find Contours → Min Area Rect → Shape Descriptors. La boîte orientée s'affiche par-dessus l'objet ; coins vides ou remplis se lisent au premier regard.

1.9. Rondeur : la forme d'ensemble, sans le bruit du bord

La circularité débarrassée de ce qui la trompait

1.9.1. L'intention

La circularité (§1.1) avait un défaut : un bord dentelé la faisait chuter, même quand la forme globale restait bien ronde. On veut mesurer la rondeur **d'ensemble**, en ignorant les aspérités du contour.

1.9.2. La forme recherchée

Le problème venait du périmètre, qui explose au moindre détail du bord. On le retire de l'équation et on le remplace par le **grand axe** de la forme — sa plus grande dimension, mesurée d'une extrémité à l'autre. Cette mesure de bout en bout se moque des petites dentelures : elle ne voit que l'enveloppe globale.



$$Rd = 4 \cdot A / (\pi \cdot L_{\max}^2)$$

1.9.3. Ce que l'écart avec la circularité révèle

Un disque dentelé garde une rondeur élevée ($Rd \approx 0,95$) tandis que sa circularité s'effondre. C'est tout l'intérêt de garder les deux : leur **différence** isole la rugosité du bord, débarrassée de la forme d'ensemble.



Une particule abrasive : $A = 3100 \text{ px}^2$, grand axe $L_{\max} = 68 \text{ px}$. Rondeur $Rd = 4 \cdot 3100 / (\pi \cdot 68^2) \approx 0,85$ — forme globalement compacte. Mais sa circularité ne vaut que $C \approx 0,55$, à cause du bord déchiqueté. La soustraction $0,85 - 0,55 = 0,30$ donne une mesure pure de l'état du bord, indépendante de la silhouette.



Canvas : Find Contours → Shape Descriptors. Les sorties circularité et rondeur sont disponibles côte à côte ; un nœud Math peut calculer leur différence pour isoler la rugosité.

1.10. Tableau récapitulatif – ce que chaque descripteur voit et ignore

Descripteur	Voit surtout	Angle mort	Invariances
Circularité C	rugosité + élongation (mêlées)	ne sépare pas les deux causes	T, R, E
Élongation E	allongement	l'intérieur de la boîte	T, R, E
Excentricité e	allongement (par la masse)	rugosité du bord	T, R, E
Solidité S	concavités, d'objets	fusions rugosité fine du bord	T, R, E
Convexité Cv	rugosité du contour	concavités de masse	T, R, E
Étendue Ext	remplissage (boîte droite)	dépend de l'orientation	T, E
Diamètre éq. D_eq	taille absolue	la forme	T, R
Rectangularité R	remplissage des coins	l'élongation	T, R, E
Rondeur Rd	forme d'ensemble	l'état du bord	T, R, E

(T = translation, R = rotation, E = échelle.)

Aucun descripteur ne suffit seul. C'est en les combinant — l'un pour l'allongement, l'autre pour le bord, un troisième pour les fusions — qu'on cerne une forme presque parfaitement.

1.11. Une invariance est une information qu'on accepte de perdre

Une invariance rend un descripteur aveugle à une transformation. C'est utile quand cette transformation ne porte aucune information pour la tâche, coûteux quand elle en porte. La circularité ignore la taille : l'utiliser, c'est déclarer que la dimension n'a aucune importance pour le problème posé. Refuser le diamètre équivalent parce qu'il varie avec la taille, c'est oublier qu'il a été conçu exactement pour ça. La bonne question de conception n'est donc pas « ce descripteur est-il invariant ? » mais « à quoi dois-je être aveugle pour cette tâche ? ».

On retrouvera cette question à chaque chapitre, sous d'autres noms. Un filtre garde une fréquence et en jette une autre ; une distance déclare ce qui se ressemble ; une caméra projette et sacrifie la profondeur. Un descripteur garde une chose et en jette une autre — choisir ses descripteurs, c'est dire ce qui, dans une forme, compte pour le problème qu'on traite.

Moments d'image

Table des matières

2.1. Moments bruts	19
2.2. Définition	19
2.3. L'idée	20
2.4. Exemple numérique — le masque jouet du chapitre	20
2.5. Piège : en image, y descend	21
2.6. Piège : un masque, pas une image	21
2.7. Code	21
2.8. Centroïde	22
2.9. Définition	22
2.10. L'idée	22
2.11. Exemple numérique	22
2.12. Applications	22
2.13. Subtilité importante : le centroïde peut tomber hors de l'objet	23
2.14. Code	23
2.15. Piège : (x, y) ou (row, col) ?	23
2.16. Moments centraux	24
2.17. Définition	24
2.18. L'idée	24
2.19. Exemple numérique (suite du masque jouet)	24

2.20. Les moments d'ordre 3 mesurent l'asymétrie	25
2.21. Code	25
2.22. Moments centraux normalisés	25
2.23. Définition	25
2.24. L'idée	25
2.25. Exemple numérique	26
2.26. Piège : l'invariance d'échelle est exacte en théorie, approchée en pratique	26
2.27. Code	26
2.28. Orientation principale	26
2.29. Définition	26
2.30. L'idée	27
2.31. Piège : $\arctan 2$, pas $\arctan$	27
2.32. Exemple numérique	27
2.33. Applications	27
2.34. Piège : l'orientation d'un objet rond n'existe pas	28
2.35. Code	28
2.36. Ellipse équivalente	29
2.37. Définition	29
2.38. L'idée	29
2.39. Exemple numérique (suite et fin du masque jouet)	29
2.40. Le piège central : l'ellipse ne mesure pas la taille de l'objet	30
2.41. Code	30
2.42. Piège de bibliothèque : des noms et une convention qui ont changé	30
2.43. Les sept invariants de Hu	31
2.44. L'idée	31
2.45. Pourquoi ces combinaisons résistent-elles à la rotation ?	31
2.46. Deux propriétés à connaître	31
2.47. Exemple : reconnaissance de caractères	32
2.48. Hu aujourd'hui : encore utile ?	32
2.49. Code	32
2.50. Moments pondérés par l'intensité	33
2.51. Code	33
2.52. Tableau récapitulatif — quel moment pour quelle question ?	34

Au chapitre précédent, plusieurs descripteurs reposaient sur des notions laissées en suspens : l'excentricité utilisait les demi-axes d'une « ellipse équivalente », et l'on a promis des invariants capables de reconnaître une forme quelle que soit sa pose. Tout cela vient d'un même outil : les **moments d'image**. Les moments transforment une région de pixels en une poignée de nombres qui en résument la masse, la position, l'orientation et l'étalement — un peu comme une fiche d'identité chiffrée de la silhouette.

L'idée centrale est empruntée à la mécanique. Si l'on découpe le masque de l'objet dans une plaque de carton d'épaisseur uniforme, alors les moments décrivent exactement les propriétés physiques de cette plaque : sa masse, son point d'équilibre, l'axe autour duquel elle tourne le plus facilement. Cette analogie n'est pas une image vague — c'est une correspondance exacte, et tout le chapitre l'exploitera pour donner un sens concret à des formules qui, écrites brutes, paraissent abstraites.

Le fil conducteur tient en une phrase : **plus l'ordre d'un moment est élevé, plus il regarde loin du centroïde, et plus il amplifie le bruit**. L'aire (ordre 0) est très robuste ; le centroïde (ordre 1) l'est presque autant ; l'orientation (ordre 2) devient fragile pour les formes rondes ; les invariants de Hu (ordre 3) peuvent varier de moitié pour deux pixels de bruit sur le contour. Chaque section illustrera un étage de cette échelle de fragilité.

Conventions pour tout le chapitre : $I(x, y)$ vaut 1 pour les pixels du masque et 0 ailleurs (moments **binares**). Les mêmes formules s'appliquent telles quelles à une image en niveaux de gris : on parle alors de moments **pondérés**, et la section 2.8 montrera ce que ce changement apporte. Côté logiciel : chaque bloc de code se colle dans une node « **Python Script** ». Le masque arrive dans la variable `a`, le résultat est renvoyé dans `out_a` — un dictionnaire qu'une node **Inspecteur** affiche à l'écran. Les modules `np` (NumPy) et `cv2` (OpenCV) sont déjà disponibles, sans import.

2.1. Moments bruts

2.2. Définition

$$M_{pq} = \sum_x \sum_y x^p \cdot y^q \cdot I(x, y)$$

Le couple (p, q) indique à quelle puissance on élève les coordonnées. La somme $p + q$ s'appelle l'**ordre** du moment.

2.3. L'idée

Un moment est une somme sur tous les pixels de l'objet, mais une somme où chaque pixel ne compte pas pareil : il est pesé par sa position élevée à une certaine puissance. En faisant varier les puissances p et q , on pose à la forme des questions de plus en plus précises.

L'ordre 0 ne pèse rien du tout ($x^0 \cdot y^0 = 1$) : il compte simplement les pixels. L'ordre 1 pèse chaque pixel par sa position : il dit où se trouve la masse. L'ordre 2 pèse par la position au carré : il dit comment la masse est étalée. C'est exactement la hiérarchie des grandeurs de la mécanique du solide, et la correspondance est terme à terme :

M_{00} = masse totale (= aire A pour un masque binaire)
 M_{10} = moment statique en x (la masse, pondérée par où elle est)
 M_{01} = moment statique en y
 M_{20}, M_{02} = moments d'inertie par rapport aux axes
 M_{11} = produit d'inertie (la masse penche-t-elle en diagonale ?)

Cette correspondance est précieuse : elle veut dire que toutes les propriétés des sections suivantes — centre de gravité, axes principaux — ne sont pas des recettes de vision inventées pour l'occasion, mais des résultats de mécanique que l'on connaît depuis longtemps, transposés tels quels à une image.

2.4. Exemple numérique — le masque jouet du chapitre

Tout le chapitre s'appuiera sur un même masque 3×3 , assez petit pour que chaque calcul tienne sur un coin de feuille :

$y \backslash x$	0	1	2
0	0	1	1
1	1	1	1
2	1	1	0

Sept pixels, avec deux coins manquants en haut à gauche et en bas à droite : la forme penche le long de la diagonale qui va du coin haut-droit au coin bas-gauche. Calculons ses moments d'ordre 0, 1 et 2.

Pour M_{00} , on compte : $M_{00} = 7$.

Pour $M_{10} = \sum x \cdot I$, le plus simple est de compter combien de pixels se trouvent dans chaque colonne : la colonne $x = 0$ en contient 2, la colonne $x = 1$ en contient 3, la colonne $x = 2$ en contient 2.

$$M_{10} = 0 \cdot (2) + 1 \cdot (3) + 2 \cdot (2) = 7$$

$$M_{01} = 0 \cdot (2) + 1 \cdot (3) + 2 \cdot (2) = 7 \quad (\text{mêmes comptes par ligne, par symétrie})$$

Pour les moments d'ordre 2, on procède de la même façon avec x^2 et y^2 , et pour M_{11} on somme le produit $x \cdot y$ pixel par pixel :

$$M_{20} = \sum x^2 = 0 \cdot (2) + 1 \cdot (3) + 4 \cdot (2) = 11$$

$$M_{02} = \sum y^2 = 11 \quad (\text{par symétrie})$$

$$M_{11} = \sum x \cdot y = (1 \cdot 0) + (2 \cdot 0) + (0 \cdot 1) + (1 \cdot 1) + (2 \cdot 1) + (0 \cdot 2) + (1 \cdot 2) = 5$$

Ces six nombres — 7, 7, 7, 11, 11, 5 — suffiront pour tout le reste du chapitre.

2.5. Piège : en image, y descend

En traitement d'image, l'axe y croît **vers le bas** : la ligne 0 est en haut. Toutes les formules de ce chapitre restent vraies, mais les angles d'orientation se retrouvent mesurés dans le sens horaire dans le repère image, à l'inverse du repère mathématique habituel. `cv2.moments` comme `regionprops` suivent cette convention. C'est la cause classique des orientations « à l'envers » : le calcul est juste, c'est le repère qui n'est pas celui auquel on pense.

2.6. Piège : un masque, pas une image

`cv2.moments` n'accepte qu'un tableau **2D à un seul canal**. Si l'on relie par mégarde une sortie *image* (3 canaux, BGR) à l'entrée du nœud, au lieu d'un *masque*, OpenCV s'interrompt brutalement :

```
cv2.moments ... moments.cpp: error: (-5:Bad argument) Invalid image
```

La parade tient en une ligne, à placer en tête de chaque script : convertir en niveaux de gris si l'entrée a trois canaux, et garantir un `uint8` contigu. Tous les codes de ce chapitre commencent désormais par cette normalisation, sous le nom `msk`.

2.7. Code

```
# msk : masque nettoyé - 2D, uint8, contigu. Convertit une image BGR en gris
# si on a relié une image au lieu d'un masque (voir le piège ci-dessus).
msk = np.ascontiguousarray(a if a.ndim == 2 else cv2.cvtColor(a,
cv2.COLOR_BGR2GRAY), np.uint8)

# cv2.moments calcule d'un coup tous les moments jusqu'à l'ordre 3.
# binaryImage=True : tout pixel non nul compte pour 1.
m = cv2.moments(msk, binaryImage=True)

# m est un dictionnaire à trois familles de clés :
# 'm00', 'm10', ... moments bruts M_pq
```

```
# 'mu20','mu11',... moments centraux  $\mu_{pq}$  (section 2.3)
# 'nu20','nu11',... moments normalisés  $\eta_{pq}$  (section 2.4)
out_a = {"M00 (aire)": m["m00"], "M10": m["m10"], "M01": m["m01"],
        "M20": m["m20"], "M02": m["m02"], "M11": m["m11"]}
```

2.8. Centroïde

2.9. Définition

$$\bar{x} = M_{10} / M_{00} \quad , \quad \bar{y} = M_{01} / M_{00}$$

2.10. L'idée

Le centroïde est le point d'équilibre de la forme : l'endroit où la plaque de carton tiendrait en équilibre sur une pointe de crayon. La formule dit exactement cela : la position moyenne des pixels, c'est-à-dire la somme des positions (M_{10}) divisée par le nombre de pixels (M_{00}). Diviser une somme par un effectif, c'est faire une moyenne — le centroïde n'est rien d'autre que la moyenne des coordonnées des pixels.

On peut vérifier que ce point a bien la propriété d'un point d'équilibre : vu depuis lui, la masse ne penche d'aucun côté, les contributions de gauche annulant exactement celles de droite. C'est précisément ce qui définit un centre de gravité.

2.11. Exemple numérique

Sur le masque jouet : $\bar{x} = 7/7 = 1$ et $\bar{y} = 7/7 = 1$. Le centroïde tombe sur le pixel central — cohérent avec la symétrie de la forme, dont les deux coins manquants se compensent.

2.12. Applications

Le centroïde est l'ancrage de presque toute analyse spatiale. En vidéo, suivre un objet revient souvent à suivre la suite de ses centroïdes d'une image à l'autre. En astronomie, le centroïde d'une étoile sur le capteur localise la source avec une précision **inférieure au pixel** (la section 2.8 expliquera pourquoi). En recalage d'images, aligner deux acquisitions commence souvent par superposer leurs barycentres.

2.13. Subtilité importante : le centroïde peut tomber hors de l'objet

Le point d'équilibre d'une forme n'est pas forcément dans la forme. Pour un masque en U, en croissant, ou pour deux objets reliés par un isthme fin, le centroïde tombe dans le creux — exactement comme le centre de gravité d'un anneau, qui se situe au milieu du trou, là où il n'y a pas de matière. Tout algorithme qui « plante un marqueur au centroïde » (marqueur de watershed, placement d'étiquette, point d'amorçage pour un modèle de segmentation) doit donc le vérifier.

2.14. Code

```
# Calcule le centroïde et, s'il tombe hors de l'objet,
# le ramène sur le pixel du masque le plus proche.
msk = np.ascontiguousarray(a if a.ndim == 2 else cv2.cvtColor(a,
cv2.COLOR_BGR2GRAY), np.uint8)
m = cv2.moments(msk, binaryImage=True)
if m["m00"] > 0:
    cx, cy = m["m10"] / m["m00"], m["m01"] / m["m00"]
    # msk[ligne, colonne] : 0 (hors objet) ou 255 (dans l'objet)
    if not msk[int(round(cy)), int(round(cx))]:
        ys, xs = np.nonzero(msk) # ys = lignes, xs = colonnes
        i = np.argmin((xs - cx) ** 2 + (ys - cy) ** 2)
        cx, cy = float(xs[i]), float(ys[i])
    out_a = {"cx": float(cx), "cy": float(cy)}
else:
    out_a = {"Erreur": "masque vide"}
```

C'est aussi pour cette raison que la transformée de distance — qui trouve le point le plus *intérieur* de la forme — est souvent préférée au centroïde pour placer une étiquette de pays sur une carte ou générer un point d'amorçage fiable.

2.15. Piège : (x, y) ou (row, col) ?

Les bibliothèques ne rangent pas les coordonnées dans le même ordre. `cv2.moments` raisonne en (x, y) ; `regionprops.centroid` renvoie (ligne, colonne), c'est-à-dire ($\bar{y}$, $\bar{x}$). Mélanger les deux conventions transpose silencieusement tous les résultats — l'erreur ne se voit que sur une forme non symétrique, donc rarement sur le cas de test.

2.16. Moments centraux

2.17. Définition

$$\mu_{pq} = \sum_x \sum_y (x - \bar{x})^p \cdot (y - \bar{y})^q \cdot I(x, y)$$

2.18. L'idée

Les moments bruts mélangent deux informations : la forme de l'objet et l'endroit où il se trouve dans l'image. Les moments centraux séparent les deux, en mesurant les coordonnées **depuis le centroïde** plutôt que depuis le coin de l'image. C'est comme décrire un objet « par rapport à son propre centre » au lieu de « par rapport au coin de la pièce » : la description ne dépend plus de l'endroit où l'objet est posé.

La conséquence immédiate est l'**invariance par translation** — la première des trois invariances que le chapitre va construire pas à pas (translation ici, échelle en 2.4, rotation en 2.7). L'intuition suffit : déplacer la forme déplace son centroïde d'autant, si bien que l'écart entre chaque pixel et le centroïde, lui, ne change pas. Les μ_{pq} sont donc aveugles à la position.

En pratique, on ne recalcule pas les μ_{pq} en reparcourant l'image : la bibliothèque les déduit des moments bruts par quelques soustractions. À l'ordre 2, par exemple :

$$\mu_{20} = M_{20} - \bar{x} \cdot M_{10}$$

$$\mu_{02} = M_{02} - \bar{y} \cdot M_{01}$$

$$\mu_{11} = M_{11} - \bar{x} \cdot M_{01}$$

(et des formules analogues, plus longues, existent à l'ordre 3 — inutile de les retenir). Il faut surtout savoir qu'elles existent : c'est ce qui permet de tout calculer en un seul passage sur l'image.

2.19. Exemple numérique (suite du masque jouet)

$$\mu_{20} = 11 - 1 \times 7 = 4$$

$$\mu_{02} = 11 - 1 \times 7 = 4$$

$$\mu_{11} = 5 - 1 \times 7 = -2$$

Le signe de μ_{11} se lit comme une tendance : il est négatif, ce qui signifie que lorsque y augmente (on descend dans l'image), x tend à diminuer. La masse s'étire donc le long de la diagonale descendante vers la gauche — exactement ce que montre le dessin du masque, avec ses coins manquants en haut-gauche et bas-droite.

2.20. Les moments d'ordre 3 mesurent l'asymétrie

À l'ordre 2, les écarts au centroïde sont élevés au carré : un pixel à gauche et un pixel à droite contribuent pareil, le carré effaçant le signe. À l'ordre 3, le cube **conserve le signe** : les deux côtés ne se compensent plus, et le moment mesure de quel côté la masse penche.

μ_{30} : asymétrie gauche/droite de la masse

μ_{03} : asymétrie haut/bas

Un caractère manuscrit comme « e » ou « a » a une asymétrie marquée que μ_{30} capture ; un « o » symétrique a $\mu_{30} \approx \mu_{03} \approx 0$. Ces moments d'ordre 3 sont la matière première des invariants de Hu (section 2.7). Notez déjà, pour le fil conducteur, que le cube amplifie les pixels lointains bien plus que le carré : l'ordre 3 est plus expressif, mais aussi plus fragile.

2.21. Code

```
# Les moments centraux sont déjà dans le dictionnaire renvoyé par cv2.moments.  
msk = np.ascontiguousarray(a if a.ndim == 2 else cv2.cvtColor(a,  
cv2.COLOR_BGR2GRAY), np.uint8)  
m = cv2.moments(msk, binaryImage=True)  
out_a = {"mu20": m["mu20"], "mu02": m["mu02"], "mu11": m["mu11"],  
         "mu30": m["mu30"], "mu03": m["mu03"]}
```

2.22. Moments centraux normalisés

2.23. Définition

$$\eta_{pq} = \mu_{pq} / \mu_{00}^{\gamma}, \quad \gamma = (p + q)/2 + 1$$

2.24. L'idée

Après la position (2.3), on veut maintenant oublier la **taille** : un même logo, imprimé en petit sur une étiquette ou en grand sur une façade, devrait donner les mêmes nombres. L'image juste est celle de deux photos d'un même objet, l'une de près, l'autre de loin : on aimerait une mesure qui les déclare identiques.

L'astuce est de diviser chaque moment par une puissance de l'aire. Pourquoi ça marche ? Quand on agrandit une forme, son moment grossit, mais son aire grossit aussi. Si on choisit la bonne puissance de l'aire au dénominateur, les deux grossissements se compensent

exactement et le rapport ne bouge plus. Tout l'enjeu est donc de trouver cette « bonne puissance » : c'est le rôle de l'exposant γ .

On peut le sentir sans calcul : agrandir une forme d'un facteur 2 multiplie son aire par 4 (la surface croît comme le carré), et gonfle un moment d'autant plus que son ordre est élevé. L'exposant $\gamma = (p + q)/2 + 1$ est précisément réglé pour que numérateur et dénominateur enflent à la même vitesse. Ce n'est pas un nombre choisi au hasard : c'est la seule valeur qui rende η insensible au zoom.

2.25. Exemple numérique

Sur le masque jouet : $\eta_{20} = \mu_{20}/\mu_{00}^2 = 4/49 \approx 0,0816$ (pour l'ordre 2, $\gamma = 2$). Agrandissons mentalement la forme d'un facteur 10 : elle compte désormais 700 pixels, μ_{20} devient environ 4×10^4 et μ_{00}^2 environ 49×10^4 . Le rapport reste $\approx 0,0816$. C'est exactement cette stabilité qui permet de reconnaître la même forme à toutes les tailles.

2.26. Piège : l'invariance d'échelle est exacte en théorie, approchée en pratique

Le raisonnement ci-dessus suppose qu'agrandir une forme multiplie proprement ses pixels. En réalité, une petite forme agrandie est re-dessinée sur la grille : les marches d'escalier du contour ne se transforment pas exactement, et η fluctue de quelques pourcents. L'invariance d'échelle est excellente entre une forme de 1 000 pixels et une de 100 000 ; elle se dégrade nettement sous quelques dizaines de pixels, où chaque pixel de contour pèse trop lourd.

2.27. Code

```
# Les moments normalisés sont déjà dans le dictionnaire renvoyé par cv2.moments.  
msk = np.ascontiguousarray(a if a.ndim == 2 else cv2.cvtColor(a,  
cv2.COLOR_BGR2GRAY), np.uint8)  
m = cv2.moments(msk, binaryImage=True)  
out_a = {"nu20": m["nu20"], "nu02": m["nu02"], "nu11": m["nu11"]}
```

2.28. Orientation principale

2.29. Définition

$$\theta = \frac{1}{2} \cdot \arctan2(2\mu_{11}, \mu_{20} - \mu_{02})$$

2.30. L'idée

L'orientation principale est l'axe le long duquel la forme s'allonge. L'analogie mécanique est parlante : c'est l'axe autour duquel la plaque de carton tourne le plus facilement. Pensez à une règle plate. Elle pivote sans effort autour de son grand axe — la matière est collée contre l'axe — et difficilement autour de son petit axe, où la matière en est éloignée. Le grand axe, celui de la rotation facile, est l'orientation de la forme.

On obtient ϑ en cherchant cet axe « de moindre effort » qui passe par le centroïde. Le calcul, qu'on ne déroule pas ici, combine les trois moments d'ordre 2 (μ_{20} , μ_{02} et μ_{11}) pour aboutir à la formule ci-dessus. Un point mérite cependant d'être compris, car il explique la forme de la formule : on y trouve l'angle **double** 2ϑ , et non ϑ . La raison est qu'un axe n'a pas de sens de parcours — une règle orientée à 30° ou à $30^\circ + 180^\circ$ pointe dans la même direction. Travailler avec 2ϑ encode naturellement cette ambiguïté, et le facteur $\frac{1}{2}$ final ramène le résultat dans l'intervalle utile.

2.31. Piège : `arctan2`, pas `arctan`

La formule s'implémente avec `arctan2(2 $\mu_{11}$ ,  $\mu_{20} - \mu_{02}$ )` et surtout pas avec `arctan` du quotient. La fonction `arctan` perd le signe du dénominateur, donc le quadrant : dès que $\mu_{20} < \mu_{02}$ (forme plus haute que large), l'orientation sort fautive de 90° . C'est une erreur silencieuse — le code tourne, les angles sont plausibles, et tout est faux pour la moitié des objets.

2.32. Exemple numérique

Masque jouet : $2\mu_{11} = -4$ et $\mu_{20} - \mu_{02} = 0$. Donc `arctan2(-4, 0) = -90°`, et $\vartheta = -45^\circ$. La forme est orientée le long de la diagonale descendante — ce que le signe de μ_{11} annonçait déjà en 2.3, et qu'on vérifie d'un coup d'œil sur le dessin. (Rappel du piège 2.1 : l'angle se lit dans le repère image, y vers le bas.)

2.33. Applications

L'orientation principale sert d'abord à **redresser** un objet avant de le reconnaître : remettre un caractère ou un code-barres à l'horizontale, aligner une empreinte digitale sur son axe, normaliser la pose d'une cellule avant classification. Elle sert aussi en tant que mesure : en microscopie, l'histogramme des orientations des fibres de collagène quantifie l'anisotropie d'un tissu ; en télédétection, celui des orientations de bâtiments révèle la trame d'une ville.

2.34. Piège : l'orientation d'un objet rond n'existe pas

Quand la forme est ronde (ou carrée vue de face), elle s'étale autant dans toutes les directions : il n'y a plus d'axe privilégié, et la formule devient indéterminée. Un seul pixel de bruit fait alors basculer ϑ de 90°. Demander l'orientation d'un disque, c'est demander dans quelle direction pointe une boule de pétanque — la question n'a pas de réponse, et l'algorithme en donnera pourtant une.

Il faut donc toujours accompagner ϑ d'une mesure de fiabilité, l'**anisotropie**, qui dit à quel point la forme a réellement une direction dominante :

$$\text{aniso} = \sqrt{(\mu_{20} - \mu_{02})^2 + 4\mu_{11}^2} / (\mu_{20} + \mu_{02})$$

Elle vaut 0 pour un disque (aucune direction privilégiée) et tend vers 1 pour un segment (une seule direction). Règle pratique : ne pas faire confiance à ϑ quand $\text{aniso} < 0,2$. C'est la première manifestation chiffrée du fil conducteur : l'ordre 2 fonctionne très bien... sauf dans son cas dégénéré, où il devient brutalement instable.

2.35. Code

```
# Renvoie l'orientation ET sa fiabilité (anisotropie).
msk = np.ascontiguousarray(a if a.ndim == 2 else cv2.cvtColor(a,
cv2.COLOR_BGR2GRAY), np.uint8)
m = cv2.moments(msk, binaryImage=True)
mu20, mu02, mu11 = m["mu20"], m["mu02"], m["mu11"]
denom = mu20 + mu02
if denom > 0:
    theta = 0.5 * np.arctan2(2 * mu11, mu20 - mu02)      # radians, repère image
    aniso = np.sqrt((mu20 - mu02) ** 2 + 4 * mu11 ** 2) / denom
    out_a = {"orientation_deg": float(np.degrees(theta)),
             "anisotropie": float(aniso),
             "fiable": bool(aniso >= 0.2)}
else:
    out_a = {"Erreur": "masque vide"}
```

2.36. Ellipse équivalente

2.37. Définition

demi-grand axe : $a = 2\sqrt{\lambda_1}$

demi-petit axe : $b = 2\sqrt{\lambda_2}$

excentricité : $e = \sqrt{(1 - \lambda_2/\lambda_1)}$

où λ_1 et λ_2 sont les deux **étalements** de la forme (voir ci-dessous).

2.38. L'idée

L'ellipse équivalente est le « résumé » le plus simple d'une silhouette : l'ellipse qui répartit sa masse exactement comme la forme réelle — même centre, même orientation, même façon de s'étaler. C'est la forme la plus dépouillée qui « pèse » comme l'originale.

Pour la trouver, on reprend l'image du nuage de points du chapitre 1. Une silhouette est une nuée de pixels ; on cherche les deux directions privilégiées de ce nuage : celle où il s'étale le plus (le grand axe) et celle, perpendiculaire, où il s'étale le moins (le petit axe). Deux nombres mesurent ces étalements — appelons-les λ_1 (le grand) et λ_2 (le petit). Inutile de savoir les calculer ; il suffit de retenir ce qu'ils disent : **λ_1 , c'est de combien la forme s'étire dans sa direction la plus longue ; λ_2 , dans sa direction la plus courte.** Les demi-axes de l'ellipse s'en déduisent, et l'excentricité, $e = \sqrt{(1 - \lambda_2/\lambda_1)}$, n'est rien d'autre que la comparaison des deux étalements — c'est exactement l'excentricité promise au chapitre 1, enfin reliée à sa source. Quand les deux étalements sont égaux (forme ronde), e tombe à 0 ; quand le petit devient minuscule face au grand (forme en aiguille), e tend vers 1.

2.39. Exemple numérique (suite et fin du masque jouet)

En combinant $\mu_{20} = 4$, $\mu_{02} = 4$ et $\mu_{11} = -2$, on obtient les deux étalements :

$\lambda_1 \approx 0,857 \rightarrow$ demi-grand axe $a = 2\sqrt{\lambda_1} \approx 1,85$

$\lambda_2 \approx 0,286 \rightarrow$ demi-petit axe $b = 2\sqrt{\lambda_2} \approx 1,07$

excentricité : $e = \sqrt{(1 - \lambda_2/\lambda_1)} = \sqrt{(1 - 1/3)} \approx 0,82$

Une forme nettement allongée, le long de la diagonale $\vartheta = -45^\circ$ trouvée en 2.5. Sept pixels ont suffi pour dérouler toute la chaîne : moments bruts $\rightarrow$ centroïde $\rightarrow$ moments centraux $\rightarrow$ orientation $\rightarrow$ ellipse.

2.40. Le piège central : l'ellipse ne mesure pas la taille de l'objet

C'est probablement l'erreur la plus répandue de tout le chapitre. L'ellipse équivalente égalise la **répartition de masse**, pas les **dimensions**. Le grand axe de l'ellipse équivalente d'un rectangle **dépasse d'environ 15 % la longueur réelle** du rectangle. L'intuition : pour qu'une ellipse, effilée à ses extrémités, loge autant de masse loin du centre qu'un rectangle dont les bouts sont pleins, elle doit être plus longue que lui. D'où la règle à encadrer :

> `axis_major_length` de `regionprops` n'est **pas** la longueur de l'objet.

Pour mesurer la dimension réelle d'une pièce sur une chaîne de contrôle, d'une bactérie ou d'un trait, il faut le **rectangle orienté minimal** (`cv2.minAreaRect`) ou le **diamètre de Feret**. L'ellipse équivalente décrit la *répartition de masse* (orientation, excentricité) ; elle ne mesure pas. Confondre les deux usages introduit une erreur systématique de l'ordre de 15 % — du genre qui passe inaperçu pendant toute une campagne de mesures.

2.41. Code

```
# regionprops donne l'ellipse équivalente clé en main.
from skimage.measure import regionprops, label

msk = np.ascontiguousarray(a if a.ndim == 2 else cv2.cvtColor(a,
cv2.COLOR_BGR2GRAY), np.uint8)
props = regionprops(label(msk > 0))
if props:
    p = props[0]
    out_a = {
        "centroide (y,x)": [float(v) for v in p.centroid], # (ligne, colonne) !
        "orientation_rad": float(p.orientation),
        "grand_axe (2a)": float(p.axis_major_length),      # ≠ longueur réelle
        "petit_axe (2b)": float(p.axis_minor_length),
        "excentricite": float(p.eccentricity),             # le e du chapitre 1
    }
else:
    out_a = {"Erreur": "masque vide"}
```

2.42. Piège de bibliothèque : des noms et une convention qui ont changé

scikit-image a fait évoluer cette partie de son interface. Les propriétés s'appellent désormais `axis_major_length` / `axis_minor_length` (les anciens noms `major_axis_length` / `minor_axis_length` sont dépréciés), et la **convention d'angle** de orientation (origine, sens de rotation) a elle aussi changé entre versions. Avant toute campagne de mesures, validez la

chaîne sur une forme test connue — un rectangle synthétique tracé à 30° , par exemple — et vérifiez que l'angle ressorti est bien celui attendu. Cette vérification de trente secondes vaut une page de documentation, et elle attrape aussi les confusions d'axes de la section 2.1.

2.43. Les sept invariants de Hu

2.44. L'idée

Il reste une invariance à conquérir : la **rotation**. Les η_{pq} sont insensibles à la position et à la taille, mais tourner la forme les mélange entre eux. Hu (1962) a trouvé sept combinaisons des moments d'ordre 2 et 3 que la rotation laisse intactes. Réunis aux invariances précédentes — translation (μ), échelle (η), rotation (φ) — ces sept nombres ($\varphi_1, \dots, \varphi_7$) constituent une **signature de la forme « pure »**, débarrassée de toute sa pose : la même pour un objet petit ou grand, déplacé ou tourné.

L'analogie utile est celle d'une empreinte digitale de la forme. Peu importe l'angle sous lequel on présente l'objet, sa signature de Hu reste la même — comme on reconnaît une chanson quelle que soit la tonalité dans laquelle on la chante.

2.45. Pourquoi ces combinaisons résistent-elles à la rotation ?

Les formules (sept expressions touffues mêlant les η) paraissent sorties d'un chapeau, et il n'est pas utile de les reproduire ni de les mémoriser : les bibliothèques s'en chargent. Ce qu'il faut comprendre, c'est qu'elles n'ont pas été *trouvées* au hasard mais *construites* pour une raison précise. Quand on tourne une forme, ses moments changent d'une façon très régulière : c'est seulement l'« angle » de certaines quantités qui bouge, pas leur taille. Hu a assemblé les η de manière que cet angle s'élimine de lui-même — un peu comme la longueur d'un vecteur ne change pas quand on le fait pivoter, même si ses coordonnées, elles, changent. Ce qui reste après élimination de l'angle est, par construction, insensible à la rotation.

2.46. Deux propriétés à connaître

φ_7 change de signe en miroir. C'est le seul des sept. Une forme et son reflet dans un miroir ont les mêmes φ_1 à φ_6 , mais des φ_7 opposés. Précieux quand la chiralité compte — distinguer « b » de « d », « p » de « q » en reconnaissance de caractères — et à exlude du vecteur quand elle ne compte pas, sous peine de séparer artificiellement des objets identiques vus de dos.

La dynamique est énorme. Les φ s'étagent typiquement de 10^{-1} à 10^{-20} : comparés directement, les petits sont écrasés par les grands. On les ramène toujours à des ordres de grandeur comparables en passant par le logarithme (en préservant le signe, crucial pour φ_7) avant toute comparaison.

2.47. Exemple : reconnaissance de caractères

C'est l'application historique. Un « A » conserve sa signature qu'il soit petit, grand, tourné ou déplacé — exactement les insensibilités voulues pour une reconnaissance robuste. Un « O » très symétrique a ses moments d'ordre 3 quasi nuls, donc φ_3 à $\varphi_7 \approx 0$; un « R » asymétrique les a nettement non nuls. La distance entre deux signatures de Hu (en échelle log — voir chapitre 3) suffit à un classifieur des plus simples.

2.48. Hu aujourd'hui : encore utile ?

Pour la reconnaissance générale, les descripteurs appris (réseaux de neurones) dominent largement. Les moments de Hu gardent trois créneaux bien réels : (1) quand on a peu ou pas de données d'entraînement, (2) quand il faut une invariance **prouvable** et non simplement constatée — métrologie, certification industrielle —, (3) quand le budget de calcul est minuscule (capteurs embarqués). Le bon réflexe n'est pas « Hu ou réseau de neurones ? » mais « mon problème exige-t-il une garantie mathématique, ou une performance statistique ? ».

2.49. Code

```
# Renvoie les 7 invariants de Hu en échelle log signée.
msk = np.ascontiguousarray(a if a.ndim == 2 else cv2.cvtColor(a,
cv2.COLOR_BGR2GRAY), np.uint8)
m = cv2.moments(msk, binaryImage=True)

# HuMoments renvoie les 7 invariants ; flatten() les met à plat.
hu = cv2.HuMoments(m).flatten()

# log10 de la valeur absolue : ramène tous les  $\varphi$  à des ordres comparables.
# np.sign préserve le signe (crucial pour  $\varphi_7$ ). Le +1e-30 évite log(0).
hu_log = -np.sign(hu) * np.log10(np.abs(hu) + 1e-30)
out_a = {f"phi{i+1}": float(v) for i, v in enumerate(hu_log)}
```

2.50. Moments pondérés par l'intensité

Toutes les formules du chapitre s'appliquent sans changement à une image en niveaux de gris : il suffit de remplacer le masque binaire par les intensités, $I(x, y)$ = niveau de gris. Seule l'interprétation change :

centroïde binaire : centre géométrique du masque

centroïde pondéré : barycentre lumineux (tiré vers les zones claires)

L'image mentale : le masque binaire traite tous les pixels comme aussi « lourds » les uns que les autres, tandis que la version pondérée rend les pixels clairs plus lourds que les pixels sombres. Le point d'équilibre se déplace alors vers les zones lumineuses.

Cet écart est exploité directement. En astronomie, une étoile s'étale sur quelques pixels avec un profil lumineux en cloche ; le centroïde pondéré moyenne ce profil et localise la source au **sous-pixel** — bien plus finement que le simple pixel le plus brillant. C'est le principe des mesures astrométriques de précision. De même, en caractérisation de faisceau laser, les moments pondérés d'ordre 2 définissent la largeur du faisceau (méthode $D4\sigma$, norme ISO 11146) — la définition de référence de l'industrie.

2.51. Code

```
# a : masque (entrée 1).  b : image en niveaux de gris ou couleur (entrée 2).
# Mesure de combien la lumière déplace le centroïde par rapport au masque seul.
msk = np.ascontiguousarray(a if a.ndim == 2 else cv2.cvtColor(a,
cv2.COLOR_BGR2GRAY), np.uint8)
m_bin = cv2.moments(msk, binaryImage=True)
gray = b if (b is not None and b.ndim == 2) else cv2.cvtColor(b,
cv2.COLOR_BGR2GRAY)

# On ne garde que les intensités de l'objet (gray là où le masque est allumé).
m_int = cv2.moments(gray.astype(np.float32) * (msk > 0), binaryImage=False)

if m_bin["m00"] > 0 and m_int["m00"] > 0:
    shift = float(np.hypot(
        m_int["m10"] / m_int["m00"] - m_bin["m10"] / m_bin["m00"],
        m_int["m01"] / m_int["m00"] - m_bin["m01"] / m_bin["m00"]))
    out_a = {"decalage_centroides_px": shift}
else:
    out_a = {"Erreur": "masque ou image vide"}
```

Piège associé : les moments pondérés héritent de tout ce qui affecte l'intensité — vignettage, fond non uniforme, pixels saturés. Un fond résiduel non soustrait tire le centroïde pondéré

vers le centre de la fenêtre de mesure ; en astrométrie, la soustraction de fond se fait *avant* tout calcul de moment.

2.52. Tableau récapitulatif – quel moment pour quelle question ?

Question	Outil	Ordre	Invariances	Fragilité
Quelle est la taille de l'objet ?	M_{00} (aire)	0	—	très robuste
Où est-il ?	centroïde M_{10}/M_{00} , M_{01}/M_{00}	1	—	très robuste
Dans quel sens est-il orienté ?	ϑ (μ_{11} , μ_{20} , μ_{02})	2	translation	instable si forme ronde
Comment sa masse est-elle répartie ?	ellipse équivalente (λ_1 , λ_2)	2	translation	$\neq$ dimensions réelles
Est-il allongé ?	excentricité e	2	translation, rotation	instable si forme ronde
Le reconnaître à toute pose ?	Hu φ_1 – φ_6	2–3	translation, rotation, échelle	sensible au bruit de bord
Le distinguer de son miroir ?	signe de φ_7	3	translation, rotation, échelle	le plus fragile de tous
Position sous-pixel d'une source ?	centroïde pondéré	1	translation	sensible au fond résiduel



Les moments héritent de toutes les erreurs de segmentation, mais pas tous au même degré. Chaque montée en ordre élève les écarts au centroïde à une puissance de plus : les pixels du bord — précisément ceux que la segmentation place mal — pèsent de plus en plus lourd. D'où l'échelle de fragilité du chapitre :

ordre 0 (aire)	: erreur $\propto$ bruit de bord	– robuste
ordre 1 (centroïde)	: erreur sub-pixel	– très robuste
ordre 2 (orientation)	: instable si forme isotrope	– filtrer par anisotropie
ordre 3 (Hu ϕ_3 – ϕ_7)	: ± 50 % pour 2 px de bruit	– fragile

La règle à retenir : **plus l'ordre est élevé, plus le moment regarde loin du centroïde, plus il amplifie le bruit de contour.** Vous connaissez peut-être déjà cette hiérarchie sous une autre forme : en statistiques, la moyenne d'un échantillon est plus stable que sa variance, elle-même plus stable que son asymétrie (skewness) puis son aplatissement (kurtosis). Ce sont les mêmes moments, appliqués à une distribution de nombres au lieu d'une image — et le parallèle, une fois vu, ne s'oublie plus.

Ce compromis prolonge la leçon du chapitre 1 : un descripteur encode un point de vue, et les moments ajoutent que **chaque supplément de détail se paie en fragilité**. L'ordre 0 voit peu mais ne se trompe jamais ; l'ordre 3 voit l'asymétrie fine mais vacille au moindre pixel. Choisir l'ordre de ses moments, c'est choisir un point d'équilibre entre ce qu'on veut voir et ce qu'on peut mesurer de façon fiable — un arbitrage que l'on retrouvera au chapitre 6, où dériver une image (l'analogie continu de monter en ordre) amplifiera le bruit exactement de la même manière.

Distances et similarités

Table des matières

3.1. Distances de Minkowski (L_p)	38
3.2. Définition	38
3.3. L'idée	39
3.4. Géométrie : la forme de la « boule unité »	39
3.5. Exemple numérique	39
3.6. Piège : la malédiction de la dimension	39
3.7. Code	40
3.8. Distance de Mahalanobis	40
3.9. Définition	40
3.10. L'idée	40
3.11. Exemple numérique	41
3.12. Applications	41
3.13. Piège : Σ singulière ou mal estimée	41
3.14. Code	42
3.15. Similarité cosinus	42
3.16. Définition	42
3.17. L'idée	42
3.18. Quand la préférer à l'euclidienne	42
3.19. Lien avec l'euclidienne sur vecteurs normalisés	43

3.20. Exemple numérique	43
3.21. Code	43
3.22. Distances entre histogrammes	43
3.23. Distance du χ^2	44
3.24. Distance de Bhattacharyya	44
3.25. Exemple numérique	44
3.26. Piège : sensibilité au découpage en cases	44
3.27. Piège de bibliothèque : <code>cv2.compareHist</code> ne calcule pas ces formules	45
3.28. Code	45
3.29. Distance de transport (Wasserstein / EMD)	45
3.30. Définition	45
3.31. L'idée fondatrice	46
3.32. Cas 1-D : une formule simple	46
3.33. Exemple numérique (cas 1-D)	46
3.34. Applications	46
3.35. Piège : coût de calcul	47
3.36. Code	47
3.37. Distance de Hausdorff	47
3.38. Définition	47
3.39. L'idée	47
3.40. Exemple numérique	48
3.41. Piège : sensibilité aux valeurs aberrantes	48
3.42. Applications	48
3.43. Code	48
3.44. Tableau récapitulatif — choisir sa mesure	49

Comparer deux objets — deux descripteurs de forme du chapitre 1, deux histogrammes de couleur, deux distributions — suppose une mesure de leur écart. Mais il n'existe pas une « distance » universelle : il en existe une famille entière, et chacune voit les données à sa façon. Choisir l'une plutôt qu'une autre n'est jamais un détail technique, car ce choix décide de ce que « proche » veut dire. Ce chapitre passe en revue les distances usuelles, de la plus simple (euclidienne) aux plus structurées (Mahalanobis, Wasserstein), avec à chaque fois la même question : *qu'est-ce que je déclare important en la choisissant ?*

Le fil conducteur tient en une phrase : **choisir une distance, c'est déclarer ce qui compte**. Une mesure rend certains écarts négligeables et d'autres décisifs ; elle impose une géométrie aux données avant même qu'on les compare. Comprendre une distance, c'est donc connaître l'hypothèse qu'elle fait dans votre dos.

Un peu de vocabulaire. Une **distance** (ou métrique) est une mesure d'écart qui respecte quatre règles de bon sens : elle n'est jamais négative ; elle vaut zéro seulement quand les deux objets sont identiques ; elle est symétrique (l'écart de x à y égale celui de y à x) ; et elle vérifie l'**inégalité triangulaire** — un détour par un troisième point ne raccourcit jamais le trajet, $d(x, z) \leq d(x, y) + d(y, z)$. Une **similarité** mesure au contraire la ressemblance (grande quand les objets se ressemblent) et ne respecte pas forcément ces règles. Plusieurs « distances » courantes (χ^2 , Bhattacharyya) ne sont d'ailleurs pas de vraies métriques — on le signalera, car cela a des conséquences pratiques.

Côté logiciel : chaque bloc de code se colle dans une node « **Python Script** ». Les deux objets à comparer arrivent dans les variables **a** et **b** (des vecteurs, des histogrammes ou des ensembles de points, selon la mesure), et le résultat est renvoyé dans **out_a** — un dictionnaire qu'une node **Inspecteur** affiche. **np** (NumPy) et **cv2** (OpenCV) sont déjà disponibles ; **scipy** s'importe au besoin.

3.1. Distances de Minkowski (L_p)

3.2. Définition

La famille L_p réunit sous une seule formule les distances les plus courantes :

$$d_p(x, y) = (\sum_i |x_i - y_i|^p)^{1/p}$$

- $p = 1$: distance de Manhattan (L1, *city-block*)
- $p = 2$: distance euclidienne (L2), la distance « à vol d'oiseau »
- $p \rightarrow \infty$: distance de Tchebychev (L ∞)

3.3. L'idée

Le paramètre p règle la façon dont on agrège les écarts coordonnée par coordonnée. Avec $p = 1$, on additionne simplement les écarts, comme un taxi qui parcourt des rues en damier (d'où le nom *Manhattan*). Avec $p = 2$, on prend la diagonale directe. Et plus p grandit, plus la formule ne retient que le plus grand écart : élever à une grande puissance écrase les petites différences devant la plus grande, jusqu'à ce qu'à la limite, **seule la coordonnée où l'écart est maximal compte encore**. C'est pourquoi L_∞ se réduit au simple maximum des écarts, sans qu'il soit besoin de dérouler le calcul.

3.4. Géométrie : la forme de la « boule unité »

La meilleure intuition est de dessiner l'ensemble des points situés à distance 1 de l'origine — la « boule unité » de chaque distance :

L_1 : losange (carré tourné de 45°)

L_2 : cercle

L_∞ : carré aligné sur les axes

Plus p augmente, plus cette boule gonfle, du losange vers le carré. Cette image n'est pas qu'esthétique : les coins du losange L_1 sont posés sur les axes, là où certaines coordonnées sont nulles. Une distance L_1 « préfère » donc les solutions à coordonnées nulles — c'est la raison géométrique pour laquelle la régularisation Lasso, bâtie sur L_1 , produit des modèles parcimonieux.

3.5. Exemple numérique

$x = (1, 2, 3)$, $y = (4, 0, 3) \rightarrow$ écarts $(3, 2, 0)$:

$$L_1 = 3 + 2 + 0 = 5$$

$$L_2 = \sqrt{9 + 4 + 0} = \sqrt{13} \approx 3,61$$

$$L_\infty = \max(3, 2, 0) = 3$$

On retrouve toujours l'ordre $L_\infty \leq L_2 \leq L_1$: plus p est grand, plus la distance est petite, car on agrège les écarts de façon de plus en plus « indulgente ».

3.6. Piège : la malédiction de la dimension

En haute dimension, les distances euclidiennes se **concentrent** : le point le plus proche et le plus lointain finissent par être presque à la même distance, et la notion de voisinage perd son sens. Concrètement, sur des descripteurs à plusieurs centaines de composantes, un «

plus proche voisin » n'est plus vraiment plus proche que les autres. Deux parades : réduire d'abord la dimension (par exemple par ACP) avant de mesurer, ou employer des distances fractionnaires ($p < 1$) qui résistent mieux — au prix de l'inégalité triangulaire, qu'elles ne respectent plus.

3.7. Code

```
# a, b : deux vecteurs (listes ou tableaux) de même longueur.
x = np.asarray(a, dtype=float).ravel()
y = np.asarray(b, dtype=float).ravel()
d = np.abs(x - y)
out_a = {
    "L1 (Manhattan)": float(d.sum()),
    "L2 (euclidienne)": float(np.linalg.norm(d)),
    "Linf (Tchebychev)": float(d.max()),
}
```

3.8. Distance de Mahalanobis

3.9. Définition

$$d(x, y) = \sqrt{(x - y)^T \Sigma^{-1} (x - y)}$$

où Σ est la matrice de covariance des données — un tableau qui résume comment les dimensions varient et se corrént entre elles.

3.10. L'idée

La distance euclidienne fait une hypothèse cachée : que toutes les directions se valent et que les dimensions sont indépendantes. C'est rarement vrai. Si vos données s'étirent beaucoup en largeur et peu en hauteur, un écart horizontal et un écart vertical de même longueur n'ont pas du tout la même signification.

L'analogie la plus juste est celle d'un écart « relatif à la normale du coin ». Un mètre d'écart est banal le long d'une autoroute, où les positions varient déjà beaucoup ; le même mètre est considérable en travers d'un couloir étroit, où tout le monde se tient sur une ligne. La Mahalanobis mesure l'écart non pas en mètres absolus, mais en **nombre de « variations typiques » propres à chaque direction**.

Techniquement, elle **redresse l'espace** : elle étire les directions où les données varient peu, comprime celles où elles varient beaucoup, et défait leurs corrélations, jusqu'à ce que le nuage de points devienne une boule bien ronde. Dans cet espace redressé, on mesure alors une simple distance euclidienne. Inutile de savoir comment se calcule ce redressement — il se lit dans la matrice de covariance ; il suffit de retenir l'effet : **la Mahalanobis est la distance euclidienne, mais mesurée après avoir rendu le nuage de données parfaitement isotrope**. Ses surfaces d'iso-distance ne sont plus des cercles, mais des ellipses épousant la forme du nuage.

3.11. Exemple numérique

Prenons des données très étirées horizontalement, dix fois plus dispersées en x qu'en y (covariance $\Sigma = \text{diag}(100, 1)$). Comparons, depuis le centre, deux points pourtant à la même distance euclidienne : $a = (10, 0)$ et $b = (0, 10)$.

$d_{\text{eucl}}(a) = d_{\text{eucl}}(b) = 10$ (identiques pour l'euclidienne)

$d_{\text{Mahal}}(a) = \sqrt{(10^2 / 100)} = 1$ (10 dans une direction très dispersée : banal)

$d_{\text{Mahal}}(b) = \sqrt{(10^2 / 1)} = 10$ (10 dans une direction peu dispersée : rare)

Pour la Mahalanobis, a est dix fois plus proche que b . Un écart de 10 dans la direction de forte variabilité est ordinaire ; le même écart dans la direction de faible variabilité est exceptionnel. C'est exactement le raisonnement de la détection d'anomalies.

3.12. Applications

Détection d'outliers (un point au-delà de $d_{\text{Mahal}} \approx 3$ est statistiquement suspect), classification gaussienne (comparer un point aux centres des classes revient à comparer des distances de Mahalanobis), et suivi par filtre de Kalman, où elle valide l'association entre une mesure et une prédiction.

3.13. Piège : Σ singulière ou mal estimée

Si le nombre d'échantillons est inférieur au nombre de dimensions, la covariance Σ n'est pas inversible — et la formule explose. Parades : ajouter un petit terme à la diagonale ($\Sigma + \epsilon I$, dit *shrinkage*), utiliser une pseudo-inverse, ou un estimateur robuste (Ledoit-Wolf). La règle : ne jamais inverser une covariance estimée sur trop peu de points sans la régulariser.

3.14. Code

```
# a : le nuage de données (n points × d dimensions, pour estimer Σ).
# b : le point à tester. Renvoie sa distance de Mahalanobis au centre du nuage.
from scipy.spatial.distance import mahalanobis

data = np.asarray(a, dtype=float)
x     = np.asarray(b, dtype=float).ravel()
mu    = data.mean(axis=0)
cov   = np.cov(data.T)
VI    = np.linalg.inv(cov + 1e-9 * np.eye(cov.shape[0])) # +εI : régularisation

dm = float(mahalanobis(x, mu, VI))
out_a = {
    "mahalanobis": dm,
    "euclidienne": float(np.linalg.norm(x - mu)),
    "anomalie (>3)": bool(dm > 3),
}
```

3.15. Similarité cosinus

3.16. Définition

$\cos(\theta) = (x \cdot y) / (\|x\| \cdot \|y\|) \in [-1, 1]$

On en tire une distance : $d = 1 - \cos(\theta)$.

3.17. L'idée

La similarité cosinus est le cosinus de l'angle entre les deux vecteurs. Elle découle directement de la définition du produit scalaire, et son trait essentiel est qu'elle **ignore les longueurs pour ne regarder que les directions**. Deux vecteurs qui pointent dans le même sens ont une similarité de 1, qu'ils soient longs ou courts ; deux vecteurs perpendiculaires ont une similarité de 0.

3.18. Quand la préférer à l'euclidienne

Chaque fois que le **profil** importe plus que l'**intensité**. L'exemple canonique est la comparaison de textes représentés par leurs fréquences de mots : un long article et son

résumé partagent le même thème (même direction) mais ont des longueurs très différentes. L'euclydienne les déclarerait éloignés à cause de l'écart de taille ; le cosinus les reconnaît proches. C'est pourquoi la recherche documentaire, les systèmes de recommandation et la comparaison d'*embeddings* (de mots, d'images, de phrases) reposent presque toujours sur le cosinus.

3.19. Lien avec l'euclydienne sur vecteurs normalisés

Il existe un pont utile entre les deux. Si l'on ramène d'abord les vecteurs à une longueur de 1 (normalisation), alors la distance euclydienne et la distance cosinus deviennent deux façons de dire la même chose :

$$d_{\text{eucl}}^2(x, y) = 2 \cdot (1 - \cos \theta) \quad (\text{quand } \|x\| = \|y\| = 1)$$

Sur des vecteurs normalisés, l'une croît exactement avec l'autre. D'où une habitude répandue : on normalise les embeddings avant de les indexer, ce qui permet d'utiliser un moteur de recherche euclydien rapide tout en raisonnant, au fond, en cosinus.

3.20. Exemple numérique

$x = (2, 0)$, $y = (3, 0)$: colinéaires $\rightarrow \cos = 6 / (2 \cdot 3) = 1$, similarité maximale malgré des longueurs différentes. $x = (1, 0)$, $y = (0, 1)$: perpendiculaires $\rightarrow \cos = 0$.

3.21. Code

```
# a, b : deux vecteurs (descripteurs, embeddings...).
x = np.asarray(a, dtype=float).ravel()
y = np.asarray(b, dtype=float).ravel()
na, nb = np.linalg.norm(x), np.linalg.norm(y)
cos = float(np.dot(x, y) / (na * nb)) if na and nb else 0.0
out_a = {"cosinus": cos, "distance (1-cos)": 1.0 - cos}
```

3.22. Distances entre histogrammes

Comparer des distributions discrètes — histogrammes de couleur, de gradients, sacs de mots — demande des mesures spécifiques. Soit deux histogrammes normalisés h et g (chacun de somme 1).

3.23. Distance du χ^2

$$d_{\chi^2}(h, g) = \frac{1}{2} \sum_i (h_i - g_i)^2 / (h_i + g_i)$$

L'idée du dénominateur $(h_i + g_i)$ est de **pondérer chaque case par son importance**. Un écart absolu de 0,05 est négligeable sur une case bien remplie (à 0,5), mais énorme sur une case presque vide (à 0,01). La χ^2 traite donc les cases rares comme plus discriminantes — un peu comme on s'étonne davantage d'une différence sur un événement rare que sur un événement banal. Ce n'est **pas une vraie métrique** (l'inégalité triangulaire peut échouer), mais elle est redoutablement efficace sur les histogrammes de couleur.

3.24. Distance de Bhattacharyya

$$BC(h, g) = \sum_i \sqrt{h_i \cdot g_i} \quad (\text{coefficient, } \in [0, 1])$$

$$d_B(h, g) = -\ln(BC(h, g))$$

Le coefficient BC a une lecture géométrique élégante. Si l'on remplace chaque case par sa racine carrée ($\sqrt{h_i}$), l'histogramme devient un vecteur de longueur 1, et BC n'est alors rien d'autre que **la similarité cosinus de ces vecteurs-racines** : on retrouve la mesure de la section 3.3, transportée dans « l'espace des racines ». Cette parenté explique sa robustesse. Une variante, la distance de Hellinger $\sqrt{1 - BC}$, est, elle, une vraie métrique.

3.25. Exemple numérique

$h = (0,5 ; 0,5 ; 0)$, $g = (0 ; 0,5 ; 0,5)$ — deux histogrammes qui ne se recouvrent que sur une seule case :

$$\chi^2 = \frac{1}{2} [0,5^2/0,5 + 0 + 0,5^2/0,5] = \frac{1}{2} [0,5 + 0 + 0,5] = 0,5$$

$$BC = \sqrt{0} + \sqrt{0,25} + \sqrt{0} = 0,5 \quad \rightarrow \quad d_B = -\ln(0,5) \approx 0,693$$

3.26. Piège : sensibilité au découpage en cases

Toutes ces mesures comparent les cases **une à une**. Deux histogrammes décalés d'une seule case — une teinte à peine plus rouge — sont jugés totalement différents, alors qu'ils sont perceptuellement presque identiques. C'est la faiblesse de fond des distances case-à-case, que corrige la distance de transport de la section suivante.

3.27. Piège de bibliothèque : `cv2.compareHist` ne calcule pas ces formules

OpenCV propose `cv2.compareHist`, mais ses conventions ne coïncident pas avec les définitions ci-dessus, ce qui est une source d'erreurs classique :

- `HISTCMP_CHISQR` calcule la χ^2 **asymétrique** $\Sigma(h_i - g_i)^2 / h_i$, et `HISTCMP_CHISQR_ALT` vaut $2 \cdot \Sigma(h_i - g_i)^2 / (h_i + g_i)$ — soit **quatre fois** la formule du livre (sur notre exemple, elle renvoie 2,0 et non 0,5).
- `HISTCMP_BHATTACHARYYA` renvoie en réalité la distance de **Hellinger** $\sqrt{1 - BC}$ (ici 0,707), et non $-\ln(BC)$.

Le code ci-dessous calcule donc les formules à la main, ce qui lève toute ambiguïté.

3.28. Code

```
# a, b : deux histogrammes (mêmes cases).
h = np.asarray(a, dtype=float).ravel()
g = np.asarray(b, dtype=float).ravel()
h, g = h / h.sum(), g / g.sum()          # normaliser (somme = 1)
eps = 1e-12

chi2 = 0.5 * np.sum((h - g) ** 2 / (h + g + eps))
bc = np.sum(np.sqrt(h * g))              # coefficient de Bhattacharyya
out_a = {
    "chi2": float(chi2),
    "BC (Bhattacharyya)": float(bc),
    "d_B (-ln BC)": float(-np.log(bc + eps)),
    "Hellinger sqrt(1-BC)": float(np.sqrt(max(0.0, 1.0 - bc))),
}
```

3.29. Distance de transport (Wasserstein / EMD)

3.30. Définition

La *Earth Mover's Distance* (distance du « déplaceur de terre ») mesure le **coût minimal pour transformer une distribution en une autre** en déplaçant de la masse, sachant qu'on paie distance $\times$ quantité déplacée. Formellement, on cherche le plan de transport le moins coûteux ; le détail de l'optimisation importe moins que l'idée.

3.31. L'idée fondatrice

Contrairement aux distances case-à-case, l'EMD **connaît la géométrie des cases** : déplacer de la masse vers une case voisine coûte peu, vers une case lointaine coûte cher. L'image est celle de tas de sable que l'on veut remodeler : combien de travail pour transformer un tas en un autre ? Deux histogrammes décalés d'une seule case ont une EMD petite (le coût d'un petit déplacement), là où la χ^2 les déclarait maximalement différents. L'EMD respecte donc la proximité perceptuelle que les mesures case-à-case ignorent.

3.32. Cas 1-D : une formule simple

En dimension 1 — comparer deux profils, par exemple deux histogrammes de niveaux de gris — l'EMD se calcule sans aucune optimisation, à partir des **fonctions de répartition cumulées** (on additionne les cases de gauche à droite) :

$$\text{EMD}_1(h, g) = \sum_i |H(i) - G(i)| \quad \text{où } H, G \text{ sont les cumuls de } h, g$$

C'est l'aire entre les deux courbes cumulées — calculable d'un trait, en parcourant les cases une fois.

3.33. Exemple numérique (cas 1-D)

$h = (0,5 ; 0,5 ; 0)$, $g = (0 ; 0,5 ; 0,5)$, cases équidistantes :

cumul $H = (0,5 ; 1,0 ; 1,0)$

cumul $G = (0,0 ; 0,5 ; 1,0)$

$$\text{EMD} = |0,5-0| + |1,0-0,5| + |1,0-1,0| = 0,5 + 0,5 + 0 = 1,0$$

L'EMD vaut 1 case : il faut déplacer toute la masse d'exactly un cran vers la droite. C'est une information d'« amplitude de déplacement » que la χ^2 (= 0,5, sans unité) ne donnait pas.

3.34. Applications

Comparaison de couleurs perceptuellement correcte (recherche d'images par le contenu), évaluation de modèles génératifs (la distance de Wasserstein entre distribution réelle et générée fonde les WGAN), transfert de couleur entre images, comparaison de nuages de points 3-D.

3.35. Piège : coût de calcul

Le cas général (2-D et au-delà) n'a pas de formule simple : il faut résoudre un vrai problème de transport optimal, coûteux quand les cases sont nombreuses. On recourt alors à l'approximation de Sinkhorn, rapide et parallélisable, au prix d'un léger lissage. À noter : la bibliothèque spécialisée POT (`import ot`) n'est **pas fournie** dans le moteur VNStudio ; pour rester autonome, on se limite ici au cas 1-D, qui couvre déjà la comparaison de profils.

3.36. Code

```
# a, b : deux histogrammes 1-D (mêmes cases équidistantes).
h = np.asarray(a, dtype=float).ravel()
g = np.asarray(b, dtype=float).ravel()
h, g = h / h.sum(), g / g.sum()

# EMD 1-D = aire entre les deux cumuls (aucune optimisation nécessaire).
emd = float(np.sum(np.abs(np.cumsum(h) - np.cumsum(g))))
out_a = {"wasserstein_1d (en cases)": emd}

# Équivalent via scipy (positions = indices de cases, poids = hauteurs) :
# from scipy.stats import wasserstein_distance
# emd = wasserstein_distance(np.arange(len(h)), np.arange(len(g)), h, g)
```

3.37. Distance de Hausdorff

3.38. Définition

C'est une distance entre deux **ensembles** de points A et B — typiquement deux contours :

$$h(A, B) = \max_{\{a \in A\}} (\min_{\{b \in B\}} d(a, b)) \quad \text{(distance dirigée)}$$
$$H(A, B) = \max(h(A, B), h(B, A)) \quad \text{(distance symétrique)}$$

3.39. L'idée

Décortiquons la formule de l'intérieur. Pour un point a de A , $\min_{\{b \in B\}} d(a, b)$ est sa distance à l'ensemble B , c'est-à-dire la distance à son **plus proche voisin** dans B . Puis $\max_{\{a \in A\}}$ retient le pire cas : le point de A le plus mal loti, le plus éloigné de tout B . La distance dirigée $h(A, B)$ répond donc à : « quelle est la pire excursion d'un point de A hors de B ? »

On a besoin des deux directions car la mesure n'est pas symétrique : A peut être entièrement blotti contre B sans que l'inverse soit vrai. La Hausdorff symétrique prend le pire des deux sens.

3.40. Exemple numérique

$A = \{(0,0), (1,0)\}$, $B = \{(0,0), (1,0), (6,0)\}$:

$h(A, B) = 0$ (chaque point de A est aussi dans B → distance nulle)
 $h(B, A) = 5$ (le point isolé (6,0) de B a pour plus proche voisin dans A
le point (1,0), à distance 5 – et non (0,0) à distance 6 !)
 $H(A, B) = \max(0, 5) = 5$

L'asymétrie est flagrante : $A \subset B$ rend une direction nulle, tandis que le point isolé de B fait exploser l'autre. Notez bien le point délicat : la distance se mesure au **plus proche** voisin (ici (1,0)), pas à un point arbitraire de A — confondre les deux est l'erreur de calcul classique sur la Hausdorff.

3.41. Piège : sensibilité aux valeurs aberrantes

Le « pire cas » rend la Hausdorff extrêmement sensible à un seul point aberrant : un unique pixel de bruit dans un contour peut doubler la distance. La parade standard est de remplacer le maximum par un quantile élevé (le 95^e percentile, noté HD95) ou par une moyenne des distances. En segmentation médicale, on rapporte presque toujours la HD95 plutôt que la Hausdorff brute, précisément pour cette raison.

3.42. Applications

Comparaison de contours et de formes, et surtout évaluation de segmentation, où elle complète l'IoU : l'IoU mesure le recouvrement global de surface, la Hausdorff mesure la **pire erreur de frontière**. Deux segmentations peuvent avoir le même IoU mais des Hausdorff très différentes si l'une a une excroissance lointaine.

3.43. Code

```
# a, b : deux ensembles de points (par ex. deux contours), forme (N, 2).
from scipy.spatial.distance import directed_hausdorff

A = np.asarray(a, dtype=float).reshape(-1, 2)
B = np.asarray(b, dtype=float).reshape(-1, 2)
```



```

hab = directed_hausdorff(A, B)[0]
hba = directed_hausdorff(B, A)[0]
out_a = {
    "hausdorff": float(max(hab, hba)),
    "dirigee A->B": float(hab),
    "dirigee B->A": float(hba),
}

```

3.44. Tableau récapitulatif – choisir sa mesure

Mesure	Compare	Vraie métrique ?	Hypothèse clé / usage type
Euclidienne (L2)	vecteurs	oui	dimensions comparables et décorrélées
Manhattan (L1)	vecteurs	oui	favorise la parcimonie ; robuste aux outliers
Tchebychev (L ∞)	vecteurs	oui	seule la pire coordonnée compte
Mahalanobis	vecteurs + Σ	oui	dimensions corrélées, d'échelles différentes
Cosinus	vecteurs	non (1-cos)	la direction compte, pas la longueur (embeddings, texte)
χ^2	histogrammes	non	les cases rares pèsent plus (couleur)
Bhattacharyya	histogrammes	non	cosinus dans l'espace des racines ; recouvrement
Wasserstein / EMD	distributions	oui	la géométrie des cases compte (couleur perceptuelle, GAN)
Hausdorff	ensembles / contours	oui	pire écart de frontière (forme, segmentation)



Le fil conducteur du chapitre, déroulé d'une mesure à l'autre :

euclidienne	→ toutes les directions se valent
Mahalanobis	→ certaines directions sont plus « surprenantes » que d'autres
cosinus	→ l'orientation compte, l'amplitude non
χ^2	→ les événements rares pèsent plus
Wasserstein	→ la proximité des catégories entre elles compte
Hausdorff	→ seul le pire cas compte

Il n'existe pas de distance universellement meilleure. Une mesure excellente sur des histogrammes de couleur (l'EMD, qui connaît la géométrie des teintes) est inadaptée à des embeddings de haute dimension (où règne le cosinus), et inversement. La première question n'est jamais « quelle est la meilleure distance ? », mais « **qu'est-ce que je veux considérer comme proche ?** » — et la réponse dicte la mesure.

C'est la même leçon que celle des chapitres précédents, vue sous un autre angle. Un descripteur (chapitre 1) choisit ce qu'il voit et ce qu'il oublie ; un moment (chapitre 2) paie chaque détail supplémentaire en fragilité ; une distance, ici, déclare ce qui mérite d'être appelé « proche ». À chaque fois, le bon outil n'est pas le plus puissant dans l'absolu, mais celui dont les hypothèses épousent le problème — un fil que l'on retrouvera au chapitre suivant, sur les métriques de segmentation, où l'on verra qu'aucune mesure unique ne capture à elle seule la qualité d'un résultat.

CHAPITRE

Métriques de segmentation et détection

4

Table des matières

4.1. IoU — Intersection over Union (indice de Jaccard)	52
4.2. Coefficient de Dice (F1 sur pixels)	54
4.3. Précision, rappel et F1	56
4.4. Average Precision (AP) et mAP	58
4.5. Panoptic Quality (PQ)	61
4.6. Boundary F1 (BF score)	62
4.7. Tableau récapitulatif — quelle métrique pour quoi ?	66
4.8. aucune métrique unique ne capture tout, et c'est inévitable	66

Lorsqu'un algorithme délimite une tumeur sur une IRM, découpe une voiture dans une scène de rue ou identifie une colonie bactérienne au microscope, comment savoir s'il a bien travaillé ? La réponse semble simple : comparer ce qu'il a trouvé à la vérité terrain. Mais cette comparaison soulève immédiatement trois questions distinctes, souvent confondues : *l'objet est-il à la bonne place ?* (localisation), *l'ai-je trouvé ?* (rappel), *ce que j'ai trouvé est-il exact ?* (précision). Ce chapitre construit les métriques qui y répondent, depuis l'IoU élémentaire jusqu'à la Panoptic Quality, en montrant comment chacune privilégie un aspect de la qualité au détriment d'un autre.

Le fil conducteur tient en une phrase : **aucune métrique unique ne capture tout**. Ce n'est pas un défaut de chaque outil — c'est une propriété fondamentale. Reporter l'IoU seul, ou l'AP seul, c'est cacher un mode d'échec. Chaque section de ce chapitre révèle l'angle mort propre à la métrique qu'elle présente, car un évaluateur rigoureux doit connaître ce que son instrument ne peut pas voir, exactement comme le §1 a montré que tout descripteur de forme encode ce qu'il choisit d'ignorer.

Quelques mots de vocabulaire avant de commencer. Un **masque binaire** est une image où chaque pixel vaut 255 (ou 1) s'il appartient à un objet, et 0 sinon — c'est la forme de données de base pour la segmentation, déjà rencontrée au chapitre 1. On notera **VP** (vrai positif), **FP** (faux positif), **FN** (faux négatif). Les vrais négatifs (VN) sont en général absents des métriques de segmentation : l'arrière-plan « non détecté » représente des millions de pixels et ferait exploser n'importe quel score de spécificité — on les ignore délibérément.

4.1. IoU – Intersection over Union (indice de Jaccard)



$$\text{IoU}(A, B) = |A \cap B| / |A \cup B|$$

A désigne la région prédite, B la vérité terrain. $\text{IoU} \in [0, 1]$.

Dérivation et propriétés

L'image mentale la plus directe est celle de deux taches peintes sur un mur. L'IoU pose une question géométrique simple : quelle fraction de la surface totale occupée par les deux taches est couverte par les deux en même temps ? Si les taches sont parfaitement superposées, la totalité est commune — $\text{IoU} = 1$. Si elles ne se touchent pas, aucune surface n'est partagée — $\text{IoU} = 0$.

En exprimant l'union à l'aide des pixels mal classés, on obtient une écriture révélatrice :

$$A \cap B = VP$$

$$A \cup B = VP + FP + FN$$

$$\boxed{?} \text{ IoU} = VP / (VP + FP + FN)$$



Cette décomposition révèle la double pénalité de l'IoU : il sanctionne simultanément les pixels oubliés (FN, sous-segmentation) et les pixels en trop (FP, sur-segmentation). C'est sa force — un seul nombre résume les deux erreurs. C'est aussi son angle mort : il ne dit pas *lequel* des deux domine. Un IoU de 0,6 peut venir d'un masque trop petit ou d'un masque trop grand ; les deux obtiennent la même note alors que les corrections à apporter sont opposées. Ici, comme pour la circularité du chapitre 1 qui confond allongement et rugosité, une seule valeur peut masquer deux causes distinctes.

Pourquoi le seuil 0,5 ?

En détection, une prédiction est comptée VP si $\text{IoU}(\text{prédiction}, \text{vérité}) \geq \tau$. Le seuil historique $\tau = 0,5$ (PASCAL VOC) est un compromis : assez permissif pour tolérer une localisation imparfaite, assez strict pour rejeter les recouvrements fortuits. Les benchmarks modernes (COCO) moyennent sur $\tau \in \{0,50 ; 0,55 ; \dots ; 0,95\}$ précisément pour ne pas dépendre de ce choix arbitraire — et parce qu'une métrique unique de seuil, c'est déjà une information partielle.



Deux carrés de côté 10 px, décalés de 3 px sur l'axe horizontal (recouvrement de 7×10 px) :

$$\text{intersection} = 7 \times 10 = 70 \text{ px}^2$$

$$\text{union} = 2 \times 100 - 70 = 130 \text{ px}^2$$

$$\text{IoU} = 70 / 130 \approx 0,538$$

Un décalage de 3 px sur 10 (soit 30 % de la taille de l'objet) ne donne qu'un IoU de 0,54. L'IoU chute rapidement dès que l'alignement n'est pas parfait, ce qui le rend sévère pour les petits objets — et ce problème d'échelle est lui-même un angle mort de la métrique (voir piège ci-dessous).



Pour un objet de quelques pixels, une erreur de 1 px de frontière fait varier l'IoU de manière considérable ; pour un grand objet, la même erreur est négligeable. L'IoU n'est donc pas comparable entre objets de tailles différentes. COCO contourne cela en rapportant des AP séparés par catégorie de taille (small / medium / large). Toujours préciser la taille des objets évalués quand on cite un IoU isolé.



```
# a : masque prédit (H×W uint8, 0/255)
# b : masque vérité (H×W uint8, 0/255)
if not isinstance(a, np.ndarray) or not isinstance(b, np.ndarray):
    out_a = {'erreur': 'deux masques requis (entrées a et b)'}
else:
    pred = a > 127
    truth = b > 127
    inter = float(np.logical_and(pred, truth).sum())
    union = float(np.logical_or(pred, truth).sum())
    iou = inter / union if union > 0 else 0.0
    out_a = {
        'IoU': round(iou, 4),
        'VP': int(inter),
        'FP': int((pred & ~truth).sum()),
        'FN': int((~pred & truth).sum()),
    }
```

4.2. Coefficient de Dice (F1 sur pixels)



$$\text{Dice}(A, B) = 2|A \cap B| / (|A| + |B|)$$

Relation exacte avec l'IoU

Dice et IoU mesurent la même chose, mais avec une pondération différente. Notons $I = |A \cap B|$ (l'intersection) et $U = |A \cup B|$ (l'union). On a $|A| + |B| = U + I$, d'où :

$$\text{Dice} = 2I / (U + I) \quad \text{et} \quad \text{IoU} = I / U$$

$$\text{Dice} = 2 \cdot \text{IoU} / (1 + \text{IoU}) \quad \text{et} \quad \text{IoU} = \text{Dice} / (2 - \text{Dice})$$


Ces deux formules montrent que Dice et IoU sont des transformations monotones croissantes l'une de l'autre. Un classement de modèles par Dice et par IoU est donc **identique** : si le modèle A bat le modèle B selon l'IoU, il le bat aussi selon Dice. Le choix entre les deux est conventionnel — la biologie médicale et la radiologie ont adopté Dice, la vision par ordinateur classique a adopté IoU — et non substantiel.

Dice est plus indulgent

Pour tout recouvrement partiel, $\text{Dice} \geq \text{IoU}$. La raison est intuitive : Dice compte l'intersection *deux fois* au numérateur (le « 2 » devant $2I$), ce qui récompense davantage

les zones bien recouvertes. À $\text{IoU} = 0,5$, par exemple, $\text{Dice} = 2 \times 0,5 / 1,5 \approx 0,667$. Cette indulgence explique sa popularité comme **fonction de coût** d'entraînement : les gradients sont plus favorables en début d'apprentissage, quand les recouvrements sont encore faibles.

Pourquoi « F1 sur pixels » ?

En réexprimant par les VP, FP, FN :

$$\text{Dice} = 2 \cdot \text{VP} / (2 \cdot \text{VP} + \text{FP} + \text{FN})$$

C'est exactement le F1-score appliqué pixel par pixel, au lieu d'objet par objet. Dice en imagerie et F1 en apprentissage automatique sont la même quantité dans deux vocabulaires différents — un exemple de plus de la façon dont le choix de représentation encode une hypothèse sur ce qui compte (ici : traiter chaque pixel comme une observation indépendante).



Reprenons les deux carrés de la section 4.1 (intersection = 70 px², $|A| = |B| = 100$ px²) :

$$\text{Dice} = 2 \times 70 / (100 + 100) = 140 / 200 = 0,700$$

À comparer à $\text{IoU} = 0,538$. Vérification par la formule de conversion : $2 \times 0,538 / (1 + 0,538) = 1,076 / 1,538 \approx 0,700$. ✓



Dice ignore les vrais négatifs — c'est un avantage quand l'objet est minuscule dans une grande image (une lésion de 0,1 % des pixels d'une IRM ne doit pas noyer son score dans un océan de VN). Mais cette propriété rend Dice instable pour les très petits objets : un segment de 5 pixels où 2 sont manqués donne $\text{Dice} = 6/8 = 0,75$, alors qu'une seule cellule de différence est en jeu. Toujours compléter par un comptage brut (VP, FN) pour les objets de taille critique.



```
# a : masque prédit (H×W uint8, 0/255)
# b : masque vérité (H×W uint8, 0/255)
if not isinstance(a, np.ndarray) or not isinstance(b, np.ndarray):
    out_a = {'erreur': 'deux masques requis (entrées a et b)'}
else:
    pred = a > 127
    truth = b > 127
    inter = float(np.logical_and(pred, truth).sum())
    denom = float(pred.sum() + truth.sum())
    dice = 2.0 * inter / denom if denom > 0 else 0.0
    iou = inter / float(np.logical_or(pred, truth).sum()) if
np.logical_or(pred, truth).sum() > 0 else 0.0
    out_a = {
        'Dice': round(dice, 4),
        'IoU': round(iou, 4),
        'verification_conversion': round(2 * iou / (1 + iou), 4),
    }
```

4.3. Précision, rappel et F1

Définitions

Précision = $VP / (VP + FP)$ « parmi mes détections, combien sont justes ? »

Rappel = $VP / (VP + FN)$ « parmi les objets réels, combien ai-je trouvés ? »

La tension fondamentale

Précision et rappel s'opposent structurellement, et cette tension est le cœur de l'évaluation en détection. Abaisser le seuil de confiance d'un détecteur augmente le rappel (on trouve plus d'objets) mais baisse la précision (on accepte plus d'erreurs). Remonter ce seuil fait l'inverse.

Une analogie éclairante : un douanier très soupçonneux qui ouvre chaque bagage inspecte tout et ne laisse rien passer (rappel = 1), mais ralentit tout le monde et crée de nombreuses fouilles inutiles (précision basse). Un douanier laxiste, lui, ne dérange personne (précision = 1 sur les personnes qu'il arrête), mais laisse passer les contrebandiers (rappel très bas). Aucune politique ne maximise les deux simultanément.

Annoncer « 95 % de précision » sans le rappel est un classique de la communication trompeuse : un détecteur qui ne signale qu'un seul objet — le plus évident — atteint 100 % de précision pour un rappel dérisoire. Les deux chiffres sont inséparables.

F1 : pourquoi la moyenne harmonique

$$F1 = 2 \cdot (P \times R) / (P + R)$$

La moyenne harmonique est dominée par la plus petite des deux valeurs. Avec $P = 1,0$ et $R = 0,01$ (détecteur qui trouve presque tout mais très rarement) :

moyenne arithmétique = $(1,0 + 0,01) / 2 = 0,505$ (trompeuse : suggère « moyen »)
 moyenne harmonique = $2 \times 1,0 \times 0,01 / (1,0 + 0,01) \approx 0,020$ (juste : le système est mauvais)

La moyenne arithmétique masquerait le déséquilibre. La moyenne harmonique refuse de récompenser un système qui sacrifie complètement l'une des deux dimensions — c'est précisément ce qu'on attend d'une métrique d'équilibre. Ici aussi, aucune métrique unique ne capture tout : le F1 réduit deux chiffres à un seul en perdant de l'information sur laquelle des deux dimensions est sacrifiée.



Un détecteur de cellules malades dans une coupe histologique trouve 80 régions suspectes : 60 sont de vraies cellules malades, 20 sont du tissu sain mal identifié, et 40 cellules malades sont passées inaperçues.

$$VP = 60, \quad FP = 20, \quad FN = 40$$

$$\text{Précision} = 60 / 80 = 0,750$$

$$\text{Rappel} = 60 / 100 = 0,600$$

$$F1 = 2 \times 0,750 \times 0,600 / (0,750 + 0,600) = 0,900 / 1,350 \approx 0,667$$

Le $F1 = 0,667$ reflète le fait que ni la précision ni le rappel ne sont excellents — il ne permet pas de décider si on doit améliorer la détection (trop de FP) ou la couverture (trop de FN).

Le F_β : pondérer la tension

Quand une dimension importe davantage que l'autre, on introduit un paramètre β :

$$F_\beta = (1 + \beta^2) \cdot P \cdot R / (\beta^2 \cdot P + R)$$

$\beta > 1$ favorise le rappel (dépistage médical : mieux vaut un faux positif qu'un cancer manqué),
 $\beta < 1$ favorise la précision (filtre anti-spam : mieux vaut laisser passer un spam que bloquer un message légitime). F1 est le cas symétrique $\beta = 1$.



```
# a : masque prédit (H×W uint8, 0/255)
# b : masque vérité (H×W uint8, 0/255)
if not isinstance(a, np.ndarray) or not isinstance(b, np.ndarray):
    out_a = {'erreur': 'deux masques requis (entrées a et b)'}
else:
    pred = a > 127
    truth = b > 127
    vp = float(np.logical_and(pred, truth).sum())
    fp = float(np.logical_and(pred, ~truth).sum())
    fn = float(np.logical_and(~pred, truth).sum())
    precision = vp / (vp + fp) if (vp + fp) > 0 else 0.0
    rappel = vp / (vp + fn) if (vp + fn) > 0 else 0.0
    f1 = 2 * precision * rappel / (precision + rappel) if (precision +
rappel) > 0 else 0.0
    out_a = {
        'precision': round(precision, 4),
        'rappel': round(rappel, 4),
        'F1': round(f1, 4),
        'VP': int(vp), 'FP': int(fp), 'FN': int(fn),
    }
```

4.4. Average Precision (AP) et mAP



$$AP = \int_0^1 p(r) \, dr$$

L'AP est l'aire sous la courbe précision-rappel. La mAP (mean Average Precision) est la moyenne de l'AP sur toutes les classes d'objets.

Construction de la courbe précision-rappel

L'idée est de traiter le détecteur à tous les seuils de confiance possibles d'un coup, plutôt que d'en choisir un seul. On trie les détections par confiance décroissante. En descendant la liste, chaque détection est classée VP ou FP (selon qu'elle dépasse le seuil IoU), et on recalcule le couple (précision, rappel) cumulé à chaque pas. On obtient une courbe en escalier : le rappel croît monotonement au fil des détections, tandis que la précision monte à chaque VP et descend à chaque FP.

L'AP résume cette courbe en un seul chiffre — son aire. Un détecteur idéal aurait une courbe plate à précision = 1 pour tout rappel : $AP = 1$. Un détecteur aléatoire produirait une courbe plate à précision = fréquence de base de la classe : AP très bas.

L'interpolation : pourquoi et comment

La courbe brute est dentelée à cause de l'ordre exact des détections. La convention (PASCAL VOC, reprise par COCO) la rend monotone décroissante en remplaçant chaque précision par le maximum des précisions à rappel égal ou supérieur :

$$p_interp(r) = \max_{\{r' \geq r\}} p(r')$$

Justification : à un niveau de rappel donné, on s'autorise à « emprunter » la meilleure précision atteignable plus loin dans la courbe. Cela lisse le bruit dû à l'ordre exact des détections et rend la métrique stable entre expériences. COCO échantillonne ensuite cette courbe sur 101 points de rappel (0 ; 0,01 ; ... ; 1) pour obtenir une intégrale numérique propre.



Cinq détections triées par confiance décroissante, sur 3 objets réels :

rang	statut	VP cumulé	précision	rappel
1	VP	1	1,000	0,333
2	VP	2	1,000	0,667
3	FP	2	0,667	0,667
4	VP	3	0,750	1,000
5	FP	3	0,600	1,000

Précisions interpolées (maximum à droite) : aux rappels {0,33 ; 0,67 ; 1,00} correspondent les précisions {1,00 ; 1,00 ; 0,75}. $AP \approx (1,00 + 1,00 + 0,75) / 3 \approx 0,917$.

Les conventions divergent : toujours les préciser

Un « mAP de 0,4 » sur COCO peut correspondre à un « mAP de 0,7 » sur VOC pour le même modèle. Comparer des AP de conventions différentes n'a aucun sens — c'est l'angle mort institutionnel de la métrique.

PASCAL VOC : mAP @ IoU = 0,5 (seuil unique)

COCO : mAP moyenné sur IoU $\in \{0,50 ; 0,55 ; \dots ; 0,95\}$

La convention COCO est beaucoup plus stricte car elle exige une localisation fine à IoU = 0,95 — un pixel de décalage sur une petite cible peut la faire basculer de VP à FP.



L'AP intègre sur tous les seuils de confiance — c'est sa vertu (pas de seuil arbitraire) et son angle mort : elle ne reflète pas la performance au seuil qu'on utilisera réellement en production. Un modèle peut avoir un excellent AP et être médiocre au seuil de déploiement. Pour le déploiement opérationnel, rapporter aussi le F1 (ou précision + rappel) au seuil de confiance effectivement choisi.



```
# a : liste Python de scores de confiance (float), port 'a' → type list
# b : liste Python de labels VP/FP (1=VP, 0=FP), port 'b' → type list
# n_positifs : nombre total d'objets réels dans l'évaluation (scalar,
port 'c')
if not isinstance(a, list) or not isinstance(b, list) or not
isinstance(c, (int, float)):
    out_a = {'erreur': 'a=scores, b=labels VP/FP, c=nb_positifs_reels'}
else:
    scores, labels, n_pos = a, b, int(c)
    ordre = sorted(range(len(scores)), key=lambda i: -scores[i])
    vp_cum, fp_cum = 0, 0
    precisions, rappels = [], []
    for i in ordre:
        if labels[i] == 1:
            vp_cum += 1
        else:
            fp_cum += 1
        precisions.append(vp_cum / (vp_cum + fp_cum))
        rappels.append(vp_cum / n_pos if n_pos > 0 else 0.0)
    # interpolation monotone décroissante
    p_interp = []
    for k in range(len(precisions)):
        p_interp.append(max(precisions[k:]))
    ap = float(sum(p_interp) / len(p_interp)) if p_interp else 0.0
    out_a = {
        'AP': round(ap, 4),
        'n_detections': len(scores),
        'n_VP': vp_cum,
        'n_FP': fp_cum,
    }
```

4.5. Panoptic Quality (PQ)

Le problème qu'elle résout

La segmentation panoptique unifie deux tâches fondamentalement différentes : segmenter les objets dénombrables (*things* — voitures, piétons, noyaux cellulaires — chacun étant une instance distincte) et les régions amorphes (*stuff* — route, ciel, fond tissulaire — non dénombrables par nature). Ni l'IoU sémantique (qui ignore les instances individuelles) ni l'AP de détection (qui ignore les régions *stuff*) ne conviennent seuls. La PQ les réconcilie en une seule formule — et son angle mort est précisément ce qu'elle combine.



On apparie d'abord prédictions et vérités terrain : un couple (prédit, réel) constitue un VP si leur IoU dépasse 0,5. Puis :

$$PQ = [\sum_{\{(p,g) \in VP\}} IoU(p,g)] / [|VP| + \frac{1}{2}|FP| + \frac{1}{2}|FN|]$$

La décomposition $SQ \times RQ$

La PQ se factorise élégamment en deux termes interprétables :

$$PQ = [\underbrace{\sum IoU}_{SQ} / |VP|] \times [|VP| / (|VP| + \frac{1}{2}|FP| + \frac{1}{2}|FN|)]$$

$\underbrace{\hspace{100px}}_{RQ}$

La **SQ** (Segmentation Quality) est l'IoU moyen des seuls couples correctement appariés : elle mesure la qualité de délimitation des objets *trouvés*. La **RQ** (Recognition Quality) est exactement le F1-score au niveau instance : elle mesure la capacité à trouver les bons objets, ni plus ni moins.

Cette factorisation sépare proprement deux questions indépendantes : « ai-je trouvé les bons objets ? » (RQ) et « les ai-je bien délimités ? » (SQ). Un modèle de segmentation médicale peut exceller en RQ (repère toutes les lésions) mais faiblir en SQ (ses contours sont flous) — la PQ seule masquerait ce diagnostic ; la décomposition le révèle. C'est la même logique que celle de l'encadré final du chapitre 1 : comprendre un descripteur, c'est connaître ce qu'il agrège et ce qu'il dissout.

Pourquoi l'appariement à $IoU > 0,5$ est unique

Si $IoU(p, g) > 0,5$, alors p et g partagent plus de la moitié de leur union. Il est impossible qu'un autre segment prédit p' atteigne aussi $IoU > 0,5$ avec le même g : il faudrait que p et p' se recouvrent à plus de 50 % l'un l'autre, or les segments d'une partition panoptique sont disjoints par définition. Le seuil 0,5 garantit donc un appariement au plus un-à-un, sans procédure d'optimisation. C'est l'élégance de ce choix précis.



Scène de télédétection avec 3 bâtiments à segmenter. Le modèle en prédit 4 : 2 bien appariés ($\text{IoU} = 0,80$ et $\text{IoU} = 0,90$), 1 prédiction sans correspondant (FP), 1 bâtiment réel non trouvé (FN).

$$\text{VP} = 2, \quad \text{FP} = 2, \quad \text{FN} = 1$$

$$\text{SQ} = (0,80 + 0,90) / 2 = 0,850$$

$$\text{RQ} = 2 / (2 + \frac{1}{2} \times 2 + \frac{1}{2} \times 1) = 2 / 3,5 \approx 0,571$$

$$\text{PQ} = \text{SQ} \times \text{RQ} = 0,850 \times 0,571 \approx 0,486$$

La lecture est immédiate : la segmentation est correcte ($\text{SQ} = 0,85$) mais la reconnaissance est insuffisante ($\text{RQ} = 0,57$, trop de FP et FN). Le diagnostic pointe vers le module de détection, pas vers les masques — information que PQ seule, à 0,49, ne permettrait pas de formuler.



```
# a : liste de IoU pour chaque paire VP (float), port 'a' → list
# b : dict avec clés 'FP' et 'FN' (int), port 'b' → dict
if not isinstance(a, list) or not isinstance(b, dict):
    out_a = {'erreur': 'a=liste IoU des VP, b=dict{"FP":int,"FN":int}'}
else:
    iou_vp = [float(x) for x in a]
    fp = int(b.get('FP', 0))
    fn = int(b.get('FN', 0))
    n_vp = len(iou_vp)
    sq = float(sum(iou_vp) / n_vp) if n_vp > 0 else 0.0
    denom_rq = n_vp + 0.5 * fp + 0.5 * fn
    rq = float(n_vp / denom_rq) if denom_rq > 0 else 0.0
    pq = sq * rq
    out_a = {
        'PQ': round(pq, 4),
        'SQ': round(sq, 4),
        'RQ': round(rq, 4),
        'VP': n_vp, 'FP': fp, 'FN': fn,
    }
```

4.6. Boundary F1 (BF score)

Motivation : l'IoU est aveugle aux frontières

Deux segmentations peuvent obtenir le même IoU avec des frontières de qualités très différentes. Un IoU de 0,90 peut venir d'un large objet aux bords approximatifs, ou d'un objet

légèrement plus petit aux bords parfaits. Pour les applications où la frontière compte — découpe précise pour la chirurgie assistée, mesure dimensionnelle en métrologie industrielle, cartographie de parcelles agricoles par satellite — il faut une métrique de **contour**.

L'image mentale utile est celle d'un calque tracé sur du papier calque posé sur le sol. L'IoU mesure si les deux silhouettes se superposent en surface ; le BF vérifie si les deux tracés de bord se suivent de près, pixel après pixel.



On calcule précision et rappel sur les seuls pixels de frontière, avec une tolérance de distance ϑ (généralement $\vartheta = 1$ à 3 px) : un pixel de contour prédit est compté correct s'il existe un pixel de contour vérité à moins de ϑ px.

$P_c = (\text{pixels frontière prédits à moins de } \vartheta \text{ d'un pixel vérité}) / (\text{total frontière prédite})$

$R_c = (\text{pixels frontière vérité à moins de } \vartheta \text{ d'un pixel prédit}) / (\text{total frontière vérité})$

$BF = 2 \cdot P_c \cdot R_c / (P_c + R_c)$

Lien avec la distance de Hausdorff (chapitre 3)

Le BF et la distance de Hausdorff (rappel §3) mesurent tous deux l'erreur de frontière, mais différemment. La Hausdorff rapporte le *pire* écart ponctuel — elle est sensible aux aberrations locales. Le BF rapporte une *proportion* de frontière correcte dans une tolérance — il est robuste et borné dans $[0, 1]$. Les deux se complètent : BF pour la qualité globale du contour, HD95 (percentile 95 de la Hausdorff) pour garantir l'absence d'excursion grave. Ici encore, aucune métrique unique ne capture tout.



Segmentation d'une lésion dermique sur image de dermoscopie. Frontière vérité : 200 px ; frontière prédite : 210 px ; tolérance $\vartheta = 2$ px. Supposons que 190 pixels prédits trouvent un voisin vérité proche, et que 185 pixels vérité trouvent un voisin prédit proche :

$P_c = 190 / 210 \approx 0,905$

$R_c = 185 / 200 = 0,925$

$BF = 2 \times 0,905 \times 0,925 / (0,905 + 0,925) \approx 0,915$

Un $BF = 0,915$ indique que 90 % du tracé prédit suit fidèlement le contour réel — bien que la frontière prédite soit légèrement plus longue (210 px vs 200 px).



Le BF dépend du couple (ϑ , résolution de l'image). Doubler la résolution sans adapter ϑ rend le score plus sévère : la même erreur physique de 0,5 mm couvre désormais deux fois plus de pixels. Toujours exprimer ϑ en unités physiques (mm, μm) si possible, et le maintenir constant entre tous les modèles comparés. Rapporter la résolution de l'image avec le score.



```
# a : masque prédit (H×W uint8, 0/255)
# b : masque vérité (H×W uint8, 0/255)
# c : tolérance en pixels (scalar int, port 'c')
if not isinstance(a, np.ndarray) or not isinstance(b, np.ndarray):
    out_a = {'erreur': 'deux masques requis (a=prédit, b=vérité)'}
else:
    theta = int(c) if isinstance(c, (int, float)) else 2
    pred = (a > 127).astype(np.uint8) * 255
    truth = (b > 127).astype(np.uint8) * 255

    # extraction des pixels de contour par dilatation-soustraction
    kernel = cv2.getStructuringElement(cv2.MORPH_CROSS, (3, 3))
    bord_pred = cv2.dilate(pred, kernel) - pred
    bord_truth = cv2.dilate(truth, kernel) - truth

    # distances signées (distance transform sur fond blanc)
    dist_pred = cv2.distanceTransform(255 - bord_pred, cv2.DIST_L2, 5)
    dist_truth = cv2.distanceTransform(255 - bord_truth, cv2.DIST_L2, 5)

    bp = (bord_pred > 0)
    bt = (bord_truth > 0)
    pc = float((dist_truth[bp] <= theta).sum()) / float(bp.sum()) if
bp.sum() > 0 else 0.0
    rc = float((dist_pred[bt] <= theta).sum()) / float(bt.sum()) if
bt.sum() > 0 else 0.0
    bf = 2 * pc * rc / (pc + rc) if (pc + rc) > 0 else 0.0

    out_a = {
        'BF': round(bf, 4),
        'P_contour': round(pc, 4),
        'R_contour': round(rc, 4),
        'theta_px': theta,
        'px_bord_pred': int(bp.sum()),
        'px_bord_verite': int(bt.sum()),
    }
```

4.7. Tableau récapitulatif – quelle métrique pour quoi ?

Outil	Ce qu'il mesure	Angle mort	Usage typique
IoU / Jaccard	recouvrement de surface, pénalise FP et FN	ne distingue pas sur- et sous-segmentation ; sévère sur petits objets	évaluation de segmentation, seuil de correspondance VP/FP
Dice / F1-pixel	recouvrement (plus indulgent que IoU)	équivalent monotone d'IoU, instable sur très petits objets	imagerie médicale, fonction de perte d'entraînement
Précision	taux de faux positifs	ignore les faux négatifs	filtrage, tri, spam
Rappel	taux de faux négatifs	ignore les faux positifs	dépistage, surveillance
F1 / F _β	équilibre précision-rappel	cache à quel seuil de confiance il est atteint	évaluation globale de détecteur ; β ajuste selon enjeu
AP / mAP	qualité intégrée sur tous les seuils	ne reflète pas le seuil de déploiement réel ; conventions VOC ≠ COCO	benchmarks de détection multi-classes
Panoptic PQ	Quality reconnaissance (RQ) × segmentation (SQ)	un seul chiffre masque le déséquilibre entre les deux	segmentation panoptique (scènes, pathologie)
Boundary F1	exactitude des contours, bornée [0,1]	dépend de ϑ et de la résolution ; ne mesure pas la surface	découpe précise, métrologie, cartographie

4.8. aucune métrique unique ne capture tout, et c'est inévitable

Le mode d'échec le plus répandu dans les publications de vision par ordinateur n'est pas un mauvais modèle, c'est une **évaluation incomplète**. Ce chapitre en a fourni la preuve section après section : l'IoU cache si l'erreur est en sur- ou sous-segmentation ; le Dice seul ne permet pas de distinguer un modèle à fort rappel d'un modèle à forte précision ;

l'AP ne reflète pas le seuil de déploiement ; la PQ masque le déséquilibre entre détection et délimitation si on ne la décompose pas en $SQ \times RQ$; le BF est muet sur la surface.

Ce n'est pas un déficit corrigeable par une meilleure formule. C'est la conséquence directe du **méta-fil du livre** : tout choix de représentation encode une hypothèse sur ce qui compte, et laisse le reste dans l'ombre. Une métrique de surface (IoU, Dice) déclare que seule la masse de pixels partagés importe, et fait l'hypothèse que les contours n'ont pas de valeur propre. Une métrique de frontière (BF, Hausdorff) déclare l'inverse. Aucune des deux n'est universellement vraie — elles sont adaptées à des usages.

Ce chapitre rejoint ainsi le chapitre 1 (chaque descripteur choisit ce qu'il voit) et le chapitre 3 (choisir une distance, c'est déclarer ce qui compte) : dans les trois cas, le choix d'un instrument de mesure est d'abord un choix épistémologique sur ce qu'on décide de faire compter.

La règle pratique qui en découle :

IoU/Dice seul	→ ajouter précision + rappel pour isoler FP et FN
F1 seul	→ donner la courbe P-R ou l'AP pour le contexte global
AP seul	→ ajouter F1 au seuil de production effectif
métrique de surface	→ compléter par BF ou HD95 pour les contours
PQ seule	→ toujours décomposer en SQ et RQ séparément

Une évaluation qui tient sur un seul chiffre est presque toujours une évaluation qui cache quelque chose. La bonne pratique est de combiner **au moins une métrique de surface** (IoU ou Dice) et **une métrique de frontière** (BF ou HD95), en précisant systématiquement les seuils IoU, la tolérance ϑ , la convention de calcul (VOC ou COCO) et la répartition par taille d'objet. Ce n'est pas de la sur-ingénierie : c'est la condition minimale pour qu'une comparaison de modèles soit honnête.

CHAPITRE

5

Filtrage et convolution

Table des matières

5.1. La convolution 2D	69
5.2. Le noyau gaussien	72
5.3. Différence de gaussiennes (DoG) et Laplacien de gaussienne (LoG)	74
5.4. Le filtre bilatéral	77
5.5. Le filtre de Gabor	79
5.6. Tableau récapitulatif — le filtre comme a priori sur le signal	82
5.7. un filtre est un a priori, et choisir c'est supposer	83

Chaque pixel d'une image n'existe pas isolément : il est entouré de voisins qui partagent son contexte. Le filtrage est l'opération qui exploite ce voisinage — remplacer chaque pixel par une combinaison pondérée de ses proches. Avec le bon choix de pondération, la même mécanique de base lisse le bruit dans une radiographie, accentue les nervures d'une feuille en microscopie, détecte les craquelures dans une pièce industrielle, ou repère les orientations dominantes d'une empreinte digitale.

Le fil conducteur de ce chapitre tient en une phrase : **un filtre est un a priori sur le signal**. Choisir un filtre, c'est déclarer ce que l'on croit savoir du signal avant même de le regarder. Le filtre gaussien suppose que le signal varie doucement — qu'entre deux pixels voisins, la valeur ne saute pas brusquement. Le filtre bilatéral suppose que le signal est lisse *par morceaux*, avec des sauts francs aux contours mais une régularité à l'intérieur des régions. Le filtre de Gabor suppose que le signal contient des structures oscillantes à une certaine fréquence et orientation. Cette hypothèse est inscrite dans les coefficients du noyau, et elle détermine ce que le filtre voit — et ce qu'il rate. Un filtre mal choisi ne produit pas seulement un mauvais résultat : il impose une hypothèse fausse sur la nature des données, et la dégrade.

Ce chapitre construit la convolution depuis ses propriétés algébriques, puis dérive les noyaux essentiels en montrant à chaque fois pourquoi la formule a cette forme et quelle hypothèse elle encode. Il prolonge naturellement le chapitre 3 (distances) — la pondération d'un filtre est une façon de mesurer la proximité pertinente entre pixels — et prépare le chapitre 6 (gradients/contours), où le choix du filtre de dérivation conditionnera la qualité des détecteurs de bords.

5.1. La convolution 2D



$$(I * K)(x, y) = \sum_i \sum_j I(x - i, y - j) \cdot K(i, j)$$

I désigne l'image, K le noyau (*kernel*). La sommation porte sur tous les décalages (i, j) couverts par le support du noyau.

L'idée

L'image mentale la plus utile est celle d'un pochoir glissant sur une feuille. Le noyau K est ce pochoir : une petite grille de nombres. On le pose centré sur chaque pixel de l'image, on multiplie chaque cellule du pochoir par l'intensité du pixel qu'elle recouvre, et on additionne tout. Le résultat est la nouvelle valeur du pixel central. On glisse ensuite le pochoir d'un pas, et on recommence pour le pixel suivant, jusqu'à couvrir toute l'image.

Ce qui fait la puissance de ce mécanisme, c'est que **le même pochoir est appliqué partout de façon identique**. Il n'y a pas de traitement spécial selon la position : la rivière dans une image satellite reçoit exactement la même opération que la forêt voisine. Cette uniformité, appelée invariance par translation, est à la fois la force et la limite de la convolution. Sa force : un filtre calibré une fois s'applique à une image entière sans recalibrage. Sa limite : si le signal a une structure différente selon la région (par exemple un fond uniforme vs un bord texturé), un seul noyau ne peut pas s'y adapter — ce qui conduit aux filtres adaptatifs comme le bilatéral (§5.4).

Convolution vs corrélation : le retournement

La vraie convolution **retourne** le noyau avant de le multiplier — c'est la signification des indices $(x-i, y-j)$ dans la formule. La corrélation croisée, elle, ne retourne pas :

corrélation : $(I * K)(x, y) = \sum_i \sum_j I(x+i, y+j) \cdot K(i, j)$

Pour les noyaux symétriques comme le gaussien ou le filtre moyennneur, retourner ne change rien : les deux opérations sont identiques. Mais pour les noyaux antisymétriques — notamment les filtres de dérivation Sobel utilisés au chapitre 6 — la distinction inverse le signe. La plupart des bibliothèques (dont `cv2.filter2D`) calculent en réalité une **corrélation**, par convention historique. Garder cela en tête évite de s'interroger pendant une heure sur un gradient qui sort « à l'envers ».

Les propriétés qui font tout fonctionner

La convolution est **linéaire** (doubler l'image double la sortie), **invariante par translation** (un filtre appliqué à une image décalée donne la sortie décalée), **associative** et **commutative**. Ces propriétés ne sont pas décoratives.

La linéarité + invariance par translation caractérisent entièrement les filtres dits LTI (*Linear Time-Invariant*) : tout traitement respectant ces deux propriétés est *nécessairement* une convolution. C'est un théorème de représentation, pas une convention : la convolution est le seul outil capable de réaliser un traitement à la fois uniforme et linéaire sur tout le domaine de l'image.

L'associativité ouvre une optimisation pratique importante : appliquer un flou gaussien puis un filtre Sobel revient à appliquer une seule fois le noyau $(G * \text{Sobel})$, pré-calculé une fois. On économise un passage complet sur l'image — qui peut être considérable en haute résolution.

La séparabilité : diviser pour régner

Un noyau 2D est dit **séparable** s'il peut s'écrire comme le produit externe de deux vecteurs 1D :

$$K(x, y) = k_v(y) \cdot k_h(x)$$

Dans ce cas, la convolution 2D se décompose en deux convolutions 1D successives : d'abord les lignes, puis les colonnes. Le coût passe de $O(n^2)$ à $O(2n)$ opérations par pixel, où n est la taille du noyau dans une direction. Pour un noyau 21×21 , c'est 441 multiplications par pixel dans le cas général contre 42 dans le cas séparable — un facteur 10 sur un traitement souvent exécuté à 30 fps.

Dérivation de la séparabilité du gaussien

Le noyau gaussien 2D se factorise exactement :

$$\begin{aligned} G(x, y) &= (1/2\pi\sigma^2) \cdot \exp(-(x^2+y^2)/2\sigma^2) \\ &= [(1/\sqrt{2\pi\sigma^2}) \cdot \exp(-x^2/2\sigma^2)] \cdot [(1/\sqrt{2\pi\sigma^2}) \cdot \exp(-y^2/2\sigma^2)] \\ &= G_{1D}(x) \cdot G_{1D}(y) \end{aligned}$$

La factorisation tient grâce à la propriété $\exp(a+b) = \exp(a) \cdot \exp(b)$, qui permet de séparer les contributions x^2 et y^2 . C'est la raison pour laquelle un flou gaussien sur une image 4K avec un grand noyau reste rapide : on filtre les lignes, puis les colonnes. ■

Gestion des bords

Au bord de l'image, le pochoir déborde vers l'extérieur, là où il n'y a pas de pixels. Quatre conventions usuelles existent. Le *zero-padding* étend l'image par des zéros, créant des bandes sombres artificielles aux bords — acceptable en intérieur d'image mais indésirable si les bords portent de l'information. Le mode *reflect* (miroir) répète les pixels de bord en les reflétant, ce qui est le comportement le plus neutre pour le lissage et le défaut le plus sûr dans la plupart des cas. Le mode *replicate* répète le pixel de bord en ligne droite. Le mode *wrap* traite l'image comme une surface torique (périodique) — cohérent avec la FFT mais rarement pertinent pour des images naturelles. Le choix affecte visiblement les quelques pixels de bordure ; pour une analyse sérieuse des bords d'une image, il faut donc documenter la convention choisie.



`cv2.filter2D` calcule une corrélation, pas une convolution. Pour les noyaux directionnels (filtres de dérivée, Gabor orienté), retourner explicitement le noyau avec `np.rot90(K, 2)` avant de l'appliquer garantit le comportement convolutionnel strict. Autre piège : travailler en `uint8` sans convertir préalablement en `float32` ou `CV_64F` provoque des saturations et troncatures silencieuses, particulièrement graves avec les noyaux dont les coefficients sont négatifs (LoG, DoG).



```
# a : image (np.ndarray HxWx3 uint8 BGR, ou HxW grayscale)
if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune image en entree'}
else:
    img = a.copy().astype(np.float32)

    # Exemple : noyau moyeneur 5x5 (a priori : localement constant)
    K = np.ones((5, 5), dtype=np.float32) / 25.0

    # cv2.filter2D calcule une corrélation ; pour un noyau symétrique
    # c'est identique
    result = cv2.filter2D(img, -1, K, borderType=cv2.BORDER_REFLECT)
    out_a = np.clip(result, 0, 255).astype(np.uint8)
```

5.2. Le noyau gaussien



$$G(x, y) = (1 / 2\pi\sigma^2) \cdot \exp(-(x^2 + y^2) / 2\sigma^2)$$

σ est l'écart-type : il contrôle la largeur de la cloche gaussienne, et donc l'étendue du voisinage pris en compte.

L'idée

Le filtre gaussien traduit l'hypothèse que le signal est **localement lisse** : la valeur d'un pixel devrait ressembler à celle de ses voisins, et cette ressemblance décroît progressivement avec la distance. La pondération en cloche garantit que les voisins proches influencent beaucoup le résultat, les voisins lointains très peu, et les pixels au-delà de $\pm 3\sigma$ presque pas du tout.

Une analogie utile est celle d'un spot lumineux flou centré sur chaque pixel : les régions éclairées les plus intensément (le centre du spot) pèsent le plus lourd dans la moyenne. Plus σ est grand, plus le spot est large et plus l'image résultante est floue — l'hypothèse de lisseur embrasse un voisinage plus étendu. Un σ petit correspond à un a priori de lisseur très local ; un σ grand, à un a priori de signal très uniforme sur de grandes zones.

Pourquoi la gaussienne et pas une autre fonction de lissage ?

Ce n'est pas un choix esthétique. La gaussienne est l'**unique** noyau de lissage vérifiant simultanément plusieurs propriétés optimales :

1. **Pas de structure créée** (*causalité d'échelle*) : en augmentant σ , le lissage gaussien ne crée jamais de nouveau maximum local dans l'image lissée. Un filtre moyeneur (boîte

carrée) viole cette propriété et peut créer des oscillations parasites à grande échelle. C'est le théorème de Koenderink qui fonde la théorie de l'espace-échelle : la pyramide gaussienne est la seule représentation multi-échelle causalement correcte.

2. **Séparabilité** (§5.1) — efficacité de calcul.
3. **Auto-similarité** : convoluer deux noyaux gaussiens d'écarts σ_1 et σ_2 donne exactement un gaussien d'écart $\sqrt{(\sigma_1^2 + \sigma_2^2)}$. Lisser progressivement revient à lisser une seule fois avec un σ plus grand.
4. **Compromis espace-fréquence optimal** : la gaussienne atteint la borne du principe d'incertitude (produit $\Delta x \cdot \Delta \omega$ minimal). Elle est aussi localisée que possible à la fois en espace et en fréquence, ce qui signifie qu'elle n'introduit pas d'artefacts oscillants et ne diffuse pas l'information sur une large plage de fréquences.

Dérivation de l'auto-similarité

Dans le domaine de Fourier, la transformée d'une gaussienne d'écart σ est une gaussienne d'écart $1/\sigma$, et la convolution dans l'espace correspond à une multiplication dans Fourier :

$$\begin{aligned}
 F(G_{\sigma 1} * G_{\sigma 2}) &= F(G_{\sigma 1}) \cdot F(G_{\sigma 2}) \\
 &= \exp(-\sigma_1^2 \omega^2 / 2) \cdot \exp(-\sigma_2^2 \omega^2 / 2) \\
 &= \exp(-(\sigma_1^2 + \sigma_2^2) \omega^2 / 2) \\
 &= F(G_{\sqrt{(\sigma_1^2 + \sigma_2^2)}})
 \end{aligned}$$

L'addition des variances découle directement de la multiplication des exponentielles via $\exp(a) \cdot \exp(b) = \exp(a+b)$. ■

Cette propriété explique pourquoi les pyramides gaussiennes sont stables : chaque niveau ne fait qu'accumuler du lissage de façon prévisible et monotone.

Choix de σ et de la taille du noyau

Le support effectif d'une gaussienne est $\pm 3\sigma$ (99,7 % de la masse). Règle pratique : taille de noyau $\approx 6\sigma + 1$, arrondie à l'impair suivant (les noyaux pairs sont asymétriques et ne peuvent pas avoir de centre). Tronquer trop court — par exemple $\pm 1\sigma$ — déforme le filtre, brise ses propriétés analytiques et crée des artefacts de bord visibles. À l'inverse, un noyau inutilement grand ($\pm 5\sigma$ ou plus) gaspille du calcul sans gain qualitatif.



Noyau gaussien 3×3 avec $\sigma = 0,85$ (approximation entière courante) :

$$\frac{1}{16} \cdot \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

La somme des coefficients vaut 16, d'où la normalisation par $1/16$. Un filtre de lissage doit préserver l'intensité moyenne ($\sum K = 1$) : si le noyau n'est pas normalisé, chaque passage de filtre assombrit ou éclaircit l'image entière de façon cumulative. Le pixel central pèse $4/16 = 25\%$, ses quatre voisins directs $2/16 = 12,5\%$ chacun, et les quatre diagonales $1/16 = 6,25\%$ chacune. La décroissance est gaussienne discrétisée.



`cv2.GaussianBlur` exige une taille de noyau impaire. Passer une taille paire génère une erreur silencieuse ou un arrondi inattendu selon les versions d'OpenCV. Quand σ est connu et la taille voulue calculable, mieux vaut passer `ksize=(0,0)` et laisser OpenCV dériver la taille à partir de σ , plutôt que de calculer la taille soi-même avec risque d'erreur de parité.



```
# a : image BGR ou grayscale
if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune image en entree'}
else:
    # sigma contrôle l'étendue du lissage ; ksize=(0,0) laisse OpenCV
    # calculer la taille
    sigma = 2.0 # modifier pour explorer l'a priori de lisseur
    blurred = cv2.GaussianBlur(a, (0, 0), sigmaX=sigma, sigmaY=sigma)
    out_a = blurred
```

5.3. Différence de gaussiennes (DoG) et Laplacien de gaussienne (LoG)



$$\text{LoG}(x,y) = \nabla^2 G = ((x^2 + y^2 - 2\sigma^2) / \sigma^4) \cdot G(x,y)$$

$$\text{DoG} = G_{\sigma 1} - G_{\sigma 2} \quad (\text{avec } \sigma 1 < \sigma 2, \text{ typiquement } \sigma 2 / \sigma 1 \approx 1.6)$$

L'idée

Ces deux filtres détectent non plus ce qui est lisse, mais ce qui *change* — les contours, les blobs, les taches. Leur logique procède en deux temps que l'on peut comprendre séparément.

Premier temps : on lisse l'image avec un gaussien. Cela supprime le bruit haute fréquence qui rendrait toute dérivation instable. C'est l'a priori fondamental : le signal utile est plus lisse que le bruit.

Second temps : on calcule la dérivée seconde du résultat lissé. La dérivée seconde d'un signal mesure la courbure de ce signal : là où l'intensité passe par un maximum local (une crête lumineuse sur fond sombre), la dérivée seconde est négative et atteint un minimum. Là où elle passe par un minimum local, elle est positive. Aux transitions brusques — les bords — la dérivée seconde s'annule et change de signe : c'est le principe du **passage par zéro** pour détecter les contours.

Par l'associativité de la convolution, on peut **pré-calculer** $\nabla^2 G$ une fois pour toutes au lieu de lisser puis dériver séquentiellement :

$$\nabla^2 (G * I) = (\nabla^2 G) * I$$

La forme du noyau LoG est un « chapeau mexicain » : positif au centre (réponse forte sur une tache lumineuse sur fond sombre), négatif en couronne autour (réponse inverse sur l'anneau de contraste opposé). Ses passages par zéro dans la réponse à l'image marquent les contours.

Dérivation du LoG

Le laplacien $\nabla^2 = \partial^2/\partial x^2 + \partial^2/\partial y^2$ est l'opérateur de dérivée seconde isotrope. En appliquant $\partial^2/\partial x^2$ à $G(x,y)$:

$$\begin{aligned}\partial G/\partial x &= (-x/\sigma^2) \cdot G(x,y) \\ \partial^2 G/\partial x^2 &= (x^2/\sigma^4 - 1/\sigma^2) \cdot G(x,y)\end{aligned}$$

Par symétrie, $\partial^2 G/\partial y^2 = (y^2/\sigma^4 - 1/\sigma^2) \cdot G(x,y)$. En sommant :

$$\nabla^2 G = ((x^2 + y^2 - 2\sigma^2) / \sigma^4) \cdot G(x,y)$$

■

La DoG : approximation économique

Le DoG approxime le LoG quand le rapport $\sigma_2/\sigma_1 \approx 1,6$ (rapport établi par Marr et Hildreth). L'intérêt est purement computationnel : deux flous gaussiens séparables suivis d'une soustraction, au lieu d'un noyau LoG non séparable. Ce rapport de 1,6 n'est pas magique — il minimise la différence entre la réponse fréquentielle du DoG et celle du LoG normalisé. En pratique, la DoG se comporte comme un filtre passe-bande : elle supprime les

basses fréquences (lissage par $G_{\sigma 2}$) et les très hautes fréquences (lissage par $G_{\sigma 1}$), et ne laisse passer qu'une bande intermédiaire correspondant aux structures de taille $\approx \sigma$.

C'est la brique de base du détecteur de points clés SIFT (Scale-Invariant Feature Transform), où une pyramide de DoG calculée à plusieurs σ localise des blobs invariants à l'échelle — ce qui permet de retrouver le même point d'intérêt dans une image zoomée ou tournée. Ce principe a révolutionné la mise en correspondance d'images dès les années 2000 et reste pertinent malgré l'arrivée des descripteurs appris, car il reste rapide et déterministe.

Détection de blobs et sélection d'échelle

Le LoG (et donc la DoG) atteint sa réponse extrême quand la taille du blob correspond à σ . En évaluant la réponse pour plusieurs σ et en cherchant les extrema dans l'espace (x, y, σ) , on détecte des blobs **et leur taille** simultanément — c'est le fondement de la détection multi-échelle. En microscopie à fluorescence, cela permet de mesurer automatiquement le diamètre de centaines de vésicules en une seule passe, quelle que soit leur taille.



Réponse du LoG au centre d'un blob clair sur fond sombre, en $x = y = 0$:

$$\text{LoG}(0,0) = ((0 + 0 - 2\sigma^2) / \sigma^4) \cdot G(0,0) = -2/\sigma^2 \cdot G(0,0)$$

Le terme est **fortement négatif** au centre d'un blob clair : la courbure de l'intensité y est maximale concave. En cherchant les minima de la réponse LoG dans l'image, on localise précisément les centres de blobs lumineux. Pour des blobs sombres sur fond clair, les maxima jouent le même rôle.



Travailler en `uint8` lors du calcul du LoG provoque une troncature silencieuse : les valeurs négatives sont ramenées à 0 avant d'être stockées, effaçant la moitié de l'information. Il faut travailler en `CV_64F` (flottant 64 bits) ou `CV_32F` pour capturer les deux polarités de la réponse. Les passages par zéro ne peuvent être détectés qu'en signant correctement la réponse.



```

# a : image (BGR ou grayscale)
if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune image en entree'}
else:
    gray = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY) if a.ndim == 3 else a
    sigma = 2.0

    # LoG = laplacien du gaussien (travailler en float64 pour les valeurs
    négatives)
    blurred = cv2.GaussianBlur(gray.astype(np.float64), (0, 0), sigma)
    log_resp = cv2.Laplacian(blurred, cv2.CV_64F)

    # DoG = approximation du LoG (rapport  $\sigma_2/\sigma_1 \approx 1.6$ )
    s1, s2 = sigma, sigma * 1.6
    dog_resp = (cv2.GaussianBlur(gray.astype(np.float64), (0,0), s1)
                - cv2.GaussianBlur(gray.astype(np.float64), (0,0), s2))

    # Visualisation : normaliser pour affichage
    def norm_display(x):
        x = x - x.min()
        mx = x.max()
        return (x / mx * 255).astype(np.uint8) if mx > 0 else
x.astype(np.uint8)

    out_a = norm_display(np.abs(log_resp)) # magnitude du LoG

```

5.4. Le filtre bilatéral



$$BF[I](p) = (1/W_p) \cdot \sum_{\{q \in \Omega\}} G_{\sigma_s}(\|p - q\|) \cdot G_{\sigma_r}(|I(p) - I(q)|) \cdot I(q)$$

W_p est le facteur de normalisation (somme des poids) ; G_{σ_s} pèse la proximité spatiale ; G_{σ_r} pèse la proximité en intensité.

L'idée

Le filtre gaussien lisse partout indistinctement — y compris à travers les contours. C'est cohérent avec son a priori de signal globalement lisse, mais cet a priori est faux aux frontières entre deux régions d'intensités différentes. Une radiographie thoracique, une image satellite, un portrait photographique : tous ont des zones intérieures relativement uniformes

séparées par des contours nets. Un filtre qui lisse à travers ces contours détruit l'information structurelle la plus précieuse.

L'a priori du filtre bilatéral est plus subtil : le signal est **lisse par morceaux** — lisse à l'intérieur de chaque région, avec des transitions brusques aux bords. Il ajoute au poids spatial du gaussien un second poids, en intensité : deux pixels ne se moyennent que s'ils sont à la fois **proches spatialement** (G_{σ_s}) ET **proches en valeur** (G_{σ_r}). Un pixel de l'autre côté d'un contour fort présente une grande différence d'intensité avec le pixel central : son poids G_{σ_r} est quasi nul, donc il ne contribue presque pas au résultat. Le contour est préservé.

Dérivation de la non-linéarité

Contrairement à tous les filtres précédents, le filtre bilatéral n'est **pas une convolution** : ses poids dépendent du contenu de l'image (le terme $G_{\sigma_r}(|I(p) - I(q)|)$), pas seulement de la position relative. C'est un filtre **non linéaire et adaptatif**. Conséquence directe : il ne se décompose pas par Fourier, n'est pas séparable, et son coût est quadratique dans la taille du voisinage. Les implémentations rapides (grille bilatérale, approximations par décomposition spectrale) sont un domaine de recherche actif. Dans OpenCV, `bilateralFilter` reste raisonnablement rapide pour des images de taille modérée mais sera lent sur du 4K en temps réel.

Les deux paramètres

σ_s (spatial) : taille du voisinage pris en compte. Un grand σ_s signifie que des pixels très éloignés peuvent contribuer — sous réserve qu'ils aient une intensité proche. σ_r (range, intensité) : tolérance sur les différences de valeur. Un petit σ_r signifie que seuls des pixels de teinte presque identique se moyennent — contours bien préservés, lissage faible. Un grand σ_r rend le filtre indifférent aux différences d'intensité : quand $\sigma_r \rightarrow \infty$, le terme G_{σ_r} devient approximativement constant, et le filtre bilatéral dégénère en filtre gaussien classique. Ces deux paramètres contrôlent deux dimensions indépendantes de l'a priori : l'étendue spatiale de la régularité supposée, et la tolérance aux transitions d'intensité.



Pixel p sur un contour, valeur $I(p) = 200$. À gauche, voisins q_1 à $I(q_1) \approx 195$ (même côté) ; à droite, voisins q_2 à $I(q_2) \approx 50$ (autre côté du contour). Avec $\sigma_r = 30$:

$\text{poids } G_{30}(|200 - 195|) = \exp(-25/1800) \approx 0.986$ (contribue fortement)
 $\text{poids } G_{30}(|200 - 50|) = \exp(-22500/1800) \approx 4 \times 10^{-6}$ (ignoré)

Le pixel p se moyenne exclusivement avec ses voisins du même côté du contour.
 Le bord reste net.

Applications

Débruitage préservant les contours en imagerie médicale (scanner, IRM), lissage de peau en retouche photo, abstraction d'image (effet « cartoon » obtenu par itérations successives), filtrage de cartes de profondeur bruitées en robotique et reconstruction 3D, étape préalable dans les pipelines HDR. La variante **guided filter** (He et al., 2013) est plus rapide, linéarisable, et élimine les artefacts de gradient inversé présents dans le bilatéral sur certains contours.



Le filtre bilatéral est sensible à la plage de valeurs des pixels. S'il est appliqué sur une image normalisée en $[0, 1]$ plutôt qu'en $[0, 255]$, le paramètre σ doit être adapté en conséquence (diviser par 255). L'oublier produit un filtre soit totalement transparent (σ trop grand par rapport à la plage), soit un filtre passant nulle part (σ trop petit). Toujours vérifier l'unité de σ par rapport à la plage effective des données.



```
# a : image BGR
if not isinstance(a, np.ndarray) or a.ndim < 2:
    out_a = {'erreur': 'image BGR requise'}
else:
    # d=9 : voisinage 9px ; sigmaColor=sigma_r ; sigmaSpace=sigma_s
    sigma_r = 30.0 # tolérance en intensité (même unité que les pixels
0-255)
    sigma_s = 7.0 # taille spatiale du voisinage
    out_a = cv2.bilateralFilter(a, d=9,
                                sigmaColor=sigma_r,
                                sigmaSpace=sigma_s)
```

5.5. Le filtre de Gabor



$$g(x,y) = \exp(-(x'^2 + y'^2) / 2\sigma^2) \cdot \cos(2\pi x' / \lambda + \psi)$$

avec les coordonnées tournées d'angle ϑ :

$$x' = x \cdot \cos\theta + y \cdot \sin\theta$$

$$y' = -x \cdot \sin\theta + y \cdot \cos\theta$$

L'idée

Le filtre de Gabor encode un a priori entièrement différent des précédents : il suppose que le signal contient une **structure oscillante** d'une fréquence donnée, dans une direction donnée. Une strie sur un tissu industriel, une nervure de feuille, un sillon dans une empreinte digitale — toutes ces structures sont des oscillations d'intensité localisées spatialement et orientées.

Le filtre de Gabor est le produit de deux facteurs que l'on peut visualiser séparément :

- Une **enveloppe gaussienne** (le facteur $\exp$) : elle sélectionne une fenêtre spatiale, centrée en $(0,0)$, de taille σ . Ce « fenêtrage » localise la détection dans l'espace — on ne cherche la strie qu'autour du pixel courant, pas dans toute l'image.
- Une **sinusoïde** (le facteur $\cos$) : elle oscille à la fréquence $1/\lambda$ le long de la direction x' (qui est la direction ϑ dans l'image). Le filtre répondra fortement aux endroits où l'image oscille à cette même fréquence dans cette même direction.

L'image mentale utile est celle d'un peigne à dents régulières, orienté à l'angle ϑ , et dont l'empreinte s'estompe progressivement vers les bords de la fenêtre. Passer ce peigne sur l'image mesure à quel point l'image « vibre » à la fréquence du peigne, dans la direction du peigne.

L'optimalité espace-fréquence

Le filtre de Gabor possède la même propriété d'optimalité que le gaussien pour le lissage : il minimise le produit de l'étalement spatial et de l'étalement fréquentiel ($\Delta x \cdot \Delta \omega$ minimal), conformément au principe d'incertitude de Heisenberg appliqué au traitement du signal. Cela signifie que, parmi tous les filtres possibles répondant à une fréquence donnée, le Gabor est celui qui localise le mieux la réponse à la fois en position (où se trouve la strie ?) et en fréquence (quelle est exactement sa période ?).

Cette optimalité explique également sa pertinence biologique remarquable : les champs récepteurs des neurones du cortex visuel primaire (V1) chez le mammifère sont modélisés avec précision par des filtres de Gabor — découverte de Hubel et Wiesel prolongée formellement par Daugman. Le filtre de Gabor n'est pas un outil inventé de façon arbitraire : c'est celui que le système visuel des vertébrés a sélectionné par évolution pour analyser les textures.

Le banc de filtres de Gabor

Un seul filtre de Gabor ne capture qu'une fréquence et une orientation. En pratique, on construit un **banc** : plusieurs valeurs de λ (différentes échelles de texture) $\times$ plusieurs valeurs de ϑ (orientations, typiquement 0° , 45° , 90° , 135°). La réponse de l'image à tout le banc forme un vecteur de descripteurs de texture riche, longtemps standard pour la classification de surfaces. L'algorithme de Daugman, encore utilisé dans les passeports biométriques

pour la reconnaissance d'iris, repose sur une banque de Gabor encodée en vecteur binaire (IrisCode). En télédétection, les bancs de Gabor permettent de discriminer les types de couverts végétaux ou les textures de sol à partir d'images satellite.

Les paramètres

λ est la longueur d'onde de la sinusoïde : elle détermine la taille de la texture détectée. ϑ est l'orientation de la structure recherchée. σ contrôle la taille de la fenêtre gaussienne (et donc combien de périodes de la sinusoïde elle couvre). γ est le rapport d'aspect de l'enveloppe ($\gamma = 1$ donne une enveloppe ronde, $\gamma < 1$ l'étire le long de l'orientation). ψ est la phase : $\psi = 0$ rend le filtre symétrique, sensible aux crêtes centrées ; $\psi = \pi/2$ le rend antisymétrique, sensible aux flancs — les bords.



Pour détecter des stries verticales espacées de 8 pixels dans une image : $\lambda = 8$ (longueur d'onde de 8 px), $\vartheta = 0^\circ$ (l'onde varie le long de x — horizontal — détectant donc des stries verticales), $\psi = 0$ (symétrique, réponse maximale sur les crêtes). Une zone de l'image avec exactement cette périodicité verticale produit une réponse maximale ; une zone lisse ou striée horizontalement produit une réponse quasi nulle. En faisant varier ϑ de 0° à 135° par pas de 45° , on obtient une carte d'orientation dominante de la texture en chaque pixel.



`cv2.getGaborKernel` retourne un noyau flottant non normalisé : sa somme n'est pas nécessairement 1, et son énergie dépend des paramètres. Lors d'un banc de Gabor comparatif, il faut normaliser chaque noyau (diviser par sa norme L2) pour que les réponses soient comparables entre les différentes orientations et fréquences. Sans normalisation, les orientations associées à des noyaux plus énergétiques domineront artificiellement la comparaison.



```
# a : image BGR ou grayscale
if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune image en entree'}
else:
    gray = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY) if a.ndim == 3 else
a.copy()
    gray_f = gray.astype(np.float32)

    # Banc de Gabor : 4 orientations, une seule échelle
    lam = 8.0      # longueur d'onde (taille de la texture cherchée)
    sigma = 4.0    # taille de l'enveloppe
    gamma = 0.5    # rapport d'aspect (allongé le long de l'orientation)
    psi = 0.0      # phase (0 = filtre symétrique, crêtes)

    responses = []
    angles_deg = [0, 45, 90, 135]
    for theta_deg in angles_deg:
        theta = np.radians(theta_deg)
        kernel = cv2.getGaborKernel((21, 21), sigma, theta, lam, gamma,
psi, ktype=cv2.CV_32F)
        kernel /= (kernel * kernel).sum() ** 0.5 # normalisation L2
        resp = cv2.filter2D(gray_f, cv2.CV_32F, kernel)
        responses.append(np.abs(resp))

    # Carte de la réponse maximale sur toutes les orientations
    max_response = np.max(np.stack(responses, axis=0), axis=0)
    out_a = np.clip(max_response / max_response.max() * 255, 0,
255).astype(np.uint8)
```

5.6. Tableau récapitulatif – le filtre comme a priori sur le signal

Filtre	Linéaire ?	Séparable ?	A priori sur le signal	Usage type
Gaussien	oui	oui	localement lisse, variation douce	débruitage, pyramide d'échelle
Moyenneur (boîte)	oui	oui	localement constant	lissage rapide, grossier

Filtre	Linéaire ?	Séparable ?	A priori sur le signal	Usage type
LoG	oui	non	contours = passages par zéro de la courbure	détection de contours, blobs
DoG	oui	oui (2 gaussiens séparables)	structures à une bande d'échelle	blobs, SIFT, multi-échelle
Bilatéral	non	non	lisse par morceaux (sauts aux contours)	débruitage préservant les bords
Gabor	oui	non	structures oscillantes orientées	texture, biométrie, cortex V1

5.7. un filtre est un a priori, et choisir c'est supposer

Chaque filtre de ce chapitre encode une déclaration sur le signal que l'on traite. Cette déclaration est inscrite dans les coefficients du noyau, et elle s'applique aveuglément à chaque pixel de l'image. Quand l'a priori est vrai — quand le signal est effectivement lisse (gaussien), effectivement à sauts nets (bilatéral), effectivement strié à telle fréquence (Gabor) — le filtre excelle. Quand l'a priori est faux, il dégrade le signal précisément là où on voudrait l'améliorer.

Cette logique relie le chapitre 5 aux deux fils conducteurs de son voisinage dans le livre. Du côté du chapitre 3 (distances), choisir un filtre revient à définir une mesure de « voisinage pertinent » : le filtre gaussien mesure la proximité spatiale, le bilatéral mesure une proximité conjointe spatial-intensité, le Gabor mesure la ressemblance oscillatoire. Du côté du chapitre 6 (gradients), tout détecteur de contour est lui-même un filtre de dérivation — et son comportement face au bruit dépend entièrement de l'a priori de lisseur qu'il porte en amont.

Plus profondément, ce chapitre rejoint le méta-fil du livre. Chaque outil présenté ici — descripteur, distance, filtre — **encode une hypothèse sur ce qui compte**. Le filtre gaussien dit : « les variations brusques sont du bruit ». Le filtre bilatéral dit : « les variations brusques aux contours sont de l'information, le reste est du bruit ». Le Gabor dit : « l'information est une oscillation à cette fréquence et cette orientation ». Aucun de ces a priori n'est universellement vrai. Mais reconnaître lequel on impose est la première étape pour choisir judicieusement, et pour comprendre pourquoi un résultat est décevant quand les données ne respectent pas l'hypothèse.

Le chapitre 10 (transformées) prolongera cette idée dans le domaine fréquentiel : **convoluer dans l'espace équivaut à multiplier dans Fourier**. Un noyau de lissage est un filtre passe-bas, un noyau de dérivation un filtre passe-haut, une DoG un filtre passe-bande, un Gabor un filtre passe-bande orienté. Penser un filtre dans le domaine fréquentiel — quelles fréquences laisse-t-il passer, lesquelles atténue-t-il ? — est souvent plus éclairant que raisonner sur ses seuls coefficients spatiaux, et permet de relier intuitivement la forme du noyau à son effet sur le contenu de l'image.

CHAPITRE

Gradients et contours

6

Table des matières

6.1. Le gradient d'image	86
6.2. Les opérateurs de Sobel et Scharr	88
6.3. Le détecteur de Canny	90
6.4. Le tenseur de structure	93
6.5. Détecteurs de coins : Harris et Shi-Tomasi	94
6.6. Tableau récapitulatif — du gradient à la structure	97
6.7. choisir une échelle, c'est choisir ce qu'on regarde	98

Un contour dans une image, c'est l'endroit où l'intensité change brusquement — la frontière entre une cellule et le fond de la boîte de Petri, le bord d'une pièce usinée sous l'objectif d'une caméra de contrôle, la crête d'une crête de relief sur une image satellite. Détecter ces frontières, c'est mesurer la **dérivée** de l'image : là où la valeur des pixels saute, la dérivée est grande ; là où l'image est uniforme, elle est nulle.

Mais dériver a un coût immédiat : toute dérivée amplifie le bruit. Un pixel légèrement trop clair à cause d'un photon parasite génère une variation locale, et la dérivée la repère avec la même ardeur qu'un vrai bord. C'est le **fil conducteur** de ce chapitre : **dériver amplifie le bruit ; tout détecteur de contours dose lissage et dérivation**. Chaque outil présenté ici — du simple gradient au pipeline de Canny, du tenseur de structure aux détecteurs de coins — est une réponse particulière à cette tension entre voir les vraies variations et ignorer les fausses.

Ce chapitre s'appuie sur le chapitre 5 (Filtrage) : la dérivation est un filtre passe-haut, le lissage un filtre passe-bas. Détecter un contour proprement, c'est appliquer un **passe-bande** qui laisse passer les fréquences spatiales des vraies structures tout en bloquant celles du bruit. La gaussienne, déjà rencontrée au chapitre 5 comme outil de lissage, jouera ici un rôle central.

6.1. Le gradient d'image



$$\nabla I = (\partial I / \partial x, \partial I / \partial y) = (I_x, I_y)$$

$$\text{magnitude : } \|\nabla I\| = \sqrt{I_x^2 + I_y^2}$$

$$\text{orientation : } \theta = \arctan2(I_y, I_x)$$

L'idée

Le gradient d'une image est un vecteur associé à chaque pixel. Ce vecteur a deux propriétés : sa **direction** indique vers où l'intensité monte le plus vite, et sa **longueur** (la magnitude) indique à quelle vitesse elle monte. Sur un bord bien net entre une zone sombre et une zone claire, ce vecteur pointe perpendiculairement au bord — il traverse la frontière depuis le sombre vers le clair — et sa magnitude est élevée. Dans une région uniforme de l'image, rien ne varie et le vecteur est de longueur nulle.

L'image mentale utile : si l'image était une carte topographique où l'intensité joue le rôle de l'altitude, le gradient est la flèche qui indique la plus forte pente, comme une boussole

de randonneur pointant vers le sommet le plus proche. Les contours sont les falaises de cette carte.

Deux quantités en découlent, à ne jamais confondre :

- la **magnitude** $\|\nabla I\|$ répond à « y a-t-il un contour ici ? »
- l'**orientation** ϑ répond à « dans quelle direction est-il orienté ? »

Dérivation discrète

Sur une grille de pixels, la dérivée continue n'existe pas — on dispose de valeurs entières espacées d'un pixel. On l'approche par **différences finies**. La plus précise pour nos usages est la différence centrée :

$$I_x(x, y) \approx [I(x+1, y) - I(x-1, y)] / 2 \quad (\text{noyau } [-1, 0, 1]/2)$$

$$I_y(x, y) \approx [I(x, y+1) - I(x, y-1)] / 2$$

Pourquoi la version centrée plutôt qu'avant ($I(x+1) - I(x)$) ou arrière ($I(x) - I(x-1)$) ? Un développement de Taylor montre que la différence centrée annule le premier terme d'erreur : son erreur est en $O(h^2)$ contre $O(h)$ pour les versions décalées. Elle est aussi **antisymétrique**, ce qui signifie qu'elle ne déphase pas la réponse : le contour détecté est localisé exactement au bon pixel, sans décalage d'un demi-pixel.



Ligne d'intensités : [10, 10, 10, 80, 90, 90, 90]. En position 3 (la transition) :

$$I_x(3) \approx (80 - 10) / 2 = 35 \quad (\text{gradient fort, la frontière est ici})$$

$$I_x(1) \approx (10 - 10) / 2 = 0 \quad (\text{région uniforme, gradient nul})$$

$$I_x(5) \approx (90 - 90) / 2 = 0 \quad (\text{région uniforme, gradient nul})$$



L'orientation DOIT utiliser $\arctan2(I_y, I_x)$ et non $\arctan(I_y/I_x)$. Le simple $\arctan$ perd le quadrant : il confond ϑ et $\vartheta+180^\circ$ (un bord montant vers la droite est identique à un bord descendant vers la gauche). De plus, $\arctan$ diverge numériquement quand $I_x = 0$ — $\arctan2$ gère ce cas proprement en retournant $\pm\pi/2$. C'est l'erreur n°1 dans les implémentations maison de détecteurs de contours.

Convention OpenCV : les axes suivent l'image, y vers le bas. L'orientation retournée par $\arctan2(g_y, g_x)$ est dans $[-\pi, \pi]$, avec 0 = direction droite, $\pi/2$ = direction bas.



```
# a : image (np.ndarray HxWx3 BGR ou HxW niveaux de gris)
if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune image en entree'}
else:
    gray = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY) if a.ndim == 3 else a
    gx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
    gy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
    mag = cv2.magnitude(gx, gy)
    ori = np.arctan2(gy, gx) # radians, plage [-π, π]

    out_a = {
        'gradient_max': float(mag.max()),
        'gradient_moyen': float(mag.mean()),
        'pixels_contour_fort': int((mag > mag.max() * 0.3).sum()),
    }
    out_b = (mag / mag.max() * 255).astype(np.uint8) # carte de
    magnitude visualisable
```

6.2. Les opérateurs de Sobel et Scharr



$$S_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} \quad S_y = S_x^T = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

L'idée

Le problème de la simple différence centrée $[-1, 0, +1]$ est que le bruit d'une ligne unique de pixels peut fausser la mesure. Sobel réintroduit le lissage là où on le combat : il dérive dans une direction tout en **lissant perpendiculairement**. Le noyau S_x se factorise de façon révélatrice :

$$S_x = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \cdot \begin{bmatrix} -1 & 0 & +1 \end{bmatrix}$$

Le vecteur colonne $[1, 2, 1]$ est un lissage gaussien-like appliqué dans la direction y (perpendiculaire au gradient horizontal cherché). Le vecteur ligne $[-1, 0, +1]$ est la dérivée centrée dans la direction x . Sobel ne fait donc pas que dériver : il dérive *en lissant transversalement*, ce qui le rend nettement plus robuste au bruit qu'une différence nue. C'est une première réponse concrète au fil conducteur : le lissage est intégré dans l'opérateur lui-même.

Dérivation : pourquoi ces coefficients ?

Le produit $[1, 2, 1]^T \times [-1, 0, +1]$ donne exactement la matrice S_x ci-dessus. La pondération $[1, 2, 1]$ accorde plus de poids au pixel central (dont la ligne est la plus fiable) et moins aux lignes adjacentes. Ce n'est pas une gaussienne exacte, mais une bonne approximation pour un noyau 3×3 .

Scharr : une meilleure approximation

Le noyau $[1, 2, 1]$ de Sobel n'est pas optimal pour l'**isotropie** : sa réponse dépend légèrement de l'orientation du contour dans l'image. Un bord à 45° ne donne pas exactement la même magnitude qu'un bord à 0° . Scharr corrige les poids pour minimiser cette erreur angulaire :

$$\text{Scharr}_x = \begin{bmatrix} -3 & 0 & +3 \\ -10 & 0 & +10 \\ -3 & 0 & +3 \end{bmatrix}$$

Les coefficients (3, 10, 3) sont calculés pour que la réponse soit identique quelle que soit l'orientation du bord. Pour des mesures d'orientation précises — en vision industrielle pour mesurer l'angle d'une pièce, ou en microscopie pour quantifier l'orientation de fibres — Scharr est préférable à Sobel.



Patch 3×3 avec un bord vertical net (gauche sombre = 0, droite claire = 100) :

```
patch = [0  0 100]
         [0  0 100]
         [0  0 100]
```

Application de S_x sur le pixel central (ligne 1, colonne 1) :

$$\begin{aligned} S_x * \text{centre} &= (-1 \cdot 0 - 2 \cdot 0 - 1 \cdot 0) + (0 \cdot 0 + 0 \cdot 0 + 0 \cdot 0) + (1 \cdot 100 + 2 \cdot 100 + 1 \cdot 100) \\ &= 0 + 0 + 400 = 400 \quad \rightarrow \text{fort gradient horizontal} \end{aligned}$$

$$\begin{aligned} S_y * \text{centre} &= (-1 \cdot 0 - 2 \cdot 0 - 1 \cdot 100) + (0 \cdot 0 + 0 \cdot 0 + 0 \cdot 100) + (1 \cdot 0 + 2 \cdot 0 + 1 \cdot 100) \\ &= -100 + 0 + 100 = 0 \quad \rightarrow \text{gradient vertical nul} \end{aligned}$$

Résultat : $I_x = 400$, $I_y = 0$. Le gradient pointe horizontalement, perpendiculaire au bord vertical — exactement ce qu'on attend. ■



Les sorties de `cv2.Sobel` doivent être calculées en `cv2.CV_64F` (flottant 64 bits), jamais en `uint8`. En `uint8`, les valeurs négatives sont tronquées à 0 : on rate tous les bords où l'intensité décroît (passage clair → sombre). Si vous voulez visualiser le résultat, convertissez après calcul : `np.uint8(np.absolute(gx))`.



```

if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune image en entree'}
else:
    gray = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY) if a.ndim == 3 else a

    # Sobel
    gx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
    gy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
    mag_sobel = cv2.magnitude(gx, gy)

    # Scharr (meilleure isotropie)
    gx_s = cv2.Scharr(gray, cv2.CV_64F, 1, 0)
    gy_s = cv2.Scharr(gray, cv2.CV_64F, 0, 1)
    mag_scharr = cv2.magnitude(gx_s, gy_s)

    out_a = {
        'gradient_max_sobel': float(mag_sobel.max()),
        'gradient_max_scharr': float(mag_scharr.max()),
    }
    # Carte visuelle normalisée (sortie image)
    vis = (mag_sobel / (mag_sobel.max() + 1e-6) * 255).astype(np.uint8)
    out_b = cv2.cvtColor(vis, cv2.COLOR_GRAY2BGR)

```

6.3. Le détecteur de Canny

L'idée

Canny (1986) n'est pas un simple noyau mais un **pipeline** en cinq étapes, conçu pour répondre à trois critères formels : bonne détection (peu de faux positifs et faux négatifs), bonne localisation (le contour est au bon endroit, pas décalé), et réponse unique (un seul pixel de large par bord, pas un épais ruban). C'est le détecteur le plus cité en vision par ordinateur classique, et sa longévité tient précisément au fait qu'il rend explicite la tension dérivation/bruit à chaque étape.

Définition — les cinq étapes

1. Lissage gaussien → réduire le bruit avant de dériver (σ contrôle l'échelle)
2. Gradient (Sobel) → magnitude et orientation en chaque pixel
3. Suppression non-maximale → amincir les contours à 1 pixel de large

- 4. Double seuillage → classer fort / faible / rejeté
- 5. Hystérésis → relier les contours faibles aux forts

Dérivation — l'étape clé : la suppression non-maximale

Le gradient brut donne des contours épais — plusieurs pixels de large — car la transition entre sombre et clair s'étale sur plusieurs colonnes. Pour les amincir, on ne garde un pixel que s'il est un **maximum local de magnitude le long de la direction du gradient**. Concrètement : on regarde les deux voisins dans la direction ϑ ; si le pixel central n'est pas le plus fort des trois, on le supprime. Le contour est ainsi réduit à sa crête, large d'exactly un pixel. C'est l'analogie de trouver le pic d'une montagne : seul le point le plus haut d'une pente est retenu.

L'hystérésis : pourquoi deux seuils ?

Un seuil unique pose un dilemme insoluble : trop haut, on coupe les portions faibles d'un contour réel qui s'interrompt ; trop bas, on garde le bruit. L'hystérésis résout ce problème par une logique connexe, avec deux seuils T_bas et T_haut :

```

pixel > T_haut      → contour certain (conservé d'office)
pixel < T_bas       → rejeté
T_bas ≤ pixel ≤ T_haut → conservé seulement s'il est connecté à un pixel certain

```

L'idée intuitive : un pixel faible isolé est probablement du bruit, mais un pixel faible qui **prolonge un contour fort** est probablement la continuation du même bord réel. On utilise la connexité spatiale comme preuve de réalité. Ratio recommandé : $T_haut / T_bas \approx 2$ à 3.

Exemple numérique (hystérésis)

Chaîne de magnitudes le long d'un contour candidat, avec $T_bas = 50$, $T_haut = 100$:

```

... 30    60    80    120    70    55    40    ...
    rej    ?    ?    sûr    ?    ?    rej

```

Le pixel à 120 est certain (>100). Les pixels à 80, 70, 55 sont dans $[50, 100]$: ils sont conservés car connectés en chaîne au pixel certain. Le pixel à 60, connecté à 80 lui-même connecté au pixel certain, est aussi conservé. Les pixels à 30 et 40 (<50) sont rejetés, coupant la chaîne. Résultat : un contour continu de 60 à 120.

Dans une application de reconnaissance de plaques d'immatriculation, ce mécanisme préserve les bords des lettres même là où l'éclairage est inégal : les portions sombres d'un bord (magnitude faible) sont conservées grâce aux portions bien éclairées qui les entourent. ■



Canny a trois paramètres couplés (σ du flou gaussien, T_{bas} , T_{haut}). Augmenter σ détecte les contours d'échelle grossière et ignore les fins détails — c'est voulu, mais il faut le choisir en connaissance de cause. Il n'existe pas de réglage universel. Astuce courante pour les images naturelles : fixer les seuils automatiquement à partir de la médiane de l'image :

```
T_bas = 0.66 × médiane(image)
T_haut = 1.33 × médiane(image)
```

Cette heuristique s'adapte à l'histogramme global et donne un point de départ raisonnable pour l'exploration.



```
if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune image en entree'}
else:
    gray = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY) if a.ndim == 3 else a

    # Seuils automatiques par la médiane
    v = float(np.median(gray))
    t_low = max(0, int(0.66 * v))
    t_high = min(255, int(1.33 * v))

    edges = cv2.Canny(gray, threshold1=t_low, threshold2=t_high,
apertureSize=3)

    out_a = {
        't_bas': t_low,
        't_haut': t_high,
        'pixels_contour': int((edges > 0).sum()),
        'densite_contour_%': float((edges > 0).mean() * 100),
    }
    out_b = cv2.cvtColor(edges, cv2.COLOR_GRAY2BGR) # visualisation
```

6.4. Le tenseur de structure



$$M(x,y) = \sum_{\text{voisinage}} w(u,v) \cdot \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

La somme est pondérée par une fenêtre gaussienne w sur un voisinage. M est une matrice 2×2 symétrique calculée en chaque pixel.

L'idée

Le gradient seul décrit un pixel isolé. Le tenseur de structure décrit un **voisinage** : il agrège l'information de plusieurs pixels pour caractériser la géométrie locale. L'image mentale utile est celle d'un détective qui, au lieu d'examiner une seule empreinte, observe toute une zone du sol pour comprendre si quelqu'un est passé, dans quelle direction, et à quelle vitesse.

Pour comprendre ce que M encode, il faut saisir l'idée de ses valeurs propres $\lambda_1 \geq \lambda_2$ — ces deux nombres qui résument les directions de variation dans le voisinage (voir chapitre 1, §1.3 sur l'excentricité pour le même outil appliqué aux formes). Ici, les valeurs propres décrivent l'énergie des gradients dans les deux directions principales du voisinage :

- $\lambda_1 \approx \lambda_2 \approx 0$ → région plate (aucune variation, fond uniforme)
- $\lambda_1 \gg \lambda_2 \approx 0$ → contour (forte variation dans une seule direction)
- $\lambda_1 \approx \lambda_2 \gg 0$ → coin ou texture (variation dans toutes les directions)

Dérivation : pourquoi les valeurs propres ?

M est la matrice de covariance des vecteurs gradients dans le voisinage. Ses vecteurs propres donnent les directions principales de variation, ses valeurs propres l'intensité de variation dans ces directions. Un contour a une direction privilégiée : forte variation perpendiculairement, nulle le long — d'où une grande et une petite valeur propre. Un coin n'a aucune direction privilégiée (les gradients pointent dans tous les sens) : deux grandes valeurs propres. C'est exactement l'analyse en composantes principales (ACP) appliquée aux gradients du voisinage. L'ACP est ici l'outil naturel pour décider « y a-t-il une, deux, ou zéro directions structurées ici ? ».

Pourquoi pondérer par une gaussienne ?

La somme Σ intègre les gradients du voisinage. Une fenêtre gaussienne (plutôt qu'un carré uniforme « box ») pondère de façon douce et décroissante depuis le centre, évite les artefacts de bord du voisinage et rend la réponse isotrope — c'est-à-dire identique quelle que soit l'orientation de la structure. C'est encore la même logique du fil conducteur : on stabilise

une mesure différentielle bruitée en l'intégrant localement. Le paramètre σ de la gaussienne contrôle la taille du voisinage intégré, et donc l'échelle des structures détectées.



Le tenseur M est rarement calculé à la main : `cv2.cornerEigenValsAndVecs` ou `cv2.cornerHarris` le calculent en interne. Si vous l'implémentez manuellement, pensez à calculer les trois termes (I_x^2 , I_y^2 , $I_x I_y$) séparément, puis à les lisser avec une gaussienne — et non à lisser l'image en premier puis à dériver, ce qui donne un résultat différent.



```
if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune image en entree'}
else:
    gray = np.float32(cv2.cvtColor(a, cv2.COLOR_BGR2GRAY) if a.ndim == 3
    else a)

    gx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
    gy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

    # Les trois composantes du tenseur, lissées par une gaussienne
    sigma = 2.0
    Ixx = cv2.GaussianBlur(gx * gx, (0, 0), sigma)
    Iyy = cv2.GaussianBlur(gy * gy, (0, 0), sigma)
    Ixy = cv2.GaussianBlur(gx * gy, (0, 0), sigma)

    # Indicateurs scalaires (det et trace de M)
    det_M = Ixx * Iyy - Ixy * Ixy
    trace_M = Ixx + Iyy

    out_a = {
        'det_M_max': float(det_M.max()),
        'trace_M_max': float(trace_M.max()),
        'det_M_moy': float(det_M.mean()),
    }
```

6.5. Détecteurs de coins : Harris et Shi-Tomasi



Harris : $R = \det(M) - k \cdot \text{trace}(M)^2$ $k \approx 0.04-0.06$
 Shi-Tomasi : $R = \min(\lambda_1, \lambda_2)$

L'idée

Un coin est un point où deux contours se rencontrent. Il est précieux en vision par ordinateur parce qu'il est **reproductible** : si vous photographiez le même objet de deux angles légèrement différents, les coins seront encore là, tandis que les bords droits peuvent disparaître. En suivi de mouvement, en reconstruction 3D, en assemblage de panoramas, les coins sont les repères que la machine cherche à retrouver d'une image à l'autre.

L'enjeu est de reconnaître un coin sans calculer explicitement les valeurs propres λ_1, λ_2 de M — ce qui serait coûteux pour chaque pixel d'une image. Harris exploite deux identités d'algèbre linéaire qui permettent d'éviter cette diagonalisation :

$$\begin{aligned}\det(M) &= \lambda_1 \cdot \lambda_2 && \text{(produit des valeurs propres)} \\ \text{trace}(M) &= \lambda_1 + \lambda_2 && \text{(somme des valeurs propres)}\end{aligned}$$

On peut donc construire un indicateur R à partir de $\det$ et trace , sans jamais calculer λ_1 et λ_2 séparément :

R grand et positif	$\Leftrightarrow$	λ_1, λ_2 tous deux grands	$\Leftrightarrow$	coin
R négatif	$\Leftrightarrow$	une valeur propre domine	$\Leftrightarrow$	contour
$R \approx 0$	$\Leftrightarrow$	les deux valeurs petites	$\Leftrightarrow$	région plate

Le terme $-k \cdot \text{trace}^2$ pénalise le cas « une seule grande valeur propre » (contour) pour ne retenir que les vrais coins. Petit k = plus de coins détectés (sensibilité élevée) ; grand k = seuls les coins très nets sont retenus (précision élevée).

Shi-Tomasi : plus simple, souvent meilleur

Shi et Tomasi (1994) montrent qu'un critère encore plus simple — la plus petite des deux valeurs propres — prédit mieux la stabilité d'un point pour le suivi :

$$R = \min(\lambda_1, \lambda_2)$$

Si $\min(\lambda_1, \lambda_2)$ est grand, c'est que les deux directions varient fortement : le point est un bon « coin à suivre » (*good feature to track*). Ce critère est le défaut de `cv2.goodFeaturesToTrack`, utilisé comme point de départ du suivi optique de Lucas-Kanade.

Propriétés : invariances et limites

Harris et Shi-Tomasi sont **invariants en rotation** (les valeurs propres ne dépendent pas de l'orientation de M) et robustes aux changements d'illumination de type affine (éclairage uniforme qui monte ou descend). Mais ils ne sont **pas invariants à l'échelle** : un coin vu de près est un coin ; le même coin vu de loin peut ressembler à une simple texture. C'est exactement cette limite qui a motivé les détecteurs multi-échelles comme Harris-Laplace, puis SIFT — qui combine détection de blobs par Différence-de-Gaussiennes (DoG) et sélection

automatique d'échelle. Ces méthodes peuvent être considérées comme une réponse plus élaborée au fil conducteur : elles dosent le lissage à plusieurs échelles simultanément.



Trois voisinages distincts, avec les valeurs propres agrégées de leur tenseur M :

région plate : $\lambda_1=2$, $\lambda_2=1$

$R_{\text{Harris}} = (2 \cdot 1) - 0.05 \cdot (2+1)^2 = 2 - 0.45 = 1.55$ (faible $\rightarrow$ pas un coin)

$R_{\text{Shi-Tomasi}} = \min(2, 1) = 1$ (faible, confirmé)

contour : $\lambda_1=100$, $\lambda_2=2$

$R_{\text{Harris}} = (100 \cdot 2) - 0.05 \cdot (102)^2 = 200 - 520 = -320$ (négatif $\rightarrow$ contour)

$R_{\text{Shi-Tomasi}} = \min(100, 2) = 2$ (faible, pas un coin)

coin : $\lambda_1=80$, $\lambda_2=70$

$R_{\text{Harris}} = (80 \cdot 70) - 0.05 \cdot (150)^2 = 5600 - 1125 = 4475$ (fort positif $\rightarrow$ coin)

$R_{\text{Shi-Tomasi}} = \min(80, 70) = 70$ (fort $\rightarrow$ bon coin)

Shi-Tomasi sur les mêmes données donne $\min = 1, 2, 70$: seul le coin a un minimum élevé — diagnostic identique à Harris, calcul plus direct. ■



La réponse de Harris R est une image flottante avec des valeurs très variables. Il faut la **normaliser** avant de seuiller, ou utiliser un seuil relatif au maximum. De plus, plusieurs pixels adjacents peuvent tous avoir une valeur R élevée (le coin « s'étale »). Appliquer une suppression non-maximale ou `cv2.dilate` + comparaison avec l'original filtre les doublons.



```

if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune image en entree'}
else:
    gray = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY) if a.ndim == 3 else a
    gray_f = np.float32(gray)

    # Harris
    R = cv2.cornerHarris(gray_f, blockSize=2, ksize=3, k=0.04)
    R_norm = cv2.normalize(R, None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)
    n_harris = int((R > 0.01 * R.max()).sum())

    # Shi-Tomasi
    corners = cv2.goodFeaturesToTrack(gray, maxCorners=200,
                                     qualityLevel=0.01, minDistance=10)
    n_shi = len(corners) if corners is not None else 0

    out_a = {
        'n_coins_harris': n_harris,
        'n_coins_shi_tomasi': n_shi,
        'harris_max': float(R.max()),
    }

    # Visualisation : coins Harris en rouge sur l'image
    vis = a.copy() if a.ndim == 3 else cv2.cvtColor(gray,
cv2.COLOR_GRAY2BGR)
    vis[R > 0.01 * R.max()] = [0, 0, 255]
    out_b = vis

```

6.6. Tableau récapitulatif – du gradient à la structure

Outil	Ce qu'il mesure	Angle mort	Usage typique
Gradient ∇	Variation locale (force et direction)	Sensible au bruit — dériver = amplifier	Carte de magnitude, orientation locale
Sobel / Scharr	Gradient robuste au bruit (lissage inté- gré)	Contours épais, pas de connexité	Première estimation de contours, entrée d'autres algos

Outil	Ce qu'il mesure	Angle mort	Usage typique
Canny	Contours fins, connectés, localisés au pixel	Trois paramètres couplés à régler (σ , T_bas, T_haut)	Détection de contours généraliste, OCR, vision industrielle
Tenseur de structure M	Géométrie locale (plat / bord / coin)	Coûteux par pixel, paramètre σ de la fenêtre	Analyse de texture, entrée de Harris
Harris	Coins stables (invariant rotation)	Pas invariant à l'échelle	Homographie, mosaïque, suivi
Shi-Tomasi	Coins optimaux pour le suivi	Idem Harris	Suivi optique Lucas-Kanade, SLAM

6.7. choisir une échelle, c'est choisir ce qu'on garde

Tout le chapitre tourne autour d'une même tension, annoncée dès l'introduction :

dériver → révèle les variations, MAIS amplifie le bruit (filtre passe-haut)
lisser → supprime le bruit, MAIS émousse les variations (filtre passe-bas)

Chaque outil est une manière de doser ces deux opérations antagonistes :

Gradient pur → dérivation nue (aucun lissage, maximum de sensibilité au bruit)
Sobel / Scharr → lissage transversal [1 2 1] intégré au noyau de dérivation
Canny → flou gaussien explicite en amont, puis dérivation + hystérésis
Tenseur M → intégration gaussienne des gradients sur un voisinage
LoG / DoG (ch. 5) → dérivée seconde d'une gaussienne (le lissage EST l'opérateur)

La question de conception n'est jamais « faut-il lisser ? » mais « à quelle échelle ? ». Le σ du lissage fixe la taille des structures que l'on verra : petit σ pour les détails fins (nervures d'une feuille en biologie, micro-fissures en contrôle qualité), grand σ pour les contours saillants des grandes structures (contours de bâtiments en télédétection, segments d'organes en imagerie médicale). Détecter des contours sans choisir une échelle n'a pas de sens.

Ce choix rejoint le méta-fil du livre : **tout choix de représentation encode une hypothèse sur ce qui compte**. Choisir $\sigma = 1$ pour Canny, c'est déclarer « les variations au niveau du pixel sont significatives ». Choisir $\sigma = 5$, c'est déclarer « seules les structures de plusieurs pixels méritent attention ». Le bon cadre rend le problème presque résolu —

mais le bon cadre, ici, c'est l'échelle, et l'échelle doit être choisie en connaissance du signal et du bruit propres à chaque application.

Ce pont entre échelle et représentation est développé dans le chapitre 5 (Filtrage, espace-échelle gaussien) et retrouvé au chapitre 10 (Transformées), où changer de base permet de lire la même image à plusieurs résolutions simultanément.

CHAPITRE

Couleur et photométrie

7

Table des matières

7.1. Luminance : du RGB au niveau de gris	101
7.2. RGB $\rightarrow$ HSV / HSL	103
7.3. CIELAB et la mesure perceptuelle	106
7.4. Gamut et conversion RGB $\rightarrow$ CMYK	108
7.5. Égalisation d'histogramme et CLAHE	111
7.6. Correction gamma et balance des blancs	113
7.7. Tableau récapitulatif — quel espace pour quelle tâche ?	116
7.8. l'espace encode une hypothèse	116

La couleur n'est pas une propriété physique des objets : c'est une construction. Un même spectre lumineux peut être encodé de dizaines de façons différentes, et deux spectres physiquement distincts peuvent produire une sensation visuelle identique. Ce chapitre traite la couleur comme un problème de représentation — quel espace pour quelle tâche — et de mesure — comment quantifier un écart colorimétrique de façon qui corresponde à la perception humaine.

Le fil conducteur tient en une phrase : **il n'existe pas d'espace colorimétrique « vrai », seulement des espaces adaptés à un usage**. RGB est conçu pour les capteurs et les écrans, HSV pour la sélection intuitive, CMYK pour l'encre, CIELAB pour la mesure perceptuelle. Convertir entre ces espaces, c'est passer d'une logique à une autre — chaque conversion implique des choix, des approximations, et des pièges spécifiques. Ce fil — *l'espace n'est pas neutre, il encode une hypothèse sur ce qui compte* — sera tissé dans chaque section.

Renvois croisés : la notion d'espace de représentation est analogue au choix d'une base dans le chapitre 10 (Transformées) — « changer de base, c'est choisir où le problème devient simple ». Le lien entre espace colorimétrique et mesure de distance sera repris au chapitre 3 (Distances) : toute distance de couleur suppose implicitement un espace.

7.1. Luminance : du RGB au niveau de gris



$Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$ (ITU-R BT.601, télévision standard)
 $Y = 0.2126 \cdot R + 0.7152 \cdot G + 0.0722 \cdot B$ (ITU-R BT.709, haute définition)

Pourquoi des coefficients inégaux ?

La conversion en niveaux de gris n'est pas une moyenne arithmétique $(R+G+B)/3$. Ces coefficients traduisent une réalité biologique : la rétine humaine n'est pas également sensible à toutes les longueurs d'onde. L'œil possède trois types de cônes — les cônes S (bleu), M (vert) et L (rouge) — dont la sensibilité est fortement asymétrique. Les cônes M et L, sensibles au vert et au rouge, sont bien plus nombreux et réactifs que les cônes S. En pratique, le vert contribue environ 59 à 72 % de la luminance perçue, le rouge environ 21 à 30 %, et le bleu seulement 7 à 11 %.

L'image mentale utile est celle d'un mélange de peinture dans lequel le vert domine massivement. Une moyenne naïve donnerait une recette aux proportions égales ; ici, la recette est calibrée sur l'instrument de mesure — l'œil humain. Conséquence concrète en imagerie médicale : un scanner anatomique ou une endoscopie converties en niveaux de gris

avec une moyenne simple aplatiront les contrastes tissulaires que l'opérateur distinguerait naturellement ; avec les bons coefficients, les structures se détachent comme l'œil les perçoit.

Pourquoi deux standards (BT.601 vs BT.709) ?

BT.601 date de la télévision à définition standard (années 1980), BT.709 de la haute définition (années 1990–2000). Les écrans ont évolué : leurs phosphores (les substances luminescentes) ont changé, modifiant les couleurs primaires de référence. Les coefficients ont donc été réajustés pour refléter ces nouveaux primaires. L'écart entre les deux standards est de quelques pour-cent — négligeable en affichage grand public, mais détectable dans toute mesure photométrique précise. OpenCV utilise BT.601 par défaut via `cv2.COLOR_BGR2GRAY`. Ce choix d'espace — BT.601 ou BT.709 — encode déjà une hypothèse : quelle « recette » de l'œil on veut imiter.

Piège d'implémentation : luminance vs luma, et le piège du gamma

C'est le piège le plus fréquemment ignoré dans la pratique. Les formules ci-dessus s'appliquent, en toute rigueur, à des valeurs **linéaires** — c'est-à-dire proportionnelles à l'énergie lumineuse réelle. Or la quasi-totalité des images numériques (JPEG, PNG, captures caméra) sont stockées en valeurs **encodées gamma** (sRGB, gamma ≈ 2.2). Ce que l'on appelle « luminance » (Y) est une grandeur physique ; ce que calcule `cv2.COLOR_BGR2GRAY` est en réalité le *luma* (Y'), une approximation sur des valeurs déjà compressées.

Pour une luminance physiquement correcte, la procédure rigoureuse est :

1. Linéariser chaque canal : $R_lin = (R_sRGB / 255)^{1/2.2}$ (gamma inverse)
2. Pondérer : $Y = 0.2126 \cdot R_lin + 0.7152 \cdot G_lin + 0.0722 \cdot B_lin$
3. Re-encoder si nécessaire : $Y_sRGB = Y^{2.2} \times 255$

L'écart entre luma et luminance est visible sur les dégradés et dans tout calcul de contraste. C'est ce calcul correct qui est exigé, par exemple, pour le calcul du ratio de contraste WCAG en accessibilité numérique, ou pour le compositing vidéo professionnel. Travailler avec des valeurs gamma comme si elles étaient linéaires revient à utiliser un thermomètre dont l'échelle serait compressée — les résultats sont plausibles à l'œil, mais faux à la mesure.



Un pixel de teinte chartreuse vive : $RGB = (180, 220, 30)$. Calcul BT.709 :

$$\begin{aligned} Y &= 0.2126 \times 180 + 0.7152 \times 220 + 0.0722 \times 30 \\ &= 38.27 + 157.34 + 2.17 \\ &\approx 198 \end{aligned}$$

Moyenne naïve : $(180+220+30)/3 = 143$

L'écart est de 55 niveaux sur 255 — plus de 20 %. Sur un contrôle qualité de surface peinte en industrie, une telle erreur de conversion fausserait entièrement la détection de défauts par variation de luminosité. ■



```
# a : image BGR (np.ndarray HxWx3 uint8)
if not isinstance(a, np.ndarray) or a.ndim < 3:
    out_a = {'erreur': 'image BGR attendue en entree a'}
else:
    gray = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY) # luma BT.601
    (approximation sur gamma)

    # Version linéarisée (luminance physique BT.709) :
    img_f = (a[:, :, ::-1] / 255.0) ** 2.2 # BGR→RGB, puis gamma
    inverse
    Y_lin = (0.2126 * img_f[:, :, 0]
             + 0.7152 * img_f[:, :, 1]
             + 0.0722 * img_f[:, :, 2])
    Y_encoded = np.clip(Y_lin ** (1/2.2) * 255, 0, 255).astype(np.uint8)

    out_a = Y_encoded # port image (masque gris, sortie image)
```

7.2. RGB → HSV / HSL



$$\begin{aligned} V &= \max(R, G, B) && \text{avec } R, G, B \in [0, 1] \\ S &= (V - \min(R, G, B)) / V && (0 \text{ si } V = 0) \\ H &= 60^\circ \times \begin{cases} (G-B)/\Delta \bmod 6 & \text{si max} = R \\ (B-R)/\Delta + 2 & \text{si max} = G \\ (R-G)/\Delta + 4 & \text{si max} = B \end{cases} && \text{où } \Delta = \max - \min \end{aligned}$$

L'idée : séparer la couleur de l'intensité

RGB mélange teinte et luminosité dans ses trois canaux simultanément : assombrir un rouge vif change R, G et B en même temps. HSV (Hue, Saturation, Value) **découple** ces trois propriétés. La teinte H est l'angle sur la roue chromatique — c'est la « couleur pure » du pixel, indépendamment de sa clarté ou de sa richesse. La saturation S est la pureté de cette couleur : un rouge très saturé est vif et pur, un rouge peu saturé tire vers le rose ou le gris. La valeur V est la clarté : à H et S constants, augmenter V éclaircit la couleur.

L'image mentale utile est celle d'un pot de peinture. H indique la couleur du pot (rouge, bleu, vert...). S indique si la peinture est couverte d'eau ou non — beaucoup d'eau la délave (faible saturation), aucune eau la laisse pure (saturation maximale). V indique si on peint sous une bonne lampe ou dans l'obscurité.

Ce découplage rend HSV idéal pour la segmentation par couleur : pour isoler tous les objets rouges d'une scène de tri industriel, quelle que soit leur luminosité, il suffit de seuiller H dans une plage autour de $0^\circ/360^\circ$. En RGB, la même opération nécessiterait de seuiller dans un volume 3D non intuitif. Le choix de l'espace HSV encode une hypothèse : la teinte est le critère qui compte, l'intensité est une nuisance à éliminer.

Géométrie : du cube RGB au cône HSV

RGB est un **cube** : chaque axe représente un canal primaire, et chaque couleur est un point à l'intérieur de ce cube. HSV est une réinterprétation géométrique de ce même cube : on pivote le cube sur son axe diagonal (la ligne grise, de $(0,0,0)$ à $(1,1,1)$), on passe en coordonnées cylindriques — H est l'angle, S le rayon, V la hauteur. La conversion ci-dessus est simplement ce changement de repère.

Piège d'implémentation : la singularité de la teinte et la circularité de H

Quand $S \rightarrow 0$ (gris, blanc, noir), la teinte H devient **indéfinie** : un gris n'a pas de couleur, donc pas d'angle. Numériquement, H devient instable et bruité pour les pixels peu saturés — n'importe quel bruit imperceptible peut faire basculer H de 0° à 180° . Toute segmentation par teinte doit donc **masquer les pixels de faible saturation** avant de seuiller H, sous peine de voir le bruit de teinte des zones grises contaminer le résultat.

De plus, H est **circulaire** : 0° et 360° désignent le même rouge. La moyenne de teintes doit se calculer en coordonnées circulaires, jamais arithmétiquement. La moyenne de 350° et 10° vaut 0° (rouge), non 180° (cyan). En pratique, dans OpenCV, H est encodé dans $[0, 179]$ en 8 bits (la plage $[0^\circ, 360^\circ]$ est divisée par 2 pour tenir dans un octet) — il faut en tenir compte pour tous les seuils.



Pixel orange vif RGB = (255, 128, 0) (valeurs sur [0, 255] ; convertir en [0, 1]) :

```
R=1.0, G=0.502, B=0.0
V = max(1.0, 0.502, 0.0) = 1.0
min = 0.0, Δ = 1.0
S = (1.0 - 0.0) / 1.0 = 1.0 (saturation maximale)
max = R → H = 60 × ((0.502 - 0.0) / 1.0 mod 6)
           = 60 × 0.502
           ≈ 30°
```

H = 30° (orange sur la roue), S = 100 % (pur), V = 100 % (lumineux). Dans OpenCV (H divisé par 2) : H_OpenCV = 15. ■

Ce résultat est conforme à l'intuition : dans le cylindre HSV, l'orange se trouve à mi-chemin entre le rouge (0°) et le jaune (60°), la saturation et la valeur maximales indiquent une couleur franche et vive.



```
# a : image BGR (np.ndarray HxWx3 uint8)
if not isinstance(a, np.ndarray) or a.ndim < 3:
    out_a = {'erreur': 'image BGR attendue en entree a'}
else:
    hsv = cv2.cvtColor(a, cv2.COLOR_BGR2HSV) # H ∈ [0,179] en 8 bits
    OpenCV

    # Segmentation par teinte : isoler les pixels dans une plage de H
    # Exemple : orange (H_ocv ∈ [5, 25]), saturation et valeur
    suffisantes
    masque_S = hsv[:, :, 1] > 60 # éliminer les gris peu saturés
    masque_H = (hsv[:, :, 0] >= 5) & (hsv[:, :, 0] <= 25)
    masque = (masque_H & masque_S).astype(np.uint8) * 255

    out_a = masque # port mask (gris, 0/255)
```

7.3. CIELAB et la mesure perceptuelle



CIELAB (ou $L^*a^*b^*$) est un espace à trois coordonnées :

L^* : clarté perceptuelle (0 = noir absolu, 100 = blanc de référence)
 a^* : axe chromatique vert(-) ↔ rouge(+) (pas de limites fixes, $\approx [-128, +127]$)
 b^* : axe chromatique bleu(-) ↔ jaune(+)

La conversion depuis RGB passe obligatoirement par l'espace intermédiaire XYZ (lié au spectre lumineux via les fonctions colorimétriques de l'observateur standard CIE 1931), puis applique une non-linéarité cubique :

$$f(t) = t^{1/3} \quad \text{si } t > (6/29)^3$$

$$f(t) = t / (3 \cdot (6/29)^2) + 4/29 \quad \text{sinon}$$

$$L^* = 116 \cdot f(Y/Y_n) - 16$$

$$a^* = 500 \cdot (f(X/X_n) - f(Y/Y_n))$$

$$b^* = 200 \cdot (f(Y/Y_n) - f(Z/Z_n))$$

où (X_n, Y_n, Z_n) est le point blanc de référence (D65 pour la lumière du jour standard).

L'idée : l'uniformité perceptuelle

Dans RGB ou HSV, une même distance numérique peut correspondre à des différences perçues très inégales selon la région de l'espace. L'œil humain distingue finement les teintes dans les verts, beaucoup moins dans les bleus saturés. CIELAB est construit précisément pour que **des distances numériques égales correspondent à des différences perçues égales** — propriété appelée *uniformité perceptuelle*.

L'image mentale utile est celle d'une carte géographique à équidistance. Sur une carte topographique, des courbes de niveau régulièrement espacées représentent des dénivellations égales partout. CIELAB est la même idée appliquée à l'espace des couleurs : deux points séparés de la même distance numérique paraissent « aussi différents » à l'œil, quelle que soit leur position dans l'espace.

C'est ce qui rend CIELAB indispensable pour *mesurer* des écarts — pas pour afficher ou stocker des couleurs. Mesurer un écart en RGB revient à mesurer une distance sur une carte dont les échelles seraient différentes selon les régions : le résultat dépend de l'endroit où l'on mesure. Ce choix d'espace encode une hypothèse forte : on veut que la mesure reflète la perception, pas l'arithmétique du capteur.

La distance ΔE_{76}

$$\Delta E_{76} = \sqrt{((L_1^* - L_2^*)^2 + (a_1^* - a_2^*)^2 + (b_1^* - b_2^*)^2)}$$

C'est la distance euclidienne dans CIELAB. Elle n'a de sens *que* parce que l'espace est perceptuellement uniforme. Les seuils d'interprétation, établis par des études psychophysiques :

$\Delta E < 1$: différence imperceptible, même pour un observateur entraîné
 $\Delta E \in [1, 2]$: perceptible par un œil exercé dans des conditions contrôlées
 $\Delta E \in [2, 10]$: perceptible au premier coup d'œil
 $\Delta E > 10$: couleurs nettement différentes

Dérivation : pourquoi ΔE_{76} ne suffit pas — la formule ΔE_{2000}

ΔE_{76} surestime les écarts dans certaines régions (les bleus très saturés notamment), car l'uniformité de CIELAB reste imparfaite. Des décennies de mesures psychophysiques ont conduit à des formules correctives : ΔE_{94} d'abord, puis **ΔE_{2000}** , qui ajoute trois termes de pondération (clarté, chroma, teinte) et un terme de rotation pour les bleus :

$$\Delta E_{2000} = \sqrt{(\Delta L' / k_L \cdot S_L)^2 + (\Delta C' / k_C \cdot S_C)^2 + (\Delta H' / k_H \cdot S_H)^2 + R_T \cdot (\Delta C' / k_C \cdot S_C) \cdot (\Delta H' / k_H \cdot S_H)}$$

où S_L , S_C , S_H sont des fonctions de position dans l'espace (la pondération varie selon la couleur), et R_T est le terme de rotation des bleus. ΔE_{2000} est aujourd'hui le standard industriel pour tout contrôle qualité couleur.



En contrôle qualité textile, un fil de coton « bleu marine de référence » a les coordonnées $Lab_1 = (22, 4, -28)$. Un lot de production donne $Lab_2 = (24, 5, -25)$.

$$\begin{aligned} \Delta E_{76} &= \sqrt{((24-22)^2 + (5-4)^2 + (-25+28)^2)} \\ &= \sqrt{4 + 1 + 9} \\ &= \sqrt{14} \approx 3.7 \end{aligned}$$

Un ΔE_{76} de 3.7 est perceptible à l'œil nu — le lot sera probablement refusé si la norme exige $\Delta E < 2$. Un calcul ΔE_{76} en RGB sur les mêmes couleurs aurait donné un résultat sans signification perceptuelle, potentiellement validant un lot visuellement défectueux. ■



La conversion BGR $\rightarrow$ LAB dans OpenCV utilise les conventions sRGB avec D65. Attention : `cv2.cvtColor(img, cv2.COLOR_BGR2LAB)` renvoie L dans $[0, 255]$, a et b* dans $[0, 255]$ avec un décalage de 128 (pour rentrer dans un uint8). Pour calculer ΔE avec les vraies valeurs Lab, il faut convertir :

$$\begin{aligned} L &= L_{\text{ocv}} \times 100/255 \\ a &= a_{\text{ocv}} - 128 \\ b &= b_{\text{ocv}} - 128 \end{aligned}$$


```
# a : image BGR1 (np.ndarray HxWx3 uint8)
# b : image BGR2 (np.ndarray HxWx3 uint8)
if not isinstance(a, np.ndarray) or not isinstance(b, np.ndarray):
    out_a = {'erreur': 'deux images BGR attendues sur a et b'}
else:
    from skimage.color import rgb2lab, deltaE_ciede2000

    # Conversion BGR→RGB puis Lab (skimage attend RGB float [0,1])
    lab1 = rgb2lab(a[:, :, ::-1].astype(np.float32) / 255.0)
    lab2 = rgb2lab(b[:, :, ::-1].astype(np.float32) / 255.0)

    dE = deltaE_ciede2000(lab1, lab2) # carte HxW des ΔE2000

    out_a = {
        'dE_moyen': float(dE.mean()),
        'dE_max': float(dE.max()),
        'dE_p95': float(np.percentile(dE, 95)),
    }
```

7.4. Gamut et conversion RGB $\rightarrow$ CMYK



Gamut : ensemble des couleurs qu'un dispositif peut reproduire. La conversion naïve RGB $\rightarrow$ CMYK :

$$\begin{aligned} C &= 1 - R/255 \\ M &= 1 - G/255 \\ Y &= 1 - B/255 \\ K &= \min(C, M, Y) \quad ; \text{ puis soustraire } K \text{ de } C, M, Y \end{aligned}$$

L'idée : synthèse additive contre synthèse soustractive

RGB et CMYK reposent sur deux logiques opposées de la lumière. RGB est **additif** : on part du noir (absence de lumière), et on ajoute des sources lumineuses colorées. Rouge + Vert + Bleu = Blanc. C'est la logique des écrans, des projecteurs, des capteurs. CMYK est **soustractif** : on part du blanc (le papier), et les encres absorbent (soustraient) des longueurs d'onde. Cyan absorbe le rouge, Magenta absorbe le vert, Yellow absorbe le bleu. Superposer les trois encres pures devrait donner le noir — mais en pratique donne un brun vaseux, d'où l'ajout d'un noir séparé (K, *key*).

Ces deux espaces ne sont pas équivalents : ils ne couvrent pas le même gamut. Un écran peut afficher des cyans et des verts saturés impossibles à imprimer ; une imprimante offset peut reproduire certains gamuts de couleurs pasteltes difficiles à afficher sur un écran médian. La conversion RGB → CMYK n'est donc pas une bijection propre : c'est une **projection d'un gamut vers un autre**, avec des pertes inévitables pour les couleurs hors-gamut. Ce choix d'espace encode une hypothèse sur le medium de sortie — ce qui « compte » n'est pas la même chose sur un écran et sur du papier.

Pourquoi la formule naïve ne suffit pas

La conversion naïve ci-dessus est mathématiquement cohérente, mais ignore tout de la physique réelle de l'impression : le **gain de point** (l'encre s'étale sur le papier, les points d'impression grossissent), les interactions entre couches d'encres humides, et le profil spectral du papier. Une encre « rouge » imprimée sur papier mat n'a pas la même apparence que sur papier brillant, même avec les mêmes valeurs CMYK.

Une conversion correcte exige un **profil ICC** : un fichier de caractérisation mesurée du dispositif (imprimante, papier, encre), construit par des mesures colorimétriques instrumentales. Ce profil encode la cartographie réelle entre les couleurs CMYK et leurs rendu Lab mesurés.

Les intents de rendu

Quand une couleur RGB tombe hors du gamut CMYK, il faut décider comment la « faire rentrer ». Quatre stratégies existent :

- **Perceptuel** : comprime l'ensemble du gamut harmonieusement, en préservant les relations relatives entre couleurs. Bon pour les photographies.
- **Colorimétrique relatif** : conserve exactement les couleurs in-gamut, ramène les hors-gamut à la frontière la plus proche. Bon pour les aplats de marque (un logo rouge Coca-Cola doit garder son rouge exact).
- **Colorimétrique absolu** : comme le relatif, mais préserve aussi la simulation du point blanc du papier source.

- **Saturation** : privilégie la vivacité sur l'exactitude. Pour les graphiques et diagrammes.

Le choix de l'intent est une décision éditoriale, pas technique — il encode une priorité sur ce qui doit être préservé.



Convertir tôt en CMYK fait perdre des couleurs irréversiblement. La bonne pratique est de travailler en RGB jusqu'au dernier moment, en activant une **épreuve écran** (*soft proof*) qui simule le rendu CMYK à l'écran. Un ΔE_{2000} (voir §7.3) entre la couleur RGB cible et son rendu CMYK simulé quantifie exactement la perte attendue avant d'imprimer.



Un vert vif de logo : RGB = (0, 210, 90). Conversion naïve :

$$C = 1 - 0/255 = 1.0$$

$$M = 1 - 210/255 \approx 0.176$$

$$Y = 1 - 90/255 \approx 0.647$$

$$K = \min(1.0, 0.176, 0.647) = 0.176$$

$$\rightarrow C' = 0.824, M' = 0, Y' = 0.471, K = 0.176$$

La simulation soft-proof avec un profil CMYK standard (FOGRA39) indique que ce vert se trouve hors gamut — le rendu imprimé sera notablement plus terne. ΔE_{2000} entre la cible et la valeur imprimée simulée vaut environ 8.5 : différence visible à l'œil nu. ■



```
# a : image BGR (np.ndarray HxWx3 uint8)
# Conversion par profil ICC via Pillow (correct pour usage réel)
if not isinstance(a, np.ndarray) or a.ndim < 3:
    out_a = {'erreur': 'image BGR attendue en entree a'}
else:
    from PIL import Image, ImageCms

    img_rgb = Image.fromarray(a[:, :, ::-1]) # BGR→RGB pour Pillow

    srgb_profile = ImageCms.createProfile('sRGB')
    # Conversion vers CMYK via profil sRGB (approximation sans profil
    imprimante)
    img_cmyk = img_rgb.convert('CMYK') # méthode interne Pillow
    (naïve)

    # Pour conversion avec profil ICC :
    # cmyk_profile = ImageCms.getOpenProfile('FOGRA39.icc')
    # xform = ImageCms.buildTransform(srgb_profile, cmyk_profile, 'RGB',
    'CMYK',
    #                                     renderingIntent=0)
    # img_cmyk = ImageCms.applyTransform(img_rgb, xform)

    c, m, y, k = img_cmyk.split()
    out_a = np.array(c) # canal C comme image de sortie (uint8)
```

7.5. Égalisation d'histogramme et CLAHE



$$T(k) = \text{round} \left((L-1)/N \cdot \sum_{j=0^k} n(j) \right)$$

L = nombre de niveaux (256 pour uint8), N = nombre total de pixels, $n(j)$ = nombre de pixels au niveau j , $T(k)$ = valeur de sortie pour les pixels de niveau k .

Dérivation : étaler l'histogramme via sa CDF

L'idée repose sur un résultat classique de théorie des probabilités : si X est une variable aléatoire de fonction de répartition cumulative (CDF) F , alors $F(X)$ suit une loi uniforme sur $[0, 1]$. La CDF d'une distribution est simplement la courbe qui donne, pour chaque valeur, la proportion de pixels inférieurs ou égaux à cette valeur.

L'image mentale utile est celle d'un accordéon. Un histogramme très concentré ressemble à un accordéon comprimé sur quelques notes. Appliquer la CDF à chaque pixel, c'est ouvrir cet accordéon pour qu'il couvre toute la gamme de notes disponibles. Les zones denses (beaucoup de pixels à une même intensité) sont étirées, les zones rares sont comprimées — le résultat est un histogramme plat, utilisant toute la dynamique disponible.

La formule discrète $T(k)$ traduit exactement cela : la somme cumulée $\Sigma n(j)$ est la CDF discrète normalisée, puis ramenée sur $[0, L-1]$.

Pourquoi l'égalisation globale échoue souvent : CLAHE

L'égalisation globale applique une seule et même transformation à toute l'image. Le problème : une région localement peu contrastée (une zone d'ombre dans une radiographie thoracique, un défaut mat sur une surface industrielle brillante) reste noyée si le reste de l'image domine l'histogramme global. L'accordéon global ne résout pas les problèmes locaux.

Le **CLAHE** (*Contrast Limited Adaptive Histogram Equalization*) introduit deux corrections :

1. **Adaptatif** : l'image est découpée en tuiles (typiquement 8×8 ou 16×16), chacune égalisée indépendamment. Les transitions entre tuiles sont interpolées bilinéairement pour éviter les artefacts de raccord visibles — sans cette interpolation, la carte ressemblerait à un damier.
2. **Contrast Limited** : avant d'égaliser chaque tuile, l'histogramme local est écrêté à une hauteur maximale (le *clip limit*) et l'excédent redistribué uniformément sur tous les niveaux. Sans cet écrêtage, une tuile quasi uniforme (comme un fond de ciel homogène) donnerait une CDF presque verticale — une transformation à pente énorme qui amplifierait dramatiquement le bruit de capteur.

Le clip limit est le paramètre clé : trop bas, l'effet est faible ; trop haut, le bruit explose. Sa valeur est empiriquement réglée à 2.0–4.0 dans la plupart des applications médicales.

Ce choix — global ou CLAHE — encode une hypothèse sur ce qui compte : l'uniformité globale de la dynamique, ou la lisibilité locale de chaque région.



Image de fond d'œil dont les intensités s'entassent dans $[0, 80]$ sur une plage $[0, 255]$ (sous-exposition, rétine sombre). La CDF globale sature à $T(80) \approx 255$: la transformation étire $[0, 80]$ sur $[0, 255]$, révélant les vaisseaux. Mais si ce segment $[0, 80]$ contient aussi du bruit de capteur, l'étirement l'amplifie d'un facteur 3.2 ($255/80$). Pour un bruit de niveau 2 niveaux, le résultat affiche un bruit de 6 niveaux — d'où l'utilité du clip limit du CLAHE pour limiter ce gain.



Appliquer CLAHE directement sur les trois canaux BGR d'une image couleur est incorrect : chaque canal est traité indépendamment, ce qui crée des dominantes colorées artificielles (les régions sombres deviennent vertes ou violettes). La bonne pratique est de convertir en Lab (ou YCrCb), appliquer CLAHE uniquement sur le canal L* (luminance), puis reconverter — les couleurs sont préservées, seul le contraste change.



```
# a : image BGR (np.ndarray HxWx3 uint8)
if not isinstance(a, np.ndarray) or a.ndim < 3:
    out_a = {'erreur': 'image BGR attendue en entree a'}
else:
    # CLAHE sur le canal L de LAB (préserve les couleurs)
    lab = cv2.cvtColor(a, cv2.COLOR_BGR2LAB)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    lab[:, :, 0] = clahe.apply(lab[:, :, 0])
    out_a = cv2.cvtColor(lab, cv2.COLOR_LAB2BGR) # port image
```

7.6. Correction gamma et balance des blancs



$I_{out} = I_{in}^{\gamma}$ (sur valeurs normalisées [0, 1])

L'idée : deux rôles distincts du gamma à ne pas confondre

Le terme « correction gamma » désigne deux opérations différentes, souvent confondues dans la pratique :

- **Gamma d'encodage** ($\gamma \approx 1/2.2$, soit environ 0.45) : compresse les valeurs avant stockage. Hérité des écrans CRT dont la réponse électronique était non-linéaire (en 2.2), il s'avère que cette compression alloue aussi plus de bits aux tons sombres, là où l'œil est le plus sensible aux gradients de luminosité. C'est un codage perceptuellement efficace, conservé dans la norme sRGB.
- **Gamma d'ajustement** (outil de retouche) : $\gamma < 1$ étire les sombres et éclaircit l'image globale ; $\gamma > 1$ compresse les sombres et assombrit. Les logiciels de retouche l'utilisent pour corriger une exposition.

Le lien avec le §7.1 est direct : les images stockées en sRGB ont déjà subi un encodage $\gamma \approx 1/2.2$. Toute opération mathématique supposant une proportionnalité entre valeurs et énergie lumineuse (flou gaussien, redimensionnement bilinéaire, mélange alpha, calcul de luminance) est **mathématiquement correcte uniquement en espace linéaire**. Flouter une image en espace gamma assombrit subtilement les transitions clair/sombre — l'erreur est invisible sur un exemple isolé, mais s'accumule dans les pipelines de rendu et de compositing. Les moteurs 3D sérieux et les logiciels de compositing professionnels linéarisent systématiquement avant traitement, puis ré-encodent.

Balance des blancs : l'hypothèse « monde gris »

$$R' = R \cdot (\bar{g} / \bar{R})$$

$$B' = B \cdot (\bar{g} / \bar{B})$$

où $\bar{R}$, $\bar{g}$, $\bar{B}$ sont les moyennes des canaux R, G, B sur l'image entière.

Dérivation de l'algorithme gray world

L'algorithme repose sur une hypothèse simple : dans une scène variée, les objets de toutes les couleurs se compensent en moyenne. La valeur moyenne de l'image devrait donc être neutre (grise) sous un éclairage neutre — $\bar{R} \approx \bar{G} \approx \bar{B}$. Si l'éclairage est teinté (une lampe tungstène jaunit la scène), la moyenne dérive : $\bar{R} > \bar{B}$. L'algorithme corrige en normalisant chaque canal sur la moyenne du vert (prise comme référence, car la sensibilité du canal vert est la plus stable).

Cette hypothèse encode une croyance sur la scène. Elle est raisonnable pour des scènes générales — un bureau, une salle d'opération, une chaîne de tri industriel avec des produits variés. Elle est **violée** par toute scène dominée par une couleur : un champ de colza (jaune), une radiographie (blanc sur noir), une vue satellitaire de forêt (verte). Pour ces cas, la correction gray world introduit une distorsion inverse. Les méthodes modernes (estimation de l'illuminant par réseau de neurones) sont plus robustes, mais reposent toutes sur une hypothèse — explicite ou apprise — sur la scène ou l'éclairage.



Image avec dominante jaune (intérieur sous lampe halogène) : $\bar{R} = 180$, $\bar{G} = 170$, $\bar{B} = 120$.

$\text{gain_R} = 170 / 180 \approx 0.94$ (légère atténuation du rouge)

$\text{gain_B} = 170 / 120 \approx 1.42$ (forte amplification du bleu)

Après correction, les trois moyennes valent 170. L'image perd sa dominante jaune. Mais si $\text{gain_B} = 1.42$ est appliqué à un pixel $B = 200$, on obtient $200 \times 1.42 = 284$, qui sera écrêté à 255 — une saturation qui crée des artefacts blancs dans les zones claires. ■



L'application naïve des gains peut saturer les canaux (valeurs > 255 écrêtées), créant des zones brûlées. Il faut vérifier les gains avant application ou normaliser différemment. De plus, l'amplification du bleu ($\text{gain} > 1$) amplifie aussi le bruit du capteur sur ce canal — le plus bruyant en conditions de faible éclairage.



```
# a : image BGR (np.ndarray HxWx3 uint8)
if not isinstance(a, np.ndarray) or a.ndim < 3:
    out_a = {'erreur': 'image BGR attendue en entree a'}
else:
    img_f = a.astype(np.float32)
    # Canaux BGR : B=0, G=1, R=2
    means = img_f.reshape(-1, 3).mean(axis=0) # [mean_B, mean_G,
mean_R]
    gray_target = means[1] # référence = canal G
    (vert)
    gains = gray_target / (means + 1e-6) # éviter division par
zéro
    corrected = np.clip(img_f * gains[np.newaxis, np.newaxis, :], 0, 255)
    out_a = corrected.astype(np.uint8) # port image
```

7.7. Tableau récapitulatif – quel espace pour quelle tâche ?

Espace / outil	Ce qu'il encode	Angle mort	Usage typique
RGB (linéaire)	Énergie lumineuse physique des trois primaires	Aucun lien direct avec la perception	Calculs physiques : flou, mélange, rendu 3D
RGB (gamma sRGB)	Valeurs compressées pour affichage/storage	Faux pour tout calcul linéaire	Capture, stockage, affichage écran
Niveaux de gris (luma)	Luminance pondérée perceptuelle	Perd toute l'information chromatique	Traitement structurel, gradients, seuillage
HSV / HSL	Teinte, saturation, valeur découplées	Teinte instable pour pixels peu saturés	Segmentation couleur, sélection intuitive
CIELAB	Couleur perceptuellement uniforme (L, a, b*)	Conversion coûteuse, pas pour le stockage	Mesure ΔE , contrôle qualité, comparaison
CMYK	Quatre encres soustractives	Gamut restreint, pas bijectif avec RGB	Impression offset, prépresse

7.8. l'espace encode une hypothèse

Ce chapitre illustre le fil conducteur du livre sur un cas particulièrement parlant. Chaque espace colorimétrique n'est pas une vérité — c'est un **contrat** qui définit ce qui compte et ce qui est ignoré.

RGB linéaire encode l'hypothèse que ce qui compte est la physique de la lumière — les proportions d'énergie dans chaque bande spectrale. HSV encode l'hypothèse que ce qui compte est la teinte perceptuelle, séparée de la luminosité. CIELAB encode l'hypothèse que ce qui compte est la différence telle que l'œil la perçoit, et non telle que le capteur la mesure. CMYK encode l'hypothèse que le medium est de l'encre sur du papier, pas de la lumière sur un écran.

Choisir un espace pour une opération, c'est formuler une hypothèse sur la tâche. Mesurer un écart de couleur en RGB revient à mesurer une distance sur une carte à l'échelle variable.

Segmenter par teinte sans masquer les gris revient à croire que les gris ont une couleur. Appliquer un flou sans linéariser revient à croire que les valeurs stockées sont de l'énergie.

Ce fil — *l'espace n'est pas neutre* — est partagé par tout le livre. Le chapitre 3 (Distances) pose la même question pour les métriques : choisir une distance, c'est déclarer ce qui compte. Le chapitre 10 (Transformées) l'énonce en termes de bases : changer de base, c'est choisir où le problème devient simple. La couleur est peut-être l'exemple le plus concret et le plus quotidien de ce principe : le même pixel, dans des espaces différents, est un objet de nature entièrement différente.

La rigueur photométrique ne consiste pas à connaître toutes les formules de conversion — elle consiste à savoir, à chaque étape d'un pipeline, dans quel espace l'on se trouve et ce qu'il suppose.

CHAPITRE

Géométrie projective et caméra

8

Table des matières

8.1. Coordonnées homogènes : l'astuce fondatrice	119
8.2. Le modèle sténopé (pinhole)	121
8.3. Calibration de caméra	124
8.4. L'homographie	126
8.5. La géométrie épipolaire	129
8.6. Stéréovision et triangulation	132
8.7. Distorsion de l'objectif et erreur de reprojection	135
8.8. Tableau récapitulatif — quelle relation pour quelle situation ?	138
8.9. l'astuce homogène et la perte de profondeur : le même geste	139

Une caméra projette un monde à trois dimensions sur un capteur plat à deux dimensions. Cette opération est irréversible : une dimension entière — la profondeur — disparaît dans la projection. C'est l'enjeu central de la vision géométrique : comprendre ce que la projection préserve, ce qu'elle détruit, et comment reconstituer la troisième dimension à partir de plusieurs vues. Ce chapitre construit le modèle de caméra standard, puis les relations géométriques entre images — homographie, géométrie épipolaire, stéréovision — qui fondent la reconstruction 3D, la réalité augmentée et l'odométrie visuelle.

Le fil conducteur tient en deux assertions complémentaires : **l'astuce homogène linéarise la projection**, et **une caméra encode une perte — la profondeur**. La première affirmation explique pourquoi les matrices sont omniprésentes dans ce domaine : en ajoutant une coordonnée fictive, on transforme une opération non linéaire (la division par la profondeur) en une simple multiplication matricielle. La seconde affirmation gouverne tout le reste : la stéréovision, la géométrie épipolaire, la reconstruction *Structure from Motion* sont autant de stratégies pour récupérer ce qui a été sacrifié. Ces deux idées seront rappelées à chaque section.

Rappel des liens : les coordonnées homogènes apparaissent implicitement dans la détection de points d'intérêt et l'appariement par RANSAC (voir §5 sur le filtrage et §10 sur les transformées). La notion de perte d'information encodée par une représentation rejoint le méta-fil des chapitres précédents : comme une distance choisit ce qui compte (chapitre 3), comme un filtre pose un a priori sur le signal (chapitre 5), une caméra choisit ce qu'elle oublie.

8.1. Coordonnées homogènes : l'astuce fondatrice



(X, Y, W) représente le point cartésien $(X/W, Y/W)$, $W \neq 0$
 (x, y) en homogène : $(x, y, 1)$, ou $(\lambda x, \lambda y, \lambda)$ pour tout $\lambda \neq 0$

L'idée — pourquoi ajouter une dimension

La projection perspective, vue crue, contient une division par la profondeur : un point 3D projeté sur un plan image donne des coordonnées de la forme $x_{\text{image}} = f \cdot X / Z$. Cette division par Z est une opération **non linéaire** — ce qui brise toute la machinerie de l'algèbre linéaire (matrices, produits, compositions). L'image mentale utile est celle d'un dossier de travail encombrant qu'on ne peut pas traiter directement : en l'encodant dans un format standard, on le rend manipulable.

L'astuce homogène consiste à ajouter une coordonnée supplémentaire et à *reporter la division à la toute fin*. On ne divise plus pendant les calculs ; on multiplie des matrices. La division par la dernière coordonnée n'intervient qu'au moment de lire le résultat final. Du coup, la projection perspective, les rotations, les translations, les changements de plan — toutes ces opérations deviennent des multiplications matricielles que l'on peut chaîner, composer et inverser avec les outils standards.

Ce principe — ajouter une dimension pour rendre linéaire ce qui ne l'était pas — est une stratégie générale en mathématiques appliquées. On la retrouve dans les noyaux des machines à vecteurs de support (chapitre sur les descripteurs appris), et de façon plus abstraite dans les transformées spectrales (chapitre 10) : le bon espace rend le problème presque résolu.

Une deuxième conséquence de ce formalisme est la capacité à représenter **les points à l'infini**. En coordonnées classiques, « l'infini » est une notion glissante. En homogène, un point à l'infini s'écrit simplement $(x, y, 0)$: la division par $W=0$ est impossible, mais l'objet existe et est manipulable. Les rails d'un chemin de fer qui semblent converger vers l'horizon ont un point de fuite bien défini en homogène — ce qui fonde la détection automatique de lignes fuyantes en réalité augmentée et en robotique de navigation.



Le point cartésien $(3, 4)$ s'écrit en homogène $(3, 4, 1)$, mais aussi $(6, 8, 2)$ ou $(30, 40, 10)$: tous ces triplets sont équivalents car ils se réduisent au même point après division par W . Retour cartésien depuis $(6, 8, 2)$: $(6/2, 8/2) = (3, 4)$. Cette classe d'équivalence — « tous les multiples d'un même triplet » — est ce qu'on appelle un **point projectif**.



Après une chaîne de multiplications matricielles, la dernière coordonnée W n'est plus nécessairement 1. Avant de lire les coordonnées image, il faut toujours diviser par W . Oublier cette normalisation finale produit des coordonnées pixel aberrantes, sans message d'erreur explicite du compilateur.



```
# a : tableau Nx3 de points homogènes (np.ndarray float32)
if not isinstance(a, np.ndarray) or a.shape[1] != 3:
    out_a = {'erreur': 'entrée attendue : tableau Nx3 homogène'}
else:
    W = a[:, 2:3]
    # normalisation : division par la dernière coordonnée
    pts_cartesiens = a[:, :2] / (W + 1e-12)
    out_a = {
        'nb_points': int(len(pts_cartesiens)),
        'premier_point_x': float(pts_cartesiens[0, 0]),
        'premier_point_y': float(pts_cartesiens[0, 1]),
    }
```

8.2. Le modèle sténopé (pinhole)



$$s \cdot [u \ v \ 1]^T = K \cdot [R \ | \ t] \cdot [X \ Y \ Z \ 1]^T$$

(X, Y, Z) est un point 3D du monde, (u, v) sa projection en pixels dans l'image, s un facteur d'échelle (la profondeur, l'inconnue perdue), K la matrice des paramètres internes, $[R \ | \ t]$ la pose de la caméra dans le monde.

L'idée — la chaîne de projection comme une suite de traductions

La projection d'un point du monde réel vers un pixel de l'image peut se décomposer en trois étapes successives, chacune étant une multiplication matricielle :

monde → caméra	: $[R \ \ t]$	– extrinsèques : où est la caméra et comment orientée
caméra → image	: division par Z	– la perte de profondeur (non-linéarité cachée)
image → pixels	: K	– intrinsèques : focale, centre optique, échelle pixels

L'astuce homogène cache la division par Z dans le facteur s , si bien que toute cette chaîne s'écrit comme un seul produit matriciel. C'est ici que le fil conducteur se noue pour la première fois : **la linéarisation homogène rend la projection calculable**, mais la profondeur $s = Z_c$ reste une inconnue.

La matrice intrinsèque K

$$K = \begin{bmatrix} fx & s & cx \\ 0 & fy & cy \\ 0 & 0 & 1 \end{bmatrix}$$

Chaque terme a une signification physique précise :

- **fx, fy** : la focale exprimée en *pixels*, c'est-à-dire la distance focale métrique divisée par la taille physique d'un pixel. Ces deux valeurs peuvent différer si les pixels sont rectangulaires (capteurs anciens ou industriels).
- **cx, cy** : le *point principal*, la projection du centre optique sur le capteur. Il est proche du centre géométrique de l'image, mais rarement exactement en son centre à cause des tolérances d'assemblage.
- **s** (ici le skew) : l'obliquité des axes du capteur. Quasi toujours nul sur les caméras modernes.

Dérivation de la projection perspective

Dans le repère caméra, un point (X_c, Y_c, Z_c) se projette sur le plan image (placé à la distance focale f) par un argument de **triangles semblables** : deux triangles rectangles partagent leur sommet au centre optique, l'un avec le point 3D, l'autre avec sa projection. Les rapports de côtés donnent :

$$x = f \cdot X_c / Z_c \qquad y = f \cdot Y_c / Z_c$$

C'est ici qu'apparaît la division par Z_c — la non-linéarité. En coordonnées homogènes, on l'écrit sans diviser :

$$Z_c \cdot [x \ y \ 1]^T = \begin{bmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix}$$

La profondeur Z_c devient le facteur d'échelle **s**. On multiplie linéairement d'abord, puis on divise par la dernière coordonnée à la toute fin. ■

La perte de profondeur — conséquence centrale

Le facteur $s = Z_c$ est **inconnu** depuis une seule image. Un point proche et petit, et un point lointain et grand, peuvent projeter exactement au même pixel : ils sont sur le même rayon depuis le centre optique. Une image seule ne donne que la *direction* du rayon vers l'objet, pas sa distance. L'astuce homogène linéarise le calcul, mais ne peut pas récupérer ce qui a été perdu dans la projection. Toute la suite du chapitre — calibration, stéréo, géométrie épipolaire — est une réponse à ce manque.



Caméra de focale $f = 500$ px, centre optique $(cx, cy) = (320, 240)$. Un point 3D à $(X_c=0.3, Y_c=0.1, Z_c=2.0)$ dans le repère caméra (en mètres). Projection :

$$\begin{aligned} x &= 500 \times 0.3 / 2.0 = 75 \text{ px} & \rightarrow & \quad u = 75 + 320 = 395 \text{ px} \\ y &= 500 \times 0.1 / 2.0 = 25 \text{ px} & \rightarrow & \quad v = 25 + 240 = 265 \text{ px} \end{aligned}$$

Si le même objet se trouve à $Z_c = 1.0$ m mais deux fois plus petit ($X_c=0.15$), il projette au même pixel 395. Deux scènes radicalement différentes, même image.



OpenCV utilise la convention (colonne, ligne) pour les coordonnées pixels, mais stocke les tableaux de points 3D au format (X, Y, Z). La fonction `cv2.projectPoints` attend un tableau de forme (N, 1, 3) ou (N, 3), pas (3, N). Un transposé silencieux produit des projections incohérentes sans lever d'exception.



```
# a : image source (np.ndarray HxWx3 BGR)
# Pipeline de référence : brancher l'image sur a, observer via Inspector
# Ici on illustre la projection analytique

import numpy as np
import cv2

if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'brancher une image sur a'}
else:
    # Paramètres intrinsèques d'une caméra typique (à adapter)
    h, w = a.shape[:2]
    fx, fy = 800.0, 800.0
    cx, cy = w / 2.0, h / 2.0
    K = np.array([[fx, 0, cx],
                  [0, fy, cy],
                  [0, 0, 1]], dtype=np.float64)

    # Point 3D de test (repère caméra)
    pt3d = np.array([[[0.1, 0.05, 1.0]]], dtype=np.float64)
    rvec = np.zeros((3,1), dtype=np.float64)
    tvec = np.zeros((3,1), dtype=np.float64)
    dist = np.zeros(5, dtype=np.float64)

    pts2d, _ = cv2.projectPoints(pt3d, rvec, tvec, K, dist)
    u, v = float(pts2d[0, 0, 0]), float(pts2d[0, 0, 1])

    out_a = {
        'point_projete_u': round(u, 2),
        'point_projete_v': round(v, 2),
        'focale_fx': fx,
        'centre_cx': cx,
    }
```

8.3. Calibration de caméra

Définition — l'objectif

Estimer la matrice K et les coefficients de distorsion à partir de plusieurs images d'une mire de géométrie connue. Sans calibration, toute mesure métrique — distance réelle, angle,

volume — est impossible. L’astuce homogène rend la projection calculable ; la calibration fournit les paramètres qui la définissent.

L’idée — la méthode de Zhang (2000)

La méthode exploite un fait clé : un damier plan vu sous des angles variés fournit, pour chaque vue, une **homographie** entre le plan du damier et l’image (§8.4). Chaque homographie impose deux contraintes sur les paramètres intrinsèques — contraintes issues de l’orthogonalité des colonnes de la matrice de rotation. Avec suffisamment de vues (en pratique 10 à 20 sous des angles et des distances variés), le système se résout pour K , puis pour les poses et les distorsions, le tout raffiné par minimisation de l’erreur de reprojection (§8.7).

L’image mentale est celle d’un témoin qui observe la même scène sous plusieurs angles et tente d’en déduire les règles de la projection. Chaque vue apporte une contrainte différente ; leur intersection définit la caméra.

Ce qu’on gagne — et l’angle mort

La calibration donne un modèle précis de la caméra, permettant de corriger la distorsion et de mesurer des distances réelles. Son angle mort est la qualité de la mire et la diversité des poses : une mire imprécise ou des vues toutes frontales laissent certains paramètres mal contraints.



La qualité de la calibration dépend de la **diversité des poses**. Des damiers tous frontaux et centrés laissent la focale et les coefficients de distorsion mal contraints. Il faut incliner la mire, la placer dans les coins de l’image, varier les distances. Une erreur de reprojection finale supérieure à 0,5 pixel signale en général une couverture insuffisante. En robotique chirurgicale, où l’on mesure des distances sub-millimétriques, on vise moins de 0,1 pixel.



Une calibration sur 15 poses d’un damier 9×6 (distance inter-cases = 25 mm) donne :

$f_x = 1412 \text{ px}$, $f_y = 1410 \text{ px}$, $c_x = 964 \text{ px}$, $c_y = 546 \text{ px}$
 $k_1 = -0.32$, $k_2 = +0.11$ (distorsion radiale – barillet modéré)
 erreur de reprojection RMS = 0.28 px

Taille physique d’un pixel : focale métrique $\approx 4,2 \text{ mm}$, donc taille pixel $\approx 4,2/1411 \approx 3,0 \mu\text{m}$ — cohérent avec un capteur Sony 1/2.3”.



```
# a : image d'une vue de damier (np.ndarray HxWx3 BGR)
if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'brancher une image de damier'}
else:
    gray = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY) if a.ndim == 3 else a
    found, corners = cv2.findChessboardCorners(gray, (9, 6), None)
    if found:
        # Affinage sub-pixel des coins
        criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER,
30, 0.001)
        corners_refined = cv2.cornerSubPix(gray, corners, (11, 11), (-1,
-1), criteria)
        out_a = {
            'coins_detectes': int(len(corners_refined)),
            'premier_coin_x': float(corners_refined[0, 0, 0]),
            'premier_coin_y': float(corners_refined[0, 0, 1]),
        }
    else:
        out_a = {'erreur': 'damier 9x6 non détecté – vérifier l\'angle et
l\'éclairage'}
```

8.4. L'homographie



$x' \sim H \cdot x$ (H : matrice 3x3, définie à un facteur d'échelle près $\rightarrow$ 8 DDL)

H relie les coordonnées homogènes d'un point dans deux images. Le signe $\sim$ indique l'égalité à un facteur d'échelle près, conséquence directe de la représentation homogène.

L'idée — quand deux vues sont-elles liées par une homographie ?

Dans exactement deux situations :

1. La scène est **plane** (tous les points sur un même plan 3D), quelle que soit la caméra.
2. La caméra subit une **rotation pure** autour de son centre optique (aucune translation), quelle que soit la scène.

Hors de ces deux cas, la relation entre deux vues ne peut pas s'écrire comme une homographie — elle relève de la géométrie épipolaire (§8.5). Confondre les deux est l'erreur conceptuelle la plus fréquente du domaine.

L'image mentale pour comprendre l'homographie est celle d'un changement de point de vue sur une affiche murale. L'affiche est plane ; vue depuis deux positions différentes, ses pixels se correspondent selon une transformation bien précise. Cette transformation est linéaire en homogène, d'où la matrice 3×3 .

Dérivation (cas du plan $Z = 0$)

Pour des points sur un plan $Z = 0$ dans le repère monde, la chaîne de projection $K \cdot [R \mid t]$ appliquée à $[X \ Y \ 0 \ 1]^T$ voit la colonne de R correspondant à Z multipliée par 0 — elle disparaît. Il reste une matrice 3×3 reliant $(X, Y, 1)$ du plan à $(u, v, 1)$ de l'image. Appelons-la H_1 . Pour une deuxième caméra, H_2 de même construction. La composition $H_2 \cdot H_1^{-1}$ relie les deux images : c'est l'homographie image $\leftrightarrow$ image. ■

La structure de cette dérivation illustre le fil conducteur : l'astuce homogène fait disparaître la profondeur Z (le plan impose $Z=0$), et la perte de profondeur devient, pour une fois, un avantage — elle simplifie la relation en une matrice carrée.

Les 8 degrés de liberté

H a 9 entrées mais est définie à l'échelle près $\rightarrow$ 8 DDL effectifs. Chaque correspondance de points fournit 2 équations $\rightarrow$ **4 correspondances** en position générale suffisent à déterminer H . En pratique, on en utilise davantage avec un ajustement aux moindres carrés (DLT, *Direct Linear Transform*) et un rejet des aberrants par RANSAC.

Applications

Assemblage de panoramas (rotation pure de la caméra $\rightarrow$ les vues se recollent par homographie) ; rectification de documents photographiés en biais (le papier est plan) ; réalité augmentée sur surface plane (incruster un objet virtuel sur une table) ; correction de perspective ; *marker tracking* en robotique.



Photographier une affiche rectangulaire de biais : ses quatre coins, rectangulaires dans la réalité, forment un quadrilatère quelconque dans l'image. L'homographie qui envoie ce quadrilatère vers un rectangle redresse l'affiche « de face ». Quatre coins = quatre correspondances = H déterminée. Si l'affiche mesure 60×40 cm et apparaît dans l'image déformée, l'application de H donne une vue frontale aux proportions correctes — c'est la base des systèmes de lecture de tableaux blancs en visioconférence ou de reconnaissance de plaques d'immatriculation.



`cv2.findHomography` avec `cv2.RANSAC` rejette les aberrants, mais le seuil (5ème argument, en pixels) doit être adapté à la qualité des correspondances. Un seuil trop large accepte de mauvaises correspondances et dégrade H. Un seuil trop strict rejette des points corrects si la localisation des coins est bruyante. En télédétection (orthophoto de zones cultivées), les points de contrôle peuvent avoir une incertitude de 2 à 3 pixels ; le seuil doit être ajusté en conséquence.



```
# a : image source, b : image destination (np.ndarray HxWx3 BGR)
if not isinstance(a, np.ndarray) or not isinstance(b, np.ndarray):
    out_a = {'erreur': 'brancher image_source sur a et image_cible sur b'}
else:
    gray_a = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY)
    gray_b = cv2.cvtColor(b, cv2.COLOR_BGR2GRAY)

    # Détection et appariement ORB
    orb = cv2.ORB_create(1000)
    kp1, des1 = orb.detectAndCompute(gray_a, None)
    kp2, des2 = orb.detectAndCompute(gray_b, None)

    if des1 is None or des2 is None or len(kp1) < 4:
        out_a = {'erreur': 'pas assez de points detectes'}
    else:
        bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
        matches = bf.match(des1, des2)
        matches = sorted(matches, key=lambda m: m.distance)[:50]

        pts_src = np.float32([kp1[m.queryIdx].pt for m in
matches]).reshape(-1, 1, 2)
        pts_dst = np.float32([kp2[m.trainIdx].pt for m in
matches]).reshape(-1, 1, 2)

        H, mask = cv2.findHomography(pts_src, pts_dst, cv2.RANSAC, 5.0)
        if H is not None:
            warped = cv2.warpPerspective(a, H, (b.shape[1], b.shape[0]))
            inliers = int(mask.sum()) if mask is not None else 0
            out_a = warped
            out_b = {'inliers': inliers, 'total_matches': len(matches)}
        else:
            out_a = {'erreur': 'homographie non calculable -
correspondances insuffisantes'}
```

8.5. La géométrie épipolaire

Définition — la matrice fondamentale F

$$x'^T \cdot F \cdot x = 0$$

pour tout couple de points correspondants $x \leftrightarrow x'$ (en pixels, coordonnées homogènes). F est une matrice 3×3 de rang 2, définie à l'échelle près $\rightarrow$ **7 DDL**.

L'idée — la contrainte épipolaire

Deux caméras observent la même scène depuis des positions différentes. Un point 3D quelconque et les deux centres optiques définissent un **plan épipolaire**. L'intersection de ce plan avec chaque image est une **ligne épipolaire**. La contrainte fondamentale est la suivante : le correspondant d'un point de l'image 1 se trouve *forcément sur la ligne épipolaire* dans l'image 2.

L'image mentale est celle d'un système de coordonnées partagé entre les deux caméras. Sans cette contrainte, chercher le correspondant d'un point revient à parcourir toute l'image 2 (un problème 2D). Avec la contrainte épipolaire, la recherche se réduit à une ligne (un problème 1D) — un gain considérable pour les algorithmes de mise en correspondance en stéréovision ou en reconstruction 3D.

La perte de profondeur est exactement ce qui fonde cette contrainte : puisqu'une image seule ne donne que la *direction* d'un rayon, tous les points possibles le long de ce rayon se projettent sur une ligne dans l'image 2. Cette ligne est la ligne épipolaire.

L'interprétation de F

Le produit $F \cdot x$ calcule directement l'équation de la ligne épipolaire dans l'image 2 associée au point x de l'image 1. La contrainte $x'^T F x = 0$ dit simplement « x' est sur cette ligne ». F encode toute la géométrie relative des deux vues **sans connaître les caméras** — uniquement à partir de correspondances de pixels.

La matrice essentielle E

$$E = K'^T \cdot F \cdot K = [t]_{\times} \cdot R$$

Quand les caméras sont **calibrées** (K et K' connues), on peut passer en coordonnées normalisées. F devient E , la **matrice essentielle**. L'avantage décisif : E se **décompose** en la rotation R et la translation t entre les deux vues — c'est la pose relative des caméras, le point de départ de toute reconstruction. La notation $[t]_{\times}$ désigne la matrice antisymétrique associée au produit vectoriel par t .

En SLAM visuel (un robot qui cartographie son environnement en navigant), E est estimée frame par frame pour calculer le déplacement incrémental de la caméra. L'accumulation de ces poses reconstitue la trajectoire du robot.

Estimation : l'algorithme des 8 points

Chaque correspondance de points fournit une équation linéaire dans les 9 entrées de F . **8 correspondances** suffisent (la 9ème contrainte étant l'échelle), d'où le nom. En pratique : normaliser les coordonnées (méthode de Hartley pour améliorer le conditionnement numérique), résoudre par SVD, puis imposer le rang 2 (annuler la plus petite valeur singulière). RANSAC gère les aberrants.

Le « 5-point algorithm » de Nistér (2004) exploite la calibration pour estimer E directement avec seulement 5 correspondances — précieux en odométrie visuelle temps réel.



La normalisation de Hartley (centrer et normaliser les coordonnées avant le calcul, renormaliser après) est **indispensable en pratique**. Sans elle, les coordonnées pixels (typiquement entre 0 et 1920) créent une matrice mal conditionnée et les solutions numériques sont instables. Ce détail est souvent omis dans les implémentations de démonstration mais critique en production.



Deux caméras séparées horizontalement. Un point de l'image 1 situé en (640, 400) pixels. Sa ligne épipolaire dans l'image 2 est approximativement horizontale (si les caméras sont bien alignées) et passe par la même hauteur. Sans rectification, la ligne peut être légèrement inclinée. La recherche du correspondant se fait le long de cette ligne, pas sur toute l'image.



```
# a : image gauche, b : image droite (np.ndarray HxWx3 BGR)
if not isinstance(a, np.ndarray) or not isinstance(b, np.ndarray):
    out_a = {'erreur': 'brancher image_gauche sur a, image_droite sur b'}
else:
    gray_a = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY)
    gray_b = cv2.cvtColor(b, cv2.COLOR_BGR2GRAY)

    orb = cv2.ORB_create(500)
    kp1, des1 = orb.detectAndCompute(gray_a, None)
    kp2, des2 = orb.detectAndCompute(gray_b, None)

    if des1 is None or des2 is None or len(kp1) < 8:
        out_a = {'erreur': 'moins de 8 points – impossible d’estimer F'}
    else:
        bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
        matches = sorted(bf.match(des1, des2), key=lambda m: m.distance)
        [:100]

        pts1 = np.float32([kp1[m.queryIdx].pt for m in matches])
        pts2 = np.float32([kp2[m.trainIdx].pt for m in matches])

        F, mask = cv2.findFundamentalMat(pts1, pts2, cv2.FM_RANSAC, 3.0)
        if F is not None:
            inliers = int(mask.ravel().sum()) if mask is not None else 0
            out_a = {
                'inliers_F': inliers,
                'total_correspondances': len(matches),
                'F_rang': int(np.linalg.matrix_rank(F)),
            }
        else:
            out_a = {'erreur': 'matrice fondamentale non calculable'}
```

8.6. Stéréovision et triangulation

Définition — la formule de profondeur

$$Z = f \cdot B / d$$

f est la focale (en pixels), B la **baseline** (distance inter-caméras en mètres), d la **disparité** (différence de position horizontale du même point dans les deux images, en pixels).

L'idée — retrouver la profondeur perdue

La vision binoculaire humaine fonctionne sur ce principe : chaque œil voit le monde sous un angle légèrement différent, et le cerveau interprète cette différence de position — la disparité — comme une information de profondeur. Plus un objet est proche, plus son image se décale entre l’œil gauche et l’œil droit.

En stéréovision artificielle, deux caméras calibrées séparées d’une baseline B jouent le rôle des deux yeux. La mesure de la disparité d (le décalage horizontal d’un même point entre les deux images) permet de recalculer la profondeur. C’est la stratégie directe pour récupérer la profondeur perdue lors de la projection : au lieu d’une seule vue qui perd Z , on en prend deux simultanément.

Dérivation

Dans chaque caméra, par triangles semblables : $x_gauche = f \cdot X / Z$ et $x_droite = f \cdot (X - B) / Z$. La disparité :

$$\begin{aligned} d &= x_gauche - x_droite \\ &= f \cdot X / Z - f \cdot (X - B) / Z \\ &= f \cdot B / Z \end{aligned}$$

D’où $Z = f \cdot B / d$. ■

Les trois conséquences de la relation inverse $Z \propto 1/d$

La relation entre profondeur et disparité est une inverse, non une proportionnelle. Cela a trois conséquences pratiques importantes :

- **Disparité nulle = infini.** Les objets très lointains ont une disparité proche de zéro. La stéréo ne mesure pas bien les grandes distances — c’est son angle mort fondamental. En astronomie ou en télédétection satellites (orbite à 500 km), la stéréovision est inutilisable ; on lui préfère le temps de vol ou la mesure radar.
- **Précision dégressive.** Une erreur fixe sur d ($\pm 0,5$ px) génère une erreur sur Z qui **croît comme Z^2** . À courte distance, la stéréo est précise ; elle devient grossière pour les objets lointains. En robotique chirurgicale, la stéréo est précise au sous-millimètre à 30 cm de distance ; en véhicule autonome, elle est utile jusqu’à 30-50 m mais supplantée par le LiDAR au-delà.
- **Compromis de la baseline.** Une grande baseline B améliore la précision en profondeur (disparités plus grandes) mais réduit le recouvrement entre vues et complique l’appariement. Choisir B est un arbitrage de conception propre à chaque application.



Caméra stéréo : $f = 800$ px, $B = 0,12$ m. Un point détecté à disparité $d = 40$ px :

$$Z = 800 \times 0,12 / 40 = 2,4 \text{ m}$$

Le même système, pour un point plus lointain à $d = 4$ px :

$$Z = 800 \times 0,12 / 4 = 24 \text{ m}$$

Une erreur de ± 1 px à $d = 4$ donne Z entre $800 \times 0,12 / 5 = 19,2$ m et $800 \times 0,12 / 3 = 32$ m — une incertitude de ± 6 m. L'erreur en Z^2 est flagrante : à $d = 40$, la même erreur de ± 1 px donne Z entre 2,18 et 2,67 m, soit $\pm 0,25$ m seulement.

Le problème d'appariement

Toute la difficulté de la stéréo est d'**appairer** chaque pixel gauche avec son homologue droit pour mesurer d . La rectification (via la géométrie épipolaire) ramène la recherche sur une même ligne horizontale. Les zones sans texture (mur uni, ciel), les occlusions (un bord d'objet visible d'un côté seulement) et les reflets spéculaires sont les ennemis classiques. Les méthodes globales (graph cut, SGM — *Semi-Global Matching*) régularisent la carte de disparité en imposant une cohérence spatiale.



`cv2.StereoSGBM_create` renvoie une disparité en unités de $1/16$ de pixel (entier multiplié par 16). Il faut diviser par 16,0 avant d'appliquer la formule $Z = fB/d$. Un oubli de ce facteur produit des profondeurs 16 fois trop petites — difficile à détecter sans valeur de référence.



```
# a : image gauche rectifiée, b : image droite rectifiée (np.ndarray
HxWx3 BGR)
if not isinstance(a, np.ndarray) or not isinstance(b, np.ndarray):
    out_a = {'erreur': 'brancher image_gauche sur a, image_droite sur b'}
else:
    gray_l = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY) if a.ndim == 3 else a
    gray_r = cv2.cvtColor(b, cv2.COLOR_BGR2GRAY) if b.ndim == 3 else b

    stereo = cv2.StereoSGBM_create(
        minDisparity=0,
        numDisparities=64,    # multiple de 16
        blockSize=9,
        P1=8 * 9 * 9,
        P2=32 * 9 * 9,
    )
    disp_raw = stereo.compute(gray_l, gray_r).astype(np.float32)
    disp = disp_raw / 16.0    # convertir en pixels réels

    # Paramètres stéréo typiques (à calibrer)
    f_px, B_m = 800.0, 0.12
    valid = disp > 1.0
    if valid.any():
        depth = np.zeros_like(disp)
        depth[valid] = f_px * B_m / disp[valid]

        out_a = depth    # carte de profondeur (float32, en mètres)
        out_b = {
            'profondeur_min_m': float(depth[valid].min()),
            'profondeur_max_m': float(depth[valid].max()),
            'profondeur_mediane_m': float(np.median(depth[valid])),
        }
    else:
        out_a = {'erreur': 'disparité nulle partout – vérifier la
rectification'}
```

8.7. Distorsion de l'objectif et erreur de reprojection

Définition — modèle de distorsion radiale

$$\begin{aligned} x_d &= x \cdot (1 + k_1 r^2 + k_2 r^4 + k_3 r^6) \\ y_d &= y \cdot (1 + k_1 r^2 + k_2 r^4 + k_3 r^6) \end{aligned} \quad r^2 = x^2 + y^2$$

(x, y) sont les coordonnées normalisées (avant mise en pixels), (x_d, y_d) les coordonnées distordues. Les k_i sont les coefficients de distorsion radiale.

L'idée — les lignes droites deviennent courbes

Les objectifs réels dévient du modèle sténopé idéal. La déviation la plus courante est la distorsion radiale : les lignes droites du monde projetées dans l'image apparaissent légèrement courbes, surtout en périphérie. L'image mentale est celle d'une carte géographique : toute projection d'une sphère sur un plan déforme les distances et les angles, plus fortement aux bords. L'objectif photographique fait de même.

Deux types s'opposent :

- **Distorsion en barillet** ($k_1 < 0$) : les bords de l'image s'incurvent vers l'extérieur, comme si l'image gonflait. Typique des grands-angles et des objectifs de surveillance. Un horizon droit apparaît bombé.
- **Distorsion en coussinet** ($k_1 > 0$) : les bords s'incurvent vers l'intérieur, comme si l'image se rétractait. Courante sur les téléobjectifs.

Le modèle polynomial en puissances paires de r (r^2, r^4, r^6) corrige ces déviations. Une **distorsion tangentielle** s'ajoute parfois si l'objectif n'est pas parfaitement parallèle au capteur — paramètres p_1, p_2 dans OpenCV.

L'erreur de reprojection — la métrique reine

$$e = \sum_i \| x_{i_observé} - \hat{x}(P, X_i) \|^2$$

Somme, sur tous les points de toutes les images, du carré de la distance entre le point **mesuré** dans l'image et le point **reprojeté** par le modèle (la caméra P appliquée au point 3D X_i).

L'erreur de reprojection mesure le tout écart du modèle à la réalité dans l'unité qui compte — le **pixel**. Elle est la quantité minimisée par :

- la **calibration** (ajuster K et la distorsion pour que la mire reprojetée colle aux coins détectés) ;
- le **bundle adjustment** (raffiner simultanément toutes les poses de caméra et toutes les positions 3D pour minimiser l'erreur sur l'ensemble d'une reconstruction *Structure from Motion*).

C'est une optimisation non linéaire (algorithme de Levenberg-Marquardt typiquement). Une erreur de reprojection finale faible (< 1 px) est le signe d'une reconstruction géométriquement cohérente. Le fil conducteur revient ici : l'astuce homogène a permis de formuler le problème en matrices ; l'erreur de reprojection mesure ce que la perte de profondeur coûte en précision de modélisation.



Lors d'une calibration, le modèle prédit qu'un coin de damier devrait se projeter en (320,0, 240,0) px, mais il est détecté en (320,8, 239,4). Contribution de ce point :

$$\begin{aligned} e_{\text{point}} &= (320,8 - 320,0)^2 + (239,4 - 240,0)^2 \\ &= 0,64 + 0,36 \\ &= 1,00 \text{ px}^2 \quad \rightarrow \quad \text{RMS} = 1,0 \text{ px sur ce point} \end{aligned}$$

Moyennée sur l'ensemble des coins et des vues, la RMS résume la qualité du modèle. En microscopie de fluorescence, où l'on mesure des distances cellulaires sub-micrométriques, on vise une $\text{RMS} < 0,1 \text{ px}$.



`cv2.undistort` applique la correction de distorsion pixel par pixel via une interpolation. L'appeler sur chaque frame d'une vidéo à 30 fps est coûteux. La bonne pratique est de précalculer les cartes de remapping avec `cv2.initUndistortRectifyMap` une seule fois, puis d'appliquer `cv2.remap` à chaque frame — ce qui divise le coût par 3 à 5 selon le matériel.



```
# a : image distordue (np.ndarray H×W×3 BGR)
if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'brancher une image sur a'}
else:
    h, w = a.shape[:2]
    # Paramètres de calibration typiques (à remplacer par votre
    calibration)
    fx, fy = 1412.0, 1410.0
    cx, cy = w / 2.0, h / 2.0
    K = np.array([[fx, 0, cx],
                  [0, fy, cy],
                  [0, 0, 1]], dtype=np.float64)
    dist = np.array([-0.32, 0.11, 0.0, 0.0, 0.0], dtype=np.float64)

    # Correction de distorsion
    undistorted = cv2.undistort(a, K, dist)

    # Calcul de l'erreur de reprojection sur un point test
    pt3d = np.array([[[0.1, 0.05, 1.0]]], dtype=np.float64)
    rvec = tvec = np.zeros((3, 1), dtype=np.float64)
    pt_proj_dist, _ = cv2.projectPoints(pt3d, rvec, tvec, K, dist)
    pt_proj_ideal, _ = cv2.projectPoints(pt3d, rvec, tvec, K,
np.zeros(5))
    err = float(np.linalg.norm(pt_proj_dist[0,0] - pt_proj_ideal[0,0]))

    out_a = undistorted
    out_b = {
        'erreur_reprojection_test_px': round(err, 4),
        'k1': float(dist[0]),
        'k2': float(dist[1]),
    }
```

8.8. Tableau récapitulatif – quelle relation pour quelle situation ?

Outil / transform	Ce qu'il encode	Angle mort	Usage
Coordonnées homogènes	linéarisation de la division par Z ; points à l'infini	ne récupère pas la profondeur	base de toute la géométrie projective

Outil / transform	Ce qu'il encode	Angle mort	Usage
Modèle sténopé $K[R t]$	projection 3D→2D complète	perd Z (ambiguïté rayon)	rendu, RA, simulation
Calibration (Zhang)	K + distorsion	nécessite une mire, diversité de poses	toute mesure métrique
Homographie H	relation entre deux vues planes ou rotation pure	invalidé si scène non plane + translation	panoramas, rectification, RA sur plan
Matrice fondamentale F	géométrie épipolaire (non calibré)	ne donne pas R, t	vérification, appariement guidé
Matrice essentielle E	pose relative R, t (calibré)	translation à échelle près	SLAM, SfM, odométrie visuelle
Stéréo $Z = fB/d$	profondeur métrique	imprécis loin, zones sans texture	robotique courte distance, chirurgie
Distorsion k_i + erreur reprojection	modèle d'écart au sténopé idéal	ne corrige pas le bougé ni la mise au point	calibration, reconstruction précise

8.9. l'astuce homogène et la perte de profondeur : le même geste

Deux idées structurent tout ce chapitre, et elles forment en réalité un seul et même geste intellectuel.

Linéariser par l'homogène. La projection perspective contient une division par Z — une opération non linéaire qui brise la machinerie matricielle. En ajoutant une coordonnée fictive, on reporte cette division à la toute fin et toute la géométrie redevient affaire de matrices : projection, homographie, composition de transformations. C'est la même stratégie que le noyau en apprentissage automatique (*ajouter une dimension pour rendre linéaire ce qui ne l'était pas*) ou que le passage en domaine de Fourier (chapitre 10 : *changer de base pour que le problème devienne simple*).

La profondeur est l'information sacrifiée. Une caméra projette un rayon, pas un point : la coordonnée Z est perdue, absorbée dans le facteur d'échelle s . L'astuce homogène ne la récupère pas — elle la dissimule proprement. Tout le reste du chapitre — calibration, stéréo, géométrie épipolaire, *bundle adjustment* — n'est qu'une collection de stratégies pour **récupérer cette dimension perdue** en combinant plusieurs vues ou en imposant des

contraintes géométriques. La géométrie de la vision 3D est, fondamentalement, l'art de défaire une projection.

Ces deux leçons rejoignent le méta-fil de l'ouvrage : comme une distance encode une hypothèse sur ce qui compte (chapitre 3), comme un filtre encode un a priori sur le signal (chapitre 5), comme un descripteur choisit ce qu'il voit et ce qu'il oublie (chapitre 1), **une caméra encode une perte — et la connaître précisément permet de l'inverser**. Dans chaque domaine — filtrage, description, projection — le bon cadre rend le problème presque résolu, et tout choix de représentation est une décision sur ce qui sera sacrifié.

CHAPITRE

Flot optique et mouvement

9

Table des matières

9.1. La contrainte du flot optique	142
9.2. Le problème d'ouverture	145
9.3. Lucas-Kanade : la solution locale	146
9.4. Horn-Schunck : la solution globale	150
9.5. Flot épars vs flot dense : choisir	154
9.6. Tableau récapitulatif — combler l'équation manquante	157
9.7. le problème mal posé, l'a priori salvateur, et le méta-fil du livre	157

Entre deux images successives d'une vidéo, les choses bougent. Un piéton traverse un carrefour, un bras de robot pivote, une cellule migre sous le microscope : dans chaque cas, l'image suivante diffère de la précédente, et cette différence porte l'empreinte du mouvement. Le flot optique est le champ de vecteurs qui tente de lire cette empreinte, pixel par pixel, en associant à chaque point une direction et une vitesse apparentes.

Le fil conducteur de ce chapitre tient en une phrase : **le problème est mal posé — une seule équation pour deux inconnues — et chaque méthode est un a priori différent ajouté pour le résoudre**. La contrainte du flot optique fournit exactement une équation par pixel, mais un déplacement dans le plan a deux composantes (u , v). Ce déficit d'information, que les mathématiciens nomment sous-détermination, est irréductible par les données seules : il faut ajouter une hypothèse sur la nature du mouvement. Lucas-Kanade suppose que le mouvement est localement constant sur un petit voisinage ; Horn-Schunck suppose qu'il varie doucement sur toute l'image. Ces deux a priori ne sont pas des artifices : ils reflètent deux visions du monde physique, avec chacune leurs forces et leurs zones d'échec.

Ce fil — **données insuffisantes + a priori = solution déterminée** — n'est pas propre au mouvement. On l'a vu en filtrage (chapitre 5 : un filtre est un a priori sur le signal) et en distance (chapitre 3 : choisir une distance, c'est déclarer ce qui compte). Le flot optique en est l'archétype le plus explicite : ici, l'insuffisance est comptable, un contre deux, et les a priori sont formulés en équations.

Lien avec les chapitres précédents. La stéréovision (chapitre 8) apparie deux vues décalées dans l'**espace** pour estimer la profondeur ; le flot optique apparie deux vues décalées dans le **temps** pour estimer le mouvement. Même structure de correspondance, dimension différente. Par ailleurs, le tenseur de structure du gradient (chapitre 6, §6.4) réapparaît ici au cœur de Lucas-Kanade : les régions où le flot est fiable sont exactement celles que Shi-Tomasi identifie comme des coins. La cohérence entre chapitres est mathématique, pas fortuite.

| 9.1. La contrainte du flot optique

L'hypothèse de constance de la luminosité

Le point de départ du flot optique est une hypothèse physique d'apparence innocente : un point physique conserve sa luminosité d'une image à la suivante. Si ce point se trouve en (x, y) à l'instant t et se déplace de (dx, dy) pendant l'intervalle dt , alors :

$$I(x, y, t) = I(x + dx, y + dy, t + dt)$$

En imagerie médicale, cette hypothèse est vérifiée si l'éclairage reste constant et si le tissu ne se déforme pas trop entre deux images. En vidéosurveillance, elle tient tant que la caméra n'est pas éblouie. En robotique mobile, elle est violée si le robot tourne brusquement et que des reflets apparaissent. C'est l'a priori zéro, celui sur lequel tout le reste se construit.

Dérivation de l'équation différentielle (OFCE)

On développe le terme de droite de l'égalité précédente en série de Taylor au premier ordre. L'image mentale utile est celle d'une surface lisse : si l'on se déplace d'un tout petit pas (dx , dy , dt) sur cette surface, la variation d'altitude est approximativement égale à la pente dans chaque direction multipliée par le pas effectué dans cette direction. Mathématiquement :

$$I(x+dx, y+dy, t+dt) \approx I(x,y,t) + (\partial I/\partial x) \cdot dx + (\partial I/\partial y) \cdot dy + (\partial I/\partial t) \cdot dt$$

En reportant dans l'égalité de constance, le terme $I(x,y,t)$ s'annule des deux côtés :

$$(\partial I/\partial x) \cdot dx + (\partial I/\partial y) \cdot dy + (\partial I/\partial t) \cdot dt = 0$$

On divise par dt . Les termes dx/dt et dy/dt sont les composantes de la vitesse (u , v) du point. On note $I_x = \partial I/\partial x$, $I_y = \partial I/\partial y$, $I_t = \partial I/\partial t$:

$$I_x \cdot u + I_y \cdot v + I_t = 0 \quad (\text{équation du flot optique, OFCE})$$

Cette équation est la contrainte fondamentale. Elle s'écrit aussi $\nabla I \cdot (u,v) = -I_t$, ce qui se lit : « la projection de la vitesse sur la direction du gradient est entièrement déterminée par les intensités ». ■

Ce que les termes signifient

- I_x , I_y : le gradient spatial (voir chapitre 6) — comment l'intensité varie dans l'espace à l'instant t . Ce sont des nombres calculables directement sur chaque image.
- I_t : la dérivée temporelle — comment l'intensité change entre les deux images, à position fixée. C'est simplement la différence $I(t+1) - I(t)$ à chaque pixel.
- (u, v) : l'inconnue cherchée — le déplacement horizontal et vertical du point.

Le problème mal posé apparaît ici clairement : une seule équation (l'OFCE) pour deux inconnues (u et v). Les données disponibles — les intensités des deux images — ne suffisent pas à lever l'ambiguïté. Il faudra un a priori.



Un bord vertical dans une image de scanner médical présente un fort gradient horizontal ($I_x = 50$) et aucun gradient vertical ($I_y = 0$). Entre deux images successives, la région s'éclaircit légèrement ($I_t = -100$) :

$$50 \cdot u + 0 \cdot v + (-100) = 0 \quad ? \quad u = 2, \quad v \text{ indéterminé}$$

On sait que le bord s'est déplacé de 2 pixels horizontalement. Mais v reste totalement libre : faire glisser ce bord vertical le long de lui-même ne modifie aucune intensité, donc aucune mesure ne peut le détecter. C'est la manifestation directe du problème mal posé : les données ne contraignent qu'une direction.



L'OFCE repose sur un développement de Taylor au premier ordre, valable uniquement pour de **petits déplacements** (typiquement inférieurs à 1–2 pixels). Un objet rapide — un bras robotique en accélération, une balle frappée — produit des déplacements inter-images de 10 à 30 pixels qui violent cette hypothèse. Les méthodes pyramidales (§9.3) résolvent ce problème, mais il faut avoir en tête que la dérivation ci-dessus s'effondre silencieusement pour les grands déplacements.



Ce premier bloc mesure I_x , I_y , I_t en un point et vérifie numériquement l'OFCE. L'entrée **a** reçoit une image BGR du port image (bleu) à l'instant t , l'entrée **b** reçoit l'image à $t+1$.

```
# a : image BGR à t (np.ndarray H×W×3 uint8)
# b : image BGR à t+1 (np.ndarray H×W×3 uint8)
if not isinstance(a, np.ndarray) or not isinstance(b, np.ndarray):
    out_a = {'erreur': 'deux images requises (a=frame t, b=frame t+1)'}
else:
    g1 = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY).astype(np.float32)
    g2 = cv2.cvtColor(b, cv2.COLOR_BGR2GRAY).astype(np.float32)

    # Gradients spatiaux (Sobel sur la moyenne des deux frames)
    gm = (g1 + g2) / 2.0
    Ix = cv2.Sobel(gm, cv2.CV_32F, 1, 0, ksize=3)
    Iy = cv2.Sobel(gm, cv2.CV_32F, 0, 1, ksize=3)
    It = g2 - g1 # dérivée temporelle

    # Résidu OFCE sur un pixel central (à titre de vérification)
    cy, cx = g1.shape[0] // 2, g1.shape[1] // 2
    out_a = {
        'Ix_centre': float(Ix[cy, cx]),
        'Iy_centre': float(Iy[cy, cx]),
        'It_centre': float(It[cy, cx]),
        'residual_ofce_si_u=0_v=0': float(It[cy, cx]), # = It si
        (u,v)=(0,0)
    }
```

9.2. Le problème d'ouverture

Énoncé

L'équation OFCE contraint la composante du mouvement **parallèle au gradient** (perpendiculaire aux contours). La composante le long du contour reste totalement libre. C'est le **problème d'ouverture** (*aperture problem*), et il s'agit d'une limitation fondamentale inhérente aux données, pas d'un défaut algorithmique.

L'intuition de la fente

L'image mentale la plus parlante est celle d'un observateur qui regarde un mur à travers un étroit judas. Une barre oblique passe derrière ce judas. L'observateur ne voit qu'une petite

portion du bord et constate que l'image dans le judas se déplace perpendiculairement à la barre. Pourtant, la barre pourrait se déplacer dans n'importe quelle direction faisant un angle non nul avec elle-même et produire exactement la même image dans le judas. Il est mathématiquement impossible de distinguer ces cas en regardant uniquement par ce trou. Ce n'est pas une illusion optique subjective : c'est une contrainte d'information objective vérifiable à tout observateur, humain ou machine.

L'analogie physique avec le déficit d'information du §9.1 est directe : la fente, c'est la fenêtre d'observation locale ; le mur, c'est l'image complète dont on se prive délibérément pour gagner en rapidité de calcul.

Où le flot est-il fiable ?

Le mouvement n'est pleinement déterminé qu'aux endroits où le gradient varie dans **plusieurs directions** — c'est-à-dire aux **coins** (rappel du chapitre 6, §6.4 : valeurs propres du tenseur de structure). Sur un contour droit, le gradient pointe toujours dans la même direction et le flot est ambigu. Dans une région uniforme (ciel, fond uni), le gradient est nul et le flot est totalement indéterminé. Ce constat n'est pas anodin : il relie directement le flot optique aux détecteurs de coins. On suit ce qui est suivable, et ce qui est suivable est exactement ce que Shi-Tomasi identifie.

Ce constat réaffirme le fil conducteur : les données (une seule équation) sont insuffisantes partout sauf aux coins. L'a priori que chaque méthode ajoute détermine comment elle traite les zones où les données se taisent.



Il est courant de vouloir calculer le flot sur toute l'image pour avoir un résultat « complet ». Le résultat dans les régions uniformes ou le long des contours droits n'est pas erroné au sens d'un bug — il est **indéterminé** : n'importe quelle valeur satisfait l'OFCE. Des vecteurs numériquement présents dans ces zones ne mesurent rien de réel. Inspecter la carte des valeurs propres du tenseur de structure avant d'interpréter un champ de flot est toujours une bonne pratique.

9.3. Lucas-Kanade : la solution locale

L'hypothèse ajoutée : mouvement constant par fenêtre

Lucas-Kanade comble le déficit d'information par un a priori spatial : le mouvement est **constant sur un petit voisinage** d'une fenêtre de $n \times n$ pixels. Tous les pixels de cette fenêtre partagent le même déplacement (u, v) . On passe alors d'une seule équation à un

système surdéterminé : n^2 équations pour 2 inconnues. Avec $n = 5$, ce sont 25 équations pour 2 inconnues — on dispose de 23 degrés de liberté pour « voter » et réduire l'influence du bruit.

C'est l'a priori le plus local et le plus simple qui soit : on suppose que le voisinage d'un point est rigide à l'échelle de la fenêtre. Pour un coin d'un panneau de signalisation ou d'un marqueur industriel, c'est très raisonnable. Pour la déformation d'un tissu biologique ou le battement d'un pavillon d'oreille, c'est une approximation grossière.

Dérivation par moindres carrés

On empile l'OFCE pour les n^2 pixels de la fenêtre dans un système matriciel :

$$A \cdot [u \ v]^T = b$$

$$\text{avec } A = \begin{bmatrix} I_{x1} & I_{y1} \\ I_{x2} & I_{y2} \\ \vdots & \vdots \end{bmatrix} \quad (\text{matrice } n^2 \times 2, \text{ une ligne par pixel de la fenêtre})$$

$$b = -[I_{t1}, I_{t2}, \dots]^T \quad (\text{vecteur } n^2 \times 1)$$

Le système est surdéterminé (plus d'équations que d'inconnues). On cherche la solution aux **moindres carrés**, c'est-à-dire le (u, v) qui minimise la somme des carrés des résidus de l'OFCE sur toute la fenêtre. Les équations normales donnent :

$$A^T A \cdot [u \ v]^T = A^T b$$

$$\boxed{?} \quad [u \ v]^T = (A^T A)^{-1} A^T b$$

La matrice 2×2 au cœur du calcul est :

$$A^T A = \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix}$$

Le lien profond avec les coins

Cette matrice est exactement le **tenseur de structure** du chapitre 6 (§6.4). Sa décomposition en valeurs propres $\lambda_1 \geq \lambda_2 \geq 0$ diagnostique la fiabilité du flot :

$$\begin{array}{lll} \lambda_1 \text{ grande}, \lambda_2 \text{ grande} & (\text{coin}) & \rightarrow A^T A \text{ inversible} \rightarrow \text{flot } (u, v) \text{ unique et stable} \\ \lambda_1 \text{ grande}, \lambda_2 \approx 0 & (\text{contour}) & \rightarrow A^T A \text{ quasi-singulière} \rightarrow \text{ambiguïté (ouverture)} \\ \lambda_1 \approx 0, \lambda_2 \approx 0 & (\text{plat}) & \rightarrow A^T A \approx 0 \rightarrow \text{flot totalement indéterminé} \end{array}$$

Le problème d'ouverture du §9.2 réapparaît ici sous forme algébrique : le flot est fiable si et seulement si $A^T A$ est inversible, ce qui exige deux grandes valeurs propres, ce qui exige un coin. C'est pourquoi Lucas-Kanade est presque toujours appliqué aux **points de Shi-Tomasi** (goodFeaturesToTrack) plutôt qu'à tous les pixels : on ne suit que là où le suivi a

du sens. Le résultat est un flot **épars** — sur quelques centaines de points — mais chaque vecteur est fiable.

Le piège des grands déplacements : la pyramide

Le développement de Taylor du §9.1 n'est valable que pour de petits déplacements, typiquement inférieurs à 1–2 pixels. Un drone qui avance rapidement ou un curseur de souris bougé vivement peuvent produire des déplacements inter-images de 15 à 40 pixels qui rendent la linéarisation incorrecte.

La solution est la **pyramide d'images** (image pyramid) : on construit plusieurs versions de plus en plus réduites de chaque image (en divisant la résolution par 2 à chaque niveau). À la résolution la plus basse, un déplacement de 20 pixels réels ne représente plus que 2 ou 3 pixels, ce qui satisfait la contrainte de petits déplacements. On calcule le flot à ce niveau grossier, puis on l'utilise comme initialisation pour affiner au niveau suivant, et ainsi de suite jusqu'à la pleine résolution. C'est le principe de `calcOpticalFlowPyrLK` d'OpenCV.



Sur une fenêtre 5×5 centrée sur un coin d'un marqueur de robotique, les gradients sont bien distribués dans toutes les directions. Le tenseur de structure a pour valeurs propres $\lambda_1 = 1200$, $\lambda_2 = 950$: les deux sont grandes, le système est bien conditionné, (u, v) sort sans ambiguïté.

Sur une fenêtre centrée sur le bord droit d'un couloir filmé par une caméra de surveillance, les gradients pointent tous dans la même direction horizontale. Le tenseur a $\lambda_1 = 1100$, $\lambda_2 \approx 3$. La matrice est quasi-singulière : $(A^T A)^{-1}$ amplifie le bruit d'un facteur $1100/3 \approx 367$. Le vecteur calculé est numériquement instable et doit être rejeté.



L'entrée `a` reçoit l'image BGR à `t` (port image bleu), l'entrée `b` l'image à `t+1`. Le résultat sur `out_a` est un dictionnaire de statistiques sur les vecteurs de flot détectés.

```
# a : image BGR à t (np.ndarray HxWx3 uint8)
# b : image BGR à t+1 (np.ndarray HxWx3 uint8)
if not isinstance(a, np.ndarray) or not isinstance(b, np.ndarray):
    out_a = {'erreur': 'deux images requises (a=frame t, b=frame t+1)'}
else:
    prev_gray = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY)
    next_gray = cv2.cvtColor(b, cv2.COLOR_BGR2GRAY)

    # Détection des points de Shi-Tomasi (coins fiables)
    p0 = cv2.goodFeaturesToTrack(prev_gray, maxCorners=200,
                                qualityLevel=0.01, minDistance=10)

    if p0 is None:
        out_a = {'erreur': 'aucun coin détecté dans la frame a'}
    else:
        # Suivi pyramidal Lucas-Kanade
        p1, status, _ = cv2.calcOpticalFlowPyrLK(
            prev_gray, next_gray, p0, None,
            winSize=(21, 21), maxLevel=3
        )
        # Filtrer les points bien suivis (status == 1)
        good_old = p0[status == 1]
        good_new = p1[status == 1]
        # Calculer les déplacements
        displacements = good_new - good_old # shape (N, 2)
        norms = np.linalg.norm(displacements, axis=1)
        out_a = {
            'points_suivis': int(np.sum(status)),
            'deplacement_moyen_px': float(np.mean(norms)) if len(norms)
        else 0.0,
            'deplacement_max_px': float(np.max(norms)) if len(norms) else
        0.0,
            'u_moyen': float(np.mean(displacements[:, 0])) if
        len(displacements) else 0.0,
            'v_moyen': float(np.mean(displacements[:, 1])) if
        len(displacements) else 0.0,
        }
    }
```

9.4. Horn-Schunck : la solution globale

L'hypothèse ajoutée : régularité globale du champ

Là où Lucas-Kanade imposait un mouvement constant fenêtre par fenêtre, Horn-Schunck adopte une perspective entièrement différente : le champ de mouvement (u, v) doit être **globalement lisse** — les pixels voisins ont des vitesses proches, sauf aux frontières d'objets où une discontinuité est admise. Ce n'est pas une hypothèse locale mais une contrainte sur l'image entière.

L'image mentale est celle d'une nappe tendue sur un paysage : la nappe peut avoir des pentes, mais elle n'a pas de déchirures ni de plis abrupts. Chaque petit décrochement de mouvement doit « coûter » quelque chose. Horn-Schunck quantifie ce coût et cherche le champ qui le minimise.

Cet a priori est physiquement justifié dans de nombreux contextes : le fond d'une scène de surveillance se déplace de façon rigide et cohérente, les organes d'un patient bougent de manière continue sur une IRM cardiaque, les nuages sur une image satellite se déforment progressivement. Là où Lucas-Kanade renonçait aux zones sans coin, Horn-Schunck les remplit par propagation depuis les zones fiables.

Formulation variationnelle

On minimise une énergie à deux termes définie sur toute l'image :

$$E(u, v) = \iint [\underbrace{(I_x u + I_y v + I_t)^2}_{\text{terme de données}} + \underbrace{\alpha^2 (\|\nabla u\|^2 + \|\nabla v\|^2)}_{\text{terme de lissage}}] dx dy$$

- **Terme de données** : l'OFCE doit être respectée au mieux — le mouvement doit être cohérent avec ce que les intensités mesurent.
- **Terme de lissage** : les gradients de u et de v doivent être petits — le champ doit varier lentement. $\|\nabla u\|^2$ mesure à quel point u varie rapidement d'un pixel au suivant.
- α : le paramètre de compromis. Grand α = forte régularisation, petit α = fidélité aux données.

Ce type de formulation, avec un terme d'attache aux données et un terme de régularisation pesés par α , est un schéma fondamental de la vision par ordinateur. On le retrouve en débruitage (chapitre 5), en segmentation par graph cut (§12), en reconstruction 3D. Horn-Schunck en est l'archétype pour le mouvement.

Le rôle du terme de lissage : propager l'information

C'est le cœur conceptuel. Dans les régions sans texture (ciel, fond uni) ou le long des contours droits, le terme de données ne contraint rien — le problème d'ouverture règne. Le terme

de lissage prend alors le relais : il **propage** le mouvement depuis les régions où il est bien déterminé (les coins, où le terme de données est fort) vers les régions ambiguës, par une sorte de diffusion spatiale. Horn-Schunck remplit les trous que Lucas-Kanade laissait vides.

Le prix à payer est double : un champ **dense** (un vecteur par pixel) mais **flou aux frontières** (un objet et son fond voient leurs vitesses mélangées là où α est grand).

Le paramètre α : le curseur du compromis

- **α grand** : forte régularisation, champ très lisse. Les discontinuités de mouvement — un objet qui se détache devant un fond statique — sont estompées. La frontière de l'objet est mal localisée, mais les zones sans texture sont cohérentes.
- **α petit** : fidélité aux données, le champ suit de près les variations d'intensité. Les discontinuités sont mieux préservées, mais le bruit dans les zones uniformes augmente et des trous apparaissent là où l'OFCE est ambiguë.

Ce dilemme attache-aux-données / régularité est le même que celui du filtre bilatéral (chapitre 5) : un filtre trop lisse perd les contours, un filtre trop conservateur garde le bruit. La seule différence est que pour Horn-Schunck, le curseur est explicitement un seul nombre α , ce qui en fait un modèle pédagogiquement transparent.

Résolution itérative

Le minimum de E s'obtient en annulant les dérivées fonctionnelles, ce qui conduit aux équations d'Euler-Lagrange. Celles-ci se résolvent par itérations de type Jacobi ou Gauss-Seidel, en alternant mise à jour de u et de v jusqu'à convergence. Chaque itération améliore le flot en tenant compte à la fois des gradients locaux et des valeurs voisines — c'est littéralement la propagation de l'information décrite ci-dessus.



Sur une image de flux sanguin par vélocimétrie à particules (PIV), $\alpha = 1.0$ donne un champ lisse où les tourbillons sont visibles mais les bords du vaisseau sont flous. $\alpha = 0.05$ restitue les gradients de vitesse au niveau de la paroi mais introduit du bruit dans le cœur du flux où le contraste est faible. Le choix optimal dépend de la fréquence spatiale du mouvement attendu — un principe qui relie ce chapitre directement au chapitre 5 (filtrage comme choix de fréquence).

Méthodes modernes

Horn-Schunck (1981) et Lucas-Kanade (1981) fondent le domaine, mais le flot dense de haute précision repose aujourd'hui sur l'apprentissage profond : FlowNet, PWC-Net, RAFT. La structure conceptuelle reste pourtant identique — attache aux données + a priori de régularité — mais l'a priori n'est plus postulé à la main : il est **appris** sur des corpus

d'entraînement. RAFT, qui représente l'état de l'art pour les benchmarks comme MPI-Sintel ou Kitty, apprend à construire un volume de corrélations dense et à itérer vers la solution. La boucle itérative de Horn-Schunck est toujours là, réimaginée dans un réseau de neurones récurrent.



L'entrée `a` reçoit l'image BGR à `t` (port image bleu), l'entrée `b` l'image à `t+1`. Le résultat sur `out_a` est un dictionnaire de statistiques sur le champ dense ; `out_b` expose l'image de visualisation du flot (port image bleu).

```
# a : image BGR à t (np.ndarray HxWx3 uint8)
# b : image BGR à t+1 (np.ndarray HxWx3 uint8)
if not isinstance(a, np.ndarray) or not isinstance(b, np.ndarray):
    out_a = {'erreur': 'deux images requises (a=frame t, b=frame t+1)'}
    out_b = a if isinstance(a, np.ndarray) else np.zeros((480, 640, 3),
dtype=np.uint8)
else:
    prev_gray = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY)
    next_gray = cv2.cvtColor(b, cv2.COLOR_BGR2GRAY)

    # Flot dense Farneback (approximation polynomiale, proche Horn-
    Schunck par l'esprit)
    flow = cv2.calcOpticalFlowFarneback(
        prev_gray, next_gray, None,
        pyr_scale=0.5, levels=3, winsize=15,
        iterations=3, poly_n=5, poly_sigma=1.2, flags=0
    )
    # flow : HxWx2 float32 (composantes u=flow[...,0], v=flow[...,1])

    u = flow[..., 0]
    v = flow[..., 1]
    magnitude = np.sqrt(u**2 + v**2)

    # Visualisation HSV : teinte = direction, saturation = 1, valeur =
    magnitude
    hsv = np.zeros_like(a)
    hsv[..., 1] = 255
    angle = np.arctan2(v, u)
    hsv[..., 0] = ((angle + np.pi) / (2 * np.pi) * 179).astype(np.uint8)
    hsv[..., 2] = cv2.normalize(magnitude, None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)
    out_b = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)

    out_a = {
        'magnitude_moyenne': float(np.mean(magnitude)),
        'magnitude_max': float(np.max(magnitude)),
        'u_moyen': float(np.mean(u)),
        'v_moyen': float(np.mean(v)),
    }
```

Note sur le port `flow`. VNStudio dispose d'un type de port dédié `flow` (couleur rouge) qui transporte un tableau `np.ndarray H×W×2 float32` représentant exactement le champ (u, v) calculé ci-dessus. Si une node aval attend un port `flow`, assigner directement `out_b = flow` (le tableau brut) plutôt que l'image de visualisation.

9.5. Flot épars vs flot dense : choisir

Le contraste structurel

Les deux familles de méthodes répondent à des questions différentes :

ÉPARS (Lucas-Kanade sur coins de Shi-Tomasi)

- + rapide, robuste, fiable là où il répond
- + idéal pour le suivi de points (tracking, odométrie visuelle, stabilisation)
- ne dit rien entre les points suivis
- ne peut pas produire de carte de mouvement complète

DENSE (Horn-Schunck, Farneback, RAFT, PWC-Net)

- + un vecteur par pixel, champ complet
- + idéal pour la segmentation de mouvement, l'interpolation d'images, les effets vidéo
- plus coûteux en calcul (surtout les réseaux profonds)
- sensible dans les zones ambiguës si α est mal réglé

La question de conception n'est pas « lequel est meilleur ? » mais « **ai-je besoin du mouvement partout, ou seulement de suivre des points fiables ?** ». Un système de stabilisation vidéo a besoin de quelques dizaines de vecteurs fiables pour estimer la transformation globale de la caméra : Lucas-Kanade suffit. Un système d'analyse du comportement de la foule à partir d'une caméra aérienne satellite a besoin d'un champ dense pour segmenter les flux de personnes : il faut une méthode dense.

Applications transversales

Ces deux familles couvrent ensemble un spectre applicatif très large :

- **Suivi d'objets et odométrie visuelle** (robotique, drones) : Lucas-Kanade sur coins, intégré dans les systèmes SLAM.
- **Stabilisation vidéo** : estimation du mouvement global de la caméra par flot épars, puis compensation par transformation inverse.
- **Compression vidéo** : les vecteurs de mouvement de MPEG et H.264 sont un flot épars par blocs de 16×16 pixels — le flot optique est au cœur de tous vos fichiers vidéo.
- **Analyse de perfusion** (IRM cardiaque, imagerie de fluorescence) : flot dense pour suivre la propagation d'un produit de contraste dans le myocarde.

- **Interpolation temporelle** (*slow motion*, conversion 24→60 fps) : générer des images intermédiaires par déformation guidée par un flot dense.
- **Reconnaissance d'actions** (surveillance, sport, gestes de réhabilitation) : le flot dense comme descripteur d'entrée d'un classificateur.
- **Télédétection** : suivi de glaciers ou de nappes d'inondation entre deux images satellitaires à quelques jours d'intervalle.



Ce script produit une superposition visuelle du flot épars (vecteurs) sur l'image courante. L'entrée `a` reçoit l'image BGR à `t`, `b` l'image à `t+1`.

```
# a : image BGR à t (np.ndarray H×W×3 uint8)
# b : image BGR à t+1 (np.ndarray H×W×3 uint8)
if not isinstance(a, np.ndarray) or not isinstance(b, np.ndarray):
    out_a = {'erreur': 'deux images requises'}
    out_b = np.zeros((480, 640, 3), dtype=np.uint8)
else:
    prev_gray = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY)
    next_gray = cv2.cvtColor(b, cv2.COLOR_BGR2GRAY)

    p0 = cv2.goodFeaturesToTrack(prev_gray, maxCorners=100,
                                qualityLevel=0.01, minDistance=15)

    vis = a.copy()
    stats = {'erreur': 'aucun coin détecté'}

    if p0 is not None:
        p1, status, _ = cv2.calcOpticalFlowPyrLK(prev_gray, next_gray,
        p0, None)
        good_old = p0[status == 1]
        good_new = p1[status == 1]

        for old_pt, new_pt in zip(good_old, good_new):
            x0, y0 = int(old_pt[0][0]), int(old_pt[0][1])
            x1, y1 = int(new_pt[0][0]), int(new_pt[0][1])
            cv2.arrowedLine(vis, (x0, y0), (x1, y1), (0, 255, 0), 1,
            tipLength=0.3)
            cv2.circle(vis, (x0, y0), 2, (0, 0, 255), -1)

        displacements = good_new - good_old
        norms = np.linalg.norm(displacements, axis=1)
        stats = {
            'points_suivis': int(len(good_new)),
            'deplacement_moyen_px': float(np.mean(norms)),
        }

    out_b = vis
    out_a = stats
```

9.6. Tableau récapitulatif – combler l'équation manquante

Méthode	A priori ajouté	Densité	Condition de fiabilité	Usage type
OFCE seule	constance de luminosité (hypothèse de base)	—	jamais seule (1 équ., 2 inc.)	base théorique
Lucas-Kanade	mouvement constant sur fenêtre $n \times n$	épars (coins)	coin (λ_1 et λ_2 de $A^T A$ grands)	suivi de points, odométrie, stabilisation
Horn-Schunck	champ globale-ment lisse (régularisation α)	dense (tous pixels)	partout, par propagation	flot dense, segmentation de mouvement, IRM
Farneback (OpenCV)	approximation polynomiale locale	dense	bonnes textures locales	flot dense rapide, effets vidéo
Réseaux (RAFT, PWC-Net)	régularités prises sur corpus	ap- dense cor-	dépend du corpus d'entraînement	flot dense haute précision, benchmarks

9.7. le problème mal posé, l'a priori salvateur, et le méta-fil du livre

Le flot optique est l'exemple le plus transparent de la vision par ordinateur d'un **problème mal posé** : les données disponibles (une équation par pixel) sont numériquement insuffisantes pour déterminer la solution (deux inconnues par pixel). La formule est impitoyable : 1 contre 2. Aucun algorithme, aussi sophistiqué soit-il, ne peut extraire d'une seule équation deux inconnues indépendantes — sauf en ajoutant de l'information extérieure aux données.

Cette information extérieure, c'est l'**a priori** : une hypothèse sur la nature du monde, formulée avant même d'avoir vu les images. Lucas-Kanade dit « le mouvement est localement rigide, à l'échelle d'une fenêtre ». Horn-Schunck dit « le mouvement varie doucement sur l'image entière ». RAFT dit « le mouvement ressemble à ce que j'ai vu dans des millions de scènes annotées ». Ces trois énoncés ne sont pas des vérités universelles — ce sont des paris sur le monde, et la qualité d'une méthode tient autant à la justesse de son pari qu'à celle de ses calculs.

La structure est toujours :

données insuffisantes + hypothèse (a priori) = solution déterminée

Ce schéma n'est pas propre au mouvement. C'est le fil conducteur de tout le livre, reformulé dans chaque chapitre selon le problème du moment :

- Le filtre (chapitre 5) résout le problème mal posé du débruitage : les données bruitées ne déterminent pas le signal original, et chaque filtre est un a priori sur les fréquences du signal (passe-bas = lisseur = a priori que le signal est lent).
- La distance (chapitre 3) résout le problème mal posé de la comparaison : deux histogrammes ou deux descripteurs ne se comparent pas de façon absolue, et chaque distance est un a priori sur ce qui compte.
- La segmentation par graph cut (chapitre 12, à venir) minimisera exactement la même énergie attache-aux-données + régularisation que Horn-Schunck, appliquée à l'appartenance d'un pixel à un segment.

Le choix de l'a priori n'est pas neutre : il décide de ce que la méthode réussit et de ce qu'elle rate. Un a priori de lissage global échoue aux frontières d'objets, là où le mouvement est discontinu par nature (un bras devant un mur). Un a priori de constance locale échoue sur les déformations non rigides à grande échelle (un drapeau qui ondule). Un a priori appris sur des scènes intérieures échoue sur des images satellitaires.

Comprendre une méthode, c'est donc comprendre son a priori — et savoir dans quelles situations cet a priori cesse d'être raisonnable. Cette compétence — identifier l'hypothèse cachée derrière chaque outil — est précisément ce que le livre cherche à construire, chapitre après chapitre.

CHAPITRE

10

Transformées

Table des matières

10.1. La transformée de Fourier discrète (DFT)	160
10.2. Le théorème de convolution	163
10.3. La transformée en cosinus discrète (DCT)	165
10.4. La transformée de Hough	168
10.5. La transformée de distance	172
10.6. Tableau récapitulatif — quelle transformée pour quel but ?	174
10.7. changer de base, c'est choisir où le problème est simple	175

Depuis le début de ce livre, les images ont été manipulées dans leur forme naturelle : un tableau de pixels, chacun portant une intensité ou une couleur. Ce point de vue spatial est le plus direct, mais il n'est pas toujours le plus efficace. Filtrer une rayure périodique pixel par pixel est laborieux ; la même opération dans le domaine fréquentiel se réduit à effacer deux coefficients. Détecter des droites dans un nuage de contours bruités est difficile ; dans l'espace des paramètres de Hough, elles se révèlent comme des pics discrets. Compresser une image en gardant l'essentiel de son contenu visuel est impossible à faire directement dans les pixels ; dans le domaine cosinus, quelques coefficients suffisent.

Le fil conducteur du chapitre tient en une phrase : **changer de base, c'est choisir où le problème devient simple**. Une transformée ne modifie pas l'image — elle la réexprime dans un nouveau système de coordonnées, choisi pour que la tâche (filtrer, détecter, compresser, mesurer) y devienne triviale. L'information est la même ; seul le cadre change. Et comme pour le choix d'une distance (chapitre 3) ou d'un filtre (chapitre 5), ce choix encode une hypothèse sur ce qui compte dans le problème.

Ce chapitre prolonge directement le chapitre 5 sur le filtrage : le théorème de convolution (§10.2) y révèle pourquoi les filtres spatiaux agissent sur les fréquences, et fournit ainsi la justification rigoureuse de ce qui n'y était qu'annoncé. On renverra aussi au chapitre sur la segmentation pour la transformée de distance, qui y joue un rôle central.

10.1. La transformée de Fourier discrète (DFT)



$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} I(x, y) \cdot \exp(-2\pi i(ux/M + vy/N))$$

L'inverse — la DFT inverse — reconstruit exactement I à partir des $F(u, v)$:

$$I(x, y) = (1/MN) \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) \cdot \exp(+2\pi i(ux/M + vy/N))$$

L'idée : décomposer une image en ondes

L'image mentale la plus utile ici est celle d'un accordeur de musique. Quand un musicien joue une note complexe, l'accordeur décompose le son en ses fréquences constitutives : une fréquence fondamentale forte, des harmoniques plus faibles. La DFT fait exactement la même chose pour une image : elle la décompose en une somme de sinusoides 2-D de fréquences, d'orientations et de phases différentes.

Chaque coefficient $F(u, v)$ décrit une onde de fréquence (u, v) . Les petites valeurs de u et v (proches de 0, dans le coin supérieur gauche du spectre) correspondent aux **basses**

fréquences : les variations lentes, les grandes plages de couleur uniforme, l'éclairage général. Les grandes valeurs de u et v correspondent aux **hautes fréquences** : les contours nets, les détails fins, le bruit de capteur. Une image très floue ne contient presque que des basses fréquences ; une image très texturée étale son énergie jusqu'aux hautes.

La DFT comme changement de base

La formulation mathématique révèle quelque chose d'important : les ondes complexes $\exp(2\pi i(ux/M + vy/N))$ forment une **base orthogonale** de l'espace des images, exactement comme les axes x, y, z forment une base de l'espace 3-D. $F(u,v)$ est la **projection** de l'image sur l'onde de fréquence (u,v) — un produit scalaire. C'est précisément en ce sens que la DFT est un changement de base : l'image et sa DFT contiennent exactement la même information, mais décrite dans deux systèmes de coordonnées différents. Dans la base des pixels, les fréquences sont invisibles ; dans la base de Fourier, elles sautent aux yeux.

Module et phase : où est l'information ?

$F(u,v)$ est un nombre complexe. Son **module** $|F(u,v)|$ indique combien d'énergie est présente à la fréquence (u,v) — c'est le spectre d'amplitude, souvent visualisé sous forme d'image. Sa **phase** $\arg(F(u,v))$ indique où cette onde est positionnée dans l'image.

Un résultat contre-intuitif mérite d'être retenu : **la phase porte l'essentiel de la structure perceptible**. Si l'on reconstruit une image en gardant la phase d'une photo A mais le module d'une photo B, l'image résultante ressemble à la photo A. La phase encode où se trouvent les contours et les structures ; le module indique seulement leur intensité. Cette asymétrie est importante pour comprendre pourquoi certaines opérations (notamment le recalage par corrélation de phase) fonctionnent si bien.

La FFT : pourquoi le calcul est praticable

La DFT naïve sur une image de N pixels demande $O(N^2)$ opérations — inacceptable pour une image 4K (≈ 8 millions de pixels). L'algorithme **FFT** (Cooley-Tukey, 1965) exploite une structure récursive : si N est une puissance de 2, on peut factoriser le calcul en sous-problèmes pairs et impairs qui se réutilisent, faisant tomber le coût à $O(N \log N)$. Pour $N = 1\,000\,000$, le rapport est de $1\,000\,000$ contre $20\,000\,000$ — un facteur 20. C'est l'un des algorithmes les plus importants du XXe siècle, et la raison pour laquelle le filtrage fréquentiel est pratique en temps réel.



Considérons une image synthétique de rayures verticales régulières espacées de 10 pixels. Sa DFT est presque entièrement nulle, **sauf deux pics** symétriques positionnés à la fréquence horizontale $u = M/10$ (et son symétrique). Toute la structure « rayures » est encodée dans exactement deux coefficients. Pour éliminer ces rayures — par exemple sur une radiographie ou un scan de document — il suffit d'annuler ces deux coefficients dans le spectre, puis d'appliquer la DFT inverse : les rayures disparaissent, sans toucher au reste de l'image. L'opération qui semblerait complexe dans l'espace spatial est triviale dans le domaine de Fourier.



La DFT suppose une extension **périodique** du signal : elle imagine que l'image se répète indéfiniment à gauche, droite, haut et bas. Si les bords gauche et droit d'une image ne sont pas identiques (ce qui est presque toujours le cas), la répétition crée une discontinuité brutale. Cette discontinuité, très riche en hautes fréquences, se retrouve dans le spectre sous forme d'énergie étalée — la **fuite spectrale**. Le remède courant est d'appliquer une **fenêtre d'apodisation** (Hann, Hamming) qui atténue doucement les bords vers zéro avant de calculer la DFT, au prix d'une légère perte d'information sur les bords.

Autre point pratique : `np.fft.fftshift` centre les basses fréquences au milieu de l'image (visualisation naturelle), mais le calcul de la DFT inverse doit être précédé de `np.fft.ifftshift` pour remettre les fréquences dans l'ordre attendu.



```
# a : image en niveaux de gris (np.ndarray HxW uint8)
if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune image en entree'}
else:
    gray = a if a.ndim == 2 else cv2.cvtColor(a, cv2.COLOR_BGR2GRAY)
    gray_f = gray.astype(np.float32)

    F = np.fft.fft2(gray_f)
    Fshift = np.fft.fftshift(F)          # basses fréquences au centre

    magnitude = 20.0 * np.log(np.abs(Fshift) + 1.0)
    # normaliser pour visualisation
    mag_norm = cv2.normalize(magnitude, None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)

    out_a = {
        'shape': list(gray.shape),
        'energie_totale': float(np.sum(np.abs(F)**2)),
        'coeff_dc': float(np.abs(F[0, 0])), # composante continue
    }
    out_b = mag_norm # spectre visualisable, à brancher sur
output_display
```

10.2. Le théorème de convolution



$$F(I * K) = F(I) \cdot F(K)$$

Autrement dit : la DFT de la convolution de deux signaux est le produit terme à terme de leurs DFT. La réciproque est également vraie : une multiplication dans l'espace correspond à une convolution dans le domaine de Fourier.

L'idée avant la dérivation

Ce théorème est le pont entre les deux langages du chapitre 5 et de ce chapitre. Convoluer deux images dans l'espace — faire glisser un noyau sur chaque pixel — est une opération locale mais souvent lente pour de grands noyaux. Multiplier deux spectres dans Fourier — chaque coefficient par son correspondant — est une opération globale mais instantanée. Le théorème dit que ces deux gestes sont identiques. La convolution, qui semble être une opération spatiale, est en réalité un filtrage fréquentiel.

Dérivation (esquisse 1-D)

Écrivons la DFT de la convolution discrète et échangeons l'ordre des sommes :

$$F(I * K)[u] = \sum_n \left(\sum_m I[m] \cdot K[n-m] \right) \cdot e^{\{-2\pi i u n / N\}}$$

Posons $k = n - m$ (donc $n = m + k$) et séparons les exponentielles :

$$\begin{aligned} &= \sum_m I[m] \cdot e^{\{-2\pi i u m / N\}} \cdot \sum_k K[k] \cdot e^{\{-2\pi i u k / N\}} \\ &= F(I)[u] \cdot F(K)[u] \quad \blacksquare \end{aligned}$$

Le changement de variable $k = n - m$ découple la double somme en un produit de deux DFT indépendantes. C'est tout : la convolution devient une multiplication.

Ce que révèle ce théorème sur le chapitre 5

Ce résultat **explique rétrospectivement** pourquoi les filtres du chapitre 5 agissent sur les fréquences. Un noyau de lissage gaussien a une DFT qui vaut ≈ 1 en basse fréquence et ≈ 0 en haute fréquence : le multiplier dans Fourier (c'est-à-dire le convoluer dans l'espace) atténue les hautes fréquences — c'est un filtre passe-bas. Un noyau de dérivation (Sobel, Laplacien) a la DFT inverse : il amplifie les hautes fréquences. Un filtre de Gabor sélectionne une bande de fréquences à une orientation donnée.

filtre spatial		réponse fréquentielle
gaussien (lissage)	→	passe-bas (atténue les hautes fréquences)
Sobel / Laplacien	→	passe-haut (amplifie les hautes fréquences)
LoG / DoG / Gabor	→	passe-bande (sélectionne une bande)

Concevoir un filtre, c'est en réalité **dessiner sa réponse fréquentielle** — souvent plus intuitif que de choisir des coefficients spatiaux à la main. La bonne base (spatiale ou fréquentielle) dépend de ce qu'on veut exprimer.

Conséquence pratique : la convolution rapide

Pour un grand noyau de taille k , la convolution spatiale coûte $O(N \cdot k^2)$ où N est le nombre de pixels. Pour $k = 50$ et une image 1 Mpx, c'est 2,5 milliards d'opérations. En passant par Fourier — FFT de l'image, FFT du noyau, multiplication terme à terme, FFT inverse — le coût tombe à $O(N \log N)$ quelle que soit la taille du noyau. C'est ainsi que sont implémentées les grandes convolutions (flous extrêmes, certaines couches de réseaux de neurones convolutifs).



La convolution via Fourier suppose une extension périodique des deux signaux. Si le noyau K est plus petit que l'image (ce qui est presque toujours le cas), il faut le **zero-padder** à la taille de l'image avant la FFT, et centrer correctement ses coefficients. `scipy.signal.fftconvolve` gère ces détails automatiquement.



```
# a : image (np.ndarray HxW uint8), b : noyau (np.ndarray float32)
if not isinstance(a, np.ndarray) or not isinstance(b, np.ndarray):
    out_a = {'erreur': 'image ou noyau manquant'}
else:
    from scipy.signal import fftconvolve
    gray = a if a.ndim == 2 else cv2.cvtColor(a, cv2.COLOR_BGR2GRAY)
    gray_f = gray.astype(np.float32)
    kernel = b.astype(np.float32)

    # convolution rapide via FFT
    result = fftconvolve(gray_f, kernel, mode='same')
    result_clipped = np.clip(result, 0, 255).astype(np.uint8)

    out_a = {
        'taille_image': list(gray.shape),
        'taille_noyau': list(kernel.shape),
    }
    out_b = result_clipped # résultat convolutionné
```

10.3. La transformée en cosinus discrète (DCT)

Définition (2-D)

$$F(u,v) = c(u) \cdot c(v) \cdot \sum_x \sum_y I(x,y) \cdot \cos[(2x+1)u\pi/2M] \cdot \cos[(2y+1)v\pi/2N]$$

avec $c(0) = \sqrt{1/N}$ et $c(k \neq 0) = \sqrt{2/N}$ (facteurs de normalisation qui garantissent l'orthonormalité).

L'idée : une DFT pour signaux réels

La DCT ressemble à la DFT, mais elle n'utilise que des cosinus — pas d'exponentielles complexes. Cette différence a deux conséquences importantes. D'abord, la DCT est **entièrement réelle** : pour une image réelle (ce qui est toujours le cas), les coefficients DCT sont des nombres réels ordinaires, deux fois moins coûteux à stocker et à manipuler que les coefficients

complexes de la DFT. Ensuite, la DCT suppose une extension **paire (symétrique)** du signal aux bords, là où la DFT suppose une extension périodique.

Pour comprendre cette différence, imaginez un morceau de tissu que l'on place côte à côte pour paver un plan. La DFT « colle » bord droit avec bord gauche : si le tissu est plus clair à droite qu'à gauche, la couture est visible — une discontinuité brutale, riche en hautes fréquences parasites. La DCT, elle, « replie » le tissu comme un miroir : le bord droit est mis en vis-à-vis avec lui-même, assurant une continuité parfaite. Cette extension symétrique évite les discontinuités et l'énergie parasite qui en résulte.

DFT : extension périodique → discontinuité possible aux bords → fuite spectrale

DCT : extension symétrique → continuité garantie aux bords → énergie mieux concentrée

La propriété reine : la compaction d'énergie

C'est la raison d'être de la DCT en traitement d'image. Pour les images naturelles — qui varient doucement, sans sauts brusques dans la plupart de leurs régions —, la DCT **concentre l'essentiel de l'énergie dans un petit nombre de coefficients** situés dans le coin basse fréquence (u,v proches de 0). Les coefficients haute fréquence sont souvent proches de zéro. Un ciel uniforme, une peau, une surface mate : leur DCT ressemble à un spectre quasi vide.

Cette concentration n'est pas magique : elle reflète le fait que les images naturelles sont corrélées localement. Les pixels voisins se ressemblent. Cette redondance spatiale se traduit, dans le domaine DCT, par peu de coefficients non nuls. Jeter les coefficients quasi nuls ne change presque rien à l'image reconstruite — c'est le fondement de toute compression avec perte.

JPEG : la DCT en action

La compression JPEG est l'application canonique de la DCT. Le pipeline illustre parfaitement le fil conducteur du chapitre :

1. Découper l'image en blocs 8×8 pixels
2. DCT de chaque bloc → énergie concentrée dans le coin supérieur gauche
3. Quantification → diviser chaque coefficient par un pas, arrondir à l'entier
(les hautes fréquences, peu visibles, sont grossièrement quantifiées)
4. Codage entropique → compresser les nombreux zéros résultants (run-length + Huffman)

L'étape de quantification est la seule destructive, et elle exploite la perception visuelle : l'œil humain tolère mal les erreurs en basse fréquence (zones lisses qui se strient) mais accepte bien

les erreurs en haute fréquence (détails fins légèrement bruités). La table de quantification JPEG quantifie donc grossièrement les hautes fréquences et finement les basses.

Pousser trop loin la quantification produit les fameux **artefacts en blocs** : quand trop de coefficients sont annulés, les frontières des blocs 8×8 deviennent visibles. Ces artefacts révèlent exactement l'unité de calcul — les 8×8 pixels — là où le changement de base avait réussi à les faire oublier.



Un bloc 8×8 prélevé dans un ciel bleu uniforme. Après DCT, le coefficient (0,0) — la **composante continue**, la moyenne du bloc — concentre presque toute l'énergie ; les 63 autres coefficients sont proches de zéro. On stocke, en pratique, un seul nombre au lieu de 64. La compression est maximale.

Un bloc 8×8 prélevé dans du feuillage serré étale son énergie sur des dizaines de coefficients. Il se compresse moins bien — la DCT l'indique honnêtement : la compressibilité d'un bloc est exactement sa concentration spectrale. La DCT n'améliore pas l'image ; elle rend visible ce qui était déjà là.



`scipy.fftpack.dct` calcule la DCT 1-D. Pour un bloc 2-D, il faut l'appliquer sur les lignes puis sur les colonnes (ou vice versa, l'ordre est indifférent). Attention au paramètre `norm='ortho'` : sans lui, les facteurs `c(u)` et `c(v)` ne sont pas normalisés et les valeurs numériques changent d'un facteur (rendant les tables de quantification JPEG inutilisables telles quelles).



```
# a : image en niveaux de gris (np.ndarray HxW uint8)
if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune image en entree'}
else:
    from scipy.fftpack import dct, idct
    gray = a if a.ndim == 2 else cv2.cvtColor(a, cv2.COLOR_BGR2GRAY)

    # découper en blocs 8x8 et calculer la DCT du premier bloc non vide
    h, w = gray.shape
    bloc = gray[:8, :8].astype(np.float32) - 128.0 # centrer sur 0

    def dct2(b):
        return dct(dct(b.T, norm='ortho').T, norm='ortho')

    def idct2(b):
        return idct(idct(b.T, norm='ortho').T, norm='ortho')

    C = dct2(bloc)
    energie_dc = float(C[0, 0]**2)
    energie_totale = float(np.sum(C**2))
    ratio_dc = energie_dc / energie_totale if energie_totale > 0 else 0.0

    # compresser : ne garder que les 4 plus grands coefficients
    C_comprese = np.zeros_like(C)
    indices = np.argsort(np.abs(C).ravel())[::-1][:4]
    for idx in indices:
        C_comprese.ravel()[idx] = C.ravel()[idx]
    bloc_reconstruit = np.clip(idct2(C_comprese) + 128.0, 0,
255).astype(np.uint8)

    out_a = {
        'coeff_dc': float(C[0, 0]),
        'ratio_energie_dc': float(round(ratio_dc, 4)),
        'erreur_quadratique_4coeff': float(np.mean((bloc -
idct2(C_comprese))**2)),
    }
```

10.4. La transformée de Hough

L'idée : voter pour des formes

Les transformées précédentes décomposaient l'image en ondes. Hough fait quelque chose de fondamentalement différent : elle change l'image vers un **espace de paramètres**, dans lequel chaque forme géométrique recherchée se révèle comme un point. Détecter des droites dans l'image revient à trouver des points dans un espace à deux paramètres. Détecter des cercles revient à trouver des points dans un espace à trois paramètres.

L'image mentale utile est celle d'un vote à bulletin secret. Chaque pixel de contour ignore quelle droite globale il appartient — il ne voit que son voisinage immédiat. Mais il peut voter pour toutes les droites qui pourraient passer par lui. Une vraie droite dans l'image rassemble de nombreux pixels qui, chacun de leur côté, votent pour les mêmes paramètres : leurs votes s'accumulent au même point dans l'espace de Hough. Le bruit et les contours parasites votent de façon dispersée et ne forment aucun pic.

Hough pour les droites : le paramétrage (ρ, ϑ)

Une droite pourrait s'écrire $y = ax + b$, mais ce paramétrage échoue pour les droites verticales ($a \rightarrow \infty$). On préfère la forme **normale** :

$$\rho = x \cdot \cos(\theta) + y \cdot \sin(\theta)$$

où ρ est la distance algébrique de la droite à l'origine, et ϑ l'angle de sa normale avec l'axe horizontal. Tout couple (ρ, ϑ) borné décrit une droite — y compris les droites verticales ($\vartheta = 0$, $\rho =$ distance à l'axe y).

Dérivation du mécanisme de vote

Un point de l'image de coordonnées (x_0, y_0) appartient à toutes les droites vérifiant $\rho = x_0 \cos(\vartheta) + y_0 \sin(\vartheta)$. Dans l'espace (ρ, ϑ) , cette relation décrit une **courbe sinusoïdale** — l'ensemble de toutes les droites passant par ce point. La dualité fondamentale s'écrit :

un POINT dans l'image $\rightarrow$ une SINUSOÏDE dans l'espace (ρ, θ)
 une DROITE dans l'image $\rightarrow$ un POINT dans l'espace (ρ, θ)

Plusieurs points alignés (appartenant à une même droite) produisent des sinusoïdes qui **se croisent toutes en un même point** (ρ, ϑ) — les paramètres de leur droite commune. L'algorithme consiste à discrétiser l'espace (ρ, ϑ) en une grille (l'**accumulateur**), et pour chaque pixel de contour, incrémenter toutes les cases de sa sinusoïde. Les pics de l'accumulateur correspondent aux droites dominantes.

Hough pour les cercles

Même principe, espace de paramètres à 3 dimensions (a, b, r) pour le cercle $(x-a)^2 + (y-b)^2 = r^2$. Un pixel de contour vote pour tous les cercles de tous les rayons possibles passant par lui. Le coût est plus élevé (accumulateur 3-D), d'où les variantes qui exploitent la direction du gradient pour réduire à 1-D le vote par pixel.

Robustesse et coût

La force de Hough est sa **robustesse aux occlusions et au bruit**. Une droite partiellement masquée accumule moins de votes mais reste détectable tant que suffisamment de pixels votent correctement. Les pixels de bruit votent de façon uniforme dans l'accumulateur et ne forment aucun pic significatif. Cette robustesse distingue Hough de la simple recherche de contours.

La faiblesse : le coût de l'accumulateur, surtout en dimension ≥ 3 , et la sensibilité à sa résolution. Des cases trop fines éparpillent les votes d'une même droite en plusieurs cases voisines ; des cases trop grossières fusionnent des droites proches. En pratique, les paramètres `threshold`, `minLineLength`, `maxLineGap` de `cv2.HoughLinesP` se règlent empiriquement selon la résolution de l'image et la densité des contours.



Trois pixels de contour alignés horizontalement à $y = 5$: $(1, 5)$, $(4, 5)$, $(8, 5)$. Dans l'espace (ρ, ϑ) , chacun génère une sinusoïde. Ces trois sinusoïdes se croisent toutes en $\vartheta = 90^\circ$ (normale verticale $\rightarrow$ droite horizontale) et $\rho = 5$. La case $(\rho = 5, \vartheta = 90^\circ)$ reçoit 3 votes, toutes les autres au plus 1. Le pic est sans ambiguïté. La droite $y = 5$ est détectée.



`cv2.HoughLines` travaille dans l'espace (ρ, ϑ) complet et renvoie des segments infinis. `cv2.HoughLinesP` (version probabiliste) est plus rapide et renvoie des **segments finis** avec longueur et saut maximaux configurables — elle est préférable en pratique. Pour les cercles, `cv2.HoughCircles` nécessite un pré-floutage de l'image (le paramètre `dp` contrôle la résolution de l'accumulateur) et produit des faux positifs si `param2` est trop bas.



```
# a : image (np.ndarray HxWx3 ou HxW uint8)
if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune image en entree'}
else:
    gray = a if a.ndim == 2 else cv2.cvtColor(a, cv2.COLOR_BGR2GRAY)
    edges = cv2.Canny(gray, 50, 150)

    # détection de droites (probabiliste)
    lines = cv2.HoughLinesP(edges, rho=1, theta=np.pi/180,
                             threshold=80, minLineLength=50,
maxLineGap=10)
    n_lines = len(lines) if lines is not None else 0

    # détection de cercles
    blurred = cv2.GaussianBlur(gray, (9, 9), 2)
    circles = cv2.HoughCircles(blurred, cv2.HOUGH_GRADIENT, dp=1,
minDist=20,
                             param1=100, param2=30, minRadius=5,
maxRadius=100)
    n_circles = len(circles[0]) if circles is not None else 0

    out_a = {
        'n_droites_detectees': n_lines,
        'n_cercles_detectes': n_circles,
    }
    # dessiner les droites sur l'image originale
    vis = a.copy() if a.ndim == 3 else cv2.cvtColor(gray,
cv2.COLOR_GRAY2BGR)
    if lines is not None:
        for l in lines:
            x1, y1, x2, y2 = l[0]
            cv2.line(vis, (x1, y1), (x2, y2), (0, 255, 0), 2)
    if circles is not None:
        for c in np.round(circles[0]).astype(int):
            cv2.circle(vis, (c[0], c[1]), c[2], (255, 0, 0), 2)
    out_b = vis
```

10.5. La transformée de distance



$$DT(p) = \min_{\{q \in \text{fond}\}} d(p, q)$$

Pour chaque pixel p d'une forme binaire, $DT(p)$ est la distance au pixel de fond (valeur 0) le plus proche. Sur le bord de la forme, $DT = 0$; à l'intérieur, DT croît à mesure qu'on s'éloigne du bord. La distance utilisée peut être euclidienne (L_2), de Manhattan (L_1) ou de Tchebychev (L_∞), selon le contexte.

L'idée : voir l'épaisseur depuis l'intérieur

La transformée de distance change radicalement le point de vue sur une forme binaire. Là où la forme originale dit seulement « appartient / n'appartient pas », la carte de distance dit « à quelle profondeur appartient ». C'est comme passer d'une carte de présence/absence à une **carte de relief** : le bord est au niveau de la mer, et l'intérieur monte en altitude proportionnellement à sa distance au bord.

Cette carte révèle deux propriétés importantes. D'une part, ses **crêtes** — les pixels localement les plus éloignés de tout bord — forment le **squelette** (ou axe médian) de la forme : le tracé central d'une lettre, l'axe d'un os ou d'un vaisseau sanguin, la ligne de faite d'une route. D'autre part, le **maximum global** de la carte donne à la fois le point le plus intérieur de la forme et son **rayon maximal inscrit** — le rayon du plus grand disque qu'on peut placer entièrement à l'intérieur sans toucher le bord.

Calcul efficace

Le calcul naïf — pour chaque pixel intérieur, chercher le pixel de fond le plus proche — est $O(N^2)$ où N est le nombre de pixels. Des algorithmes en deux passes (Meijster, Felzenszwalb-Huttenlocher) calculent la transformée euclidienne exacte en $O(N)$ — linéaire en nombre de pixels. OpenCV implémente ces algorithmes via `cv2.distanceTransform`.

Applications

La transformée de distance intervient dans plusieurs contextes distincts. En **segmentation**, elle fournit le relief pour l'algorithme watershed : en traitant la carte de distance comme un terrain où l'eau monte, les bassins versants correspondent aux objets individuels, séparant des objets accolés que le seuillage seul fondrait ensemble (voir le chapitre sur la segmentation pour les détails). En **analyse de forme**, le squelette extrait par crêtes de la DT permet de réduire une forme à son essence linéaire — indispensable pour reconnaître des chiffres manuscrits ou analyser des réseaux vasculaires. En **robotique et navigation**, une carte

de distance dans l'espace libre donne immédiatement la distance à l'obstacle le plus proche pour tout point du terrain.



Un disque binaire de rayon 10 pixels, centré en (cx, cy) . Sur son bord, $DT = 0$. En s'approchant du centre, DT croît linéairement. Exactement au centre, $DT = 10$ — le rayon du disque. Le maximum de la carte DT donne simultanément le centre du disque et son rayon, sans jamais avoir calculé de contour ni de centroïde. La transformée de distance **mesure la forme de l'intérieur**.

Si l'on prend maintenant un rectangle de 40×20 pixels, la carte de distance forme une pyramide tronquée : les crêtes courent le long de l'axe médian du rectangle, et le maximum (10, au centre) correspond au demi-côté court.



`cv2.distanceTransform` attend une image binaire où les pixels d'avant-plan valent **255 et non 1** (convention OpenCV pour les masques). Si le masque est en 0/1, il faut le multiplier par 255 ou passer par `cv2.threshold`. Le paramètre `maskSize` de 3, 5 ou `cv2.DIST_MASK_PRECISE` détermine la précision de l'approximation de la distance euclidienne ; `DIST_MASK_PRECISE` calcule la distance exacte au prix d'un temps légèrement plus long.



```
# a : masque binaire (np.ndarray HxW uint8, valeurs 0/255)
if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucun masque en entree'}
else:
    m = a if a.ndim == 2 else cv2.cvtColor(a, cv2.COLOR_BGR2GRAY)
    _, m = cv2.threshold(m, 127, 255, cv2.THRESH_BINARY)

    dt = cv2.distanceTransform(m, cv2.DIST_L2, cv2.DIST_MASK_PRECISE)

    # point le plus intérieur et rayon maximal inscrit
    max_val = float(np.max(dt))
    max_idx = np.unravel_index(dt.argmax(), dt.shape)
    centre_y, centre_x = int(max_idx[0]), int(max_idx[1])

    # normaliser pour visualisation
    dt_vis = cv2.normalize(dt, None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)
    dt_color = cv2.applyColorMap(dt_vis, cv2.COLORMAP_JET)

    out_a = {
        'rayon_maximal_inscrit': max_val,
        'centre_interieur': {'x': centre_x, 'y': centre_y},
    }
    out_b = dt_color # carte de distance visualisée, à brancher sur
output_display
```

10.6. Tableau récapitulatif – quelle transformée pour quel but ?

Transformée	Domaine cible	Ce qu'elle révèle	Angle mort	Usage type
Fourier (DFT/FFT)	Fréquences (ondes complexes)	Périodicité, spectre d'amplitude et de phase, fréquences parasites	Localisation spatiale (un coin est global)	Filtrage fréquentiel, recadrage, analyse de texture
DCT	Fréquences (cosinus réels)	Compaction d'énergie	Discontinuités (artefacts blocs)	Compression JPEG, repré-

Transformée	Domaine cible	Ce qu'elle ré- vèle	Angle mort	Usage type
		signaux corrélés, coefficients compressibles		sensation compacte de blocs
Hough	Espace de paramètres $(\rho, \theta$ ou $a, b, r)$	Formes géométriques par accumulation de votes	Formes non paramétriques, coût $O(N \cdot \text{dim})$	Détection de droites, cercles, contours occultés
Distance	Carte de proximité	Épaisseur, squelette, rayon inscrit, intérieur de forme	Bruit de bord ($DT = 0$ partout sur le contour)	Watershed, squelettisation, axe médian, navigation

10.7. changer de base, c'est choisir où le problème est simple

Le fil qui traverse ce chapitre — et qui dépasse la vision par ordinateur — peut s'énoncer ainsi : un problème difficile dans un domaine peut devenir trivial dans un autre.

Filtrer une fréquence parasite	→ impossible à l'œil dans l'espace pixel, deux coefficients à annuler dans Fourier.
Compresser une image naturelle faciles à jeter.	→ 64 pixels corrélés, difficiles à tronquer, quelques coefficients DCT quasi nuls,
Trouver des droites dans le bruit (dur),	→ chercher des alignements dans l'espace chercher des pics dans (ρ, θ) (facile).
Trouver le centre le plus profond d'une forme	→ géométrie complexe dans l'espace pixel, un maximum dans la transformée de distance.

Aucune transformée n'ajoute d'information. L'image reste la même ; seul le système de coordonnées change. Ce que chaque transformée fait, c'est **réorganiser l'information** pour qu'une question précise trouve sa réponse dans une opération simple : annuler un coefficient, lire un maximum, détecter un pic. La maîtrise vient de savoir reconnaître, face à un problème, *dans quelle base il devient facile.*

Ce principe relie ce chapitre aux autres. Choisir une **distance** (chapitre 3), c'est déclarer quelle différence compte. Choisir un **filtre** (chapitre 5), c'est dessiner un a priori sur le signal à traiter. Choisir un **descripteur** (chapitre 1), c'est décider quel aspect d'une forme mérite d'être vu. Choisir une **base** ici, c'est décider dans quel espace une propriété — fréquence, compacité, paramètre géométrique, profondeur — devient lisible. À chaque fois, la même leçon : **le bon cadre rend le problème presque résolu**, et tout choix de représentation encode une hypothèse sur ce qui compte.

CHAPITRE

Morphologie mathématique

11

Table des matières

11.1. Érosion et dilatation	179
11.2. Ouverture et fermeture	181
11.3. Gradient morphologique	184
11.4. Top-hat (chapeau haut-de-forme) et black-hat	186
11.5. Transformée tout-ou-rien et squelette	188
11.6. Tableau récapitulatif — chaque opérateur est un choix de sonde	190
11.7. tester une forme, c'est déclarer ce qui compte	191

Tous les filtres du chapitre 5 partagent une même grammaire : pondérer les valeurs voisines et les sommer. La morphologie mathématique change radicalement de grammaire. Elle ne calcule pas de moyenne ; elle pose une question géométrique — *cette forme tient-elle ici ?* — et répond par un minimum ou un maximum. L'outil central n'est plus un noyau de coefficients flottants mais un **élément structurant** : une forme-sonde, conçue par le praticien, que l'on promène pixel par pixel sur l'image. Partout où la sonde s'insère, on lit une réponse ; partout où elle bute, on lit l'autre. Ce chapitre construit les opérateurs de base depuis cette idée unique, en montrant à chaque étape que c'est le *choix de la sonde* — sa forme, sa taille, son orientation — qui décide du résultat.

Le fil conducteur tient en une phrase : **la morphologie remplace la pondération par le test de forme**. Là où la convolution vit dans un espace vectoriel (somme linéaire, superposition), la morphologie vit dans un **treillis** — une structure algébrique fondée sur l'ordre plutôt que sur l'addition. Tester si une forme tient, c'est déclarer quelle géométrie compte ; choisir un élément structurant, c'est encoder une hypothèse sur l'échelle et la direction des structures pertinentes. On retrouve, habillé différemment, le méta-fil constant du livre : le bon cadre rend le problème presque résolu.

Rappels de liens avec les autres chapitres. La morphologie est le prolongement non linéaire du filtrage (chapitre 5) : là où le filtre gaussien pondère pour lisser, l'ouverture morphologique sélectionne par la forme pour nettoyer. Le gradient morphologique (§11.3) mesure le même phénomène que Sobel (chapitre 6) — la transition d'intensité aux contours — mais par min/max au lieu de dérivation, ce qui change sa robustesse au bruit. Le top-hat (§11.4) accomplit en espace géométrique ce que le passe-haut fréquentiel fait en espace de Fourier (chapitre 10) : isoler les détails fins. Le squelette (§11.5) coïncide avec les crêtes de la transformée de distance (chapitre 10), unissant les deux langages. Enfin, la morphologie est l'outil canonique de post-traitement des masques produits par le seuillage (chapitre 12) et la segmentation (chapitre 4) : nettoyer, reboucher, séparer — les garanties géométriques que les réseaux de neurones ne fournissent pas.

11.1. Érosion et dilatation



binaire :

$$A \ominus B = \{ z : B_z \subseteq A \} \quad (\text{érosion})$$

$$A \oplus B = \{ z : \check{B}_z \cap A \neq \emptyset \} \quad (\text{dilatation})$$

niveaux de gris :

$$(I \ominus B)(x) = \min_{\{b \in B\}} I(x + b)$$

$$(I \oplus B)(x) = \max_{\{b \in B\}} I(x + b)$$

B désigne l'élément structurant (*structuring element*, SE), $\check{B}$ son symétrique par réflexion à l'origine, B_z le translaté de B au point z.

L'idée

L'image mentale la plus directe est celle d'un tampon-sonde que l'on pose en chaque point de l'image. En binaire, l'érosion répond à la question : « le tampon tient-il entièrement dans la forme à cet endroit ? » Si oui, le point est conservé ; sinon, il est effacé. La dilatation renverse la perspective : « la forme touche-t-elle au moins un point du tampon réfléchi positionné ici ? » Si oui, le point est allumé.

Pour les niveaux de gris, « tenir entièrement » se traduit par « la valeur minimale de la fenêtre » (érosion) et « toucher » par « la valeur maximale » (dilatation). L'érosion abaisse tout vers le bas ; la dilatation pousse tout vers le haut. Ce n'est pas une approximation : c'est la généralisation exacte, en algèbre de l'ordre, de ce qui était en binaire une logique ensembliste.

Dérivation : infimum et supremum de translatés

En binaire, réécrire $B_z \subseteq A$ comme « tout point de B translaté tombe dans A » permet d'exprimer l'érosion comme une intersection de translatés de A, et la dilatation comme une union (somme de Minkowski) :

$$A \ominus B = \cap_{\{b \in B\}} A_{-b}$$

$$A \oplus B = \cup_{\{b \in B\}} A_b$$

Pour les niveaux de gris, « $\cap$ » devient « min » et « $\cup$ » devient « max » : on passe du treillis $\{0, 1\}$ au treillis $[0, 255]$, où l'ordre remplace l'appartenance. La linéarité disparaît au passage — d'où le fait que la morphologie est fondamentalement différente de la convolution.



La dualité — l'identité à ne jamais oublier

Éroder l'objet revient à dilater le fond, et réciproquement :

$$(A \ominus B)^c = A^c \oplus \check{B}$$

$$(\text{niveaux de gris} : I \ominus B = -((-I) \oplus \check{B}))$$

Les deux opérateurs sont **adjoints** : comprendre l'érosion suffit à déduire la dilatation, et toute la suite (ouverture/fermeture, top-hat/black-hat) se décline en paires symétriques selon ce principe.

Ce que ça mesure / l'angle mort

L'érosion rétrécit les régions claires, supprime les détails clairs plus fins que B, élargit les zones sombres et déconnecte les ponts minces. La dilatation fait exactement l'inverse : épaissit, comble, fusionne. Angle mort majeur : **ce sont des opérations à perte, non inversibles**. Un détail plus étroit que la sonde, une fois effacé par l'érosion, est définitivement perdu. Éroder puis dilater ne reconstruit pas l'image d'origine — c'est l'ouverture (§11.2), qui est structurellement plus petite. Le fil conducteur du chapitre s'illustre ici : la sonde décide ce qui est « trop fin pour survivre ».



Signal 1-D $I = [3, 1, 4, 1, 5, 9, 2, 6]$, élément structurant plat de taille 3 (un pixel de chaque côté du centre), bords par réplication :

érosion (min sur la fenêtre glissante) : $[1, 1, 1, 1, 1, 2, 2, 2]$

dilatation (max sur la fenêtre glissante) : $[3, 4, 4, 5, 9, 9, 9, 6]$

Vérifions l'indice 5 (valeur 9) : fenêtre = $\{5, 9, 2\}$, min = 2, max = 9. L'érosion ramène 9 à 2 ; la dilatation propage 9 vers ses voisins. Le pic isolé disparaît en érosion mais contamine son entourage en dilatation. L'asymétrie est flagrante : $I \ominus B \leq I \leq I \oplus B$ point à point.



- **Convention d'objet.** En binaire, « éroder » signifie rétrécir les pixels *non nuls* (l'objet clair). Sur un masque inversé — fond blanc, objet sombre — l'érosion élargit l'objet. Toujours vérifier ce qui constitue le premier plan.
- **Réflexion de B.** La définition de la dilatation utilise $\check{B}$ (le SE réfléchi). `cv2.dilate` ne réfléchit pas le SE — sans conséquence pour un SE symétrique (carré, disque, croix), mais source d'erreur pour un SE asymétrique, exactement comme la confusion convolution/corrélation signalée au chapitre 5.
- **Bords.** L'érosion a besoin de valeurs hors image. Un `cv2.erode` avec `borderValue=0` consomme le bord de l'objet comme s'il était entouré de fond. Utiliser `morphologyDefaultBorderValue()` ou prolonger manuellement si les bords sont critiques.
- **Point d'ancrage.** Par défaut au centre du SE ; pour un SE rectangulaire non carré, l'ordre (row, col) vs (x, y) peut surprendre.



```
# a : masque binaire (HxW uint8, 0/255) ou image niveaux de gris
if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune entrée'}
else:
    B = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5)) # disque
    ~5 px
    ero = cv2.erode(a, B)
    dil = cv2.dilate(a, B)
    # différence : zones effacées par l'érosion
    diff = cv2.subtract(a, ero)
    out_a = {'pixels_retières_erosion': int(np.count_nonzero(diff))}
    out_b = ero      # masque érodé
    out_c = dil      # masque dilaté
```

11.2. Ouverture et fermeture



ouverture : $I \circ B = (I \ominus B) \oplus B$ (supprime les petits objets clairs)
 fermeture : $I \bullet B = (I \oplus B) \ominus B$ (referme les petits trous sombres)

L'idée

L'image mentale de l'ouverture est celle d'une sonde qui roule à l'intérieur de la forme. Partout où elle peut entrer, la forme est préservée ; partout où elle bute — parce que le passage est trop étroit, ou l'objet trop petit pour la contenir — la forme disparaît. Ce qui reste est exactement l'union de toutes les positions que la sonde a pu occuper :

$I \circ B = \text{union de tous les translatés de } B \text{ contenus dans } I$

C'est la caractérisation géométrique exacte : l'ouverture conserve tout ce qui est *plus grand que la sonde*, et supprime le reste. La fermeture est la duale : la sonde roule à l'extérieur, comblant les passages trop étroits pour qu'elle y pénètre.

Le fil conducteur s'exprime ici avec une netteté particulière : choisir B de taille 5 pixels, c'est déclarer que tout objet plus étroit que 5 pixels est du bruit ; choisir un SE en forme de trait horizontal, c'est déclarer que seules les structures horizontales méritent de survivre.

Dérivation et idempotence

L'ouverture et la fermeture héritent d'une propriété fondamentale : elles sont **idempotentes**. Réappliquer l'ouverture ne change plus rien :

$$(I \circ B) \circ B = I \circ B$$

Preuve : l'érosion est anti-extensive ($I \ominus B \leq I$) et la dilatation est extensive ($I \oplus B \geq I$) ; leur composition $\circ$ est croissante, anti-extensive ($I \circ B \leq I$) et idempotente — la définition algébrique d'un **filtre morphologique**. ■

Conséquence pratique souvent ignorée : itérer une ouverture est inutile. Pour nettoyer davantage, il faut agrandir B , pas répéter l'opération. `cv2.morphologyEx(..., iterations=3)` n'itère pas l'ouverture comme un tout — il itère l'érosion et la dilatation internes séparément, ce qui revient à ouvrir avec un SE plus grand.

Ordre garanti

$$I \circ B \leq I \leq I \bullet B$$

L'ouverture ne peut qu'abaisser, la fermeture qu'élever. Cela permet une vérification immédiate : si l'ouverture produit des pixels plus clairs que l'original, il y a un bogue (mauvais paramètre, convention inversée).

Granulométrie : la sonde comme tamis

En ouvrant successivement avec des SE de taille croissante et en mesurant l'aire perdue à chaque pas, on obtient une **distribution de tailles** des structures présentes — un tamisage virtuel. La courbe « aire survivante en fonction du rayon du SE » est l'équivalent morphologique d'un histogramme granulométrique (grains, gouttes, pores, cellules). C'est le

lien naturel avec le diamètre équivalent du chapitre 1 : ici, la taille d'un objet est définie par la plus grande sonde qui y entre.

Ce que ça mesure / l'angle mort

L'ouverture est un nettoyage par le bas : elle retire les saillies et taches claires plus fines que B. La fermeture est un colmatage par le haut : elle bouche les fissures et trous sombres plus étroits que B. L'angle mort réside entièrement dans le choix du SE : un B trop grand efface des structures voulues, un B mal orienté laisse passer le bruit. Et $\circ \neq \bullet$: ouvrir puis fermer ou l'inverse ne produit pas le même résultat ; il faut décider ce qui prime (nettoyage d'abord, ou colmatage d'abord).



Signal I = [3, 1, 4, 1, 5, 9, 2, 6], B plat de taille 3 :

érosion(I) = [1, 1, 1, 1, 1, 2, 2, 2]

ouverture = dil(ero(I)) = [1, 1, 1, 1, 1, 2, 2, 2] $\leq I$ ✓ anti-extensive

dilatation(I) = [3, 4, 4, 5, 9, 9, 9, 6]

fermeture = ero(dil(I)) = [3, 3, 4, 4, 5, 9, 6, 6] $\geq I$ ✓ extensive

L'ouverture a rasé le pic isolé 9 (indice 5, ramené à 2) parce qu'il est plus étroit que B. La fermeture a rebouché les vallées étroites 1 (indices 1 et 3, remontés à 3 et 4). C'est littéralement le retrait des petits objets et le comblement des petits trous.



- $\circ \neq \bullet$ **en pratique.** Sur un masque de cellules en microscopie, ouvrir d'abord (retirer le bruit blanc) puis fermer (reboucher les trous dans les noyaux) est l'ordre habituel ; l'inverser peut créer de faux positifs.
- **Forme du SE.** Une ouverture par un segment horizontal ne conserve que les structures horizontales : utile pour isoler les lignes d'une table de données numérisée ou les traits d'un texte imprimé.
- **Idempotence.** Sur OpenCV, `iterations=3` dans `morphologyEx` itère l'érosion puis la dilatation séparément — c'est une ouverture par un SE plus grand, pas une triple ouverture.



```
# a : masque binaire (HxW uint8, 0/255)
if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune entrée'}
else:
    B = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (7, 7))
    opened = cv2.morphologyEx(a, cv2.MORPH_OPEN, B)
    closed = cv2.morphologyEx(a, cv2.MORPH_CLOSE, B)
    retires = int(np.count_nonzero(a) - np.count_nonzero(opened))
    rebouche = int(np.count_nonzero(closed) - np.count_nonzero(a))
    out_a = {'pixels_supprimes_ouverture': retires,
            'pixels_ajoutes_fermeture': rebouche}
    out_b = opened
    out_c = closed
```

11.3. Gradient morphologique



$$\nabla_m I = (I \oplus B) - (I \ominus B)$$

gradient interne : $I - (I \ominus B)$	(bord côté objet)
gradient externe : $(I \oplus B) - I$	(bord côté fond)

L'idée

Pour comprendre le gradient morphologique, il faut penser à ce qui se passe dans une zone plate de l'image. Si l'intensité est constante autour d'un pixel, le maximum et le minimum de sa fenêtre locale sont égaux : leur différence est nulle. Près d'une transition entre une zone sombre et une zone claire, en revanche, la dilatation capture la valeur haute et l'érosion capture la valeur basse : leur différence révèle l'amplitude du saut. Le gradient morphologique mesure donc le **contraste local dans le voisinage défini par B**, sans jamais calculer de dérivée explicite.

Le lien avec le chapitre 6 est immédiat. Le gradient de Sobel calcule deux dérivées directionnelles (selon x et selon y) par convolution, puis combine leurs modules et angles pour obtenir magnitude et direction du gradient. Le gradient morphologique est plus simple : il ne donne que la magnitude, par min/max, sans orientation. Ce choix encode une hypothèse géométrique — on traite toutes les directions de façon identique (si B est un disque). C'est une nouvelle illustration du fil conducteur : la sonde encode l'isotropie ou l'anisotropie voulue.

Dérivation

Sur une zone plate (constante = c) : $(c \oplus B) = c$ et $(c \ominus B) = c$, donc $\nabla_m = 0$. Près d'une marche de hauteur h entre deux zones constantes : la dilatation monte d'un côté et l'érosion descend de l'autre, étalant la différence sur une largeur égale à la taille de B . La magnitude est exacte (h) mais la largeur du contour détecté est imposée par B , non par l'image. ■

Ce que ça mesure / l'angle mort

Comparé au gradient de Sobel (chapitre 6), le gradient morphologique est **plus robuste au bruit gaussien** (min/max sont moins sensibles à la moyenne que la dérivée) mais **plus sensible aux valeurs aberrantes** (un seul pixel très brillant gonfle le max de la fenêtre). Il ne fournit aucune information de direction. L'angle mort principal : l'épaisseur du contour détecté est proportionnelle à la taille de B — un grand SE donne des bords épais, ce qui nuit à la localisation fine (par exemple pour un watershed par marqueurs qui attend des contours d'un pixel).



Marche d'intensité $I = [10, 10, 10, 50, 50, 50]$, B plat de taille 3, bords répliqués :

```
dilatation = [10, 10, 50, 50, 50, 50]
érosion    = [10, 10, 10, 10, 50, 50]
∇m I      = [ 0,  0, 40, 40,  0,  0]
```

Le contour de hauteur 40 ($= 50 - 10$) est correctement détecté, mais étalé sur 2 pixels ($=$ taille de $B - 1$) de part et d'autre de la marche. La valeur est juste, la largeur est un artefact de la sonde — illustration directe du fil conducteur : le choix de B détermine la résolution spatiale du résultat.



- **Épaisseur** **taille de B**. Pour un watershed par marqueurs (lien chapitre 12), on veut un gradient fin : prendre $B = 3 \times 3$. Un gradient épais crée de faux bassins versants.
- **Sensibilité aux pics**. Un seul pixel aberrant gonfle le max et donc le gradient. Pré-filtrer avec une médiane (chapitre 5) si l'image est bruitée par impulsions sel-et-poivre.



```
# a : image niveaux de gris ou masque (HxW uint8)
if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune entrée'}
else:
    img = a if a.ndim == 2 else cv2.cvtColor(a, cv2.COLOR_BGR2GRAY)
    B = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3)) # fin,
    pour watershed
    grad = cv2.morphologyEx(img, cv2.MORPH_GRADIENT, B)
    out_a = {'gradient_max': int(grad.max()),
            'pixels_bord': int(np.count_nonzero(grad > 10))}
    out_b = grad # carte de gradient (image)
```

11.4. Top-hat (chapeau haut-de-forme) et black-hat



top-hat (white) :	$TH = I - (I \circ B)$	(objets clairs plus petits que B)
black-hat	: $BH = (I \bullet B) - I$	(objets sombres plus petits que B)

L'idée

L'idée repose sur une observation simple : si l'on ouvre une image avec un SE *plus grand que les objets d'intérêt*, l'ouverture efface ces petits objets et ne conserve que les variations lentes (l'éclairage de fond, le vignettage de l'objectif, un dégradé). Ce fond estimé est ensuite soustrait à l'image originale. Ce qui reste dans la différence, ce sont précisément les petits détails clairs que l'ouverture avait supprimés : ils réapparaissent, isolés du fond.

L'analogie utile est celle d'un projecteur éclairant une table de biais. Sur la table reposent de petits objets brillants (des pastilles de métal, des cellules fluorescentes, des étoiles). Le fond lui-même est inégalement éclairé. Un seuillage global ne peut pas séparer les objets du fond variable. Le top-hat, lui, estime le fond par la forme (l'ouverture ne « voit » que ce qui est plus grand que B, donc elle voit le fond et pas les objets) et le soustrait localement. Le seuil devient alors trivial.

C'est un **passer-haut géométrique** : il extrait les détails clairs plus petits que la sonde, quelles que soient les variations lentes du fond. Le black-hat est le dual et extrait les détails sombres. On retrouve le fil conducteur : B fixe la frontière entre « objet » et « fond » — choisir B, c'est déclarer à quelle échelle un détail mérite d'être vu.

Dérivation

Puisque $I \circ B \leq I$ (anti-extensif), la différence $TH = I - (I \circ B) \geq 0$ est toujours non négative. Les points où $TH = 0$ sont ceux où l'ouverture a préservé l'intensité (zones larges, fond) ; les points où $TH > 0$ sont ceux que l'ouverture avait abaissés (détails clairs plus fins que B). ■

Différence essentielle avec le passe-haut fréquentiel (chapitre 10) : le top-hat soustrait un fond estimé *par la forme*, localement, sans aucun calcul de fréquence. Il est insensible à un gradient d'éclairage lent, là où un passe-haut de Fourier laisserait des artefacts en bord de domaine.

Ce que ça mesure / l'angle mort

Top-hat = petits objets clairs sur fond quelconque ; black-hat = petits objets sombres. La condition de réussite est une **fenêtre d'échelle** bien choisie : B doit être plus grand que les objets à extraire, mais plus petit que les variations du fond. Si le fond varie à la même échelle que les objets, aucun SE ne sépare les deux — la méthode échoue.



Fond en rampe (éclairage non uniforme) $I = [10, 12, 14, 16, 48, 20]$ avec un détail clair à l'indice 4 (valeur 48 au lieu des 18 attendus). Ouverture par un grand B (taille 5, bords répliqués) :

ouverture (grand B) = $[10, 12, 14, 16, 16, 16] \approx$ la rampe (B trop large pour le pic)

top-hat = $I - \text{ouverture} = [0, 0, 0, 0, 32, 4]$

La rampe $[10, 12, 14, 16]$ est annulée ; seul le détail ressort (32 à l'indice 4). Le résidu 4 en fin est un artefact de bord (réplication) — signal d'avertissement : les bords sont la zone fragile de toute morphologie, sur une bande de largeur $\approx$ rayon du SE.



- **Choix de B critique.** Trop petit : le top-hat « voit » l'objet comme un élément du fond et le perd dans l'ouverture. Trop grand : coûteux en calcul et sensible aux artefacts de bord.
- **Bords.** Recadrer ou ignorer une bande de largeur $\approx$ rayon(B) en périphérie.
- **Espace linéaire vs gamma** (rappel chapitre 7). Soustraire des intensités a un sens photométrique en espace linéaire (après décodage gamma) ; en sRGB brut, la différence est biaisée par la courbe de gamma.



```
# a : image niveaux de gris (HxW uint8)
if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune entrée'}
else:
    img = a if a.ndim == 2 else cv2.cvtColor(a, cv2.COLOR_BGR2GRAY)
    B = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (31, 31)) # plus
    grand que les détails visés
    tophat = cv2.morphologyEx(img, cv2.MORPH_TOPHAT, B)
    blackhat = cv2.morphologyEx(img, cv2.MORPH_BLACKHAT, B)
    out_a = {'tophat_max': int(tophat.max()),
            'blackhat_max': int(blackhat.max())}
    out_b = tophat # détails clairs isolés du fond
    out_c = blackhat # détails sombres isolés du fond
```

11.5. Transformée tout-ou-rien et squelette

L'idée : deux sondes au lieu d'une

Tous les opérateurs précédents *transforment* l'image. La transformée tout-ou-rien (*hit-or-miss*, HMT) fait autre chose : elle **détecte** une configuration géométrique exacte. Au lieu d'une sonde, elle en utilise deux, disjointes. La première (B_1) décrit la forme que le premier plan doit respecter ; la seconde (B_2) décrit la forme que le fond doit respecter autour du premier plan :

$$I \otimes (B_1, B_2) = (I \ominus B_1) \cap (I^c \ominus B_2)$$

Un pixel survit uniquement si B_1 tient dans l'objet *et simultanément* B_2 tient dans le fond environnant. C'est de la mise en correspondance de motif binaire : détection de coins, d'extrémités de traits, de pixels isolés.

L'idée centrale est encore une fois le test de forme, mais élevé à une précision maximale. Le fil conducteur atteint son expression la plus pure : non seulement on déclare la forme qui doit être présente, mais aussi celle qui doit être absente.

Le squelette : l'axe médian d'une forme

Le squelette morphologique réduit une forme à son **axe médian** — le lieu des centres des plus grands disques inscrits dans la forme. C'est l'ensemble de tous les points qui sont simultanément les plus éloignés possibles de plusieurs parties du bord.

L'image mentale : prenez une lettre imprimée, par exemple un « B ». Son squelette trace une ligne verticale centrale (la colonne) et deux petites branches horizontales (les panses).

Tout ce qui détermine la topologie de la forme — ses connexions, ses branches, sa longueur — est présent dans ce squelette d'un pixel d'épaisseur.

La formulation mathématique par seuillage de l'érosion itérée est :

$$S(A) = \bigcap_k [(A \ominus kB) - (A \ominus kB) \circ B] \quad (kB = B \text{ érodé } k \text{ fois})$$

C'est le pont annoncé avec le chapitre 10 : le squelette coïncide exactement avec les **crêtes de la transformée de distance** (les points où la DT est localement maximale), et éroder par un disque de rayon r revient à seuiller la DT à la valeur r . Morphologie et transformée de distance décrivent la même réalité — l'épaisseur et la centralité d'une forme — l'une par sondes successives, l'autre par carte de distances.

Ce que ça mesure / l'angle mort

La détection tout-ou-rien est précise au pixel près : très efficace pour les motifs réguliers (texte, circuits électroniques, biologie structurée), mais fragile au bruit — un seul pixel manquant dans le motif annule la détection. Le squelette est topologiquement riche (il préserve le nombre de branches et la connectivité) mais **instable géométriquement** : une petite bosse sur le contour crée une branche parasite (« barbe »). Ces deux opérateurs supposent un masque binaire propre — d'où la nécessité d'un nettoyage préalable par ouverture ou fermeture (§11.2).



Détecter les **pixels isolés** sur un masque : B_1 = un seul pixel central allumé, B_2 = ses 8 voisins (anneau de fond). Un pixel d'objet entouré uniquement de fond satisfait les deux conditions ; tout pixel ayant au moins un voisin objet est rejeté. La HMT renvoie exactement la carte des pixels solitaires — utile pour compter ou éliminer le bruit « poivre » d'un masque binaire, complément du bruit « sel » que l'ouverture retire.



- **Encodage du noyau** dans `cv2.MORPH_HITMISS` : +1 (premier plan requis), -1 (fond requis), 0 (indifférent). L'image doit être binaire mono-canal `CV_8U` (valeurs 0 ou 255) — pas 0/1.
- **`skimage.morphology.skeletonize` vs `medial_axis`**. `medial_axis` renvoie également la transformée de distance (épaisseur le long de l'axe médian), utile pour reconstruire la forme originale ou mesurer l'épaisseur locale. `skeletonize` ne donne que la topologie. Les algorithmes (Zhang-Suen, Lee) produisent des squelettes légèrement différents : comparer sur les données réelles.



```
# a : masque binaire (H×W uint8, 0/255)
if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune entrée'}
else:
    from skimage.morphology import skeletonize, medial_axis
    m = (a > 127) # bool H×W
    skel = skeletonize(m).astype(np.uint8) * 255
    skel_ma, dist = medial_axis(m, return_distance=True)
    epaisseur_max = float(dist[skel_ma].max()) * 2 # diamètre maximal
inscrit
    out_a = {'longueur_squelette_px': int(skel.astype(bool).sum()),
            'epaisseur_max_px': epaisseur_max}
    out_b = skel # axe médian (image binaire)

# détection de pixels isolés (hit-or-miss)
k = np.array([[-1, -1, -1], [-1, 1, -1], [-1, -1, -1]], np.int8)
_, m_bin = cv2.threshold(a, 127, 255, cv2.THRESH_BINARY)
isoles = cv2.morphologyEx(m_bin, cv2.MORPH_HITMISS, k)
out_c = isoles
out_a['pixels_isoles'] = int(np.count_nonzero(isoles))
```

11.6. Tableau récapitulatif – chaque opérateur est un choix de sonde

Opérateur	Test de forme	Effet géométrique	Angle mort	Usage type
Érosion $\ominus$	min local sur B	rétrécit le clair, perd les détails fins	perte irréversible	séparation d'objets proches, nettoyage grossier
Dilatation $\oplus$	max local sur B	épaissit le clair, comble les lacunes fines	fusion d'objets proches	reconnexion, marqueurs Watershed
Ouverture $\circ$	sonde qui roule à l'intérieur	supprime objets clairs < B (anti-extensif)	B mal calibré efface le signal utile	retrait de bruit, granulométrie

Opérateur	Test de forme	Effet géométrique	Angle mort	Usage type
Fermeture •	sonde qui roule à l'extérieur	bouche trous sombres < B (extensif)	B mal calibré noie les détails	colmatage de masques (vaisseaux, fissures)
Gradient ∇_m	amplitude du saut local	contour isotrope, magnitude seule	épaisseur imposée par B, pas d'orientation	pré-Watershed, détection de bords robuste
Top-hat	différence après ouverture	détails clairs < B sur fond quelconque	B doit encadrer l'échelle objet/fond	OCR sur fond inégal, microscopie fluorescente
Black-hat	différence après fermeture	détails sombres < B sur fond quelconque	idem top-hat	défauts sombres en contrôle qualité, télédétection
Hit-or-miss •	deux sondes simultanées (fg + bg)	détecte un motif géométrique exact	fragile au bruit (1 pixel manquant = échec)	coins, extrémités, pixels isolés
Squelette	crêtes de la transformée de distance	axe médian, topologie préservée	barbes sur les contours gueux	signature de forme, mesure d'épaisseur

11.7. tester une forme, c'est déclarer ce qui compte

Le fil conducteur de ce chapitre — **la morphologie remplace la pondération par le test de forme** — est la déclinaison géométrique du méta-fil constant du livre. À chaque chapitre, un objet central condense l'hypothèse sur « ce qui compte » :

chapitre 3 (distances) : une métrique → ce qui rend deux choses proches
 chapitre 5 (filtrage) : un noyau → un a priori sur la régularité du signal
 chapitre 6 (gradients) : une dérivée → ce qui constitue une transition significative
 chapitre 10 (transformées) : une base → le domaine où le problème devient simple
 chapitre 11 (morphologie) : un élément structurant → la forme et l'échelle qui comptent

La morphologie pousse l'idée à sa forme la plus géométrique. On ne pondère pas, on ne projette pas : on teste si une forme tient. Tout le travail réside dans le choix de la sonde — un disque pour rester isotrope, un segment orienté pour ne garder qu'une direction, un grand SE pour estimer un fond lentement variable, deux SE pour reconnaître un motif exact. Ce choix encode une hypothèse géométrique aussi précise qu'une dérivée ou une base de Fourier, mais formulée dans le langage de l'ordre plutôt que du calcul.

La conséquence pratique est directe : les opérateurs eux-mêmes (un min, un max, une différence) sont triviaux à calculer. Ce qui est difficile, c'est de choisir B — de déclarer, avant de toucher à l'image, quelle géométrie mérite d'être vue et à quelle échelle. Une fois B bien posé, le résultat est presque acquis.

Ce positionnement détermine aussi la place de la morphologie dans les pipelines modernes. Les réseaux de neurones profonds apprennent *quoi* segmenter, mais n'offrent aucune garantie de forme sur les masques produits. La morphologie impose *quelle géométrie* le masque doit respecter : combler les trous, séparer les jonctions, nettoyer le bruit géométrique. Les deux approches sont complémentaires — le réseau repère l'objet, la morphologie lui donne une forme propre. La morphologie reste ainsi indispensable comme post-traitement géométrique, quel que soit le moteur de segmentation en amont (seuillage du chapitre 12, clustering du chapitre 4, ou segmentation profonde).

Et la boucle se ferme avec les descripteurs du chapitre 1 : éroder, ouvrir, squelettiser, c'est déjà définir des propriétés de forme (épaisseur, taille, topologie) avant même de calculer un descripteur. La morphologie n'est pas une étape de pré-traitement accessoire — c'est déjà une lecture de la géométrie de l'image, formulée dans l'algèbre de l'ordre.

CHAPITRE

Seuillage et segmentation classique

12

Table des matières

12.1. Seuillage d'Otsu — la décision sur l'histogramme	194
12.2. Seuillage adaptatif — la décision locale	197
12.3. K-means — la décision dans l'espace de caractéristiques	199
12.4. Mean-shift — la décision par les modes de densité	201
12.5. Contours actifs (snakes) — la décision sur une frontière	202
12.6. Coupe de graphe (graph cut) — la décision globale et cohérente	205
12.7. Tableau récapitulatif — méthode, a priori, forces, limites, usage	207
12.8. données contre régularité, le curseur de toute segmentation	208

Segmenter, c'est partitionner l'image : décider, pour chaque pixel, à quelle région il appartient. Avant l'apprentissage profond, les méthodes classiques couvrent tout un spectre de stratégies — trancher pixel par pixel sur l'intensité (seuillage), regrouper dans un espace de caractéristiques (*clustering*), ou faire émerger les régions d'une optimisation globale (contours actifs, coupe de graphe). Ce chapitre les construit dans cet ordre, en montrant que la difficulté n'est jamais le calcul mais le **choix de ce qu'on suppose** d'une bonne segmentation.

L'image seule ne dit pas où couper. Le bruit la rend ambiguë, les objets se fondent dans des fonds similaires, l'éclairage crée des dégradés qui masquent les vraies frontières. Fil conducteur : **toute segmentation arbitre entre fidélité aux données et a priori de régularité**. Chaque méthode ajoute une hypothèse — deux classes d'intensité, des amas compacts, une frontière lisse, des voisins cohérents — et place la décision dans l'espace où cette hypothèse devient une simple optimisation. La progression du chapitre va du **purement local et sans a priori spatial** (seuillage : chaque pixel seul, donc bruité) au **globalement cohérent** (graph cut : données + lissage, donc robuste).

Rappel des liens. La segmentation classique est un carrefour du livre. Le couple attache aux données / régularité de Horn-Schunck (§9.4, flot optique) trouve ici son pendant exact dans l'énergie du graph cut (§12.6) : c'est le même archétype, appliqué à des étiquettes plutôt qu'à des vecteurs de mouvement. Les méthodes par *clustering* (§12.3–§12.4) reposent sur une distance choisie entre pixels (chapitre 3) ; segmenter en couleur se fait souvent en espace Lab plutôt qu'en RGB (chapitre 7). Les masques que le seuillage produit se nettoient ensuite par morphologie (chapitre 11), et toute segmentation s'évalue par IoU et Dice (chapitre 4).

12.1. Seuillage d'Otsu – la décision sur l'histogramme



$$\sigma^2_{\text{inter}}(t) = \omega_0(t) \cdot \omega_1(t) \cdot [\mu_0(t) - \mu_1(t)]^2$$

$$T^* = \operatorname{argmax}_t \sigma^2_{\text{inter}}(t)$$

ω_0, ω_1 = proportions des deux classes (pixels $\leq t$ et $> t$) ; μ_0, μ_1 = leurs moyennes d'intensité.

L'idée — couper au creux de l'histogramme

Un histogramme d'image, c'est simplement le portrait-robot statistique de l'image : pour chaque valeur d'intensité possible (0 à 255), il indique combien de pixels ont cette valeur. Quand une image contient un objet clair sur un fond sombre — un document imprimé, une

cellule colorée sous microscope, un grain de métal sur tapis d'inspection — le résultat est un histogramme en forme de double bosse : une première bosse pour le fond, une creuse séparée, une seconde bosse pour l'objet. Otsu cherche automatiquement le fond de cette vallée.

L'image mentale utile est celle d'un arbitre qui coupe une file d'attente en deux groupes aussi homogènes que possible. Pour chaque seuil t candidat, il divise les pixels en « classe sombre » (tout ce qui vaut $\leq t$) et « classe claire » (tout ce qui vaut $> t$), puis calcule à quel point les deux groupes sont intérieurement cohérents. Quand le seuil tombe dans la vallée, les deux groupes sont séparés et chacun est homogène : c'est le bon seuil.

Dérivation

La variance totale de l'histogramme se décompose en deux parts :

$$\sigma^2_{\text{total}} = \sigma^2_{\text{intra}}(t) + \sigma^2_{\text{inter}}(t)$$

σ^2_{total} ne dépend pas de t — c'est une propriété fixe de l'image. Donc **maximiser la variance inter-classes équivaut à minimiser la variance intra-classes** : trouver le seuil qui rend chaque groupe le plus homogène possible. La variance inter-classes admet la forme fermée ci-dessus (ne nécessitant que des moyennes cumulées), ce qui permet de la calculer pour les 256 seuils possibles en un seul balayage de l'histogramme. ■

Le seuil d'Otsu place donc la décision dans l'espace de l'**histogramme** plutôt que dans l'espace de l'image. C'est un résumé qui jette la position des pixels — exactement comme les moments du chapitre 2 — et ne garde que la distribution des intensités.

Ce que ça mesure / l'angle mort

Otsu suppose un histogramme **bimodal** : deux modes (objet, fond) séparés par une vallée. Son angle mort est entier dans cette hypothèse. Sur un histogramme unimodal (fond sans contraste net), ou quand une classe est minuscule — un petit défaut sur une grande surface lisse : l'histogramme est écrasé par la surface —, Otsu place le seuil n'importe où. Il suppose aussi des classes de variances comparables ; un fond très dispersé le biaise. Le fil conducteur l'exprime ainsi : l'a priori ici est « deux classes d'intensité bien séparées » — quand cet a priori est faux, la fidélité aux données (l'histogramme) ne suffit plus.



Histogramme bimodal sur les niveaux 0–4, 16 pixels au total : effectifs [5, 3, 0, 3, 5].

seuil $t = 1$ (classe 0 = {0,1}, classe 1 = {2,3,4}) :

$$\omega_0 = 8/16 = 0.5 \quad \mu_0 = (0 \cdot 5 + 1 \cdot 3)/8 = 0.375$$

$$\omega_1 = 8/16 = 0.5 \quad \mu_1 = (3 \cdot 3 + 4 \cdot 5)/8 = 3.625$$

$$\sigma^2_{\text{inter}} = 0.5 \cdot 0.5 \cdot (0.375 - 3.625)^2 = 0.25 \cdot 10.5625 \approx 2.64 \quad \leftarrow \text{maximum}$$

seuil $t = 0$ (ou $t = 3$) :

$$\sigma^2_{\text{inter}} \approx 1.82$$

Le maximum tombe dans la **vallée** de l'histogramme (entre les deux modes), exactement là où l'intuition placerait le seuil. Otsu n'a fait que formaliser « couper au creux ».



`cv2.THRESH_OTSU` opère sur une image **mono-canal en 8 bits** (histogramme à 256 cases) ; convertir en niveaux de gris avant de l'appeler. Sur une image à éclairage inégal, le seuil global est le mauvais outil ($\rightarrow$ §12.2) : regarder toujours l'histogramme avant de faire confiance à Otsu. Un flou léger (chapitre 5) avant l'appel réduit le bruit qui creuse de fausses vallées dans l'histogramme.



```
# a : image BGR (bleu) ou masque – on convertit en niveaux de gris
if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune image en entree'}
else:
    gray = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY) if a.ndim == 3 else a
    T, mask = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY +
cv2.THRESH_OTSU)
    out_a = {'seuil_otsu': float(T)}
    out_b = mask # masque binaire résultant (connecter à
output_display)
```

Domaines : binarisation de documents imprimés (OCR), tri de grains sur fond contrasté, masquage de cellules colorées en microscopie après passage en niveaux de gris.

12.2. Seuillage adaptatif – la décision locale



$$T(x, y) = \text{moyenne_locale}(x, y) - C$$

Le pixel est objet si $I(x, y) < T(x, y)$ (convention texte sombre sur fond clair). La variante gaussienne pondère les voisins par une gaussienne centrée en (x, y) .

L'idée — suivre le fond plutôt que le trancher

Otsu pose **un** seuil pour toute l'image. Si l'éclairage varie lentement — une page photographiée sous une lampe oblique, une plaque industrielle éclairée par le côté —, aucun seuil global ne convient : trop bas dans la zone d'ombre, on perd l'objet ; trop haut dans la zone lumineuse, on noie le fond dans l'objet.

L'image mentale est celle d'un niveau à bulle qui s'adapte à la pente du terrain plutôt que de chercher une horizontale unique. On estime le fond localement par une moyenne de voisinage, et on seuille par rapport à lui. L'a priori est explicite : **le fond varie doucement à l'échelle de la fenêtre** — un a priori de régularité spatiale que le top-hat morphologique (chapitre 11) exploitait déjà différemment. La constante C est une marge qui évite de classer le bruit de fond comme objet.

Dérivation

Si la luminosité locale suit un fond lentement variable $f(x, y)$ et que l'objet est toujours C niveaux en dessous de ce fond, alors $I(x, y) - f(x, y) < -C$ quand et seulement quand le pixel appartient à l'objet. Estimer $f(x, y)$ par la moyenne de voisinage suffit quand la taille de la fenêtre est grande devant les détails de l'objet mais petite devant les variations du fond. ■

Ce que ça mesure / l'angle mort

On gagne la robustesse à l'éclairage inégal, on perd la vision globale : un objet **plus grand que la fenêtre** devient invisible (son intérieur ressemble localement à du fond uniforme). L'angle mort est la **taille de fenêtre**, paramètre critique. Ici encore le fil conducteur joue : l'a priori spatial (fond lentement variable) résout l'ambiguïté que la seule donnée locale ne pouvait trancher.



Une ligne de pixels avec un fond en dégradé [200, 200, 100, 100] et du texte toujours 50 niveaux sous le fond local : un pixel de texte vaut 150 en zone claire, 50 en zone sombre.

seuil global (Otsu $\approx$ 125) :

texte clair 150 > 125 $\rightarrow$ classé FOND (raté)

fond sombre 100 < 125 $\rightarrow$ classé TEXTE (raté)

seuil adaptatif $T = \text{moyenne_locale} - 20$ ($\approx$ suit le fond) :

zone claire : $T \approx 180 \rightarrow$ texte 150 < 180 $\rightarrow$ TEXTE $\checkmark$, fond 200 $\rightarrow$ FOND $\checkmark$

zone sombre : $T \approx 80 \rightarrow$ texte 50 < 80 $\rightarrow$ TEXTE $\checkmark$, fond 100 $\rightarrow$ FOND $\checkmark$

Le seuil local **suit** le fond et tranche correctement partout ; le seuil global échouait des deux côtés.



`blockSize` doit être **impair** dans `cv2.adaptiveThreshold`. Trop petit, l'intérieur des traits épais se vide (le seuil regarde si peu de voisins qu'il confond l'intérieur d'un trait avec le fond) ; trop grand, on retombe sur du global et l'éclairage inégal ressurgit. La variante gaussienne (`ADAPTIVE_THRESH_GAUSSIAN_C`) lisse les transitions et évite les artefacts de bloc visibles avec la variante moyenne.



```
# a : image BGR (bleu)
if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune image en entree'}
else:
    gray = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY) if a.ndim == 3 else a
    mask = cv2.adaptiveThreshold(
        gray, 255,
        cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY_INV,
        blockSize=21, C=8
    )
    out_a = {'pixels_objet': int(np.sum(mask > 0))}
    out_b = mask
```

Domaines : OCR sur pages photographiées sous éclairage oblique, lecture de plaques sous éclairage inégal, détection de défauts sur surfaces non uniformément éclairées.

12.3. K-means – la décision dans l'espace de caractéristiques



$$\operatorname{argmin}_S \sum_k \sum_{\{x \in S_k\}} \|x - \mu_k\|^2$$

On cherche la partition $S = \{S_1, \dots, S_K\}$ des pixels (décrits par un vecteur x de caractéristiques) qui minimise la dispersion intra-amas autour des centres μ_k .

L'idée — regrouper ce qui se ressemble

Jusqu'ici, chaque pixel décidait seul sur la base de son intensité. K-means fait un saut conceptuel : il projette chaque pixel dans un **espace de caractéristiques** (sa couleur RGB, ou sa couleur + sa position), puis y cherche des **groupes naturels**. L'image mentale est celle de nuages de points dans l'espace des couleurs : K-means place K centres et attire chaque point vers le centre le plus proche, comme K aimants.

La décision quitte l'espace image pour un **espace de caractéristiques**. Choisir ces caractéristiques et leur échelle, c'est déclarer ce qui rapproche deux pixels — l'esprit même du chapitre 3. Inclure les coordonnées spatiales (x, y) en plus de la couleur favorise des régions compactes ; les omettre permet à des zones disjointes de couleur similaire d'être dans le même groupe.

Dérivation — descente alternée

Le problème est combinatoire (NP-difficile en général), mais l'algorithme de Lloyd descend l'énergie par **deux étapes alternées** dont chacune ne peut que la diminuer :

1. Affectation : chaque pixel va au centre le plus proche (fixe μ , optimise S)
2. Mise à jour : chaque centre devient la moyenne de ses pixels (fixe S , optimise μ)

L'étape 2 minimise exactement $\sum \|x - \mu_k\|^2$ à affectation fixée, car la moyenne est le point qui minimise la somme des carrés des distances (rappel chapitre 3 : la moyenne minimise l'erreur L2). L'énergie décroît à chaque tour et le procédé converge — vers un **minimum local**. ■

L'a priori ici est explicite : K amas **sphériques de tailles comparables**. Quand les groupes réels ont des formes allongées, imbriquées, ou des densités très différentes, cet a priori se trompe et le résultat est décevant.

Ce que ça mesure / l'angle mort

K-means impose K amas et une norme L2 isotrope. Il échoue sur des amas allongés ou imbriqués. Surtout, **K doit être fixé a priori** et le résultat dépend de l'initialisation : deux lancements différents peuvent donner deux segmentations différentes sur la même image.



Points 1-D : $\{1, 2, 9, 10, 11\}$, $K = 2$, centres initiaux $c_1 = 1$, $c_2 = 10$.

Affectation : $\{1, 2\} \rightarrow c_1$ $\{9, 10, 11\} \rightarrow c_2$

Mise à jour : $c_1 = (1+2)/2 = 1.5$ $c_2 = (9+10+11)/3 = 10$

Ré-affectation : inchangée $\rightarrow$ convergence

Amas $\{1, 2\}$ (centre 1.5) et $\{9, 10, 11\}$ (centre 10). Une initialisation différente ($c_1 = 2$, $c_2 = 9$) converge ici au même résultat, mais sur des données réelles un mauvais départ piège l'algorithme dans un minimum local — d'où **k-means++** qui disperse les centres initiaux.



Combiner couleur (0–255) et position (0–2000 px) sans normaliser laisse la position écraser la couleur : l'espace couleur n'a plus aucune influence. Pondérer explicitement. Segmenter en **Lab** (distances $\approx$ perceptuelles, chapitre 7) plutôt qu'en RGB donne des amas plus proches de la perception humaine. Lancer plusieurs fois (attempts=5) et garder la meilleure énergie avec KMEANS_PP_CENTERS.



```
# a : image BGR (bleu)
if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune image en entree'}
else:
    K = 4 # nombre de classes
    Z = a.reshape(-1, 3).astype(np.float32)
    crit = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 20, 1.0)
    _, labels, centers = cv2.kmeans(
        Z, K, None, crit, attempts=5, flags=cv2.KMEANS_PP_CENTERS
    )
    seg = centers[labels.flatten()].reshape(a.shape).astype(np.uint8)
    out_a = {'nb_classes': K, 'pixels_par_classe':
[int(np.sum(labels==k)) for k in range(K)]}
    out_b = seg # image segmentée
```

Domaines : segmentation couleur en télédétection (couverture végétale, zones urbaines), quantification de couleurs pour compression, classification de types de tissus en histologie.

12.4. Mean-shift – la décision par les modes de densité



$$m(x) = \left[\sum_i x_i \cdot K(\|x - x_i\|^2 / h^2) \right] / \left[\sum_i K(\|x - x_i\|^2 / h^2) \right] - x$$

À chaque pas, on déplace x vers la moyenne pondérée de ses voisins (noyau K , fenêtre de largeur h), puis on itère jusqu'à un point fixe.

L'idée — remonter vers les sommets de densité

K-means cherche K centres prédéfinis. Mean-shift ne suppose pas le nombre de groupes : il **détecte les zones denses** de l'espace de caractéristiques et fait converger chaque point vers le sommet de densité le plus proche. L'image mentale est celle de billes dans un paysage en relief : chaque bille remonte naturellement vers la bosse la plus proche. Les billes qui finissent sur le même sommet appartiennent au même groupe.

Le vecteur $m(x)$ pointe vers la zone la plus dense du voisinage. On démontre qu'il est **proportionnel au gradient de l'estimateur à noyau de la densité** des points : itérer $x \leftarrow x + m(x)$ est une montée de gradient qui converge vers un **mode** (maximum local de densité). Les points qui convergent vers le même mode forment un amas. Le nombre de modes — donc de groupes — **émerge de la densité et de la seule largeur h** .

Ce que ça mesure / l'angle mort

Mean-shift épouse des amas de **forme arbitraire** (pas d'hypothèse sphérique) et trouve K automatiquement. L'angle mort se déplace sur h : tout le comportement en dépend. h petit $\rightarrow$ beaucoup de petits modes (sur-segmentation) ; h grand $\rightarrow$ tout fusionne (sous-segmentation). Et le coût est élevé ($\approx O(n^2)$ par itération sur des données non structurées), ce qui le rend lent sur de grandes images. Ici encore, l'a priori « des modes de densité existent et sont séparés par une largeur h » est la clé de voûte.



Points 1-D groupés autour de 2 et de 10, largeur $h = 3$, départ en $x = 4$:

Voisins dans $[1, 7]$: surtout $\{1, 2, 3\} \rightarrow$ moyenne $\approx 2 \rightarrow x$ glisse vers 2

Itération suivante depuis ~ 2 : point fixe atteint $\rightarrow$ mode = 2

Un départ en $x = 8$ converge vers le mode 10. Deux modes émergent **sans qu'on ait dit « $K = 2$ »** : c'est la densité qui décide, pas le programmeur.



`h` est le paramètre et n'a pas de bon réglage universel. Le choisir par l'échelle attendue des structures : si les objets d'intérêt couvrent 30 niveaux de gris, $h \approx 15-20$ est un bon point de départ. `cv2.pyrMeanShiftFiltering` prend deux rayons (spatial `sp`, couleur `sr`) : c'est un mean-shift conjoint position+couleur, qui lisse en préservant les bords — cousin du filtre bilatéral (chapitre 5). Pour un clustering pur, `sklearn.cluster.MeanShift` est plus flexible.



```
# a : image BGR (bleu)
if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune image en entree'}
else:
    # lissage par mean-shift (préserve les bords, comme un filtre
    # bilatéral adaptatif)
    shifted = cv2.pyrMeanShiftFiltering(a, sp=20, sr=30)
    # compter les couleurs uniques restantes donne une idée du nombre de
    # régions
    flat = shifted.reshape(-1, 3)
    unique_colors = len(np.unique(flat, axis=0))
    out_a = {'couleurs_distinctes_apres_shift': int(unique_colors)}
    out_b = shifted
```

Domaines : simplification d'images satellite pour pré-traitement, détection de régions d'intérêt sans K connu a priori, lissage de textures en conservant les contours.

12.5. Contours actifs (snakes) – la décision sur une frontière



$$E = \int \left[\underbrace{\alpha \|v'(s)\|^2}_{\text{élasticité}} + \underbrace{\beta \|v''(s)\|^2}_{\text{rigidité}} + \underbrace{E_{\text{image}}(v(s))}_{\text{attache aux données}} \right] ds$$

$v(s)$ = courbe paramétrée ; on cherche la courbe qui minimise E .

L'idée — une courbe élastique attirée par les bords

Toutes les méthodes précédentes étiquettent les pixels **indépendamment** (chaque pixel décide pour lui-même, puis on agrège). Le snake adopte une philosophie radicalement différente : on **place une courbe** dans l'image et on la laisse se déformer jusqu'à ce qu'elle s'accroche aux bords de l'objet. L'image mentale est celle d'un anneau en caoutchouc qu'on

pose à proximité d'un objet : il se contracte, reste souple, mais est attiré par les forts gradients — les contours de l'objet.

Trois forces s'équilibrent en permanence :

- **L'élasticité** (terme α) tend à rapprocher les points de la courbe, comme un élastique qui se contracte.
- **La rigidité** (terme β) pénalise les courbures trop brusques, comme un ressort qui résiste aux pliures.
- **L'attache aux données** ($E_{\text{image}} = -\|\nabla I\|^2$) attire la courbe vers les zones de fort gradient, c'est-à-dire les contours détectés (chapitre 6).

Dérivation

Les deux premiers termes sont l'a priori de régularité : **une frontière d'objet physique est lisse**. Le troisième est l'attache aux données. La minimisation passe par les équations d'Euler-Lagrange, résolues itérativement en discrétisant la courbe en N points — exactement la mécanique variationnelle de Horn-Schunck (§9.4), appliquée cette fois à une courbe plutôt qu'à un champ de vecteurs. Le fil conducteur est littéral : α et β dosent la régularité, l'énergie image dose la fidélité aux données. ■

Ce que ça mesure / l'angle mort

Le snake produit un contour **lisse et fermé**, idéal quand il y a un objet unique aux frontières douces — un organe en imagerie médicale, une silhouette en vidéo. Angles morts : il faut l'**initialiser près** de l'objet (sinon il se coince sur un faux contour ou dégénère), il ne **change pas de topologie** (une courbe ne peut se scinder pour entourer deux objets séparés), et il bute dans les concavités profondes (d'où la *gradient vector flow* qui propage l'attraction loin des bords).



Le terme d'élasticité préfère un espacement **régulier** des points. Pour trois points consécutifs, $\|v'\|^2 \approx$ somme des carrés des écarts :

Espacement (10, 10) : $\alpha(10^2 + 10^2) = 200 \alpha$

Espacement (5, 15) : $\alpha(5^2 + 15^2) = 250 \alpha$ (plus coûteux)

À longueur totale égale, la configuration régulière a l'énergie interne la plus basse. Le snake répartit ses points uniformément, tandis que l'énergie image les tire vers les bords : la forme finale est le **compromis** entre ces deux tendances, réglé par α et β .



L'initialisation est critique : `skimage.segmentation.active_contour` part d'une courbe fournie ; mal placée, le résultat est faux. Pour des objets multiples ou de topologie variable (une lésion qui se divise, des cellules en mitose), utiliser les **ensembles de niveaux** (*level sets*, `morphological_chan_vese`), qui gèrent fusions et scissions automatiquement. Lisser $\|\nabla I\|^2$ par un flou gaussien avant calcul élargit le bassin d'attraction du contour et stabilise la convergence.



```
# a : image BGR (bleu) – le snake s'initialise comme un cercle
from skimage.segmentation import active_contour
from skimage.filters import gaussian

if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune image en entree'}
else:
    gray = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY) if a.ndim == 3 else a
    gray_f = gray.astype(np.float32) / 255.0
    h, w = gray_f.shape

    # cercle initial centré sur l'image
    s = np.linspace(0, 2 * np.pi, 200)
    r_init = h // 3
    init = np.array([
        h / 2 + r_init * np.sin(s),
        w / 2 + r_init * np.cos(s)
    ]).T

    snake = active_contour(
        gaussian(gray_f, sigma=3),
        init, alpha=0.015, beta=10, gamma=0.001
    )
    out_a = {
        'nb_points_snake': len(snake),
        'centre_approx_y': float(np.mean(snake[:, 0])),
        'centre_approx_x': float(np.mean(snake[:, 1]))
    }
```

Domaines : segmentation d'organes en imagerie médicale (contours lisses et réguliers), suivi de silhouette en vidéo, délimitation de cellules isolées en microscopie de fluorescence.

12.6. Coupe de graphe (graph cut) – la décision globale et cohérente



$$E(L) = \underbrace{\sum_p D_p(L_p)}_{\text{terme de données}} + \lambda \underbrace{\sum_{\{p,q\}} V(L_p, L_q)}_{\text{terme de lissage}}$$

L = étiquetage de tous les pixels (objet/fond) ; D_p = coût de donner l'étiquette L_p au pixel p ; V = coût d'un désaccord entre voisins p et q ; λ = poids de régularité.

L'idée — l'image comme un réseau de tuyaux

Toutes les méthodes précédentes prennent des décisions **locales** ou en **espace de caractéristiques sans géométrie**. Le graph cut prend la décision **globalement**, en modélisant l'image entière comme un réseau. L'image mentale est celle d'un réseau de tuyaux hydrauliques : chaque pixel est un carrefour, deux terminaux représentent « objet » et « fond », et chaque tuyau a une capacité. Segmenter revient à couper les tuyaux pour séparer les deux terminaux en dépensant le moins possible — c'est le **minimum cut**, dual du maximum flow de la théorie des graphes.

Concrètement : les tuyaux vers les terminaux portent les **coûts de données** (à quel point ce pixel ressemble à l'objet ou au fond, d'après sa couleur ou des graines tracées). Les tuyaux entre pixels voisins portent les **coûts de lissage** (élevés si les deux pixels se ressemblent, pour décourager une frontière au milieu d'une zone homogène). Minimiser l'énergie revient à trouver la coupe minimale — **résolue exactement et en temps polynomial** par l'algorithme max-flow/min-cut de Boykov-Kolmogorov. ■

Dérivation

Un étiquetage L correspond à une coupe du graphe : pour chaque pixel étiqueté « objet », on coupe le tuyau vers le terminal « fond », et réciproquement. La capacité totale coupée est exactement $E(L)$. Minimiser E revient donc à trouver la min-cut — problème polynomial dans le cas binaire. C'est l'aboutissement du fil conducteur : la décision n'est plus locale (§12.1) ni dans un espace sans géométrie (§12.3), mais **globale**, en équilibrant explicitement données et régularité spatiale. La structure est identique à Horn-Schunck (§9.4) — le terme de lissage du flot optique devient ici le coût de désaccord V entre voisins.

Ce que ça mesure / l'angle mort

Le terme de lissage **propage** les décisions confiantes vers les pixels ambigus, exactement comme le terme de régularisation du flot optique remplissait les zones sans texture. Angles morts : le cas **multi-étiquettes** est NP-difficile (approché par α -expansion) ; l'énergie doit

être **sous-modulaire** pour une solution exacte ; et le résultat dépend de λ et, pour GrabCut, de **graines** fournies par l'utilisateur (rectangle ou traits objet/fond).

Le curseur λ — le fil conducteur rendu visible

$\lambda = 0$ → chaque pixel décide seul = SEUILLAGE (§12.1), bruité
 λ petit → données dominantes, frontières fidèles mais découpées
 λ grand → lissage dominant, frontières nettes mais détails perdus
 $\lambda \rightarrow \infty$ → une seule région (tout est lissé)

λ est littéralement le **curseur entre fidélité aux données et a priori de régularité** — le fil conducteur du chapitre rendu visible en un seul paramètre.



Deux pixels voisins. p ressemble nettement à l'objet : $D_p(\text{obj}) = 1$, $D_p(\text{fond}) = 8$. q est ambigu : $D_q(\text{obj}) = 5$, $D_q(\text{fond}) = 4$. Coût de désaccord $V = 2$ ($\lambda = 1$).

$(\text{obj}, \text{obj}) : 1 + 5 + 0 = 6 \quad \leftarrow \text{minimum}$
 $(\text{obj}, \text{fond}) : 1 + 4 + 2 = 7$
 $(\text{fond}, \text{obj}) : 8 + 5 + 2 = 15$
 $(\text{fond}, \text{fond}) : 8 + 4 + 0 = 12$

Seul, q préférerait *fond* ($4 < 5$). Mais le terme de lissage et la forte confiance de p **basculent q vers objet** : la cohérence spatiale l'emporte sur une préférence individuelle fragile. C'est précisément ce qu'on attend d'une bonne segmentation.



Seules certaines formes de V garantissent la sous-modularité (et donc l'optimum exact) ; $V(L_p, L_q) = |L_p - L_q|$ pour des étiquettes binaires est toujours valide. GrabCut nécessite un **rectangle initial** (graine) autour de l'objet ; sans lui, rien à séparer. λ n'est pas normalisé : le calibrer relativement à l'échelle des coûts de données (si D_p varie de 0 à 10, un λ de 0.1 à 5 couvre tout le spectre du curseur).



```
# a : image BGR (bleu)
if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune image en entree'}
else:
    h, w = a.shape[:2]
    # rectangle graine : 10 % de marge de chaque côté
    margin_x, margin_y = w // 10, h // 10
    rect = (margin_x, margin_y, w - 2 * margin_x, h - 2 * margin_y)

    mask_gc = np.zeros((h, w), np.uint8)
    bgd = np.zeros((1, 65), np.float64)
    fgd = np.zeros((1, 65), np.float64)
    cv2.grabCut(a, mask_gc, rect, bgd, fgd, 5, cv2.GC_INIT_WITH_RECT)

    seg = np.where((mask_gc == 1) | (mask_gc == 3), 255,
0).astype(np.uint8)
    out_a = {'pixels_objet': int(np.sum(seg > 0)), 'pixels_fond':
int(np.sum(seg == 0))}
    out_b = seg
```

Domaines : détournage semi-automatique en retouche photo, segmentation interactive d'organes en imagerie médicale (avec graines tracées par un radiologue), séparation objet/fond en robotique de saisie industrielle.

12.7. Tableau récapitulatif – méthode, a priori, forces, limites, usage

Méthode	A priori	Forces	Limites	Usage type
Otsu	2 classes d'intensité bimodales	automatique, rapide, aucun paramètre	échoue si histogramme unimodal ou classes déséquilibrées	binarisation de documents, tri de grains
Adaptatif	fond spatialement lentement variable	robuste à l'éclairage inégal	à objets plus grands que la fenêtre invisibles ; paramètre block-Size	OCR sur pages photographiées, contrôle qualité

Méthode	A priori	Forces	Limites	Usage type
K-means	K amas sphériques de taille comparable	segmentation multi-classes, flexible	K fixé a priori, min. local, lent sur grandes images	segmentation couleur, télé-détection
Mean-shift	modes de densité bien séparés par h	K automatique, formes d'amas libres	h critique, coût $O(n^2)$	images satellite, histologie sans K connu
Snakes	frontière lisse, objet unique	contour précis et régulier	init. proche requise, topologie fixe	organes médicaux, silhouettes vidéo
Graph cut	données + voisins spatialement cohérents	optimum global exact (cas binaire), robuste au bruit	graines requises (GrabCut), λ à calibrer	détourage interactif, segmentation médicale guidée

Note sur l'état de l'art : ces méthodes précèdent l'apprentissage profond, qui domine aujourd'hui la segmentation générale (U-Net, Mask R-CNN, SAM). Elles gardent leur créneau : **aucune donnée d'entraînement requise**, interprétables, rapides, fiables sur des problèmes bien définis. Elles persistent aussi à l'intérieur des pipelines profonds — un CRF dense (cousin du graph cut) raffine les sorties de réseaux de segmentation, et les graines de SAM sont une segmentation interactive dont l'énergie sous-jacente ressemble à celle du graph cut.

12.8. données contre régularité, le curseur de toute segmentation

Le chapitre raconte une seule histoire, déclinée six fois : l'image seule ne tranche pas, il faut **ajouter une hypothèse** et choisir **où** placer la décision.

Otsu : a priori « deux classes d'intensité » – décision sur l'histogramme
Adaptatif : a priori « le fond varie lentement » – décision locale
K-means : a priori « K amas compacts » – décision en espace de caractéristiques
Mean-shift : a priori « des modes de densité » – décision en espace de caractéristiques
Snakes : a priori « une frontière lisse » – décision sur une courbe
Graph cut : a priori « données + voisins cohérents » – décision globale

D'un bout à l'autre, le même axe : à gauche le **pur attachement aux données** — le seuillage, où chaque pixel décide seul et le bruit passe tout entier —, à droite la **régularité dominante** — le lissage qui propage et nettoie, au risque d'effacer le détail. Le graph cut le rend littéral avec son λ ; c'est le même curseur que le α de Horn-Schunck pour le mouvement (chapitre 9), la même tension qu'un filtre encode comme a priori sur le signal (chapitre 5), la même question qu'une distance pose en déclarant ce qui rapproche deux pixels (chapitre 3).

Ce fil conducteur rejoint le méta-fil du livre : **le bon a priori rend le problème presque résolu**. Un filtre choisit ce qu'il lisse (chapitre 5). Une distance déclare ce qui compte (chapitre 3). Un descripteur choisit ce qu'il voit et ce qu'il oublie (chapitre 1). Une segmentation déclare ce qu'elle suppose d'une frontière. Dans chaque cas, la représentation encode une hypothèse sur le monde — et tout l'art consiste à choisir, pour l'image qu'on a, ce qu'on accepte de supposer vrai.

CHAPITRE

Texture

13

Table des matières

13.1. Statistiques du premier ordre — la texture que l’histogramme ne capte pas . .	212
13.2. Matrice de cooccurrence (GLCM) — la statistique du second ordre	214
13.3. Descripteurs d’Haralick — compresser la GLCM en quelques nombres	217
13.4. Local Binary Pattern (LBP) — le micro-motif local	219
13.5. Bancs de filtres et énergie de Gabor — la texture comme spectre orienté	222
13.6. Tableau récapitulatif — quelle relation, quelle échelle, quel résumé	224
13.7. la texture est une relation, pas une valeur	225

La texture est ce que l'histogramme ne voit pas. Deux régions peuvent avoir exactement la même distribution de niveaux de gris — la même moyenne, la même variance, la même entropie — et pourtant l'une est lisse et l'autre rugueuse, l'une rayée et l'autre tachetée. La différence n'est pas dans les valeurs des pixels, mais dans leur **arrangement spatial**. Ce chapitre construit les descripteurs de texture dans l'ordre où ils répondent à une question de plus en plus fine : statistique marginale (premier ordre), cooccurrence de paires (GLCM), micro-motif local (LBP), réponse fréquentielle orientée (bancs de filtres).

Le **fil conducteur** de ce chapitre tient en une phrase : **décrire une texture, c'est choisir une relation de voisinage et une échelle, puis résumer la statistique de cette relation**. Un pixel seul n'a pas de texture ; elle vit dans le rapport entre pixels. Tout descripteur fixe donc trois choses — *quelle relation* (paires à un décalage, centre contre couronne, bande de fréquence), *à quelle échelle* (décalage d , rayon R , longueur d'onde λ), *quel résumé* (un histogramme, cinq scalaires, une énergie) — et chacun de ces choix jette la position individuelle pour ne garder qu'une statistique. Ce fil traversera chaque section : du résumé qui ignore toute géométrie (premier ordre) à ceux qui l'encodent explicitement (cooccurrence, motif, fréquence), la marque commune est que l'invariance gagnée (à la position, à l'illumination, à la rotation) est exactement l'information délibérément oubliée.

Rappel des liens avec les autres chapitres. La texture est un carrefour qui réutilise presque tout le livre. Les statistiques du premier ordre ne sont que des moments de l'histogramme (chapitre 2 : un moment intègre une grandeur pondérée par une densité). Le filtre de Gabor du §12.5 est le filtre orienté du chapitre 5 (§5.6) transposé à l'analyse de texture. Le tenseur de structure (chapitre 6, §6.4) mesure déjà l'orientation locale : LBP et Gabor y ajoutent, l'un la forme du micro-motif, l'autre la sélectivité d'échelle. Comparer deux textures revient à comparer deux histogrammes (LBP) ou deux vecteurs (Haralick) selon les distances du chapitre 3 (χ^2 , Bhattacharyya, Mahalanobis). Le spectre de puissance de Fourier, introduit au chapitre 10, est lui-même une description spectrale de texture. L'entropie du §12.3 est la même entropie de Shannon que le chapitre 14 mobilise comme descripteur de cooccurrence. Le couple *information gardée / invariance gagnée* est le pendant exact du fil du chapitre 1 (« un descripteur choisit ce qu'il voit et ce qu'il oublie ») — ici appliqué non plus à une silhouette, mais à un motif statistique.

13.1. Statistiques du premier ordre – la texture que l’histogramme ne capte pas



À partir de l’histogramme normalisé $p(g)$, $g = 0..L-1$:

moyenne $\mu = \sum_g g \cdot p(g)$
 variance $\sigma^2 = \sum_g (g-\mu)^2 \cdot p(g)$
 asymétrie $\gamma = \sum_g (g-\mu)^3 \cdot p(g) / \sigma^3$ (skewness)
 uniformité $U = \sum_g p(g)^2$
 entropie $H = -\sum_g p(g) \cdot \log p(g)$

L’idée Ces grandeurs ne sont rien d’autre que les moments de la **distribution marginale** des intensités (rappel du chapitre 2 : un moment intègre une grandeur pondérée par une densité). On part de l’image, on construit l’histogramme — opération qui **jette toute information de position** — puis on en calcule moyenne, variance, etc. L’image mentale est celle d’un sac de billes colorées : on vide l’image pixel par pixel dans le sac, on secoue, et on compte combien de billes portent chaque nuance. L’histogramme dit *combien*, jamais *où*. La variance mesure alors l’amplitude des fluctuations de niveau (un fond uniforme a $\sigma^2 \approx 0$, une texture contrastée un σ^2 élevé) ; l’uniformité U est maximale ($= 1$) pour une zone d’un seul niveau et décroît quand l’histogramme s’étale.

Du point de vue du fil conducteur : le premier ordre choisit la **relation la plus pauvre possible** — un pixel seul, sans voisin. C’est précisément pourquoi il échoue sur la texture : il n’a fixé aucune géométrie de voisinage. Le résumé qu’il produit (μ, σ^2, H) est juste, mais il porte sur une tout autre question que « comment les pixels se succèdent-ils ? ». ■

Ce que ça mesure — et son angle mort total L’angle mort est *définitionnel* : par construction, ces descripteurs ignorent l’arrangement spatial. Une zone lisse et une zone poivre-et-sel, si elles partagent le même histogramme, leur sont rigoureusement indiscernables. Deux images avec exactement la même distribution de niveaux obtiennent des statistiques d’ordre 1 identiques, quel que soit leur aspect visuel. C’est précisément ce vide que les sections suivantes comblent, en réintroduisant la géométrie qu’on vient de jeter.



Deux patches 1×4 strictement de même histogramme $\{0, 1, 2, 3\}$ (chaque niveau exactement une fois) :

$A = [0, 1, 2, 3]$ (rampe lisse – les valeurs montent progressivement)

$B = [0, 3, 1, 2]$ (alternance abrupte – les valeurs sautent)

$$\mu_A = \mu_B = (0+1+2+3)/4 = 1.5$$

$$\sigma^2_A = \sigma^2_B = (1.5^2 + 0.5^2 + 0.5^2 + 1.5^2) / 4 = 5/4 = 1.25$$

$H_A = H_B$, $U_A = U_B$... identiques sur TOUS les descripteurs d'ordre 1

Aucune statistique du premier ordre ne sépare A de B : même moyenne, même variance, même entropie. Pourtant A monte doucement (voisins proches se ressemblent) et B saute brutalement (voisins proches diffèrent). Il faut regarder les **paires de pixels** — c'est la GLCM (§12.2). ■



Deux pièges classiques. D'abord l'**espace gamma** (rappel du chapitre 7) : moyenne et variance calculées sur des valeurs encodées en gamma (ce qu'on lit directement dans un fichier JPEG ou PNG standard) ne sont pas celles du signal physique lumineux. Pour une vraie mesure de rugosité radiométrique — par exemple comparer la réflectance de matériaux en télédétection — il faut linéariser l'image avant le calcul. Ensuite la **quantification** : réduire L de 256 à 16 niveaux stabilise les estimateurs sur des petits patches mais lisse le contraste — le même arbitrage que l'on retrouvera, démultiplié, dans la GLCM.



```
# a : image en niveaux de gris (HxW uint8) connectée sur l'entrée a
# Calcul des statistiques du premier ordre sur la région entière
if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune image en entree'}
else:
    gray = a if a.ndim == 2 else cv2.cvtColor(a, cv2.COLOR_BGR2GRAY)
    hist = np.bincount(gray.ravel(), minlength=256).astype(float)
    p = hist / hist.sum()
    g = np.arange(256)
    mu = float((g * p).sum())
    var = float(((g - mu) ** 2 * p).sum())
    U = float((p ** 2).sum())
    mask = p > 0
    H = float(-(p[mask] * np.log2(p[mask])).sum())
    out_a = {
        'moyenne': mu,
        'variance': var,
        'uniformite': U,
        'entropie_bits': H,
    }
```

13.2. Matrice de cooccurrence (GLCM) – la statistique du second ordre



$P(i, j \mid d, \theta) = \# \{ \text{paires de pixels distantes de } (d, \theta) \text{ dont l'un vaut } i \text{ et l'autre } j \}$ puis normalisation
 $\sum P = 1$

La GLCM est une matrice $L \times L$ ($L = \text{nombre de niveaux quantifiés}$) ; $P(i, j)$ est la probabilité qu'un pixel de niveau i ait, au décalage (d, θ) , un voisin de niveau j .

L'idée Le premier ordre estimait $p(g) = \text{fréquence du niveau } g$. Le second ordre estime la loi **jointe** $p(g_1, g_2)$ du couple (pixel, voisin-à- (d, θ)). C'est exactement l'information manquante au §12.1 : non plus la distribution d'un pixel isolé, mais celle d'une **paire** liée par une relation géométrique fixée.

L'image mentale : au lieu de compter combien de billes portent chaque couleur (ordre 1), on compte maintenant combien de *paires de billes voisines* portent telle et telle couleur. Si deux billes adjacentes se ressemblent souvent (texture lisse), la diagonale de la matrice

est chargée. Si les niveaux voisins sont souvent très différents (texture contrastée), la masse s'écarte de la diagonale.

On choisit la relation — un décalage d (l'échelle) et une orientation θ (la direction) — et on dénombre toutes les paires de l'image qui la réalisent. La matrice obtenue est l'estimateur empirique de $p(g_1, g_2)$ pour cette relation. On la rend en général **symétrique** (compter (i, j) et (j, i)) pour ne pas privilégier un sens de parcours. Du point de vue du fil conducteur : la GLCM fixe la relation (paire à distance d dans la direction θ), l'échelle (le décalage d) et produit un résumé (la matrice entière, ou les cinq scalaires d'Haralick). ■

Ce que ça mesure — et son angle mort La GLCM encode la **co-variation locale** : sur la diagonale ($i \approx j$) se concentrent les paires de niveaux semblables (zones lisses) ; loin de la diagonale se trouvent les transitions brusques (bords, rugosité). Angles morts : elle dépend du couple (d, θ) choisi — une texture rayée verticalement est invisible si on n'interroge que $\theta = 0^\circ$; elle explose en taille ($L \times L$) et se vide si L est grand, d'où la quantification quasi obligatoire à 8–32 niveaux ; elle n'est pas invariante en rotation sauf à moyenner sur plusieurs θ .



Image-jouet 3×3 à trois niveaux, décalage horizontal $d = 1$, $\theta = 0^\circ$, matrice symétrique :

```
img =  0 0 1
        0 1 2
        1 2 2
```

Paires horizontales adjacentes (non orientées) :

ligne 0 : (0,0) (0,1)

ligne 1 : (0,1) (1,2)

ligne 2 : (1,2) (2,2)

→ 6 paires, comptées dans les deux sens = 12 cooccurrences

GLCM symétrique, puis normalisée ($\div 12$) :

comptes	normalisée $P(i, j)$
[2 2 0]	[1/6 1/6 0]
[2 0 2] →	[1/6 0 1/6]
[0 2 2]	[0 1/6 1/6]

(6 cases à $1/6 \approx 0.167$)

Les six paires se rangent **le long de la diagonale et juste à côté** : niveaux voisins majoritairement proches (0-0, 1-2, 2-2) ou distants d'un cran (0-1). C'est la signature d'une texture à transitions douces — lisse. On en extrait les scalaires d'Haralick au §12.3. ■



Quantifier d'abord : passer de 256 à `levels` niveaux avant le calcul ; sinon la matrice 256×256 est creuse et statistiquement non fiable sur tout patch de taille raisonnable. Compromis : trop peu de niveaux écrase le contraste utile, trop de niveaux vide la matrice. **Symétrie et normalisation** sont presque toujours souhaitées (`symmetric=True`, `normed=True` dans `scikit-image`) ; sans normalisation les descripteurs ne sont pas comparables entre patches de tailles différentes. **(d, θ) est un choix, pas un réglage neutre** : pour l'invariance directionnelle, calculer sur $\theta \in \{0^\circ, 45^\circ, 90^\circ, 135^\circ\}$ et moyenner les descripteurs. Enfin, les paires qui débordent de l'image ne sont pas comptées — sur un petit patch, l'effet de bord est substantiel.



```
# a : image en niveaux de gris (HxW uint8) connectée sur l'entrée a
from skimage.feature import graycomatrix

if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune image en entree'}
else:
    gray = a if a.ndim == 2 else cv2.cvtColor(a, cv2.COLOR_BGR2GRAY)
    # Quantification 16 niveaux
    q = (gray.astype(int) // 16).astype(np.uint8)
    glcm = graycomatrix(q,
                        distances=[1],
                        angles=[0, np.pi/4, np.pi/2, 3*np.pi/4],
                        levels=16,
                        symmetric=True,
                        normed=True)

    # glcm : (16, 16, 1, 4) — on l'exporte pour le §12.3
    out_a = {'glcm_shape': list(glcm.shape), 'glcm_somme':
float(glcm.sum())}
```


13.3. Descripteurs d'Haralick – compresser la GLCM en quelques nombres



$$\begin{aligned}\text{Contraste} &= \sum_{i,j} (i-j)^2 \cdot P(i,j) \\ \text{Énergie/ASM} &= \sum_{i,j} P(i,j)^2 \\ \text{Homogénéité} &= \sum_{i,j} P(i,j) / (1 + |i-j|) \\ \text{Entropie} &= -\sum_{i,j} P(i,j) \cdot \log P(i,j) \\ \text{Corrélation} &= \sum_{i,j} (i-\mu_i)(j-\mu_j) \cdot P(i,j) / (\sigma_i \cdot \sigma_j)\end{aligned}$$

μ_i, σ_i : moyenne et écart-type des marges de la GLCM.

L'idée Une GLCM 16×16 contient 256 nombres : illisible et redondante. Chaque descripteur d'Haralick est une **somme pondérée** de la matrice par un noyau qui souligne un aspect géométrique précis. Voici la bonne façon de se les représenter :

Le **contraste** pèse par $(i-j)^2$: plus les paires de niveaux sont éloignées des voisines diagonales, plus il est grand — c'est l'amplitude des sauts locaux. L'image mentale : si on regarde la GLCM comme un relief 3D, le contraste mesure à quel point la masse est répandue loin du sillon central.

L'**énergie** ($\sum P^2$, aussi nommée *Angular Second Moment*) est maximale quand la masse est concentrée sur peu de cases : une texture régulière et prévisible, comme le quadrillage d'un tissu, « vote » toujours pour les mêmes paires et concentre la probabilité. C'est l'**inverse conceptuel de l'entropie** (chapitre 14) : énergie haute = désordre bas.

L'**homogénéité** pèse par $1/(1+|i-j|)$: elle récompense les paires proches de la diagonale et punit doucement les paires éloignées — l'opposé souple du contraste. Une surface de papier mat poli obtient une homogénéité proche de 1.

L'**entropie** est l'entropie de Shannon de la loi jointe : combien de bits faut-il pour coder une paire tirée au hasard selon la GLCM ? Une texture aléatoire (sable fin, bruit) étale la masse sur toutes les cases et maximise l'entropie.

La **corrélation** est le coefficient de corrélation linéaire entre le niveau d'un pixel et celui de son voisin : elle dit dans quelle mesure on peut *prédire* le voisin connaissant le pixel central. Sur une surface rayée régulièrement, la corrélation est proche de 1 dans la direction des stries.

Du point de vue du fil conducteur : les cinq scalaires héritent du choix de (d, θ) de la GLCM ; ils constituent le **résumé** de la statistique de la relation paire. ■



Sur la GLCM normalisée du §12.2 (six cases à 1/6) :

$$\begin{aligned}
 \text{Énergie} &= 6 \cdot (1/6)^2 &= 6/36 = 1/6 &\approx 0.167 \\
 \text{Contraste} &= 4 \text{ cases hors-diag à } |i-j|=1 \\
 &= 4 \cdot 1^2 \cdot (1/6) &= 4/6 &\approx 0.667 \\
 \text{Homogén.} &= 2 \cdot (1/6)/1 + 4 \cdot (1/6)/2 &= 2/6 + 4/12 = 2/3 &\approx 0.667 \\
 \text{Entropie} &= -6 \cdot (1/6) \cdot \ln(1/6) = \ln 6 &&\approx 1.79 \text{ nats} \\
 \text{Corrél.} &: \text{marges } \mu_i = \mu_j = 1, \sigma_i^2 = \sigma_j^2 = 2/3 \\
 &\text{num} = \Sigma (i-1)(j-1)P = (0,0):+1/6 + (2,2):+1/6 = 1/3 \\
 &= (1/3) / (2/3) &&= 0.5
 \end{aligned}$$

Lecture : contraste modéré et corrélation positive (0,5) confirment des transitions douces et ordonnées — exactement ce que la disposition de la GLCM montrait visuellement. Les cinq nombres redisent, sous forme compacte et comparable, ce que la matrice exprimait en clair. ■



Deux pièges de nomenclature piègent régulièrement. Premièrement : **'energy' de scikit-image $\neq$ ASM**. `graycoprops(gldcm, 'ASM')` rend ΣP^2 (notre énergie = 0,167) ; `graycoprops(gldcm, 'energy')` rend $\sqrt{\text{ASM}}$ ($\approx 0,408$). Deux noms, deux valeurs : toujours vérifier lequel attend votre pipeline avant de comparer des résultats. Deuxièmement : **deux « homogénéités » dans la littérature**. *Inverse Difference* utilise $1/(1+|i-j|)$ (la formule ci-dessus) ; *Inverse Difference Moment* — ce que calcule `graycoprops('homogeneity')` — utilise $1/(1+(i-j)^2)$. Sur cet exemple ($|i-j| \in \{0, 1\}$) les deux coïncident, mais elles divergent dès que $|i-j| = 2$. Ne jamais comparer des valeurs issues des deux conventions. Troisièmement : la **base du logarithme** de l'entropie (nats avec $\ln$, bits avec $\log_2$) change la valeur d'un facteur $\ln 2 \approx 0,693$ selon l'implémentation.



```
# a : image en niveaux de gris (HxW uint8) connectée sur l'entrée a
from skimage.feature import graycomatrix, graycoprops

if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune image en entree'}
else:
    gray = a if a.ndim == 2 else cv2.cvtColor(a, cv2.COLOR_BGR2GRAY)
    q = (gray.astype(int) // 16).astype(np.uint8)
    glcm = graycomatrix(q, distances=[1],
                        angles=[0, np.pi/4, np.pi/2, 3*np.pi/4],
                        levels=16, symmetric=True, normed=True)
    # Moyenne sur les 4 orientations → invariance directionnelle
    contraste = float(graycoprops(glcm, 'contrast').mean())
    homogeneite = float(graycoprops(glcm, 'homogeneity').mean())
    asm = float(graycoprops(glcm, 'ASM').mean()) # Σ P² (≠
    'energy')
    correlation = float(graycoprops(glcm, 'correlation').mean())
    # Entropie : non fournie par graycoprops, calcul manuel
    p = glcm[... , 0, 0]
    masque = p > 0
    entropie = float(-(p[masque] * np.log(p[masque])).sum())
    out_a = {
        'contraste': contraste,
        'homogeneite': homogeneite,
        'asm': asm,
        'correlation': correlation,
        'entropie_nats': entropie,
    }
```

13.4. Local Binary Pattern (LBP) – le micro-motif local



$$\text{LBP}_{\{P,R\}}(c) = \sum_{p=0}^{P-1} s(g_p - g_c) \cdot 2^p, \quad s(x) = 1 \text{ si } x \geq 0, \text{ sinon } 0$$

g_c = niveau du pixel central, g_p = niveaux de P voisins échantillonnés sur un cercle de rayon R .

L'idée Plutôt que de compter des paires (GLCM), LBP décrit chaque pixel par la **forme** de son voisinage immédiat. La démarche est simple à visualiser : on place le pixel central au milieu d'une couronne de P voisins équidistants sur un cercle de rayon R . Pour chaque

voisin, on pose une question binaire — est-il plus clair que le centre ? — et on note 1 pour oui, 0 pour non. La chaîne de P bits ainsi obtenue, lue comme un entier en pondérant chaque position par 2^p , est un **code de micro-motif**. Un bord franc, par exemple, génère un code du type 00001111 (une moitié sombre, une moitié claire). Un coin génère 00000111. Une zone uniforme génère 00000000 ou 11111111.

L'image entière devient une carte d'étiquettes LBP, dont on prend l'**histogramme** : c'est ce vecteur — non pas l'entier d'un pixel isolé — qui décrit la texture. On compte la fréquence de chaque micro-motif dans la région analysée.

Ce seuillage relatif au centre confère à LBP une propriété précieuse : l'**invariance à toute transformation monotone des niveaux de gris**. Ajouter une constante d'éclairage (passer de jour à ombre), multiplier par un gain (changer d'exposition), appliquer un gamma — aucune de ces opérations n'inverse le signe de $g_p - g_c$. Le code est donc inchangé. C'est l'invariance à l'illumination « gratuite », offerte par le choix de la relation (le signe d'une différence) plutôt que par sa valeur.

Du point de vue du fil conducteur : LBP choisit la **relation centre/couronne** (comparer chaque voisin au centre), l'**échelle R** (le rayon), et produit comme résumé l'histogramme des codes — une distribution de micro-motifs. ■

Ce que ça mesure — et son angle mort LBP capte les **micro-structures** (bords, coins, points, zones plates) à l'échelle R . Deux variantes importantes : le **LBP uniforme** (motifs à ≤ 2 transitions $0 \leftrightarrow 1$ dans le code circulaire — ces motifs « propres » correspondent à des coins, bords, arcs ; les autres, bruités, sont regroupés dans une seule case fourre-tout), qui réduit la dimension et la sensibilité au bruit ; et le **LBP rotation-invariant** (ror), qui prend la plus petite rotation circulaire du code pour ne pas distinguer 00001111 de 00111100. Angles morts : LBP est mono-échelle (un seul R à la fois ; on empile plusieurs R pour le multi-échelle) ; il est **fragile au bruit** quand $g_p \approx g_c$, car un grain de bruit peut basculer le signe et changer le code ; et il jette l'**amplitude** des différences (ne garde que leur signe), perdant ainsi l'information sur le contraste absolu.



Voisinage 3×3 (P = 8, R = 1), centre **g_c** = 50, voisins parcourus depuis le coin haut-gauche dans le sens horaire :

60	20	90	positions :	p0	p1	p2
40	50	10			p7	c p3
30	70	80		p6	p5	p4

g_p :	60	20	90	10	80	70	30	40	(p0..p7)
s :	1	0	1	0	1	1	0	0	(g_p ≥ 50 ?)

$$\begin{aligned} \text{LBP} &= 1 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2 + 0 \cdot 2^3 + 1 \cdot 2^4 + 1 \cdot 2^5 + 0 \cdot 2^6 + 0 \cdot 2^7 \\ &= 1 + 4 + 16 + 32 = 53 \end{aligned}$$

Le code circulaire **1 0 1 0 1 1 0 0** présente **6 transitions** 0↔1 (en bouclant : 0→1, 1→0, 0→1, 1→0, ..., 0→1 au boucllement). Ce motif n'est **pas uniforme** (seuil d'uniformité : ≤ 2 transitions) : il tombe dans la case fourre-tout du LBP uniforme — typique d'un voisinage irrégulier ou bruité. Un bord franc, lui, donnerait un motif type **00001111** à 2 transitions exactes, uniforme, clairement identifiable. ■



L'ordre des voisins est une convention. scikit-image démarre à l'angle 0 (voisin de droite) et tourne dans le sens trigonométrique ; l'exemple ci-dessus part du coin haut-gauche en sens horaire. L'entier produit diffère donc selon l'implémentation — mais l'histogramme (le vrai descripteur) est équivalent à une permutation des cases près. Ne jamais comparer des codes LBP bruts entre deux bibliothèques différentes. Pour P = 8 et R = 1 les voisins tombent exactement sur la grille de pixels ; pour R fractionnaire ou P ≠ 8, scikit-image interpole bilinéairement les positions hors grille. Enfin, l'histogramme LBP se compare en χ^2 ou Bhattacharyya (chapitre 3), **pas** en distance euclidienne : les cases sont des catégories nominales (codes de motifs), pas une échelle ordonnée.



```
# a : image en niveaux de gris (HxW uint8) connectée sur l'entrée a
from skimage.feature import local_binary_pattern

if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune image en entree'}
else:
    gray = a if a.ndim == 2 else cv2.cvtColor(a, cv2.COLOR_BGR2GRAY)
    P, R = 8, 1
    lbp = local_binary_pattern(gray, P, R, method='uniform')
    # P+2 bins pour LBP uniforme : P+1 motifs uniformes + 1 fourre-tout
    hist, _ = np.histogram(lbp, bins=np.arange(P + 3), density=True)
    out_a = {
        'lbp_hist': [float(v) for v in hist], # vecteur descripteur (10
valeurs)
        'P': P,
        'R': R,
    }
```

13.5. Bancs de filtres et énergie de Gabor — la texture comme spectre orienté



$$g(x, y) = \exp(-(x'^2 + y'^2) / 2\sigma^2) \cdot \cos(2\pi x' / \lambda + \psi)$$

avec $x' = x \cdot \cos\theta + y \cdot \sin\theta$
 $y' = -x \cdot \sin\theta + y \cdot \cos\theta$

Énergie de texture en (λ, θ) : $E(\lambda, \theta) = \| I * g_{\{\lambda, \theta\}} \|^2$ (sur la fenêtre analysée)

L'idée Une texture est une répétition : les stries d'un tissu, les alvéoles d'une éponge, les grains d'une roche métamorphique. Toute répétition a un contenu fréquentiel (une échelle, une période) et souvent une orientation privilégiée. L'idée spectrale — introduite au chapitre 10 avec la transformée de Fourier — est de mesurer *combien d'énergie* la texture place à chaque fréquence et dans chaque direction.

Mais la transformée de Fourier globale perd la localisation : elle dit *quelles* fréquences sont présentes dans toute l'image, pas *où* elles se trouvent. Le filtre de Gabor est le compromis : une sinusoïde de longueur d'onde λ et d'orientation ϑ , **fenêtrée** par une gaussienne d'étendue σ . L'image mentale est celle d'un détecteur pointu : il répond fort là où la texture ressemble *localement* à une sinusoïde de sa fréquence et de son orientation. On le décline en **banc** (plusieurs $\lambda \times$ plusieurs ϑ) et on mesure l'énergie de réponse (magnitude carrée, intégrée

localement) à chaque combinaison. Le vecteur de ces énergies décrit la texture par sa *signature fréquence-orientation*.

C'est le pendant orienté du tenseur de structure (chapitre 6, §6.4), qui mesure l'orientation locale dominante ; Gabor y ajoute la **sélectivité d'échelle** via λ . Du point de vue du fil conducteur : le banc de Gabor choisit la **relation avec une sinusoïde** (λ, ϑ) comme relation de voisinage, λ comme échelle, et l'énergie locale comme résumé. ■

Ce que ça mesure — et son angle mort Le banc répond fort là où la texture possède une périodicité à la fréquence et à l'orientation du filtre. Une trame régulière (tissu, grille métallique, mire) allume un pic net dans le banc ; une texture aléatoire (sable, bruit blanc) étale l'énergie uniformément. Angles morts : le choix du pas d'échantillonnage (λ, ϑ) fixe la résolution — un motif de période intermédiaire entre deux bandes λ est mal vu ; le banc est **coûteux** (autant de convolutions que de filtres dans le banc) ; et l'énergie seule ignore la **phase**, donc deux textures de même spectre mais d'agencement spatial différent peuvent se confondre — le même type d'angle mort que l'histogramme du §12.1, transposé au domaine fréquentiel.



Mire sinusoïdale verticale de **période 4 pixels** → fréquence spatiale $f_0 = 1/4 = 0,25$ cycle/pixel, orientation 0° (variation horizontale) :

Banc de Gabor, $\theta = 0^\circ$, longueurs d'onde $\lambda \in \{2, 4, 8\}$ px :

$\lambda = 4 \rightarrow$ fréquence $1/4 = f_0 \rightarrow$ RÉSONANCE, énergie maximale

$\lambda = 2 \rightarrow$ fréquence $1/2$ (trop haute) → réponse faible

$\lambda = 8 \rightarrow$ fréquence $1/8$ (trop basse) → réponse faible

Même mire interrogée à $\lambda = 4$ mais $\theta = 90^\circ$ (filtre horizontal) :

la sinusoïde est constante le long de la verticale → énergie ≈ 0

Le pic d'énergie tombe sur le filtre dont λ et ϑ **coïncident** avec la mire ($\lambda = 4, \vartheta = 0^\circ$). Le banc localise ainsi la texture dans le plan (échelle, orientation) : c'est la signature fréquence-direction de la répétition. ■



Normaliser les noyaux (somme nulle pour la composante cosinus, énergie unité) ; sinon les énergies ne sont pas comparables entre échelles différentes. **σ et λ sont liés** : la largeur de bande dépend du rapport σ/λ ; le fixer constant (par exemple $\sigma \approx 0,56 \lambda$ pour environ 1 octave de largeur de bande) garde une sélectivité homogène d'une échelle à l'autre. **Toujours utiliser la magnitude** des deux phases ($\psi = 0$ et $\psi = \pi/2$, ce qui donne partie réelle et partie imaginaire du filtre complexe) : la magnitude est insensible à la position du motif

sous le filtre, alors qu’une seule phase oscille avec cette position. **Repliement** : λ proche de 2 px touche la limite de Nyquist (rappel du chapitre 10) — les réponses aux plus hautes fréquences sont peu fiables.



```
# a : image en niveaux de gris (HxW uint8) connectée sur l'entrée a
from skimage.filters import gabor_kernel
from scipy import ndimage as ndi

if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'aucune image en entree'}
else:
    gray = a if a.ndim == 2 else cv2.cvtColor(a, cv2.COLOR_BGR2GRAY)
    img_float = gray.astype(float)
    feats = {}
    for theta_deg in [0, 45, 90, 135]:
        theta = np.deg2rad(theta_deg)
        for lam in [2, 4, 8]:
            k = gabor_kernel(frequency=1.0 / lam, theta=theta)
            re = ndi.convolve(img_float, k.real, mode='reflect')
            im = ndi.convolve(img_float, k.imag, mode='reflect')
            mag = np.hypot(re, im) # magnitude insensible à la phase
            energie = float(mag.mean())
            feats[f'E_lam{lam}_t{theta_deg}'] = energie
    out_a = feats # 12 valeurs : signature
fréquence-orientation
```

13.6. Tableau récapitulatif – quelle relation, quelle échelle, quel résumé

Descrip- teur	Relation interrogée	Échelle	Résumé produit	Invariances	Angle mort principal
1er ordre	aucune (pixel seul)	—	μ, σ^2, H (scalaires)	translation, rotation (to- tales)	ignore tout arrangement spatial
GLCM / Ha- ralick	paires à ϑ	$(d, \text{décalage } d)$	$5 \text{ scalaires} \times (d, \vartheta)$	translation ; rotation si moyenné sur ϑ	dépend de (d, ϑ) ; coû- teuse en L

Descrip- teur	Relation interrogée	Échelle	Résumé produit	Invariances	Angle mort principal
LBP	centre couronne P points	vs rayon R de	histogramme de codes	niveaux mo- notones ; ro- tation (ror)	bruit près du seuil ; jette l'amplitude
Gabor banc	/ corrélation à une sinusoïde (λ, ϑ)	longueur d'onde λ	énergies en (λ, ϑ)	translation (via magni- tude)	ignore la phase ; coût- eux

Note sur l'état de l'art. Ces familles précèdent l'apprentissage profond, qui domine aujourd'hui la classification de textures (réseaux convolutifs, *bilinear pooling* — héritier direct de la cooccurrence —, *scattering transforms* cousins du banc de Gabor). Les méthodes classiques gardent pourtant leur créneau : sans données d'entraînement, elles sont interprétables, légères, robustes sur des problèmes contraints (microscopie, télédétection, contrôle industriel), et souvent imbattables en rapport performance/coût sur de petits jeux. On les retrouve aussi à l'intérieur des pipelines modernes — un LBP ou un banc de Gabor comme couche d'entrée fixe, une matrice de Gram (cooccurrence de canaux profonds) au cœur du transfert de style neural.

13.7. la texture est une relation, pas une valeur

Ce chapitre raconte une seule histoire, déclinée quatre fois : un pixel n'a pas de texture, seule une **relation entre pixels** en a une. Chaque section a construit cette relation un peu plus finement :

1er ordre : relation « aucune » — résume la distribution d'UN pixel (et échoue)
 GLCM : relation « paire à (d, θ) » — résume la loi jointe de DEUX pixels liés
 LBP : relation « centre/couronne » — résume le SIGNE des différences au voisinage
 Gabor : relation « sinusoïde (λ, θ) » — résume l'ÉNERGIE par fréquence et direction

D'un bout à l'autre, le même mouvement : on **ajoute de la géométrie** que le premier ordre avait jetée (la position du voisin, la forme du motif, la fréquence du motif), puis on la **re-jette partiellement** sous forme d'une statistique. Et l'invariance qu'on récolte — à la position, à l'illumination, à la rotation — est exactement l'information qu'on a délibérément perdue. LBP achète l'invariance à l'éclairage en ne gardant que le signe des différences ;

Gabor achète l'invariance de position en ne gardant que la magnitude ; la GLCM achète l'invariance directionnelle en moyennant sur les angles.

Ce mouvement est le fil du chapitre 1 transposé du contour au motif : « un descripteur choisit ce qu'il voit et ce qu'il oublie ». C'est la même question qu'une distance pose en déclarant ce qui rapproche deux points (chapitre 3), le même principe qu'un filtre encode comme a priori sur le signal (chapitre 5), la même tension que la transformée de Fourier résout en choisissant sa base (chapitre 10). Décrire une texture, comme estimer un flot, débruiter une image ou segmenter une scène, revient à choisir une représentation où la régularité cherchée devient visible : **le bon couple relation-échelle rend la texture presque évidente** — et tout l'art consiste à décider, pour le motif observé, quelle relation porte l'information et laquelle on peut sacrifier sans perte.

CHAPITRE

14

Qualité d'image

Table des matières

14.1. MSE et PSNR — la fidélité, ou l'observateur aveugle à la structure	229
14.2. SSIM — modéliser l'observateur : luminance, contraste, structure	231
14.3. Entropie d'image — la qualité sans référence, ou le contenu informationnel . .	234
14.4. Mesures de netteté sans référence — variance du Laplacien, énergie de gradient	237
14.5. Métriques perceptuelles apprises (LPIPS) — l'observateur ajusté sur l'humain	240
14.6. Tableau récapitulatif — quel observateur, quelle référence, quel angle mort . .	243
14.7. une métrique de qualité est un observateur déguisé	244

Demander si une image est « bonne » n'a pas de sens sans préciser : bonne *pour qui*, et *à comparer à quoi* ? Une compression qui décale uniformément la luminance de quelques niveaux est catastrophique pour un capteur qui somme des pixels, mais invisible pour un œil humain. Un flou léger ruine une page d'OCR mais passe inaperçu sur une photo de paysage. Une dégradation de texture inventée par un réseau génératif reçoit le pire score PSNR possible alors que les observateurs humains la préfèrent à l'image floue fidèle. Ce chapitre construit les métriques de qualité dans l'ordre où elles se dotent d'un modèle d'observateur de plus en plus explicite : la fidélité pixel à pixel (MSE/PSNR), qui ne modélise presque rien ; la similarité structurelle (SSIM), qui code à la main une approximation du système visuel humain ; l'entropie et la netteté, qui se passent de toute référence et mesurent une propriété intrinsèque de l'image ; enfin les métriques apprises (LPIPS), qui ajustent le modèle d'observateur sur des jugements humains réels.

Fil conducteur : mesurer la qualité d'une image, c'est postuler un observateur et déclarer ce qui le dérange. Aucune métrique n'est neutre. Chacune incarne un modèle de qui regarde — un comparateur de pixels indépendants, un système visuel humain approximé, un détecteur de mise au point sans mémoire de l'original, un réseau calibré sur des préférences humaines — et ne pénalise que les dégradations que ce modèle juge importantes, restant délibérément aveugle aux autres. Tout le chapitre déroule le même arbitrage : ce que la métrique punit, c'est ce que son observateur perçoit ; ce qu'elle ignore, c'est ce qu'il tolère. Le passage du *full-reference* (on dispose de l'original) au *no-reference* (on ne l'a pas) puis à l'*appris* est une montée en explicitation du modèle d'observateur — de l'implicite naïf « les pixels doivent coïncider » au réseau calibré sur des jugements.

Rappel des liens

La qualité d'image recoud des outils dispersés dans tout le livre :

chapitre 3 (distances)	: MSE = carré de la distance L2 par pixel ; terme « structure » de SSIM = corrélation/cosinus ; LPIPS = distance dans un espace de descripteurs
appris	
chapitre 4 (métriques seg.)	: fil jumeau — « aucune métrique unique ne capture tout »
chapitre 5 (filtrage)	: le flou est un filtre passe-bas ; les fenêtres locales de SSIM sont gaussiennes
chapitre 6 (gradients)	: Tenengrad somme des gradients de Sobel ; le Laplacien est un opérateur passe-haut
chapitre 7 (couleur)	: MSE en espace gamma $\neq$ MSE physique ; SSIM standard ne voit que la luminance
chapitre 10 (transformées)	: flou = multiplication du spectre par un passe-bas ; netteté = énergie haute-fréquence
chapitre 13 (texture)	: l'entropie d'image y est un descripteur du 1er

ordre,

aveugle à l'arrangement — même angle mort ici
 chapitre 15 (coûts profonds): SSIM et LPIPS deviennent des fonctions de coût
 (perceptual loss, DSSIM)

Le couple « ce qui est puni / ce qui est toléré » est le pendant exact du fil du chapitre 3 (« choisir une distance, c'est déclarer ce qui compte ») et du chapitre 4 (« aucune métrique unique ne capture tout ») — ici, ce qui compte est dicté par un observateur supposé.

14.1. MSE et PSNR — la fidélité, ou l'observateur aveugle à la structure



$$\begin{aligned}\text{MSE} &= (1/N) \sum_i (I_i - \hat{I}_i)^2 \\ \text{PSNR} &= 10 \cdot \log_{10} (\text{MAX}^2 / \text{MSE}) \quad [\text{dB}]\end{aligned}$$

N = nombre de pixels, MAX = valeur maximale représentable (255 en 8 bits, 1.0 en flottant normalisé). PSNR = ∞ pour deux images identiques (MSE = 0).

Dérivation : du carré de l'erreur au décibel

Le MSE est la moyenne empirique des erreurs au carré — c'est-à-dire, à la normalisation N près, le **carré de la distance euclidienne par pixel** entre les deux images (rappel du chapitre 3). L'image mentale utile : on empile les deux images l'une sur l'autre, on lit la différence de chaque case du damier, on la met au carré, et on fait la moyenne. Les cases les plus éloignées pèsent d'autant plus qu'elles sont éloignées — c'est l'effet du carré : une erreur de 10 niveaux pèse 100 fois plus qu'une erreur de 1 niveau, pas simplement 10 fois plus.

Le PSNR n'est qu'un changement d'échelle logarithmique du MSE, hérité des calculs de rapport signal/bruit en transmission :

$$\text{PSNR} = 10 \cdot \log_{10} (\text{MAX}^2 / \text{MSE}) = 20 \cdot \log_{10} (\text{MAX}) - 10 \cdot \log_{10} (\text{MSE})$$

On prend pour « puissance du signal » la plus grande valeur représentable au carré (MAX^2) et pour « puissance du bruit » l'erreur quadratique moyenne. Le logarithme convertit ce rapport — par nature multiplicatif, comme l'est une plage dynamique — en une échelle additive et lisible : plus la valeur est haute, meilleure est la qualité. ■

Ce que ça mesure / l'angle mort

L'observateur modélisé pose les deux images l'une sur l'autre et somme les écarts au carré, **chaque pixel traité indépendamment et à l'identique**. Il n'a aucune notion de structure, de voisinage, ni de *où* se trouve l'erreur dans l'image. D'où ses deux échecs symétriques : une dégradation globale mais perceptuellement bénigne — léger décalage

de luminance, faible recalage géométrique — gonfle le MSE alors que l'œil ne voit rien ; à l'inverse, une atteinte locale mais sémantiquement grave — une lettre effacée dans un document OCR, une micro-fissure sur une pièce industrielle — reste noyée dans la moyenne de millions de pixels intacts. Le MSE dit *combien* les valeurs diffèrent en moyenne, jamais *comment* ni *où*.



Patch 2×2 (8 bits), un seul pixel altéré de 8 niveaux :

$$\begin{aligned} \mathbf{I} &= \begin{bmatrix} 10 & 10 \\ 10 & 10 \end{bmatrix} & \hat{\mathbf{I}} &= \begin{bmatrix} 10 & 10 \\ 10 & 18 \end{bmatrix} \\ \text{MSE} &= (0 + 0 + 0 + 8^2)/4 = 64/4 = 16 \\ \text{PSNR} &= 10 \cdot \log_{10}(255^2/16) = 10 \cdot \log_{10}(4064.06) \approx 36.1 \text{ dB} \end{aligned}$$

36 dB est une « bonne » qualité. Voici maintenant l'angle mort, sur deux dégradations opposées :

(a) +1 sur TOUS les pixels (décalage de luminance imperceptible) :

$$\text{MSE} = 1 \rightarrow \text{PSNR} \approx 48.1 \text{ dB} \quad (\text{« excellent »})$$

(b) damier 0/255 décalé d'1 pixel (image visuellement identique) :

presque tout pixel bascule 0→255

$$\text{MSE} \approx 255^2 = 65025 \rightarrow \text{PSNR} = 0 \text{ dB} \quad (\text{« pire cas »})$$

Le cas (b) est la démonstration canonique : une image visuellement *inchangée* — juste glissée d'un pixel — reçoit le pire score possible, parce que l'observateur-MSE compare des positions absolues. Pixel par pixel, tout a changé ; structurellement, rien. C'est ce modèle d'observateur qui parle, pas la réalité.



- **Débordement uint8** : $(\mathbf{I} - \hat{\mathbf{I}})**2$ sur des tableaux uint8 boucle modulo 256 en NumPy — $200 - 10$ calculé en uint8 peut produire n'importe quoi. **Caster en float64 avant de soustraire**. Bug classique et silencieux.
- **Choix de MAX** : 255 pour uint8, 1.0 pour des flottants normalisés. Calculer le MSE sur des flottants [0,1] mais conserver MAX = 255 fausse le PSNR de $20 \cdot \log_{10}(255) \approx 48 \text{ dB}$ — un glissement invisible qui invalide toute comparaison.
- **Espace linéaire vs gamma** (rappel chapitre 7) : un MSE calculé sur des valeurs encodées en gamma ne pèse pas l'erreur physique — il sur-pénalise les tons clairs et sous-pénalise les tons sombres. Pour une mesure radiométrique correcte, linéariser l'image d'abord.
- **Vidéo** : la moyenne des PSNR par image $\neq$ PSNR du MSE moyen (le logarithme n'est pas linéaire). Préciser la convention de moyennage dans tout rapport.



```
# a : image de référence (np.ndarray H×W×3 uint8 BGR ou H×W uint8)
# b : image dégradée (même format)
if not isinstance(a, np.ndarray) or not isinstance(b, np.ndarray):
    out_a = {'erreur': 'deux images attendues sur a et b'}
else:
    ref = a.astype(np.float64)
    deg = b.astype(np.float64)
    mse = float(np.mean((ref - deg) ** 2))
    if mse == 0:
        psnr = float('inf')
    else:
        psnr = float(10 * np.log10(255.0 ** 2 / mse))
    out_a = {
        'MSE': round(mse, 4),
        'PSNR_dB': round(psnr, 2),
        'interpretation': 'excellent >40dB / bon 30-40 / mediocre <30',
    }
```

Domaines d'usage : évaluation de codecs (JPEG, HEVC, AV1), bancs d'essai de débruitage, transmission sur canal bruité, où le PSNR reste la mesure historique et la référence de comparaison, malgré ses limites bien documentées.

14.2. SSIM — modéliser l'observateur : luminance, contraste, structure



$$\text{SSIM}(x, y) = (2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2) / ((\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2))$$

$c_1 = (k_1L)^2$, $c_2 = (k_2L)^2$, $k_1 = 0.01$, $k_2 = 0.03$, $L = \text{plage dynamique (255)}$

Calculée localement sur des fenêtres gaussiennes 11×11 , puis moyennée sur l'image entière (MSSIM). Bornée par 1, atteint 1 si et seulement si $x = y$.

L'idée : factoriser la similarité en trois comparaisons

SSIM part d'un constat simple : l'œil humain ne somme pas des erreurs de pixels — il perçoit des *structures*. Une légère dérive globale de luminosité sur toute une photo de satellite ne choque pas ; une ligne de texte effacée dans un document administratif est catastrophique, même si les pixels restants sont parfaits.

Pour capturer cela, SSIM décompose la ressemblance en trois questions séparées posées localement sur chaque petite fenêtre de l'image :

$$\begin{aligned} l(x, y) &= (2\mu_x\mu_y + c_1)/(\mu_x^2 + \mu_y^2 + c_1) && \text{luminance (comparaison des moyennes)} \\ c(x, y) &= (2\sigma_x\sigma_y + c_2)/(\sigma_x^2 + \sigma_y^2 + c_2) && \text{contraste (comparaison des écarts-} \\ &&& \text{types)} \\ s(x, y) &= (\sigma_{xy} + c_3)/(\sigma_x\sigma_y + c_3) && \text{structure (corrélation entre les} \\ &&& \text{patches)} \\ \text{SSIM} &= l^\alpha \cdot c^\beta \cdot s^\gamma, \quad \alpha=\beta=\gamma=1, \quad c_3=c_2/2 \rightarrow \text{formule compacte ci-dessus} \end{aligned}$$

Chaque terme a la forme $2ab/(a^2+b^2)$, qui vaut 1 si et seulement si $a = b$, et décroît sinon. Le terme de structure est exactement le **coefficient de corrélation de Pearson** entre les deux patches — la covariance normalisée par le produit des écarts-types, soit le cosinus des patches centrés (rappel chapitre 3 : similarité cosinus, corrélation). Les constantes c_1 , c_2 stabilisent les rapports là où les dénominateurs frôlent zéro (zones uniformes).

Le geste décisif est la **séparation** : isoler la moyenne (luminance) et la variance (contraste) de la corrélation (structure) signifie qu'un décalage global de luminosité dégrade l mais laisse s intact. SSIM sous-pondère ainsi précisément les changements que l'œil tolère, là où le MSE les pénalisait à plein. ■

Ce que ça mesure / l'angle mort

L'observateur modélisé juge la **similarité structurelle locale**, peu sensible aux offsets de luminance et de contraste — c'est un modèle artisanal du système visuel humain. Angles morts : (1) c'est une *approximation* codée à la main, pas la perception réelle ; elle ne capte ni le masquage de texture, ni la couleur (SSIM standard ne travaille que sur la luminance) ; (2) elle est **mono-échelle** — la fenêtre 11×11 par défaut confond des flous de niveaux différents, d'où MS-SSIM qui combine plusieurs niveaux de sous-échantillonnage (écho du choix d'échelle σ aux chapitres 5 et 6) ; (3) comme le MSE, elle suppose les images **recalées** — un décalage d'un pixel abîme s ; (4) elle reste *full-reference* — sans l'original, rien à mesurer.



Reprenons la dégradation que le MSE punissait sévèrement : un décalage uniforme de luminance de +10. Patch de moyenne $\mu_x = 100$ et variance $\sigma_x^2 = 25$ ($\sigma_x = 5$) ; on pose $\hat{y} = x + 10$, donc $\mu_y = 110$, $\sigma_y = 5$, et $\sigma_{xy} = \text{cov}(x, x+10) = \text{var}(x) = 25$ (corrélation parfaite, la structure est intacte).

$$c_1 = (0.01 \cdot 255)^2 = 6.50 \quad c_2 = (0.03 \cdot 255)^2 = 58.52 \quad c_3 = c_2/2 = 29.26$$

$$\begin{aligned} l &= (2 \cdot 100 \cdot 110 + 6.50) / (100^2 + 110^2 + 6.50) = 22006.5 / 22106.5 \approx 0.9955 \\ c &= (2 \cdot 25 + 58.52) / (25 + 25 + 58.52) = 108.52 / 108.52 = 1.000 \\ s &= (25 + 29.26) / (25 + 29.26) = 1.000 \end{aligned}$$

$$\text{SSIM} = 0.9955 \cdot 1 \cdot 1 \approx 0.9955$$

Pour la **même** dégradation de +10, le MSE donne 100, soit PSNR ≈ 28.1 dB (« médiocre »). Deux verdicts opposés sur un même décalage de luminosité : l'observateur-MSE crie à la dégradation, l'observateur-SSIM répond « structure intacte, quasi identique ». C'est la signature du chapitre : chaque métrique punit ce que son observateur perçoit, et tolère le reste.



- **data_range obligatoire** : `structural_similarity(a, b, data_range=255)` pour du `uint8`, `=1.0` pour des flottants. Oublié, c_1 et c_2 sont calculés avec la mauvaise plage et le score dérive silencieusement.
- **Reproduire l'article de référence** : par défaut `scikit-image` utilise une fenêtre uniforme ; pour retrouver les valeurs de Wang et al. (2004), passer `gaussian_weights=True`, `sigma=1.5`, `use_sample_covariance=False`.
- **Couleur** : fournir `channel_axis` ; SSIM standard reste une mesure de luminance — pour la fidélité chromatique, lui adjoindre un ΔE (chapitre 7).
- **Pas une métrique au sens mathématique** : SSIM ne vérifie pas l'inégalité triangulaire ; $\text{DSSIM} = (1 - \text{SSIM})/2$ sert de coût borné dans les fonctions de perte (chapitre 15), pas de distance.
- **Recalage préalable** : comme le MSE, SSIM exige des images alignées — recalculer d'abord si nécessaire (chapitre 8).



```
# a : image de référence (np.ndarray H×W uint8 niveaux de gris)
# b : image dégradée (même format)
# Pour des images couleur BGR, convertir en niveaux de gris avant (ou
# utiliser channel_axis)
from skimage.metrics import structural_similarity

if not isinstance(a, np.ndarray) or not isinstance(b, np.ndarray):
    out_a = {'erreur': 'deux images attendues sur a et b'}
else:
    gray_a = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY) if a.ndim == 3 else a
    gray_b = cv2.cvtColor(b, cv2.COLOR_BGR2GRAY) if b.ndim == 3 else b
    score, smap = structural_similarity(
        gray_a, gray_b,
        data_range=255,
        gaussian_weights=True,
        sigma=1.5,
        use_sample_covariance=False,
        full=True,
    )
    # smap est la carte locale de SSIM : montre OÙ la dégradation siège
    out_a = {'SSIM': round(float(score), 4)}
    out_b = smap.astype(np.float32) # carte spatiale (connecter à
output_display)
```

Domaines d'usage : réglage et évaluation des codecs image/vidéo (SSIM et MS-SSIM y ont supplanté le PSNR comme étalon perceptuel), bancs d'essai de super-résolution et de restauration, validation de la compression en imagerie médicale (où la préservation de la structure est diagnostique).

14.3. Entropie d'image – la qualité sans référence, ou le contenu informationnel



$$H = -\sum_{g=0}^{L-1} p(g) \cdot \log_2 p(g) \quad [\text{bits/pixel}]$$

$p(g)$ = fréquence du niveau g dans l'histogramme normalisé. $H \in [0, \log_2 L]$; au plus 8 bits pour une image 8 bits.

Dérivation : l'entropie de Shannon de l'histogramme

L'entropie de Shannon mesure la « surprise » moyenne d'un message. L'image mentale utile est celle d'un chiffreur : combien de bits faut-il en moyenne pour noter la valeur d'un pixel

tiré au hasard dans cette image ? Si l'image ne contient qu'un seul niveau (fond uni), chaque pixel est prévisible à 100 % — 0 bit suffit, $H = 0$. Si tous les niveaux sont équiprobables (histogramme parfaitement plat), chaque pixel est une surprise totale — H atteint son maximum $\log_2(L)$.

Par la concavité de $-x \cdot \log x$ (inégalité de Gibbs), H est **maximale pour l'histogramme uniforme** et **nulle pour une image constante**. C'est l'entropie de la distribution marginale des intensités — la même grandeur que l'entropie du chapitre 13 (§13.1), ici calculée sur l'image entière. ■

Le changement de régime est fondamental : contrairement à MSE et SSIM, l'entropie est **sans référence**. Il n'y a pas d'original à approcher ; on mesure une propriété de l'image elle-même — l'étalement de son histogramme, son occupation de la plage dynamique. Comme proxy de qualité, elle capte le contraste global : une image délavée, sous-exposée ou saturée a une entropie basse ; une image bien exposée et contrastée, une entropie haute.

Ce que ça mesure / l'angle mort

L'observateur modélisé ne pose qu'une seule question : « y a-t-il de l'information ? » — sans jamais regarder *où* elle se trouve. C'est la mesure du premier ordre par excellence, **aveugle à l'arrangement spatial** (même angle mort qu'au §13.1) : deux images de même histogramme, l'une nette, l'autre obtenue en mélangeant ses pixels aléatoirement, obtiennent exactement la même entropie. Pire, le **bruit augmente l'entropie** — il étale l'histogramme — si bien qu'une image plus bruitée peut scorer « mieux » qu'une image propre. L'entropie confond information utile et bruit. Son vrai créneau : l'exposition/contraste, le seuillage à entropie maximale (chapitre 12), et l'information mutuelle pour le recalage multimodal (CT $\leftrightarrow$ IRM).



Deux images 2×2 calculables à la main :

`img = [0 64 ; 128 192]` → 4 niveaux distincts, $p = 1/4$ chacun
 $H = -4 \cdot (1/4) \cdot \log_2(1/4) = -\log_2(1/4) = 2$ bits (maximum pour 4 symboles)

`img2 = [100 100 ; 100 200]` → niveau 100 (×3, $p=3/4$), niveau 200 (×1, $p=1/4$)
 $H = -(3/4) \cdot \log_2(3/4) - (1/4) \cdot \log_2(1/4)$
 $= 0.75 \cdot 0.415 + 0.25 \cdot 2 = 0.311 + 0.500 = 0.811$ bits

L'image à histogramme plat exploite toute sa plage dynamique (2 bits, le maximum) ; l'image piquée n'en utilise qu'une fraction (0.811 bit). L'avertissement tient : mélangez les pixels de `img`, l'entropie reste 2 bits — l'arrangement lui est invisible ; et un patch poivre-et-sel scorerait tout aussi haut qu'une image nette richement texturée.



- **Base du logarithme** : bits ($\log_2$) ou nats ($\ln$) sont deux conventions incompatibles — facteur $\ln 2 \approx 0.693$ entre les deux. `skimage.measure.shannon_entropy(image, base=2)` garantit des bits.
- **Binning** : pour des images 16 bits ou flottantes, le nombre de classes de l'histogramme change H fortement. Fixer les bins explicitement et documenter le choix.
- **Entropie $\neq$ qualité perceptuelle** : ne jamais l'utiliser seule comme critère de « meilleure image ». La coupler à une mesure de structure ou de netteté.
- **Global vs local** : l'entropie globale ignore la localisation. Pour la mise au point ou la texture, préférer une **carte d'entropie locale** calculée par filtre de voisinage.



```
# a : image (np.ndarray HxW uint8 niveaux de gris, ou HxWx3)
from skimage.measure import shannon_entropy

if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'image attendue sur a'}
else:
    gray = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY) if a.ndim == 3 else a
    h_global = float(shannon_entropy(gray, base=2))

    # carte locale d'entropie sur fenêtre 9x9
    from skimage.filters.rank import entropy as rank_entropy
    from skimage.morphology import disk
    h_map = rank_entropy(gray, disk(4)).astype(np.float32)

    out_a = {
        'entropie_globale_bits': round(h_global, 3),
        'max_theorique_bits': 8.0,
        'taux_occupation': round(h_global / 8.0, 3),
    }
    out_b = h_map # carte locale (connecter à output_display +
                  colormap)
```

Domaines d'usage : optimisation d'exposition automatique, seuillage à entropie maximale (chapitre 12), recalage multimodal par information mutuelle (CT ↔ IRM en imagerie médicale), pré-tri grossier de contraste dans les grandes bases d'images.

14.4. Mesures de netteté sans référence – variance du Laplacien, énergie de gradient



Variance du Laplacien	: $S = \text{Var}(\nabla^2 I)$
Tenengrad (Sobel)	: $S = \sum (G_x^2 + G_y^2)$
Brenner	: $S = \sum (I(x+2, y) - I(x, y))^2$

Trois mesures *sans référence* de netteté ou de mise au point : elles montent avec le contenu haute-fréquence et s'effondrent sous le flou.

L'idée : le flou est un passe-bas, la netteté une énergie haute-fréquence

Un flou est un filtrage **passé-bas** — rappel chapitre 5 : une gaussienne atténue les hautes fréquences ; rappel chapitre 10 : la convolution par un noyau lissant correspond à multiplier le spectre de Fourier par un filtre qui écrase les hautes fréquences. La netteté est donc proportionnelle à l'énergie haute-fréquence qui survit.

L'image mentale utile est celle d'un bord net : une frontière tranchée entre une zone sombre et une zone claire. Sous un bon objectif, ce bord dessine une marche presque verticale dans le profil d'intensité. Sous un flou, la marche s'étale en une pente douce. Le Laplacien ∇^2 est un opérateur **passé-haut** — sa transformée de Fourier croît avec la fréquence spatiale. Appliqué à une image nette, il réagit fortement aux bords ; appliqué à une image floue, ses réponses s'affaiblissent. La variance de ces réponses quantifie ce qui reste de hautes fréquences : grande pour une image nette (nombreux fronts marqués), proche de zéro sous le flou (fronts étalés). Tenengrad somme les gradients de Sobel au carré (chapitre 6), Brenner utilise une simple différence entre pixels distants de 2 — trois proxys de la même grandeur, l'énergie haute-fréquence. ■

Ce que ça mesure / l'angle mort

L'observateur modélisé est un **détecteur de mise au point**, sans mémoire de l'original. Angle mort majeur : la valeur absolue n'est **pas comparable d'une scène à l'autre** — un ciel uniforme parfaitement net a peu de hautes fréquences, donc un score bas malgré sa netteté ; ces mesures ne sont monotones qu'au sein d'une *même* scène parcourue en focus. Comme l'entropie, elles sont dupées par le **bruit**, qui injecte des hautes fréquences (une image bruitée floue peut battre une image nette propre). Et elles ne disent rien des autres dégradations : couleur, artefacts de compression, distorsion géométrique.



Laplacien 1-D discret, noyau $[1 \ -2 \ 1]$, sur un front net puis flouté :

Front net : $I = [0, 0, 0, 100, 100, 100]$

$L_i = I_{i-1} - 2I_i + I_{i+1}$ aux 4 positions internes : $[0, 100, -100, 0]$

$\text{Var} = (0^2 + 100^2 + 100^2 + 0^2)/4 = 5000$

Front flouté : $I_b = [0, 0, 33, 67, 100, 100]$

$L = [33, 1, -1, -33]$

$\text{Var} = (33^2 + 1^2 + 1^2 + 33^2)/4 = (1089 + 1 + 1 + 1089)/4 = 545$

La variance du Laplacien chute d'un facteur ≈ 9 entre le front net (5000) et le même front étalé (545). C'est ce signal que l'autofocus exploite : on balaie la mise au point mécaniquement et on retient la position qui maximise cette variance, dans les microscopes comme dans les smartphones.



- **Valeur non comparable entre scènes** : ne jamais fixer un seuil universel de netteté ; n'utiliser que l'ordre relatif sur une *même* scène (pile de focus, sweep d'autofocus).
- **Le bruit gonfle la mesure** : débruiter légèrement avant le calcul, ou restreindre la ROI à une zone texturée — sur un fond uni, la mesure ne capte que le bruit thermique.
- **Débordement** : `cv2.Laplacian(gray, cv2.CV_64F)` — la réponse laplacienne peut être négative et dépasser 255 en valeur absolue. Utiliser `CV_8U` écrête silencieusement et fausse la variance.
- **Échelle du noyau et seuil de Tenengrad** modifient la valeur absolue. Les figer dans tout système comparatif.



```
# a : image (np.ndarray HxW uint8 niveaux de gris, ou HxWx3)
if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'image attendue sur a'}
else:
    gray = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY) if a.ndim == 3 else a

    # Variance du Laplacien
    lap = cv2.Laplacian(gray, cv2.CV_64F)
    var_lap = float(lap.var())

    # Tenengrad (énergie de gradient Sobel)
    gx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
    gy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
    tenengrad = float(np.mean(gx ** 2 + gy ** 2))

    out_a = {
        'var_laplacien': round(var_lap, 2),
        'tenengrad': round(tenengrad, 2),
        'interpretation': 'comparer uniquement sur meme scene/objet',
    }
```

Domaines d'usage : autofocus (microscopie à fluorescence, smartphones, endoscopie), sélection d'images nettes pour le focus-stacking (macro, astrophotographie), détection de flou en contrôle qualité (rejet de scans/pages OCR hors mise au point), criblage du flou de bougé en vidéo.

14.5. Métriques perceptuelles apprises (LPIPS) – l’observateur ajusté sur l’humain



$$\text{LPIPS}(x, y) = \sum_l \left(\frac{1}{H_l W_l} \sum_{h, w} \| w_l \odot (\hat{\phi}_l(x)_{hw} - \hat{\phi}_l(y)_{hw}) \|_2^2 \right)$$

$\hat{\phi}_l$ = activations normalisées de la couche l d’un réseau pré-entraîné (VGG, AlexNet) ; w_l = poids appris pour corrélérer avec le jugement humain. Plus petit = plus proche perceptuellement.

Dérivation : comparer dans un espace de représentation appris

Plutôt que les pixels bruts (MSE) ou des statistiques locales codées à la main (SSIM), LPIPS compare les images dans l’**espace de features d’un réseau profond** — une représentation où la distance euclidienne corrèle avec le jugement humain. L’image mentale utile est celle d’un espace de descripteurs (rappel chapitre 3 et chapitre 1) : deux images proches dans cet espace ressemblent à quelque chose d’analogue pour un observateur humain, même si leurs pixels diffèrent.

Les activations d’un réseau entraîné sur ImageNet encodent texture, structure, forme et sémantique à différentes échelles — des bords en couche 1 aux objets reconnaissables en couche 5. Un écart dans cet espace pénalise les changements perceptuellement saillants et ignore les imperceptibles. Les poids w_l sont calibrés sur des triplets humains « laquelle ressemble le plus à la référence ? ». C’est le geste de tout le livre — choisir la représentation où la bonne distance devient signifiante — mais la représentation est ici **apprise des données humaines** plutôt que conçue par un ingénieur. ■

Ce que ça mesure / l’angle mort

L’observateur modélisé est un réseau calibré sur la préférence humaine ; LPIPS suit la perception bien mieux que PSNR/SSIM sur de nombreuses distorsions, en particulier les artefacts « perceptuels » des modèles génératifs et de la super-résolution. Angles morts : (1) il hérite des **biais de son réseau de base et de son jeu d’entraînement** — il peut se tromper sur du contenu hors distribution (imagerie médicale, télédétection) ; (2) il reste *full-reference*, lourd (GPU), **non interprétable** — on ne sait pas *pourquoi* deux images sont jugées dissemblables ; (3) c’est encore un *modèle* de l’observateur, pas l’observateur — il existe des exemples adverses, et l’optimiser directement dans une fonction de perte crée des artefacts qui flattent la métrique sans plaire à l’œil ; (4) pour le sans-référence, des alternatives existent — BRISQUE et NIQE (statistiques) ou MUSIQ et MANIQA (appris récents).

Exemple — le compromis perception/distorsion

Une métrique apprise ne se calcule pas à la main ; l'exemple est qualitatif mais vérifiable sur n'importe quel banc de super-résolution. Soit une image de document agrandie $\times 4$ par deux méthodes :

Bicubique : pixels « moyens », image floue mais fidèle
→ PSNR élevé, SSIM correct, LPIPS ÉLEVÉ (mauvais)

Super-résolution GAN : texture plausible mais inventée
→ PSNR plus bas, SSIM plus bas, LPIPS BAS (bon)

Les observateurs humains préfèrent presque toujours la sortie GAN ; pourtant elle a le pire PSNR, car ses pixels ne coïncident pas avec la vérité — elle *hallucine* une texture crédible de caractères. C'est le **compromis perception/distorsion** : fidélité pixel et qualité perçue peuvent s'opposer radicalement. Chaque métrique tranche selon son observateur — l'observateur-pixel récompense le flou fidèle, l'observateur-humain récompense la netteté plausible.



- **Backbone et version** : LPIPS-VGG $\neq$ LPIPS-Alex — ne jamais comparer des scores issus de réseaux différents dans le même tableau.
- **Normalisation d'entrée** : LPIPS attend du RGB dans $[-1, 1]$, forme (N, 3, H, W). Une mauvaise normalisation — oublier de passer de $[0, 255]$ à $[-1, 1]$, ou garder l'ordre BGR — corrompt le score silencieusement.
- **Pas une métrique au sens mathématique** : l'inégalité triangulaire n'est pas garantie — usage de classement uniquement, pas de combinaison arithmétique.
- **Jouabilité** : optimiser un modèle génératif *directement* contre LPIPS produit des artefacts qui flattent la métrique sans plaire à l'œil — l'observateur appris, comme tout observateur, peut être trompé.



```
# LPIPS requiert le module lpips (PyTorch) – non disponible dans le
# moteur VNStudio standard.
# Ce script illustre la logique ; exécuter depuis un environnement Python
# complet.
# a : image de référence (np.ndarray H×W×3 uint8 BGR)
# b : image dégradée (même format)

# Équivalent léger dans VNStudio : comparer les histogrammes de features
# ORB
# ou utiliser la node cv_features + comparaison de descripteurs

if not isinstance(a, np.ndarray) or not isinstance(b, np.ndarray):
    out_a = {'erreur': 'deux images attendues sur a et b'}
else:
    # Proxy perceptuel accessible sans GPU : corrélation d'histogrammes
    # par canal
    scores = []
    for ch in range(3):
        ha = cv2.calcHist([a], [ch], None, [64], [0, 256])
        hb = cv2.calcHist([b], [ch], None, [64], [0, 256])
        cv2.normalize(ha, ha); cv2.normalize(hb, hb)
        scores.append(float(cv2.compareHist(ha, hb, cv2.HISTCMP_CORREL)))
    proxy = float(np.mean(scores))
    out_a = {
        'proxy_perceptuel_correl': round(proxy, 4),
        'note': '1.0 = identiques, <0.9 = divergence chromatique ou
structurelle notable',
    }
```

Domaines d'usage : évaluation de la super-résolution et des modèles génératifs (GANs, diffusion), coût perceptuel pour entraîner des réseaux de restauration d'image, classement d'images quand la cible est la préférence humaine plutôt que la fidélité pixel.

14.6. Tableau récapitulatif – quel observateur, quelle référence, quel angle mort

Métrique	Modèle d'observation	Quat-quel pénalise	Angle mort principal	Référence	Usage typique
MSE / PSNR	/ comparateur de pixels indépendants	tout écart pixel à pixel	aveugle à la structure et à la <i>position</i> de l'erreur	pleine	codecs, dé-bruitage, transmission
SSIM / MS-SSIM	SVH ap-proximité, sensible à la structure locale	perte de corrélation structurelle	modèle artisanal, mono-échelle, sensible au décalage	pleine	codecs image/vidéo, imagerie médicale
Entropie	« y a-t-il de l'information ? »	contenu / plage dynamique non exploitée	ignore l'arrangement spatial ; le bruit l'augmente	aucune	exposition, recalage multimodal
Var. Laplacien / Tenen-grad	détecteur de mise au point	énergie haute-fré-quence insuffisante	non comparable entre scènes ; le bruit la gonfle	aucune	autofocus, focus-stacking, OCR
LPIPS / IQA appris	réseau calibré sur le jugement humain	ce qui dérange un observateur humain	biais du backbone ; non interprétable ; jouable	pleine (NIQE/MANI-QA : aucune)	super-résolution, génératif

Note sur l'état de l'art : le PSNR domine encore les rapports de codecs par tradition, mais SSIM et MS-SSIM s'imposent dès qu'on vise la perception, et les métriques apprises (LPIPS, DISTs) sont devenues l'étalon pour la super-résolution et le génératif. Les méthodes classiques gardent un créneau net : **interprétables, sans entraînement, légères** — indispensables là où l'on n'a ni GPU ni données (autofocus embarqué, contrôle industriel, microscopie). D'autres outils complètent la boîte sans figurer ici : VIF et FSIM (full-reference perceptuels), BRISQUE/NIQE (sans-référence statistiques), VMAF (qualité vidéo, fusion

apprise de plusieurs métriques), et le ΔE du chapitre 7 pour la fidélité chromatique que SSIM ignore.

14.7. une métrique de qualité est un observateur déguisé

Le chapitre raconte une seule histoire, déclinée cinq fois. Il n'existe pas de « qualité » dans l'absolu, seulement une qualité *pour un observateur*, et chaque métrique en est un, plus ou moins explicite :

MSE/PSNR	: observateur « les pixels doivent coïncider »	– punit tout écart, ignore la structure
SSIM	: observateur « l'œil voit des structures »	– punit la perte de corrélation, tolère luminance/contraste
Entropie	: observateur « y a-t-il de l'information ? »	– sans référence, punit le manque de contenu, gobe le bruit
Netteté	: observateur « est-ce net ? »	– sans référence, punit le flou, gobe le bruit
LPIPS	: observateur « voici ce qu'un humain préfère »	– appris, punit ce qui dérange un humain, hérite ses biais

D'un bout à l'autre, le même mouvement : **ce que la métrique pénalise est exactement ce que son observateur perçoit ; ce qu'elle ignore est ce qu'il tolère.** Le MSE achète sa simplicité en restant aveugle à la position de l'erreur ; SSIM achète l'invariance à la luminance en séparant moyenne, variance et corrélation ; l'entropie et la netteté achètent l'indépendance à toute référence au prix de se laisser duper par le bruit ; LPIPS achète la corrélation à l'humain au prix de l'opacité et du biais. Le passage du *full-reference* au *no-reference* puis à l'*appris* n'est rien d'autre qu'une explicitation croissante du modèle d'observateur — de l'implicite naïf au réseau calibré sur des jugements.

C'est, transposé de la comparaison de vecteurs à la comparaison d'images, le fil du **chapitre 3** (« choisir une distance, c'est déclarer ce qui compte »), et la reprise directe du **chapitre 4** (« aucune métrique unique ne capture tout » — d'où l'usage conjoint PSNR + SSIM + LPIPS selon la tâche). Comme il n'existe pas d'espace couleur vrai mais des espaces adaptés à un usage (**chapitre 7**), comme un filtre encode un a priori sur le signal (**chapitre 5**), une métrique de qualité encode un a priori sur **qui regarde**. Mesurer la qualité d'une image, comme décrire une forme (**chapitre 1**), choisir un descripteur (**chapitre 13**) ou estimer un flot (**chapitre 9**), c'est choisir un cadre où ce qui compte devient mesurable : **nommer l'observateur, c'est déjà résoudre la moitié du problème** — et tout l'art consiste à choisir, pour l'usage visé, l'observateur dont les angles morts sont ceux qu'on peut se permettre.

CHAPITRE

Apprentissage profond (fonctions de coût)

15

Table des matières

15.1. Entropie croisée et softmax — quand le gradient est l'erreur elle-même	247
15.2. Dice loss — transformer une métrique de segmentation en objectif	249
15.3. Focal loss — déplacer le gradient vers les exemples difficiles	252
15.4. Smooth L1 (Huber) — le compromis entre stabilité et robustesse	254
15.5. IoU loss et GIoU — optimiser la métrique, et combler ses zones plates	257
15.6. Loss contrastive (InfoNCE) — structurer un espace de représentation	260
15.7. Tableau récapitulatif — quel coût pour quel objectif	263
15.8. le coût n'est pas la cible, c'est la pente vers elle	263

Un réseau de neurones n'apprend rien de l'objectif qu'on lui fixe en pensée — il apprend de la **dérivée** de cet objectif. Entre la métrique qui nous intéresse (la justesse d'une classification, le chevauchement d'une détection, la ressemblance perceptuelle d'une image générée) et ce que la descente de gradient peut réellement minimiser, il y a un fossé : la plupart des métriques sont discrètes, non dérivables, ou plates sur de vastes régions. La fonction de coût est le pont. Elle n'est pas un score mais une **source de gradient** — ce qui compte n'est pas tant sa valeur que la forme de sa pente : où elle pousse fort, où elle s'aplatit, où elle pointe à côté de la cible.

Le fil conducteur du chapitre tient en une phrase : **on n'optimise jamais la métrique qu'on vise, mais un substitut dérivable**. Concevoir un coût, c'est sculpter un paysage de gradients qui conduit vers la métrique sans jamais la toucher directement. Chaque section pose la même question : quelle est la forme du gradient, et où trahit-il la métrique cible — par saturation (gradient nul là où il faudrait apprendre), par explosion (un exemple aberrant écrase tous les autres), ou par désalignement (la pente descend, mais pas vers ce qu'on veut) ?

Rappel des liens avec les chapitres précédents. Les fonctions de coût recyclent presque tout le livre, mais en changeant le statut : un score d'évaluation y devient un objectif d'optimisation, ce qui impose une contrainte nouvelle — la dérivabilité — absente quand on se contentait de mesurer.

chapitre 3 (distances) : la distance L2 redevient le coût MSE ; la similarité cosinus est le cœur d'InfoNCE
 chapitre 4 (métriques seg.) : IoU et Dice – métriques d'évaluation – deviennent ici des coûts ; le passage métrique → coût EST le sujet du chapitre
 chapitre 8 (caméra) : l'erreur de reprojection du bundle adjustment est déjà un coût ; smooth L1 y régularise les grosses erreurs
 chapitre 14 (qualité) : SSIM et LPIPS deviennent des coûts perceptuels (perceptual loss, DSSIM)
 chapitre 16 (stats robustes) : smooth L1 EST le M-estimateur de Huber ; focal loss et Huber partagent l'idée de pondérer les exemples

Le couple *métrique / coût* est le pendant exact du fil du chapitre 4 (« aucune métrique unique ne capture tout ») : ici, non seulement aucun coût ne capture tout, mais le coût n'est même pas la chose qu'on veut — il en est l'approximation dérivable. Et il prolonge le chapitre 3 (« choisir une distance, c'est déclarer ce qui compte ») d'un cran : choisir un coût, c'est déclarer ce qui compte *et* comment l'apprendre.

15.1. Entropie croisée et softmax – quand le gradient est l’erreur elle-même



softmax : $\hat{y}_i = \exp(z_i) / \sum_j \exp(z_j)$
entropie croisée : $L = -\sum_i y_i \cdot \log(\hat{y}_i)$

z désigne les **logits** : les sorties brutes du réseau avant toute normalisation, des nombres qui peuvent être positifs, négatifs, grands ou petits. $\hat{y}$ est la distribution de probabilités obtenue après le softmax, et y est la cible (format **one-hot** : un vecteur rempli de zéros sauf à l’indice de la bonne classe, où il vaut 1). Les deux formules vont presque toujours ensemble — softmax transforme les logits en distribution valide (tous positifs, somme égale à 1), l’entropie croisée mesure l’écart à la distribution cible.

L’idée Pour comprendre l’entropie croisée, il faut d’abord saisir ce qu’elle pénalise. Imaginons un réseau qui classe des radiographies thoraciques et qui doit sortir une probabilité pour chaque diagnostic. L’entropie croisée punit non pas le fait de se tromper en général, mais la **confiance mal placée** : se tromper avec certitude (prédire 99 % pour la mauvaise classe) coûte infiniment plus cher que se tromper avec hésitation. La formule $-\log(\hat{y})$ encode directement cette idée — quand $\hat{y}$ tend vers 0, $-\log(\hat{y})$ explose vers l’infini ; quand $\hat{y}$ vaut 1 (confiance parfaite dans la bonne réponse), le coût tombe à zéro.

La métrique qu’on vise en réalité est la **justesse de classification** (le taux de bonnes réponses), mais cette métrique est un escalier : elle ne change pas quand les logits bougent légèrement, elle change d’un bond quand la classe prédite bascule d’une autre classe vers la bonne. Un escalier n’a pas de pente exploitable — on le remplace donc par l’entropie croisée, qui est lisse et toujours dérivable. C’est le premier exemple du fil conducteur.

Dérivation Le couple softmax + entropie croisée est célèbre pour son gradient d’une élégance désarmante. Posons la dérivée partielle du softmax par rapport au logit z_k :

$$\partial \hat{y}_i / \partial z_k = \hat{y}_i (\delta_{ik} - \hat{y}_k) \quad (\delta_{ik} = 1 \text{ si } i=k, \text{ sinon } 0)$$

Puis dérivons le coût L par rapport à z_k (en appliquant la règle de la chaîne) :

$$\begin{aligned} \partial L / \partial z_k &= -\sum_i y_i \cdot (1/\hat{y}_i) \cdot \partial \hat{y}_i / \partial z_k \\ &= -\sum_i y_i \cdot (1/\hat{y}_i) \cdot \hat{y}_i (\delta_{ik} - \hat{y}_k) \\ &= -\sum_i y_i (\delta_{ik} - \hat{y}_k) \\ &= -y_k + \hat{y}_k \cdot \sum_i y_i \\ &= \hat{y}_k - y_k \quad (\text{car } \sum_i y_i = 1 \text{ pour un vecteur one-hot}) \quad \blacksquare \end{aligned}$$

Le gradient sur les logits est **exactement la différence entre la prédiction et la cible**. Pas de facteur parasite, pas de saturation cachée : si le réseau prédit 0,7 là où il fallait 1,

le logit reçoit une poussée de $-0,3$; s'il a déjà raison ($\hat{y}_k = y_k$), le gradient s'annule de lui-même. C'est cette propriété qui rend l'entropie croisée si docile à entraîner.



Un classifieur pour la reconnaissance de chiffres manuscrits sort les logits $z = [2,0 ; 1,0 ; 0,1]$ pour les classes {« 3 », « 8 », « 5 »}, la vérité étant « 3 » ($y = [1, 0, 0]$).

```
exp(z)    = [7,389 ; 2,718 ; 1,105]      somme = 11,212
softmax   = [0,659 ; 0,242 ; 0,099]
L         = -log(0,659) = 0,417
gradient  =  $\hat{y} - y = [0,659-1 ; 0,242 ; 0,099] = [-0,341 ; 0,242 ; 0,099]$ 
```

Le logit de la bonne classe est poussé vers le haut ($-0,341$), ceux des deux distracteurs vers le bas. La somme du gradient est nulle ($-0,341 + 0,242 + 0,099 = 0$) : le softmax conserve la masse totale de probabilité — on redistribue, on n'en crée pas.

Angle mort L'entropie croisée traite chaque exemple à poids égal, quelle que soit sa difficulté ou la fréquence de sa classe. Sur un jeu déséquilibré — 99 % de pixels de fond et 1 % d'objet en segmentation, ou des milliers d'images saines pour quelques anomalies en contrôle qualité — la masse des exemples faciles domine la somme et le gradient. Le réseau apprend à bien traiter le cas majoritaire et ignore les cas rares. C'est précisément le problème que la focal loss (§15.3) et le Dice loss (§15.2) corrigent.



Calculer **softmax** puis **log** séparément est numériquement instable : un logit grand fait déborder **exp** (résultat infini), un logit très négatif donne $\log(0) = -\infty$. La parade est l'astuce **log-sum-exp**, qui soustrait le maximum des logits avant l'exponentielle — mathématiquement neutre, numériquement salvateur :

$$\log \sum_j \exp(z_j) = m + \log \sum_j \exp(z_j - m), \quad m = \max(z)$$

En pratique : ne jamais enchaîner soi-même **softmax** + **log**. Passer directement les **logits bruts** à la fonction **cross_entropy**. Un bug classique consiste à normaliser d'abord en probabilités, puis à les passer à **cross_entropy** — c'est une double application du softmax qui écrase les gradients sans faire planter le programme.



```
# a : vecteur de logits bruts (list ou np.ndarray, shape [C])
# b : indice de la classe correcte (int)
if not isinstance(a, np.ndarray) or not isinstance(b, (int, float)):
    out_a = {'erreur': 'logits (a) et classe cible (b) requis'}
else:
    z = np.array(a, dtype=float)
    k = int(b)
    # log-sum-exp stable
    m = float(np.max(z))
    log_sum_exp = m + float(np.log(np.sum(np.exp(z - m))))
    log_softmax_k = float(z[k]) - log_sum_exp      # log(ŷk), stable
    softmax = np.exp(z - log_sum_exp)              # ŷ

    # gradient = ŷ - y (one-hot)
    gradient = softmax.copy()
    gradient[k] -= 1.0

    out_a = {
        'logits': [float(x) for x in z],
        'softmax': [float(x) for x in softmax],
        'cout': float(-log_softmax_k),
        'gradient_logit_cible': float(gradient[k]),
    }
```

15.2. Dice loss – transformer une métrique de segmentation en objectif



Dice « soft » : $L = 1 - 2 \cdot \sum(p \cdot g) / (\sum p + \sum g + \epsilon)$

p désigne la carte de probabilités prédite par le réseau — des valeurs continues entre 0 et 1 pour chaque pixel. g est le masque de vérité binaire (1 = objet, 0 = fond). Les sommes portent sur tous les pixels de l'image. ϵ est un petit terme de stabilité numérique, typiquement 1.

L'idée Le coefficient de Dice du chapitre 4 opère sur des masques **binaires** : l'intersection compte des pixels entiers, c'est une fonction en escalier, non dérivable. Le réseau ne peut pas l'optimiser directement. L'astuce du Dice loss consiste à **relâcher** cette contrainte binaire : on remplace l'intersection de pixels par leur produit $p \cdot g$, et les cardinaux par des sommes de probabilités. Quand p est binaire (0 ou 1), on retrouve exactement le Dice du chapitre 4 ; entre les deux extrêmes, la version « soft » est continue et dérivable.

L'image mentale utile est celle d'un thermomètre gradué. Le Dice binaire dit « l'objet est là ou il n'est pas ». Le Dice soft dit « l'objet est là à 73 % ». Cette nuance de gris est exactement ce dont le gradient a besoin pour savoir dans quelle direction pousser le réseau.

Le point crucial qui distingue le Dice loss de l'entropie croisée se lit dans son gradient. Pour un pixel p_j , en notant $n = 2\Sigma(p \cdot g)$ et $d = \Sigma p + \Sigma g$:

$$\partial L / \partial p_j = -(2g_j \cdot d - n) / d^2$$

Le dénominateur d **normalise par la taille des régions**. Que l'objet occupe 1 % ou 50 % de l'image, l'échelle du gradient reste comparable. C'est là la raison d'être du Dice loss : là où l'entropie croisée serait noyée par les millions de pixels de fond (chacun produisant un gradient minuscule mais s'additionnant), le Dice rapporte tout au chevauchement relatif. Ce substitut dérivable ne cherche pas à copier fidèlement la formule du chapitre 4 — il cherche à en imiter le *comportement* tout en restant manipulable par la descente de gradient.



Une coupe d'imagerie médicale (IRM) montre une petite lésion. Le masque de vérité g compte 4 pixels positifs. Le réseau prédit les probabilités suivantes :

pixels objet ($g=1$) : $p = 0,9 ; 0,8 ; 0,6 ; 0,3 \rightarrow \Sigma(p \cdot g) = 2,6$

pixels fond ($g=0$) : un seul faux positif $p = 0,2$

$$\Sigma(p \cdot g) = 2,6$$

$$\Sigma p = 2,6 + 0,2 = 2,8$$

$$\Sigma g = 4$$

$$\text{Dice} = 2 \cdot 2,6 / (2,8 + 4) = 5,2 / 6,8 = 0,765$$

$$L = 1 - 0,765 = 0,235$$

Le pixel objet à $p = 0,3$ (mal détecté) reçoit le gradient le plus fort vers le haut ; le faux positif à $p = 0,2$ reçoit une poussée vers le bas. Les milliers de vrais négatifs corrects sont totalement ignorés par le coût — c'est tout l'intérêt face au déséquilibre objet/fond.



Sans le terme ε , une image **sans objet** (g entièrement nul) et une prédiction également vide donnent 0/0 — ce NaN (Not a Number) contamine alors tout le lot d'images en cours d'entraînement. Le ε placé au numérateur *et* au dénominateur rend le cas vide bien défini (Dice = 1, coût = 0 : prédire « rien » sur une image sans objet est correct). Autre piège : appliquer le Dice loss sur les logits bruts sans les passer par une fonction sigmoïde ou softmax d'abord produit des résultats aberrants. En multiclasse, calculer le Dice par classe puis faire la moyenne — sinon une grande classe (le fond) masque les petites (les lésions).



```
# a : probabilités prédites par le réseau, np.ndarray HxW float, valeurs
# dans [0,1]
# b : masque de vérité binaire, np.ndarray HxW uint8 (0 ou 255)
if not isinstance(a, np.ndarray) or not isinstance(b, np.ndarray):
    out_a = {'erreur': 'probas (a) et masque verite (b) requis'}
else:
    p = a.astype(float).flatten()
    g = (b.astype(float) / 255.0).flatten() # normalise 0/255 → 0/1

    eps = 1.0
    inter = float(np.sum(p * g))
    sum_p = float(np.sum(p))
    sum_g = float(np.sum(g))
    dice = (2.0 * inter + eps) / (sum_p + sum_g + eps)
    cout = 1.0 - dice

    out_a = {
        'intersection_soft': round(inter, 4),
        'sum_pred': round(sum_p, 4),
        'sum_verite': round(sum_g, 4),
        'dice_soft': round(dice, 4),
        'dice_loss': round(cout, 4),
    }
```

15.3. Focal loss – déplacer le gradient vers les exemples difficiles



$L = -\alpha \cdot (1 - p_t)^\gamma \cdot \log(p_t)$ avec p_t = proba prédite de la VRAIE classe

α pondère les classes (équilibre de fréquences), $\gamma \geq 0$ est le **facteur de focalisation**. À $\gamma = 0$, on retrouve exactement l'entropie croisée pondérée par α .

L'idée Le problème de l'entropie croisée sur les jeux déséquilibrés est que les exemples faciles, bien que chacun produise un gradient minuscule, sont si nombreux qu'ils dominent la mise à jour totale. Sur un détecteur d'objets examinant des dizaines de milliers de zones candidates à chaque image, 99,9 % des zones sont du fond (exemple facile) et 0,1 % contiennent un objet (exemple difficile). Le réseau apprend surtout à dire « c'est du fond ».

L'idée de la focal loss est d'ajouter un **facteur modulateur** $(1 - p_t)^\gamma$ devant l'entropie croisée. Lisons son effet selon la difficulté de l'exemple :

exemple BIEN classé : $p_t \rightarrow 1$ $\Rightarrow (1-p_t)^\gamma \rightarrow 0$ le coût (et son gradient) s'effacent

exemple MAL classé : $p_t \rightarrow 0$ $\Rightarrow (1-p_t)^\gamma \rightarrow 1$ le coût reste celui de l'entropie croisée

Le facteur **éteint progressivement** la contribution des exemples déjà maîtrisés. Le gradient se concentre sur le petit nombre d'exemples résistants. C'est l'inverse d'une simple pondération de classe — focal loss pondère par la *difficulté courante*, qui évolue au fil de l'entraînement. Un exemple difficile au début peut devenir facile en milieu d'entraînement, et son poids diminue automatiquement. C'est encore le fil conducteur : la justesse de classification (la métrique visée) reste non dérivable et insensible au déséquilibre, mais ce substitut remodèle la pente pour que le gradient mène quand même où on veut.

Le parallèle avec le M-estimateur de Huber (chapitre 16) est instructif **par opposition** : Huber réduit l'influence des grandes erreurs (les valeurs aberrantes à ignorer) ; focal loss réduit l'influence des *petites* erreurs (les exemples faciles, déjà appris). Deux repondérations, deux philosophies — robustesse contre les aberrations d'un côté, focalisation sur le difficile de l'autre.

Exemple numérique ($\gamma = 2$) Comparaison sur deux exemples tirés d'un système de contrôle qualité industriel (détection de défauts rares sur des pièces conformes) :

exemple FACILE $p_t = 0,9$: entropie croisée = $-\log(0,9) = 0,105$
 focal = $(1-0,9)^2 \cdot 0,105 = 0,01 \cdot 0,105 = 0,00105$
 → atténué d'un facteur 100

exemple DIFFICILE $p_t = 0,5$: entropie croisée = $-\log(0,5) = 0,693$
 focal = $(1-0,5)^2 \cdot 0,693 = 0,25 \cdot 0,693 = 0,173$
 → atténué d'un facteur 4

L'exemple facile est divisé par 100, le difficile seulement par 4. Le rapport des poids effectifs entre difficile et facile passe de 6,6 (en entropie croisée pure) à 165 : le gradient bascule massivement vers ce qui résiste. C'est ce qui permet d'entraîner un détecteur à une seule étape sans échantillonnage explicite des négatifs.



γ et α s'ajustent ensemble, pas isolément : augmenter γ abaisse l'amplitude globale du coût, qu'il faut souvent compenser par α . Les valeurs de référence $\alpha = 0,25$ et $\gamma = 2$ ne sont pas universelles. Par ailleurs, calculer la focal loss à partir des probabilités plutôt que des logits bruts réintroduit l'instabilité numérique déjà rencontrée en §15.1 : utiliser toujours la forme stable basée sur l'entropie croisée avec logits. Enfin, un γ trop élevé peut **mémoriser le bruit d'étiquettes** — un exemple mal étiqueté ressemble à un exemple difficile et capte tout le gradient.



```
# a : vecteur de logits bruts (list ou np.ndarray, shape [C])
# b : indice de la classe correcte (int)
# gamma et alpha fixés ici pour la démonstration
gamma = 2.0
alpha = 0.25

if not isinstance(a, np.ndarray) or not isinstance(b, (int, float)):
    out_a = {'erreur': 'logits (a) et classe cible (b) requis'}
else:
    z = np.array(a, dtype=float)
    k = int(b)
    m = float(np.max(z))
    log_sum_exp = m + float(np.log(np.sum(np.exp(z - m))))
    softmax = np.exp(z - log_sum_exp)

    pt = float(softmax[k]) # proba de la vraie classe
    ce = float(-np.log(pt + 1e-12)) # entropie croisée stable
    focal = alpha * ((1.0 - pt) ** gamma) * ce
    modulateur = float((1.0 - pt) ** gamma)

    out_a = {
        'proba_vraie_classe': round(pt, 4),
        'entropie_croisee': round(ce, 4),
        'focal_loss': round(focal, 4),
        'facteur_modulation': round(modulateur, 4),
        'attenuation': round(ce / (focal + 1e-12), 2),
    }
```

15.4. Smooth L1 (Huber) – le compromis entre stabilité et robustesse



$$L(x) = \begin{cases} 0,5 \cdot x^2 / \beta & \text{si } |x| < \beta \\ |x| - 0,5 \cdot \beta & \text{sinon} \end{cases}$$

x est l'erreur de régression (par exemple le décalage entre la coordonnée prédite d'une boîte et la coordonnée cible). β est le seuil de transition, souvent fixé à 1. C'est, au facteur d'échelle près, exactement le **M-estimateur de Huber** du chapitre 16 : une parabole près de zéro, une ligne droite au-delà.

L'idée Pour comprendre pourquoi cette forme par morceaux, il faut lire le **gradient**, qui est la vraie pente que le réseau descend :

$$L'(x) = \begin{cases} x/\beta & \text{si } |x| < \beta \\ \text{sign}(x) & \text{sinon} \end{cases} \quad \begin{array}{l} \text{(proportionnel à l'erreur, comme L2)} \\ \text{(constant et borné, comme L1)} \end{array}$$

La forme quadratique près de zéro donne un gradient **doux et continu** qui s'annule à l'optimum — une erreur de 0,01 produit un gradient de 0,01, pas un gradient de ± 1 qui ferait vibrer le paramètre à chaque mise à jour. La forme linéaire au loin **borne le gradient** à ± 1 : une erreur énorme (une boîte complètement aberrante, une correspondance fautive issue d'une annotation imprécise) ne peut plus écraser le batch. Smooth L1 prend le meilleur des deux : la précision de la L2 près de la cible, la robustesse de la L1 contre les valeurs aberrantes. ■

On retrouve ici le fil conducteur sous un nouvel angle : la métrique qu'on vise est l'erreur de localisation absolue, mais la L2 pure (son substitut dérivable le plus naturel) trahit cette métrique dès qu'une annotation est imprécise. Smooth L1 est un substitut mieux sculpté, qui préserve la précision locale tout en neutralisant l'explosion de gradient.

Le lien avec le chapitre 8 est direct : l'erreur de reprojection du bundle adjustment — la mesure d'écart entre un point 3D projeté et sa correspondance observée — est une somme de carrés L2, dominée par une seule correspondance aberrante. La remplacer par un noyau de Huber est précisément la correction à apporter pour rendre le recalage robuste aux faux appariements.

Exemple numérique ($\beta = 1$) Quatre résidus de coordonnées issus d'un détecteur embarqué en robotique, dont un aberrant dû à une annotation imprécise, comparés en L2 et en smooth L1 :

erreur x	L2 (x^2)	gradient (2x)	smooth L1	gradient
0,5	0,25	1,0	0,125	0,5
0,8	0,64	1,6	0,320	0,8
0,3	0,09	0,6	0,045	0,3
6,0 (★)	36,0	12,0	5,500	1,0

En L2, l'erreur aberrante (★) pèse 36 sur 37 du coût total et fournit un gradient de 12 — elle dicte à elle seule la mise à jour et tire tous les paramètres dans sa direction. En smooth L1, sa contribution tombe à 5,5 sur 6,0 et son gradient est plafonné à 1,0 — du même ordre que les trois autres résidus normaux. L'aberration ne pilote plus l'apprentissage.



La valeur de β change radicalement le comportement : un β trop petit transforme tout en L1 (gradient constant, convergence lente près de l'optimum) ; trop grand, tout redevient L2 (sensible aux aberrations). Attention aux conventions des bibliothèques : certaines implémentations utilisent **beta** comme paramètre, d'autres **delta**, et **multiplient différemment** la branche linéaire — les deux coïncident à $\beta = 1$ mais divergent d'un facteur d'échelle sinon. Toujours vérifier laquelle on utilise avant de comparer des magnitudes de coût. En détection, régresser en coordonnées **normalisées** par l'ancre est essentiel, faute de quoi le choix de β dépend de la résolution de l'image.



```
# a : valeur prédite (float ou np.ndarray)
# b : valeur cible (float ou np.ndarray)
beta = 1.0 # seuil de transition

if not isinstance(a, (int, float, np.ndarray)) or not isinstance(b, (int,
float, np.ndarray)):
    out_a = {'erreur': 'valeur prédite (a) et valeur cible (b) requises'}
else:
    pred = np.atleast_1d(np.array(a, dtype=float))
    tgt = np.atleast_1d(np.array(b, dtype=float))
    x = pred - tgt # résidus

    # gradient smooth L1 (pour illustrer la pondération)
    quadratic = np.abs(x) < beta
    loss = np.where(quadratic, 0.5 * x**2 / beta, np.abs(x) - 0.5 * beta)
    grad = np.where(quadratic, x / beta, np.sign(x))

    out_a = {
        'résidus': [round(float(v), 4) for v in x],
        'cout_smooth_l1': [round(float(v), 4) for v in loss],
        'gradients': [round(float(v), 4) for v in grad],
        'cout_total': round(float(np.mean(loss)), 4),
        'cout_L2_total': round(float(np.mean(x**2)), 4),
    }
```


15.5. IoU loss et GIoU – optimiser la métrique, et combler ses zones plates



$$\begin{aligned}L_{\text{IoU}} &= 1 - \text{IoU} \\L_{\text{GIoU}} &= 1 - \text{IoU} + |C \setminus (A \cup B)| / |C|\end{aligned}$$

A désigne la boîte prédite, B la boîte cible. $\text{IoU} = |A \cap B| / |A \cup B|$ (rappel chapitre 4, §4.1). C est la plus petite boîte englobant à la fois A et B.

L'idée La situation est classique et illustre parfaitement le fil conducteur. La métrique qu'on vise est l'IoU — directement. Pourquoi ne pas simplement l'optimiser ? Le problème surgit aussitôt : quand A et B **ne se chevauchent pas**, l'IoU vaut 0 partout, quelle que soit la distance entre les boîtes.

A et B disjointes $\text{IoU} = 0$ $L_{\text{IoU}} = 1$ $\text{gradient} \approx 0$

C'est une **zone plate** : le substitut colle à la métrique si bien qu'il en hérite aussi le défaut — l'incapacité à distinguer « boîtes presque adjacentes » de « boîtes aux antipodes de l'image ». La descente de gradient ne sait pas dans quelle direction bouger A pour se rapprocher de B.

GIoU corrige cela en greffant un terme supplémentaire : la fraction de la boîte englobante C non couverte par l'union. Ce terme reste **actif même sans chevauchement** et décroît quand les boîtes se rapprochent (C rétrécit). Il sculpte un gradient là où la métrique seule était aveugle — exactement la logique du substitut dérivable.

Dérivation du terme correctif Quand A et B sont disjointes, $\text{IoU} = 0$. La quantité $|C \setminus (A \cup B)|$ est l'aire de C moins l'aire de l'union : c'est la région que C englobe mais que ni A ni B n'occupent. Plus les boîtes sont éloignées, plus cette région est vaste, plus le terme correctif est grand, plus L_{GIoU} est élevé — et son gradient tire A vers B. ■



Deux boîtes **disjointes**, format (x_1, y_1, x_2, y_2) , issues d'un détecteur en imagerie aérienne (objets petits souvent ratés à l'initialisation) :

$A = (0, 0, 2, 2)$ aire = 4

$B = (3, 3, 5, 5)$ aire = 4

intersection = 0 union = 8 IoU = 0 $\rightarrow$ L_IoU = 1 (zone plate, gradient nul)

$C = \text{englobante} = (0, 0, 5, 5)$ aire = 25

$|C \setminus (A \cup B)| = 25 - 8 = 17$

GIoU = 0 - 17/25 = -0,68

L_GIoU = 1 - (-0,68) = 1,68

Là où l'IoU loss reste bloqué à 1 sans indiquer de direction, le GIoU vaut 1,68 et **diminue dès que B se déplace vers A** : en approchant B en (2,5 ; 2,5 ; 4,5 ; 4,5), C rétrécit, l'aire non couverte baisse, L_GIoU diminue $\rightarrow$ le gradient tire activement les boîtes l'une vers l'autre.

Angle mort GIoU converge lentement quand une boîte en contient une autre — le terme correctif s'annule alors que l'alignement n'est pas encore optimal. D'où les raffinements ultérieurs (DIoU, CIoU) qui ajoutent la distance des centres et le rapport d'aspect. Aucun ne « capture tout » : le chapitre 4 résonne encore — aucune métrique unique ne suffit, et son substitut dérivable hérite toujours d'une partie de ses angles morts.



Les conventions de coordonnées sont une source d'erreur permanente : (x, y, w, h) (coin + dimensions) ou (x_1, y_1, x_2, y_2) (deux coins) selon les bibliothèques. Toujours **clamper** les largeurs et hauteurs à ≥ 0 avant de calculer une aire — des boîtes dégénérées produisent des intersections fantômes. L'aire de l'union se calcule par inclusion-exclusion $|A| + |B| - |A \cap B|$, jamais par somme naïve (l'intersection serait comptée deux fois). Ajouter ϵ aux dénominateurs pour les boîtes de taille nulle en début d'entraînement.



```
# a : boîte prédite [x1, y1, x2, y2] (list de 4 floats)
# b : boîte cible [x1, y1, x2, y2] (list de 4 floats)
if not (isinstance(a, (list, np.ndarray)) and isinstance(b, (list,
np.ndarray))):
    out_a = {'erreur': 'boîte prédite (a) et boîte cible (b) requises,
format [x1,y1,x2,y2]'}
else:
    ax1, ay1, ax2, ay2 = [float(v) for v in a[:4]]
    bx1, by1, bx2, by2 = [float(v) for v in b[:4]]

    # intersection
    ix1, iy1 = max(ax1, bx1), max(ay1, by1)
    ix2, iy2 = min(ax2, bx2), min(ay2, by2)
    inter = max(0.0, ix2 - ix1) * max(0.0, iy2 - iy1)

    # aires individuelles
    aire_a = max(0.0, ax2 - ax1) * max(0.0, ay2 - ay1)
    aire_b = max(0.0, bx2 - bx1) * max(0.0, by2 - by1)
    union = aire_a + aire_b - inter + 1e-6
    iou = inter / union

    # boîte englobante C
    cx1, cy1 = min(ax1, bx1), min(ay1, by1)
    cx2, cy2 = max(ax2, bx2), max(ay2, by2)
    aire_c = max(0.0, cx2 - cx1) * max(0.0, cy2 - cy1) + 1e-6

    giou = iou - (aire_c - union) / aire_c
    l_iou = round(1.0 - iou, 4)
    l_giou = round(1.0 - giou, 4)

    out_a = {
        'IoU': round(iou, 4),
        'GIoU': round(giou, 4),
        'L_IoU': l_iou,
        'L_GIoU': l_giou,
        'aire_englobante': round(aire_c, 2),
        'terme_correctif': round((aire_c - union) / aire_c, 4),
    }
```

15.6. Loss contrastive (InfoNCE) – structurer un espace de représentation



$$L = -\log(\exp(\text{sim}(z_i, z_{j^+})/\tau) / \sum_k \exp(\text{sim}(z_i, z_k)/\tau))$$

z_i désigne la **représentation ancre** — un vecteur numérique extrait par le réseau d'une image. z_{j^+} est la représentation **positive** : la même image vue sous un autre angle, un autre éclairage, un recadrage différent. z_k est l'ensemble de tous les candidats (le positif plus les **négatifs** — des images d'autres objets). **sim** est une mesure de similarité, presque toujours le **cosinus** (rappel chapitre 3, §3.6). τ est la **température**, un paramètre qui règle la « dureté » de la comparaison.

L'idée Ce coût s'attaque à un problème fondamentalement différent : il n'y a pas de vérité binaire (bonne classe / mauvaise classe), pas de masque de vérité, pas de boîte cible. On a seulement la relation « ces deux images montrent le même contenu » ou « ces deux images montrent des contenus différents ». La métrique visée — la qualité de l'espace de représentation — n'a pas de formule directe. Comment sculpter un gradient à partir de rien ?

L'idée est de **fabriquer un problème de classification artificiel** : « parmi tous les candidats, lequel est le vrai positif ? » Les logits de ce problème sont les similarités cosinus divisées par τ . La cible est l'indice du positif. Tout le gradient propre de l'entropie croisée (§15.1) s'applique alors : il rapproche l'ancre de son positif (similarité cosinus $\uparrow$) et éloigne l'ancre des négatifs (similarité cosinus $\downarrow$), avec une force proportionnelle à l'erreur de chaque candidat.

La température τ règle la dureté de la comparaison. L'image mentale est celle d'un microscope avec un réglage de contraste : une τ basse est comme un contraste élevé — les différences légères entre candidats sont amplifiées, le réseau s'entraîne sur les quasi-confusions, mais le moindre bruit est fatal. Une τ élevée est comme un faible contraste — tout est traité également, le signal est doux et stable mais moins discriminant. C'est le pendant, dans l'espace latent, du choix de distance du chapitre 3 : on ne déclare plus seulement *ce qui compte*, mais à *quel point les quasi-confusions comptent*.

Dérivation Malgré son allure différente, InfoNCE est une **entropie croisée** appliquée à un problème artificiel. Notons les logits $s_{ik} = \text{sim}(z_i, z_k)/\tau$. Le coût devient :

$$\begin{aligned} L &= -\log(\exp(s_{ij^+}) / \sum_k \exp(s_{ik})) \\ &= -s_{ij^+} + \log \sum_k \exp(s_{ik}) \end{aligned}$$

C'est exactement la forme log-sum-exp de l'entropie croisée du §15.1. Le gradient par rapport à s_{ij^+} (la similarité avec le positif) est $-1 + \text{softmax}(s_{ij^+})$ — il pousse la similarité vers le

haut. Le gradient sur un négatif s_{ik} vaut $+\text{softmax}(s_{ik})$ — il pousse la similarité vers le bas, avec d'autant plus de force que le négatif est, à tort, jugé proche. ■



Une ancre microscopique (une cellule dans une vue), son positif (même cellule, contraste différent, $\text{sim} = 0,9$) et deux négatifs (autres cellules, $\text{sim} = 0,3$ et $0,2$). Effet de la température :

$\tau = 0,1$: logits = [9 ; 3 ; 2]
 exp = [8103 ; 20,1 ; 7,4] somme = 8130
 $p(\text{positif}) = 0,997 \rightarrow L = -\log(0,997) = 0,003$

$\tau = 1,0$: logits = [0,9 ; 0,3 ; 0,2]
 exp = [2,46 ; 1,35 ; 1,22] somme = 5,03
 $p(\text{positif}) = 0,489 \rightarrow L = -\log(0,489) = 0,715$

Pour des similarités identiques, le coût varie de 0,003 à 0,715 selon τ . À τ basse, le réseau se juge déjà excellent (gradient ténu) ; à τ haute, il « voit » les négatifs comme des concurrents sérieux et reçoit un gradient fort. La température n'est pas un détail de réglage — c'est elle qui décide à quel point les quasi-confusions doivent être combattues.



Trois pièges récurrents. **(1)** La similarité cosinus suppose des vecteurs **normalisés** ($\|z\| = 1$) — oublier la normalisation fait dépendre le coût de la norme des vecteurs, pas seulement de leur direction. **(2)** La température basse amplifie les **faux négatifs** : dans un grand lot d'images astronomiques, deux clichés de nébuleuses différentes du même type peuvent se ressembler — les traiter comme négatifs avec τ très petite leur inflige un gradient violent à tort. **(3)** Le nombre de négatifs gouverne la qualité du signal — un lot trop petit fournit trop peu de candidats pour structurer l'espace de représentation.



```
# a : vecteur représentation ancre      (list ou np.ndarray, shape [D])
# b : vecteur représentation positif   (list ou np.ndarray, shape [D])
# (les négatifs seraient des entrées c, d, e... pour un vrai lot)
tau = 0.1 # température

if not (isinstance(a, (list, np.ndarray)) and isinstance(b, (list,
np.ndarray))):
    out_a = {'erreur': 'ancree (a) et positif (b) requis'}
else:
    zi = np.array(a, dtype=float)
    zp = np.array(b, dtype=float)

    # normalisation L2 (cosinus)
    zi_n = zi / (np.linalg.norm(zi) + 1e-12)
    zp_n = zp / (np.linalg.norm(zp) + 1e-12)

    sim_pos = float(np.dot(zi_n, zp_n)) # similarité cosinus ancre/
positif

    # avec uniquement le positif, InfoNCE = 0 (pas de négatifs)
    # on illustre la formule et l'effet de tau
    logit_pos = sim_pos / tau
    # Avec N négatifs supplémentaires, on ajouterait leurs logits ici
    # Pour la démo sans négatifs : p(positif) = 1, L = 0
    # On montre l'effet de tau sur la similarité
    out_a = {
        'sim_cosinus': round(sim_pos, 4),
        'logit_pos_tau': round(logit_pos, 4),
        'tau': tau,
        'norme_ancree': round(float(np.linalg.norm(zi)), 4),
        'norme_positif': round(float(np.linalg.norm(zp)), 4),
        'note': 'connecter d, e... pour ajouter des negatifs (entrées c,
d...)',
    }
```

15.7. Tableau récapitulatif – quel coût pour quel objectif

Coût	Métrique substituée	Forme du gradient	Problème de saturation/explosion	Usage typique
Entropie croisée + softmax	justesse de classification (escalier)	$\hat{y} - y$ (l'erreur elle-même, jamais nul tant que $\hat{y} \neq y$)	poids égal → noyé par les classes majoritaires	classification, OCR, détection multiclasse
Dice loss	coefficient de Dice binaire (non dérivable)	normalisé par la taille des régions	instable si objet absent (0/0 sans ϵ) ; ignore la qualité des frontières	segmentation médicale, objets minuscules
Focal loss	justesse sous déséquilibre extrême	éteint les exemples faciles $(1-p_t)^\gamma$	sensible au bruit d'étiquettes à γ élevé	détection dense, défauts rares, télédétection
Smooth L1 (Huber)	erreur de localisation sans explosion	doux près de 0 (L2), borné au loin (L1 : $\pm\beta$)	gradient L2 explose sur les aberrations → remplacé par borne L1	régression de boîtes, reprojection robuste
IoU loss	IoU des boîtes (non dérivable si disjointes)	nul hors chevauchement → zone plate comblée par GIoU	zone plate complète si $A \cap B = \emptyset$	détection, petits objets, imagerie aérienne
InfoNCE (contrastive)	qualité de l'espace de représentation	rapproche positifs, éloigne négatifs (force $\propto \tau$)	faux négatifs à τ basse ; signal faible si trop peu de négatifs	pré-entraînement auto-supervisé, CLIP

15.8. le coût n'est pas la cible, c'est la pente vers elle

Le fil conducteur du chapitre, et un principe qui structure tout l'apprentissage profond :

on ne minimise jamais ce qu'on veut ;
on minimise un substitut dont le gradient y conduit.

Mais ce substitut n'est pas choisi au hasard. Chaque coût de ce chapitre répond à une faille précise de la métrique qu'il remplace :

classer juste partout	→ la justesse est un escalier (non dérivable) ; l'entropie croisée fournit $\hat{y} - y$, continu et actif
segmenter (Dice/IoU)	→ la métrique est binaire et plate ; sa relaxation continue redevient dérivable
apprendre malgré le faciles ; déséquilibre	→ l'entropie croisée se noie dans la masse des exemples focal et Dice répondèrent pour rendre la pente utile
régresser une boîte chevauchement	→ la L2 explose sur une aberration ; Huber borne le gradient, GIoU comble la zone sans
structurer un espace de représentation	→ « se ressembler » n'a pas de gradient en soi ; InfoNCE en fabrique un par comparaison artificielle

Chaque coût est un **arbitrage entre fidélité à la métrique et exploitabilité du gradient** : l'IoU est la vraie cible, mais c'est GIoU qu'on optimise parce que lui a une pente partout ; le Dice du chapitre 4 est la mesure, mais c'est sa version soft qu'on dérive. Toute la conception d'un coût tient dans une question : *où le gradient s'annule-t-il, explose-t-il, ou pointe-t-il à côté ?* — et comment remodeler le paysage pour que la descente mène quand même à la métrique.

Ce fil résonne avec tout le livre. Choisir une distance (chapitre 3), c'est déclarer ce qui compte. Choisir une base (chapitre 10), c'est choisir où le problème devient simple. Choisir un filtre (chapitre 5), c'est imposer un a priori sur le signal. Choisir un coût, c'est déclarer ce qui compte *et fabriquer la pente qui y mène*. La métrique dit si c'est bon ; le coût dit dans quelle direction s'améliorer.

Et comme partout dans ce livre — distance, filtre, descripteur, a priori, base, coût — **le bon cadre rend le problème presque résolu** : un coût bien sculpté transforme un objectif insaisissable en une simple descente.

CHAPITRE

Statistiques robustes

16

Table des matières

16.1. Médiane et point de rupture — la robustesse comme influence bornée	267
16.2. MAD — l'échelle robuste qui définit l'aberration	269
16.3. M-estimateurs — de Huber à Tukey, doser l'influence	271
16.4. IRLS — comment on résout un M-estimateur en pratique	274
16.5. RANSAC — la robustesse par consensus (et son compte d'itérations)	277
16.6. Au-delà du RANSAC « vanilla » — situer chaque variante	280
16.7. Tableau récapitulatif — qui a le droit de déplacer l'estimation ?	283
16.8. être robuste, c'est décider à l'avance ce qu'on refuse de croire	283

La vision par ordinateur produit des données truffées d'aberrations : appariements faux entre deux images, pixels saturés ou morts, occlusions partielles, annotations imprécises, reflets spéculaires qui font briller un point là où il n'y a rien. Or les outils classiques — la moyenne, les moindres carrés — accordent à chaque observation une influence **non bornée** : une seule valeur aberrante peut emporter l'estimation arbitrairement loin. Un faux appariement dans un calcul d'homographie, un pixel brûlé dans une mesure de dispersion, un point 3D mal reconstruit dans un bundle adjustment — une seule donnée corrompue peut détruire un résultat qui était juste pour les quatre-vingt-dix-neuf autres.

Les statistiques robustes refondent l'estimation autour d'une question de **confiance** : combien de poids accorder à une observation selon qu'elle est plausible ou suspecte ? Le fil conducteur de ce chapitre tient en une phrase : **être robuste, c'est borner l'influence d'une donnée**. L'outil pour lire cette influence est la **fonction d'influence** ψ , qui quantifie combien un résidu de taille e pèse sur le résultat final. Les estimateurs s'ordonnent alors sur un spectre que ce chapitre parcourt du début à la fin : influence **non bornée** (moyenne), **bornée constante** (médiane), **écrêtée** (Huber), **redescendante vers zéro** (Tukey), puis **binaire** — gardée ou jetée (RANSAC). À chaque section, la même question : jusqu'où une seule donnée a-t-elle le droit de déplacer le résultat ?

Rappel des liens avec les autres chapitres. Les statistiques robustes ne sont pas un domaine à part : elles traversent tout le livre et en révèlent la face « méfiance ».

Chapitre 3 (distances)	: la moyenne minimise L2, la médiane minimise L1 le choix L1/L2 est déjà le choix non-robuste/robuste
Chapitre 5 (filtrage)	: le filtre médian est un estimateur robuste appliqué localement c'est pourquoi il efface le bruit poivre-et-sel là où le gaussien l'étale
Chapitre 8 (caméra)	: RANSAC estime l'homographie (4 points) et la matrice fondamentale (8 points) au milieu d'appariements faux
Chapitre 9 (flot optique)	: champs de mouvement robustes aux occlusions ; rejet d'outliers
Chapitre 15 (coûts)	: Huber = smooth L1 ; la fonction d'influence ψ EST la dérivée du coût ρ , c'est-à-dire le gradient – même objet, deux lectures

Le lien avec le chapitre 15 est exact, pas analogique : $\psi = \rho'$ est la dérivée du coût. Là où le chapitre 15 demandait *où le gradient sature ou explose*, ce chapitre demande *où l'influence d'une donnée est bornée*. C'est la même dérivée, lue dans l'autre sens. Et ce chapitre prolonge le chapitre 3 d'un cran : choisir L1 plutôt que L2 (« choisir une distance, c'est déclarer ce qui compte »), c'est déjà déclarer *à quel point on se méfie des écarts extrêmes*.

16.1. Médiane et point de rupture — la robustesse comme influence bornée



moyenne : $\hat{\mu} = \operatorname{argmin}_m \sum_i (x_i - m)^2$ (minimise L2)
 médiane : $\hat{m} = \operatorname{argmin}_m \sum_i |x_i - m|$ (minimise L1)

La médiane est la valeur centrale d'un échantillon trié ; la moyenne, son barycentre.

Dérivation — pourquoi la médiane minimise L1

L'idée visuelle est la suivante : la moyenne est sensible aux distances, la médiane l'est seulement aux côtés. Pour le voir, dérivons la somme L1 par rapport à m , en notant que $d|x_i - m|/dm = -\operatorname{sign}(x_i - m)$:

$$d/dm \sum_i |x_i - m| = -\sum_i \operatorname{sign}(x_i - m) = -(\#\{x_i > m\} - \#\{x_i < m\})$$

La dérivée s'annule quand il y a **autant de points au-dessus qu'en dessous** de m — c'est exactement la médiane. La moyenne, elle, annule la dérivée de la somme L2 : $d/dm \sum (x_i - m)^2 = -2\sum (x_i - m) = 0 \iff m = (1/n)\sum x_i$. La différence est tout entière dans le poids accordé à un écart : L2 le met au carré (les grands écarts dominent), L1 le compte linéairement — chaque point pèse par son seul **signe** au voisinage de l'optimum. Un outlier à 1000 m de la médiane tire autant qu'un point à 1 m de la médiane. ■

Le point de rupture et la fonction d'influence

Deux notions chiffrent la robustesse. Le **point de rupture** est la fraction de données qu'on peut corrompre arbitrairement avant que l'estimation puisse partir à $\pm\infty$. Pour la moyenne, **un seul** point suffit : point de rupture $1/n \rightarrow 0\%$. Pour la médiane, il faut corrompre **plus de la moitié** de l'échantillon pour la pousser à l'infini : point de rupture 50 %, le maximum théorique possible.

La **fonction d'influence** $\psi(x)$ mesure l'effet, sur l'estimation, d'ajouter une observation en x . L'image mentale utile est celle d'un levier : combien la balance de la décision bascule-t-elle quand on pose un nouveau poids en position x ? Pour la moyenne, $\psi(x) = x - \mu$: **non bornée**, linéaire — une donnée lointaine tire d'autant plus fort qu'elle est éloignée du centre. Pour la médiane, $\psi(x) \propto \operatorname{sign}(x - m)$: **bornée** — une observation n'influe que par le côté où elle tombe, jamais par son amplitude. *C'est la signature du chapitre : borner ψ , c'est être robuste.* L'angle mort de la médiane est réel : en ne lisant que le signe, elle jette une information utile quand les données sont propres (la moyenne est alors plus efficace, c'est-à-dire de plus faible variance), et son gradient **sign** est discontinu en zéro — c'est précisément le défaut que Huber corrige au §16.3.



Cinq mesures d'un même capteur de distance (en cm), dont la dernière déraile suite à un reflet spéculaire.

propre : `x = [2, 3, 5, 7, 8]`
 `moyenne = 5.0` `médiane = 5`

corrompu : `x = [2, 3, 5, 7, 800]` (1 outlier sur 5 = 20 % < seuil 50 %)
 `moyenne = 163.4` `médiane = 5`

Un seul point aberrant déplace la moyenne de 5 à 163 — elle est détruite. La médiane ne bouge pas d'un iota : tant que les outliers restent minoritaires (ici 20 % < 50 %), ils ne peuvent pas franchir le centre. La fonction d'influence ψ de la médiane est bornée : le capteur fou ne tire qu'avec un poids de 1 (son signe), jamais proportionnellement à ses 800 cm d'écart.



La médiane d'un nombre **pair** d'éléments est, par convention, la moyenne des deux centraux — elle redevient donc localement sensible à ces deux valeurs. Pour des données entières (intensités 8 bits), `np.median` renvoie un flottant (.5) : vérifier le type avant de réindexer une image. Et attention au coût : une médiane exacte trie en $O(n \log n)$, ce qui pèse pour un filtre médian sur grande image — les implémentations rapides (`cv2.medianBlur`) utilisent des histogrammes glissants (voir chapitre 5). Enfin, la médiane n'est **pas** la moyenne tronquée ni le mode : ne pas les confondre quand une bibliothèque parle de « valeur centrale ».



```
# a : tableau 1D de scalaires (list ou np.ndarray)
# Brancher sur le port a un port scalar ou list

if not isinstance(a, (list, np.ndarray)):
    out_a = {'erreur': 'entree invalide'}
else:
    x = np.array(a, dtype=float)
    mu = float(np.mean(x))
    med = float(np.median(x))
    # Fonction d'influence : non bornée pour la moyenne, bornée pour la
    médiane
    influence_moy = float(x[-1] - mu)          # croît sans limite avec
    l'outlier
    influence_med = float(np.sign(x[-1] - med)) # vaut +1, jamais x[-1]
    out_a = {
        'moyenne': mu,
        'mediane': med,
        'influence_derniere_valeur_sur_moy': influence_moy,
        'influence_derniere_valeur_sur_med': influence_med,
    }
```

16.2. MAD — l'échelle robuste qui définit l'aberration



$MAD = \text{médiane}(|x_i - \text{médiane}(x)|)$
 $\sigma^{\wedge} = 1.4826 \cdot MAD$

La MAD (*median absolute deviation*) est la médiane des écarts absolus à la médiane. Le facteur 1.4826 la calibre pour estimer l'écart-type σ d'une loi normale.

Dérivation — d'où vient le 1.4826

Avant de pouvoir borner l'influence à $\pm k$, encore faut-il savoir ce que « k » signifie en termes de bruit. L'image mentale utile est celle d'un mètre-étalon : on ne peut pas dire qu'une mesure est « loin » sans d'abord définir l'unité de distance. L'écart-type classique est le mètre-étalon habituel, mais il est lui-même détruit par un seul outlier (il contient un $\Sigma(\cdot)^2$, donc une influence non bornée). La MAD est l'échelle robuste naturelle, mais brute elle sous-estime σ ; on la recalibre. Pour $X \sim N(\mu, \sigma^2)$, la MAD de population est la valeur m telle que $P(|X - \mu| \leq m) = \frac{1}{2}$. Or $|X - \mu|/\sigma$ suit une demi-normale, donc :

$$P(|X - \mu| \leq m) = \frac{1}{2} \quad \Leftrightarrow \quad P(|Z| \leq m/\sigma) = \frac{1}{2} \quad \Leftrightarrow \quad m/\sigma = \Phi^{-1}(0.75) \approx 0.6745$$

Donc $\text{MAD} = 0.6745 \cdot \sigma$, d'où $\hat{\sigma} = \text{MAD} / 0.6745 = \mathbf{1.4826 \cdot \text{MAD}}$, avec $1.4826 = 1/\Phi^{-1}(3/4)$. ■

Le point de rupture de la MAD est 50 % (c'est une double médiane, comme celle de la médiane elle-même). La fonction d'influence de la MAD est, elle aussi, bornée : l'aberration ne peut pas gonfler l'échelle estimée au-delà d'un plafond. C'est cette propriété qui rend la MAD indispensable pour paramétrer Huber (§16.3) et le seuil d'inlier de RANSAC (§16.5) : sans mètre-étalon robuste, « borner l'influence à $\pm k$ » n'a pas de sens — k doit s'exprimer en unités de bruit, pas en pixels bruts.



Reprenons les deux échantillons du §16.1.

propre : $x = [2, 3, 5, 7, 8]$, médiane = 5
 écarts $|x-5| = [3, 2, 0, 2, 3]$ trié $[0, 2, 2, 3, 3] \rightarrow \text{MAD} = 2$

$\hat{\sigma} = 1.4826 \cdot 2 = 2.97$ (écart-type classique : 2.55)

corrompu : $x = [2, 3, 5, 7, 800]$, médiane = 5
 écarts $|x-5| = [3, 2, 0, 2, 795]$ trié $[0, 2, 2, 3, 795] \rightarrow$
 MAD = 2

$\hat{\sigma} = 2.97$ (écart-type classique : 356)

L'échelle robuste $\hat{\sigma}$ ne bouge pas ($2.97 \rightarrow 2.97$) là où l'écart-type classique explose de 2.55 à 356. On peut alors **détecter** l'outlier par un score z modifié $z_i = 0.6745 \cdot (x_i - \text{médiane})/\text{MAD}$, signalé si $|z| > 3.5$. La fonction d'influence de la MAD est bornée : l'aberration à 800 ne tire sur l'échelle que par son signe, pas par son amplitude.

pour $x = 800$: $z = 0.6745 \cdot (800 - 5) / 2 = 268 \gg 3.5 \rightarrow$ aberration flagrante



MAD = 0 dès que plus de la moitié des valeurs coïncident — cas fréquent en vision : capteur saturé (majorité de pixels à 255), zone uniforme, données quantifiées en paliers. La division $(x - \text{médiane})/\text{MAD}$ produit alors des `inf/NaN`. Parade : ajouter un ϵ , ou basculer sur un estimateur d'échelle plus fin (IQR, ou Q_n de Rousseeuw–Croux). Autre piège, le facteur de calibration : `scipy.stats.median_abs_deviation(x, scale='normal')` applique 1.4826 automatiquement, mais un calcul maison `np.median(np.abs(x - np.median(x)))` renvoie la

MAD **brute**, sans le facteur — comparer des seuils issus des deux conventions introduit un facteur 1.48 d'erreur silencieuse.



```
# a : tableau 1D de scalaires (list ou np.ndarray)

if not isinstance(a, (list, np.ndarray)):
    out_a = {'erreur': 'entree invalide'}
else:
    x = np.array(a, dtype=float)
    med = float(np.median(x))
    mad_brute = float(np.median(np.abs(x - med)))
    sigma_hat = 1.4826 * mad_brute # calibré pour
    bruit gaussien
    eps = 1e-9
    z_scores = 0.6745 * (x - med) / (mad_brute + eps) # score z modifié
    outliers = [bool(v) for v in (np.abs(z_scores) > 3.5)]
    out_a = {
        'mediane': med,
        'MAD': mad_brute,
        'sigma_hat': float(sigma_hat),
        'z_scores': [float(z) for z in z_scores],
        'outliers_detectes': outliers,
    }
```

16.3. M-estimateurs — de Huber à Tukey, doser l'influence



estimation : $\hat{\theta} = \operatorname{argmin}_{\theta} \sum_i p(e_i / \sigma)$ $e_i = \text{résidu}, \sigma = \text{MAD}$
(§16.2)

influence : $\psi = p'$ (dérivée du coût, cf. chapitre 15)

Huber	:	$p(e) = e^2/2$	si $ e \leq k$	$\psi(e) = e$	si $ e \leq k$
		$k e - k^2/2$	sinon	$k \cdot \text{sign}(e)$	sinon

Tukey	:	$p(e) = (k^2/6)[1 - (1 - (e/k)^2)^3]$	si $ e \leq k$	$\psi(e) = e(1 - (e/k)^2)^2$	si $ e \leq k$
		$k^2/6$	sinon	0	

sinon

Dérivation — sculpter la fonction d'influence

Les moindres carrés ordinaires (MCO) minimisent $\sum e_i^2$, soit $\rho(e) = e^2/2$ et donc $\psi(e) = e$: l'influence d'un résidu **croît sans limite**. Un M-estimateur (*maximum-likelihood-type*) remplace ce ρ par une fonction qui plafonne ou redescend. Annuler le gradient donne l'équation estimante :

$$\sum_i \psi(e_i) \cdot \partial e_i / \partial \theta = 0$$

Tout se joue dans la forme de ψ . L'image mentale utile est celle d'un bras de fer : jusqu'à quelle distance un adversaire a-t-il le droit de tirer ?

Pour Huber, ψ vaut e tant que $|e| \leq k$ (régime MCO, précis près de la cible), puis se **fige à $\pm k$** : un résidu énorme ne tire plus que d'une force constante k . L'influence est **bornée mais non nulle** — Huber atténue l'outlier, ne le rejette pas. On retrouve ici exactement le « smooth L1 » du chapitre 15, §15.4 : borner le gradient pour stabiliser l'entraînement, borner l'influence pour stabiliser l'estimation — même dérivée, deux lectures.

Pour Tukey (biweight / bisquare), ψ monte, culmine, puis **redescend à zéro** au-delà de k : un outlier franc reçoit une influence *strictement nulle*, il disparaît de l'équation. C'est la différence cruciale entre ψ **monotone** (Huber) et ψ **redescendante** (Tukey). ■

Le spectre complet de l'influence

On peut maintenant résumer le fil conducteur en un seul tableau de ψ :

MCO	→ $\psi(e) = e$	non bornée, un outlier emporte tout
médiane/L1	→ $\psi = \text{sign}(e)$	bornée constante, compte les côtés, pas les distances
Huber	→ ψ écrêtée à $\pm k$	bornée et monotone, atténue sans rejeter
Tukey	→ ψ redescend à 0	influence annulée au-delà de c , rejette franchement

Réglage des seuils : $k = 1.345 \cdot \hat{\sigma}$ pour Huber et $c = 4.685 \cdot \hat{\sigma}$ pour Tukey donnent tous deux **95 % d'efficacité** sous bruit gaussien — on ne perd que 5 % de précision par rapport aux MCO quand les données sont propres, tout en gagnant la robustesse. Le ρ de Tukey étant non convexe, la minimisation a des minima locaux et exige une bonne initialisation (voir §16.4).



Quatre résidus d'un ajustement de ligne de profil en imagerie industrielle, dont un aberrant issu d'un éclat de métal ; seuil $k = 2$.

résidu e	$\psi_{\text{MCO}} (=e)$	$\psi_{\text{Huber}} (\text{écrêté } \pm 2)$	$\psi_{\text{Tukey}} (k=2)$
0.5	0.5	0.5	0.439
-0.8	-0.8	-0.8	-0.564
0.3	0.3	0.3	0.287
6.0 (★)	6.0	2.0	0.000

L'outlier (★) pèse 6.0 en MCO — il dicte l'ajustement. Huber rabote son influence à 2.0, du même ordre que les résidus sains. Tukey l'annule : $6.0 > k = 2$, donc $\psi = 0$, l'éclat de métal disparaît de l'équation. Détail du calcul Tukey pour $e = 0.5$: $0.5 \cdot (1 - (0.5/2)^2)^2 = 0.5 \cdot (1 - 0.0625)^2 = 0.5 \cdot 0.879 = 0.439$. C'est exactement le mécanisme d'un ajustement de matrice fondamentale ou d'un bundle adjustment robustes (chapitre 8) : un faux appariement à $e = 6.0$ ne fausse plus la géométrie estimée.



Le seuil k doit s'exprimer **en unités de $\hat{\sigma}$** (recalculé via la MAD à chaque problème), jamais en pixels fixes : $k = 2$ px est lâche sur une image bruitée et strict sur une image propre. Le ρ de Tukey étant non convexe, **ne jamais l'initialiser par les MCO** (qui sont déjà tirés par l'outlier) : partir de la médiane ou d'un fit L1. `statsmodels` propose ces noyaux (`HuberT`, `TukeyBiweight`) avec leurs propres conventions de `tuning_constant` — vérifier qu'on compare des k homogènes avant de comparer des fits.



```
# a : image (np.ndarray BGR) pour une régression sur le profil central
# On extrait un profil de ligne et on l'ajuste par MCO vs Huber

if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'entree invalide'}
else:
    import statsmodels.api as sm

    # Profil horizontal central de l'image en niveaux de gris
    gray = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY) if a.ndim == 3 else a
    row = gray[gray.shape[0] // 2, :].astype(float)
    n = len(row)
    x_coords = np.arange(n, dtype=float)

    X = sm.add_constant(x_coords)
    # MCO : influence non bornée
    mco = sm.OLS(row, X).fit()
    # Huber : influence écrêtée (ψ bornée monotone)
    huber = sm.RLM(row, X, M=sm.robust.norms.HuberT()).fit()

    out_a = {
        'pente_MCO': float(mco.params[1]),
        'pente_Huber': float(huber.params[1]),
        'ordonnee_MCO': float(mco.params[0]),
        'ordonnee_Huber': float(huber.params[0]),
    }
```

16.4. IRLS – comment on résout un M-estimateur en pratique



poids : $w_i = \psi(e_i) / e_i$
 itérer : $\theta \leftarrow \operatorname{argmin}_{\theta} \sum_i w_i \cdot e_i(\theta)^2$ (moindres carrés pondérés)

IRLS = *iteratively reweighted least squares* — moindres carrés repondérés itérativement.

Dérivation — d'une influence à un poids

Un M-estimateur n'a pas de solution explicite (le ψ est non linéaire). L'astuce : factoriser $\psi(e) = w(e) \cdot e$ avec $w(e) = \psi(e)/e$, ce qui transforme l'équation estimante en :

$$\sum_i w(e_i) \cdot e_i \cdot \partial e_i / \partial \theta = 0$$

Ce sont exactement les équations normales de **moindres carrés pondérés** par les w_i . Comme les poids dépendent des résidus, donc de θ , on itère : (1) fit pondéré $\rightarrow$ (2) nouveaux résidus $\rightarrow$ (3) nouveaux poids $\rightarrow \dots$ jusqu'à convergence. L'image mentale utile : on joue une négociation. Au premier tour, tout le monde parle à égalité. Après chaque round, celui dont la voix semble trop extrême voit son micro baissé. On répète jusqu'à ce que les voix restantes s'accordent. Les poids traduisent directement la forme de ψ :

$$\text{Huber : } w(e) = \begin{cases} 1 & \text{si } |e| \leq k \\ k/|e| & \text{sinon} \end{cases} \quad \text{Tukey : } w(e) = \begin{cases} (1-(e/k)^2)^2 & \text{si } |e| \leq k \\ 0 & \text{sinon} \end{cases}$$

Huber **dégrade** le poids des grands résidus comme $1/|e|$ (jamais à zéro) ; Tukey le met carrément à **zéro** au-delà de k . Un fit robuste n'est donc rien d'autre qu'un MCO où les outliers sont progressivement éteints par leur propre influence bornée. ■



Estimer une position en reconstruction 3D — un seul paramètre, la profondeur — à partir de $y = [10.0, 10.2, 9.8, 25.0]$, dont **25.0** est aberrant (mauvaise triangulation sur un fond spéculaire).

MCO (moyenne)	: $\theta = 55/4 = 13.75$	← tiré vers le haut par 25.0
init médiane	: $\theta_0 = 10.1$	
résidus	: $e = [-0.1, 0.1, -0.3, 14.9]$	
échelle (MAD)	: $\hat{\sigma} = 1.4826 \cdot 0.2 = 0.297$	
poids Tukey ($c=4.685$)	: $w \approx [0.99, 0.99, 0.91, 0.00]$	← 25.0 reçoit le poids 0
fit pondéré	: $\theta_1 = \sum w_i y_i / \sum w_i \approx 10.01$	

Les MCO donnent 13.75 (faux) ; une seule passe d'IRLS, en annulant le poids de l'outlier — c'est-à-dire en bornant son influence à zéro — ramène l'estimation à **10.01**, la vraie profondeur. Les itérations suivantes ne bougent plus : l'aberration est définitivement éteinte.



La **division par e** dans $w = \psi(e)/e$ explose pour les résidus quasi nuls : protéger par $w = \psi(e)/\max(|e|, \epsilon)$, ou utiliser directement les formes fermées de w ci-dessus (qui valent 1 près de zéro). Ré-estimer $\hat{\sigma}$ **avant** les poids à chaque tour, sinon une échelle mal initialisée fausse tous les poids. Et fixer un nombre maximal d'itérations + un seuil de convergence : avec Tukey, des allers-retours de pondération (un point au bord de c qui entre/sort) peuvent empêcher la stabilisation.



```
# a : image (np.ndarray BGR) avec des points lumineux aberrants
# On calcule la luminosité moyenne de chaque ligne de façon robuste
(IRLS)

if not isinstance(a, np.ndarray):
    out_a = {'erreur': 'entree invalide'}
else:
    gray = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY) if a.ndim == 3 else a
    y = gray.mean(axis=1).astype(float) # luminosité moyenne par ligne

    def irls_location(y_arr, c=4.685, iters=10):
        theta = float(np.median(y_arr)) # init robuste
        (jamais la moyenne)
        for _ in range(iters):
            e = y_arr - theta
            sigma = 1.4826 * float(np.median(np.abs(e - np.median(e)))) +
1e-9
            u = e / sigma
            w = np.where(np.abs(u) <= c, (1 - (u / c) ** 2) ** 2, 0.0) #
poids Tukey
            denom = float(np.sum(w))
            if denom < 1e-9:
                break
            theta = float(np.sum(w * y_arr) / denom)
        return theta

    luminosite_robuste = irls_location(y)
    luminosite_mco = float(y.mean())

    out_a = {
        'luminosite_IRLS_robuste': luminosite_robuste,
        'luminosite_MCO': luminosite_mco,
        'biais_outliers': round(luminosite_mco - luminosite_robuste, 2),
    }
```

16.5. RANSAC – la robustesse par consensus (et son compte d'itérations)



$$N = \log(1 - p) / \log(1 - w^n)$$

p = probabilité souhaitée d'obtenir au moins un bon tirage, w = proportion d'inliers, n = taille de l'échantillon **minimal** pour ajuster le modèle (2 pour une droite, 4 pour une homographie, 8 pour la matrice fondamentale — chapitre 8).

Dérivation — combien de tirages pour toucher un échantillon pur

RANSAC pousse l'idée d'influence bornée à l'extrême : au lieu d'atténuer les outliers, il les **exclut** par hypothèse-et-test. L'image mentale utile est celle d'un jury : au lieu d'écouter tous les témoins et de pondérer leurs déclarations, on tire au sort un petit groupe, on lui demande une version cohérente des faits, et on compte combien d'autres témoins confirment cette version. On répète jusqu'à trouver le jury le plus consensuel. Sa boucle : tirer au hasard un échantillon minimal de n points → ajuster le modèle → compter les inliers (les points à moins de t du modèle) → garder le modèle au plus grand consensus → ré-ajuster sur tous ses inliers. Reste à choisir N , le nombre de tirages. Raisonçons sur les probabilités :

$P(\text{les } n \text{ points d'un tirage sont tous inliers})$	$= w^n$
$P(\text{un tirage est contaminé, } \geq 1 \text{ outlier})$	$= 1 - w^n$
$P(\text{les } N \text{ tirages sont TOUS contaminés})$	$= (1 - w^n)^N$
$P(\text{au moins un tirage pur})$	$= 1 - (1 - w^n)^N \geq p$

On veut la dernière ligne $\geq p$. En isolant N (les deux logarithmes sont négatifs, leur quotient est positif) :

$$(1 - w^n)^N \leq 1 - p \quad \Leftrightarrow \quad N \cdot \log(1 - w^n) \leq \log(1 - p) \quad \Leftrightarrow \quad N \geq \log(1 - p) / \log(1 - w^n) \quad \blacksquare$$

Point capital : N dépend **exponentiellement de n** et de w , mais **pas de la taille du jeu de données**. La fonction d'influence de RANSAC est **binaire** : un point est inlier (poids 1, il fonde le modèle) ou outlier (poids 0, totalement ignoré). C'est le bout du spectre — non plus atténuer ni redescendre, mais **partitionner**. Sa force : un point de rupture qui peut **dépasser 50 %** (RANSAC tolère une majorité d'outliers, à condition d'allonger N), là où aucun M-estimateur ne va.



Combien de tirages pour $p = 0.99$, dans trois scénarios courants en vision (chapitre 8) ?

homographie ($n=4$), $w=0.5$: $N = \log(0.01)/\log(1-0.5^4) = -4.605 / -0.0645 = 72$

homographie ($n=4$), $w=0.3$: $N = \log(0.01)/\log(1-0.3^4) = -4.605 / -0.0081 = 567$

mat. fondamentale ($n=8$), $w=0.5$: $N = \log(0.01)/\log(1-0.5^8) = -4.605 / -0.0039 = 1177$

Trois leçons en trois chiffres. À taux d'inliers fixe, **doubler n ($4 \rightarrow 8$) multiplie N par 16** ($72 \rightarrow 1177$) : d'où la règle d'or « toujours le plus petit échantillon minimal ». Et **faire chuter w ($0.5 \rightarrow 0.3$) multiplie N par 8** ($72 \rightarrow 567$) : la moindre dégradation du taux d'inliers coûte cher en tirages. C'est exactement le calcul qu'on pose pour budgéter un recalage d'images ou un *structure-from-motion* en télédétection ou en reconstruction 3D (chapitre 8).



Le seuil d'inlier t est le paramètre le plus délicat : trop serré, le bon modèle est rejeté ; trop lâche, des outliers passent pour inliers et corrompent le consensus. Le caler en unités de bruit via la MAD (§16.2) plutôt qu'à la main. Toujours utiliser le **minimum** de points par tirage (en mettre plus gonfle N exponentiellement, à l'encontre de l'intuition). Fixer une graine aléatoire pour la reproductibilité, et **rejeter les configurations dégénérées** (colinéarité de 4 points pour une homographie) avant d'ajuster. La formule suppose w connu ; en pratique on **l'estime en cours de route** (RANSAC adaptatif).



```
# a : image BGR (np.ndarray), b : deuxième image BGR
# Calcul d'homographie robuste entre a et b par RANSAC, avec
visualisation

if not isinstance(a, np.ndarray) or not isinstance(b, np.ndarray):
    out_a = {'erreur': 'deux images requises (ports a et b)'}
else:
    # Détection de points clés ORB et appariement
    orb = cv2.ORB_create(500)
    kp1, des1 = orb.detectAndCompute(a, None)
    kp2, des2 = orb.detectAndCompute(b, None)

    if des1 is None or des2 is None or len(kp1) < 4 or len(kp2) < 4:
        out_a = {'erreur': 'pas assez de points cles'}
    else:
        bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
        matches = bf.match(des1, des2)
        matches = sorted(matches, key=lambda m: m.distance)[:100]

        src_pts = np.float32([kp1[m.queryIdx].pt for m in
matches]).reshape(-1, 1, 2)
        dst_pts = np.float32([kp2[m.trainIdx].pt for m in
matches]).reshape(-1, 1, 2)

        # RANSAC : influence binaire — inlier ou rien
        # t = 3 px ; N calculé en interne selon la formule du §16.5
        H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC,
ransacReprojThreshold=3.0)

        n_total = len(matches)
        n_inliers = int(mask.sum()) if mask is not None else 0
        w_estime = n_inliers / n_total if n_total > 0 else 0.0

        out_a = {
            'n_appariements': n_total,
            'n_inliers': n_inliers,
            'taux_inliers': round(w_estime, 3),
            'homographie_valide': H is not None,
        }
```

16.6. Au-delà du RANSAC « vanilla » – situer chaque variante



LMedS : $\hat{\theta} = \operatorname{argmin}_{\theta} \text{ médiane}_i (e_i^2)$ (pas de seuil ; point de rupture 50 %)
MSAC : $\text{score} = \sum_i \min(e_i^2, t^2)$ (coût tronqué au lieu d'un comptage 0/1)
MLESAC : $\text{score} = \text{vraisemblance d'un mélange inliers/outliers}$

Dérivation — pourquoi raffiner le comptage binaire

Le RANSAC d'origine **compte** les inliers : tous les inliers se valent (poids 1), un point à $t - \epsilon$ et un point à 0 contribuent autant. C'est une perte d'information. L'image mentale utile : RANSAC compte les mains levées, MSAC mesure l'enthousiasme des participants. **MSAC** remplace le comptage par un coût tronqué $\sum \min(e_i^2, t^2)$: un inlier presque parfait coûte moins qu'un inlier de justesse, ce qui départage deux modèles de même nombre d'inliers — pour un surcoût quasi nul. **MLESAC** maximise la vraisemblance d'un modèle de mélange. **LMedS** (least median of squares, Rousseeuw) supprime le seuil t en minimisant la **médiane** des carrés des résidus : point de rupture 50 %, aucun paramètre à régler, mais il échoue dès que les inliers passent sous 50 % et reste peu efficace.

État de l'art moderne. **PROSAC** trie les tirages par qualité d'appariement (scores SIFT/ORB) et touche un échantillon pur bien plus vite ; **MAGSAC++** marginalise sur le seuil σ et supprime ainsi le choix fragile de t — c'est aujourd'hui un défaut solide dans les pipelines d'appariement d'images. À l'horizon, **l'estimation robuste apprise** (NG-RANSAC qui pondère les correspondances par un réseau, estimateurs de pose de bout en bout) remplace le tirage uniforme par un a priori entraîné — le pendant, pour la robustesse, de « descripteurs appris vs Hu » (chapitre 1) ou de « RAFT vs Horn-Schunck » (chapitre 9). Rien de tout cela ne périme le RANSAC classique, simple et universel : chaque variante occupe un créneau (vitesse, absence de seuil, robustesse extrême) plutôt qu'elle ne le détrône.



Deux modèles candidats, même nombre d'inliers (3 sur 4, $t = 2$), départagés par MSAC :

résidus modèle A : [0.2, 0.5, 1.9, 8.0] modèle B : [0.1, 0.2, 0.3, 8.0]

RANSAC (comptage) : A et B $\rightarrow$ 3 inliers ÉGALITÉ (ne sait pas choisir)

MSAC $\sum \min(e^2, t^2)$: A $\rightarrow 0.04 + 0.25 + 3.61 + 4 = 7.90$ B $\rightarrow 0.01 + 0.04 + 0.09 + 4 = 4.14$

$\rightarrow$ B gagne (résidus inliers plus serrés)

Là où RANSAC bute sur une égalité, MSAC préfère sans coût supplémentaire le modèle dont l'influence des inliers est mieux concentrée. La fonction d'influence de MSAC n'est plus strictement binaire : elle vaut e_i^2 pour les inliers (croissante, mais plafonnée à t^2) et t^2 pour les outliers (constante). C'est un pont entre l'influence binaire de RANSAC et l'influence écrêtée de Huber.



Ne pas attendre de miracle de LMedS sous fort taux d'outliers : son point de rupture plafonne à 50 %, en deçà de ce que RANSAC + N suffisant atteint. Les seuils des variantes ne sont **pas interchangeables** : MSAC réutilise t comme borne de troncature, MAGSAC s'en passe — comparer deux méthodes à « même t » n'a pas toujours de sens. Et toujours **ré-ajuster** le modèle final sur l'ensemble des inliers (moindres carrés, voire IRLS du §16.4) : le modèle issu du seul échantillon minimal est robuste mais peu précis.



```
# a : image BGR, b : deuxième image BGR
# Comparaison RANSAC vs MAGSAC++ pour l'homographie

if not isinstance(a, np.ndarray) or not isinstance(b, np.ndarray):
    out_a = {'erreur': 'deux images requises (ports a et b)'}
else:
    orb = cv2.ORB_create(500)
    kp1, des1 = orb.detectAndCompute(a, None)
    kp2, des2 = orb.detectAndCompute(b, None)

    if des1 is None or des2 is None or len(kp1) < 8 or len(kp2) < 8:
        out_a = {'erreur': 'pas assez de points cles'}
    else:
        bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
        matches = sorted(bf.match(des1, des2), key=lambda m: m.distance)
        [:100]
        src_pts = np.float32([kp1[m.queryIdx].pt for m in
matches]).reshape(-1, 1, 2)
        dst_pts = np.float32([kp2[m.trainIdx].pt for m in
matches]).reshape(-1, 1, 2)

        # RANSAC classique (influence binaire)
        _, mask_ransac = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC,
3.0)

        # MAGSAC++ (marginalise sur le seuil, recommandé pour
l'appariement)
        _, mask_magsac = cv2.findHomography(src_pts, dst_pts,
cv2.USAC_MAGSAC, 3.0)

        n = len(matches)
        out_a = {
            'n_appariements': n,
            'inliers_RANSAC': int(mask_ransac.sum()) if mask_ransac is
not None else 0,
            'inliers_MAGSAC': int(mask_magsac.sum()) if mask_magsac is
not None else 0,
        }
```

16.7. Tableau récapitulatif – qui a le droit de déplacer l'estimation ?

Estimateur	Fonction d'influence ψ	Sort réservé à un outlier extrême	Point de rupture	Usage typique
Moyenne / MCO	$\psi(e) = e$, non bornée	le laisse tout dominer	0 %	bruit gaussien pur, données propres
Médiane / L1	$\psi = \text{sign}(e)$, bornée constante	ignoré en amplitude	50 %	localisation robuste, filtre médian (ch. 5)
MAD	(échelle robuste, ψ bornée)	insensible	50 %	normaliser, définir le seuil d'outlier
Huber	écrêtée à $\pm k$, monotone	influence plafonnée à k (jamais nulle)	élevé*	régression douce, bundle adjustment, smooth L1 (ch. 15)
Tukey biweight	redescend à 0 au-delà de c	rejeté (poids nul)	(poids élevé, insensible*)	rejet d'aberrations franches
RANSAC / variantes	binaire (inlier 1 / outlier 0)	exclu du modèle	> 50 % possible	homographie, matrice F , recalcage (ch. 8)

* Pour les M-estimateurs, le point de rupture dépend de l'estimation conjointe de l'échelle (MAD) et, pour Tukey, de l'initialisation. La hiérarchie qui compte est celle de l'influence : non bornée $\rightarrow$ bornée constante $\rightarrow$ écrêtée $\rightarrow$ redescendante $\rightarrow$ binaire.

16.8. être robuste, c'est décider à l'avance ce qu'on refuse de croire

Le fil du chapitre tient en une fonction et une question :

$\psi(e)$ = combien le résidu e pèse sur l'estimation

question : jusqu'où une seule donnée a-t-elle le droit de déplacer le résultat ?

Tous les estimateurs de ce chapitre ne sont que des **réponses différentes à cette question**, lisibles sur la forme de ψ . Chaque section a montré comment borner cette influence un peu plus sévèrement :

moyenne / MCO	$\rightarrow \psi(e) = e$	non bornée : un point emporte tout
médiane / L1	$\rightarrow \psi = \text{sign}(e)$	bornée constante : le seul côté compte
MAD	$\rightarrow$ mètre-étalon robuste : « loin » se mesure en unités de bruit	
Huber	$\rightarrow \psi$ écrêtée à $\pm k$	on plafonne l'influence sans la nier
Tukey	$\rightarrow \psi$ redescend à 0	au-delà de c , l'outlier n'existe plus
RANSAC	$\rightarrow \psi$ binaire	inlier ou rien : on partitionne, on jette

Le lien avec le chapitre 15 boucle une équivalence exacte : ψ **est** la dérivée du coût ρ , c'est-à-dire le gradient. Le chapitre 15 sculptait des paysages de gradients pour qu'une descente atteigne une métrique ; ici, on sculpte la **même dérivée** pour qu'une observation suspecte ne puisse pas détourner l'estimation. *Borner le gradient* (stabilité d'entraînement) et *borner l'influence* (robustesse statistique) sont une seule décision regardée sous deux angles. Et le lien avec le chapitre 3 est tout aussi direct : choisir L1 plutôt que L2 (« choisir une distance, c'est déclarer ce qui compte »), c'est déjà déclarer à quel point on se méfie des écarts extrêmes — c'est déjà borner ψ .

On reconnaît enfin le **méta-fil de tout l'ouvrage**. Choisir une distance (ch. 3), c'est déclarer ce qui compte ; un filtre (ch. 5), c'est poser un a priori sur le signal ; un descripteur (ch. 1), c'est choisir ce qu'on voit et ce qu'on oublie ; une base (ch. 10), c'est choisir où le problème devient simple ; un coût (ch. 15), c'est fabriquer la pente qui mène à la cible ; et choisir un estimateur robuste, c'est **déclarer ce qu'on refuse de croire** — à l'avance, en bornant le poids qu'une observation pourra prendre. À chaque fois, le même geste : *le bon cadre rend le problème presque résolu*, et tout choix de représentation encode une hypothèse sur ce qui compte. Les statistiques robustes y ajoutent la dernière pièce — une hypothèse sur ce dont il faut **se méfier**. C'est, au fond, ce que le livre entier répétait : voir, c'est toujours décider ce à quoi l'on accorde de l'importance, et ce à quoi l'on n'en accorde pas.